SYNOPSYS®

CUSTOMER EDUCATION SERVICES

Design Compiler: RTL Synthesis Workshop

Student Guide

10-I-011-SSG-024 2017.09

Synopsys Customer Education Services
690 E. Middlefield Road
Mountain View, California 94043

Workshop Registration: http://training.synopsys.com

Copyright Notice and Proprietary Information

© 2018 Synopsys, Inc. All rights reserved. This software and documentation contain confidential and proprietary information that is the property of Synopsys, Inc. The software and documentation are furnished under a license agreement and may be used or copied only in accordance with the terms of the license agreement. No part of the software and documentation may be reproduced, transmitted, or translated, in any form or by any means, electronic, mechanical, manual, optical, or otherwise, without prior written permission of Synopsys, Inc., or as expressly provided by the license agreement.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at http://www.synopsys.com/Company/Pages/Trademarks.aspx.
All other product or company names may be trademarks of their respective owners.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

Synopsys, Inc.
690 E. Middlefield Road
Mountain View, CA 94043
www.synopsys.com

Document Order Number: 10-I-011-SSG-024
Design Compiler: RTL Synthesis Student Guide

Table of Contents

Unit i: Introduction

Facilities i-2
Workshop Goal i-3
Course Materials i-4
Workshop Target Audience i-5
Workshop Prerequisite Knowledge i-6
Introductions i-7
Agenda i-8
Agenda i-9
Agenda i-10
What Does "Synthesis" Mean? i-11
What do WE Mean by "Synthesis"? i-12
RTL Synthesis Flow i-13
Synthesis Transformations i-14
Two Modes Available for Interconnect RCs i-15
Workshop Based on Topographical Mode i-16
Two-pass Synthesis Flow i-17
Three Interfaces to Design Compiler i-18
Helpful UNIX-like DC_Shell commands i-19
Summary: Exercise i-20

Unit 1: Data Setup for DC Topographical Mode

Agenda 1-1
Unit Objectives 1-2
RTL Synthesis Flow 1-3
Unit Agenda 1-4
Technology-Specific Logic Libraries 1-5
Example: Standard Cell Definition 1-6
Generic Logic Libraries 1-7
Target Library Used During Compile 1-8
Link Library Used to Resolve Instantiated 'References' 1-9
Search Path Used for Generic Libs and Search Directories 1-10
Application Variables Are Global and Non-Persistent 1-11
One Startup File Name – Up to Three File Locations 1-12
Default .../admin/setup/.synopsys_dc.setup 1-13
Example: Project-specific (CWD) .synopsys_dc.setup 1-14
Invoking DC and Reading a Verilog RTL File 1-15
Reading a VHDL RTL File 1-16
Define a UNIX Path for the 'VHDL Library' 1-17
Good Practice: Explicitly Define the current_design 1-18
Good Practice: Explicitly link the Design 1-19
Good Practice: Run check_design 1-20

Table of Contents

HTML-based check_design report 1-21
Good Practice: Save the ddc Design Before compile 1-22
Read .ddc Design File Instead of RTL File 1-23
Save the Design After compile 1-24
Reading Designs with analyze & elaborate 1-25
Modifying Parameters with elaborate 1-26
Example: Switching Technology Libraries 1-27
Exercise: Migrating a Netlist 1-28
Exercise: Library Setup 1-29
Test for Understanding 1/2 1-30
Test for Understanding 2/2 1-31
Unit Agenda 1-32
Recall: Topographical Mode 1-33
Two-pass Synthesis Flow 1-34
Physical Data Needed by DC-Topographical 1-35
What is a Standard Cell Library? 1-36
"Layout" vs. "Abstract" Views 1-37
UNIX Structure of a Milkyway Physical Cell Library 1-38
Technology File (.tf file) 1-39
TLUPlus files 1-40
Layer Mapping File 1-41
Milkyway Design Library 1-42
Loading Physical Technology Data 1st Time 1-43
Re-Loading an Existing Design Library 1-44
Default Floorplan Constraints - 1st Synthesis 1-45
Modified FP Constraints – 1st Synthesis 1-46
Default Core Size and Shape 1-47
Defining an Exact Rectangular/Rectilinear Die/Core Area 1-48
Example: Physical Constraints File 1-49
Loading Actual Floorplan – 2nd Synthesis 1-50
Consistency of DC vs. ICC/ICC II Settings 1-51
Typical DC-T Flow: Preparing to Compile 1-52
Handy Alternative Setup Files (1 of 2) 1-53
Handy Alternative Setup Files (2 of 2) 1-54
For Further Investigation 1-55
SolvNet 1-56
Test for Understanding 1/2 1-57
Test for Understanding 2/2 1-58
Summary 1-59
Lab 1: Data Setup for DC Topographical Mode 1-60

Unit 2: Accessing Design and Library Objects

Agenda 2-1
Unit Objectives 2-2

Table of Contents

Design Objects: Verilog Perspective 2-3
Design Objects: VHDL Perspective 2-4
Design Objects: Schematic Perspective 2-5
Ports Versus Pins 2-6
Multiple Objects with the Same Name 2-7
The "get_*" Command 2-8
Library Objects 2-9
"get_*" Command Exercise 1/2 2-10
"get_*" Command Exercise 2/2 2-11
Some Handy all_* Commands 2-12
"all_*" Command Exercise 1/3 2-13
"all_*" Command Exercise 2/3 2-14
"all_*" Command Exercise 3/3 2-15
Summary 2-16
Appendix: Unit-2 Design and Library Objects 2-17
Objects and Attributes 2-18
Accessing the Synopsys Database 2-19
Accessing and Manipulating Collections 2-20
Filtering Collections 2-21
Iterating Over a Collection 2-22
Collection Versus Tcl List Commands 2-23

Unit 3: Constraints: Reg-to-Reg and I/O Timing

Agenda 3-1
Unit Objectives 3-2
RTL Synthesis Flow 3-3
Default Design Scenario 3-4
Timing Path Definition 3-5
Constraining Register-to-Register Paths 3-6
Constraining Reg-to-Reg Paths: Example 3-7
create_clock Required Arguments 3-8
Ideal Clock Behavior 3-9
Modeling Clock Trees 3-10
Modeling Clock Skew 3-11
set_clock_uncertainty and Setup Timing 3-12
Modeling Latency or Insertion Delay 3-13
Modeling Transition Time 3-14
Pre/Post Layout Clock 3-15
Constraining Input Paths 3-16
Constraining Input Paths: Example 1 3-17
Constraining Input Paths: Example 2 3-18
Constraining Output Paths 3-19
Constraining Output Paths : Example 1 3-20
Constraining Output Paths : Example 2 3-21

Table of Contents

Default I/O Clock Latency and Uncertainty 3-22
Multiple Inputs/Outputs - Same Constraints 3-23
Different Port Constraints 3-24
Exercise: Combinational Path 3-25
Constraining a Purely Combinational Design 3-26
Answer: Use a Virtual Clock! 3-27
Exercise: Combinational Design 3-28
Time Budgeting (1/2) 3-29
Time Budgeting (2/2) 3-30
Time Budgeting Example 3-31
Registered Outputs 3-32
Timing Constraint Summary 3-33
Area Constraint 3-34
Executing Commands Interactively 3-35
Sourcing Constraints Files 3-36
Executing Run Scripts in "Batch Mode" 3-37
Constraints File Recommendations (1 of 3) 3-38
Constraints File Recommendations (2 of 3) 3-39
Constraints File Recommendations (3 of 3) 3-40
Check the Syntax of Constraints 3-41
Check for Missing/Inconsistent Constraints 3-42
Check Values/Options of Clock Constraints 3-43
Check Values/Options of Port Constraints 3-44
Redirect Checks and Reports to a File 3-45
Need Help with Commands and Variables? 3-46
Summary 3-47
Lab 3: Constraints: Reg-to-Reg and I/O Timing 3-48

Unit 4: Constraints: Input Transition and Output Loading

Agenda 4-1
Unit Objectives 4-2
RTL Synthesis Flow 4-3
Factors Affecting Timing 4-4
Effect of Output Capacitive Load 4-5
Modeling Output Capacitive Load: Example 1 4-6
Modeling Output Capacitive Load: Example 2 4-7
Effect of Input Transition Time 4-8
Modeling Input Transition: Example 1 4-9
Modeling Input Transition: Example 2 4-10
Load Budgeting 4-11
Load Budgeting Assumptions 4-12
Load Budget Script 4-13
Summary 4-14
Lab 4: Constraints: Input Transition and Output Loading 4-15

Table of Contents

Unit 5: DC Ultra Synthesis Techniques

Agenda 5-1
Unit Objectives 5-2
RTL Synthesis Flow 5-3
Unit Agenda 5-4
Requirements for Optimal Synthesis Results 5-5
Default compile_ultra Optimizations 5-6
What is the DesignWare Library? 5-7
Singleton Arithmetic Optimization 5-8
Datapath Optimization: CSA Transformations 5-9
Arithmetic Expression Optimization 5-10
Critical Path Re-Synthesis 5-11
Load Splitting and Combo Logic Duplication 5-12
Library Analysis (ALIB) 5-13
Test for Understanding 5-14
Unit Agenda 5-15
Preservation of Sub-Block Pins 5-16
Auto-Ungrouping - ON by Default 5-17
Auto-Ungrouping Example 5-18
Auto-Ungrouping of DesignWare Hierarchies 5-19
Boundary Optimization - ON by Default 5-20
Controlling Boundary Optimization 5-21
Design Partitioning 5-22
Partitioning Guidelines for Design Compiler 5-23
Scan Registers: The Problem 5-24
Test-Ready Synthesis - The Solution 5-25
Automatic Shift-Register Identification 5-26
Adaptive Retiming 5-27
The Fundamentals of Adaptive Retiming 5-28
Adaptive Retiming Details 5-29
Test for Understanding 1/2 5-30
Test for Understanding 2/2 5-31
Unit Agenda 5-32
High-Effort Timing Optimization 5-33
Prioritizing Setup Timing over DRCs 5-34
Clock Network: Default DRC-Disabling 5-35
Disabling DRCs on Entire Clock Network 5-36
Default Path Groups 5-37
Problem: Sub-Critical Paths Ignored 5-38
Problem: Reg-to-Reg Paths Ignored 5-39
Solution: User-Defined Path Groups 5-40
Creating User-defined Path Groups 5-41
Enabling Near-Critical Path Optimization 5-42

Table of Contents

Prioritizing Path Groups: -weight 5-43
Example: -weight 5-44
Multi-Clock Example 5-45
TNS-Driven Placement 5-46
Reducing Large Datapath Delay 5-47
Some Things to Try … 5-48
What if the Pipeline is Still Violating Timing? 5-49
The Solution: Pipeline or Register Retiming1 5-50
RTL Recommendations for Pipelines 5-51
Optimization Phases of Register Retiming 5-52
Adaptive + Register Retiming Flow Example 5-53
Resulting Pipeline 5-54
Maintaining Registered Outputs 5-55
Critical Loop Limits Min Period Retiming 5-56
Solution #1: Separate Pipeline Hierarchies 5-57
Solution #2: Threshold Optimization 5-58
Multi-Core Optimization 5-59
Multi-Core License Requirements 5-60
Disabling Runtime-intensive Settings 5-61
Test for Understanding 1/2 5-62
Test for Understanding 2/2 5-63
Unit Agenda 5-64
Generate a Constraint Report After Compile 5-65
Generate Timing Reports for More Detail 5-66
Parallel Command Execution 5-67
Subsequent Incremental Compile 5-68
What if Still Violating Constraints? 5-69
Final Area Recovery 5-70
Synthesis Optimization Overview 5-71
Summary: Unit Objectives 5-72
Lab 5: DC Ultra Synthesis Techniques 5-73
Appendix 5-74
The Importance of Quality Source Code 5-75
Example: Coding to Allow Resource Sharing 5-76
Example: Preventing Resource Sharing 5-77
Example: 'Forced' Resource Sharing 5-78
For More Information - Documentation 5-79

Unit 6: Timing Analysis

Agenda 6-1
Unit Objectives 6-2
RTL Synthesis Flow 6-3
Timing Reports 6-4
Timing Report: Path Information Section 6-5

Table of Contents

Timing Report: Path Delay Section 6-6
Timing Report: Path Required Section 6-7
Timing Report: Summary Section 6-8
Timing Report: Options 6-9
Example -nworst vs. -max_paths 6-10
Timing Analysis Exercise (1 of 3) 6-11
Timing Analysis Exercise (2 of 3) 6-12
Timing Analysis Exercise (3 of 3) 6-13
Analysis Recommendations 6-14
Summary 6-15

Unit 7: Constraints: Multiple Clocks and Exceptions

Agenda 7-1
Unit Objectives 7-2
RTL Synthesis Flow 7-3
Multiple Synchronous Clocks 7-4
Multiple Synchronous Clock Designs 7-5
Multiple Clock Input Delay 7-6
Maximum Internal Input Delay Calculation 7-7
Multiple Clock Output Delay: Example 7-8
Maximum Internal Output Delay Calculation 7-9
Inter Clock Uncertainty 7-10
Generated Clocks 7-11
Clock Propagation Through Sequential Logic 7-12
Generated Clocks – Instantiated Register 7-13
Generated Clocks – RTL Register 7-14
Mutually Exclusive Synchronous Clocks 7-15
Logically Exclusive Clocks: Problem #1 7-16
Logically Exclusive Clocks: Solution #1 7-17
Logically Exclusive Clocks: Problem #2 7-18
Logically Exclusive Clocks: Solution #2 7-19
Multiple Clocks per Register: Problem #3 7-20
Multiple Clocks per Register: Solution #3 7-21
Multiple Clocks per Register: Example #4 7-22
Multiple Clocks per Register: Example #5 7-23
Crosstalk Analysis in ICC (II) and PT-SI: Problem #6 7-24
Crosstalk Analysis in ICC (II) and PT-SI: Solution #6 7-25
Challenge Exercise 7-26
Challenge Exercise: Solution 7-27
Asynchronous Clocks 7-28
Asynchronous Clock Designs 7-29
Synthesizing with Asynchronous Clocks 7-30
Example: Asynchronous Design Constraints 7-31
Multi-Cycle Paths 7-32

Table of Contents

Example Multi-Cycle Design 7-33
Default Single-Cycle Timing 7-34
Multi-Cycle Timing Constraint 7-35
Default Hold Check 7-36
Setting the Proper Hold Constraint 7-37
Another Example 7-38
Always Check for Invalid Exceptions 7-39
Summary 7-40
Lab 7: Constraints: Multiple Clocks and Exceptions 7-41

Unit 8: DC Graphical: SPG Flow, Congestion, Layout GUI

Agenda 8-1
Unit Objectives 8-2
What is DC-Graphical? 8-3
Synopsys Physical Guidance (SPG) Flow 8-4
Automatic Register Replication 8-5
Register Replication across Hierarchy 8-6
Problem: Resistance Variation Across Metal Layers 8-7
Solution: Net Layer Optimization (NLO) 8-8
1. Automatic Net Layer Optimization 8-9
2. Pattern-Based NLO 8-10
2. Pattern-Based NLO Examples 8-11
3. Manual NLO 8-12
NLO Results Passed on to ICC/ICC II 8-13
What is Congestion? 8-14
Detouring Around Localized Congestion 8-15
The Problem with Severe Congestion 8-16
Netlist Topology Congestion Optimization 1/3 8-17
Netlist Topology Optimization Examples 2/3 8-18
Netlist Topology Optimization Examples 3/3 8-19
Targeting/Limiting Congestion Optimization 8-20
Settings that Affect Placement/Congestion 8-21
Congestion Analysis After Synthesis 8-22
Understanding the Congestion "Heat Map" 8-23
Textual Congestion Analysis 8-24
Opening the Layout Window 8-25
Generating a Congestion Heat Map 8-26
Analyzing Congestion in the GUI 8-27
Improved Congestion Correlation 8-28
Congestion Alleviation Flow 8-29
Floorplan Exploration in DC-G 8-30
Example: Modifying Floorplan in DC-G 8-31
Trace from GUI back to RTL source 8-32
Physical Guidance 8-33

Table of Contents

Improved Timing Correlation with ICC or ICC II 8-34
Summary 8-35
Appendix 8-36
Invoking ICC Design Planning 8-37
ICC-DP Simplified and Full Version 8-38
Saving the Floorplan 8-39
Return to Design Compiler 8-40
Exiting IC Compiler Floorplanning 8-41
Incremental Synthesis 8-42
Full Re-synthesis 8-43

Unit 9: Constraints: Complex Design Considerations

Agenda 9-1
Unit Objectives 9-2
RTL Synthesis Flow 9-3
Defining a Clock: Recall Default Behavior 9-4
Defining a Clock: Different Clock Name 9-5
Input Delay with Different Clock Name 9-6
Defining a Clock: Duty-cycle 9-7
Defining a Clock: Offset 9-8
Defining a Clock: Complex 9-9
Defining a Clock: Exercise 9-10
Input Delay: Recall – Basic Options 9-11
Input Delay: Falling Clock Edge 9-12
Input Delay: Falling Clock Edge Exercise 9-13
Input Delay: Multiple Input Paths 9-14
Multiple Input Path Timing Analysis 9-15
Effect of Driving Cell on Input Delay 9-16
set_driving_cell Recommendation 9-17
Modeling External Capacitive Load on Inputs 9-18
Output Delay: Recall – Basic Options 9-19
Output Delay: Complex Output Paths 9-20
Complex Output Path Timing Analysis 9-21
Argument Ordering of TCL Commands 9-22
Pre- versus Post-CTS Clock Latencies 9-23
Pre-CTS: Ideal Clock Latencies 9-24
Post-CTS: Propagated Clock Sources 9-25
Post-CTS: Propagated Clock Objects 9-26
Independent I/O Clock Latencies (1 of 2) 9-27
Independent I/O Clock Latencies (2 of 2) 9-28
Exercise: Output Delay with Network Latency 9-29
Exercise: Output Delay with Source Latency 9-30
Different I/O versus Internal Latencies 9-31
Default External or I/O Clock Latencies 9-32

Table of Contents

Handling Different I/O vs Internal Latencies 9-33
Virtual External Clock Latencies 9-34
Included External Clock Latencies 9-35
Unit Objectives Review 9-36
OPTIONAL Lab 9: Constraints: Complex Design Considerations 9-37

Unit 10: Post-Synthesis Output Data

Agenda 10-1
Objectives 10-2
RTL Synthesis Flow 10-3
Unit Agenda 10-4
Data Needed for Physical Design or Layout 10-5
What is SDC? 10-6
What is SCAN-DEF? 10-7
Unit Agenda 10-8
Problem: Verilog assign Statements 10-9
Multiple Port Nets Cause assign Statements 10-10
Preventing Multiple Port Nets 10-11
Verilog tri Declarations Cause assign Statements 10-12
Unit Agenda 10-13
Special Characters in Netlists 10-14
Special Characters Solution: change_names 10-15
Avoiding Bit-Blasted Ports: change_names 10-16
Unit Objectives Review 10-17

Unit 11: Conclusion

Agenda 11-1
Recall: Workshop Goal 11-2
Achieving Best Timing, Congestion and Correlation 11-3
RTL Code & Steps After Loading the Design 11-4
Top-down Compile Strategy and Constraints 11-5
Constraints – continued 1/2 11-6
Constraints – continued 2/2 11-7
Topographical Mode 11-8
Ultra Optimization techniques 11-9
Synopsys Physical Guidance Flow 11-10
Congestion Analysis 11-11
DCG-DP Link 11-12
Log Files and RM Scripts 11-13
Post-Synthesis Analysis 11-14
Generate Output for Layout Tool 11-15
Getting Help Within DC 11-16

Table of Contents

Introducing DC Explorer 11-17
Design Explorer in Two-pass Synthesis Flow 11-18

Unit CS: Customer Support

Synopsys Support Resources CS-2
SolvNet Online Support CS-3
SolvNet Registration CS-4
Support Center CS-5
Other Technical Sources CS-6
Summary: Getting Support CS-7

Table of Contents

This page was intentionally left blank.

SYNOPSYS®
Silicon to Software™

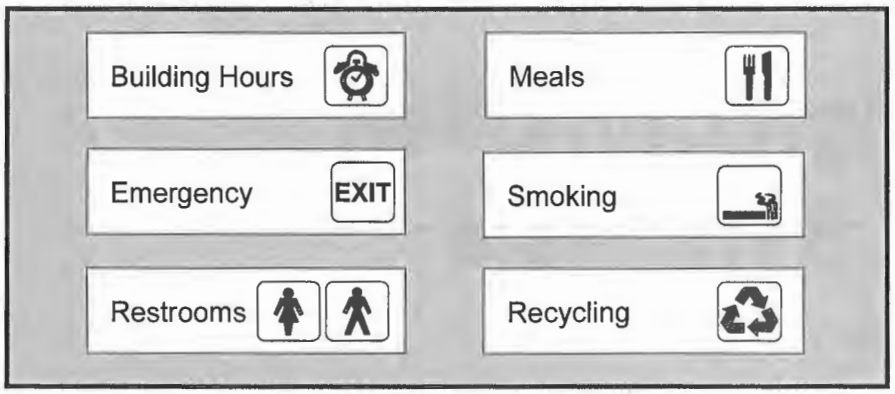
Design Compiler: RTL Synthesis

2017.09-SP4



Synopsys Customer Education Services
© 2018 Synopsys, Inc. All Rights Reserved
Synopsys 10-I-011-SSG-024

Facilities





Please turn off or silence your cell phone

i-2

©2018 Synopsys, Inc. All Rights Reserved.

Workshop Goal

DESIGN COMPILER

- **Use *Design Compiler* to synthesize a block-level RTL design to generate a gate-level netlist with acceptable post-placement timing and congestion by:**
  - Reading hierarchical block-level RTL designs
  - Loading libraries, technology data and floorplan constraints
  - Applying and checking timing constraints
  - Using timing- and congestion-focused DC Ultra and SPG optimization features
  - Analyzing timing reports
  - Analyzing layout congestion (DC Graphical feature)
  - Generating output required by physical design tools

i-3

©2018 Synopsys, Inc. All Rights Reserved.

Course Materials

- Student Workbook
- Lab Book
- Job Aid

i-4

©2018 Synopsys, Inc. All Rights Reserved.

Workshop Target Audience

ASIC digital designers who will be using *Design Compiler* to synthesize RTL designs to gates



i- 5

©2018 Synopsys, Inc. All Rights Reserved.

Workshop Prerequisite Knowledge

- **Prior experience with *Design Compiler* is not needed**
- **An understanding of basic digital ASIC design concepts is assumed, including:**
  - Combinational and sequential logic functionality
  - Setup and hold timing
- **The ability to work in a *Unix/X-windows* environment, using a text editor such as *emacs*, *vi*, *pine,* is required for labs**

i-6

©2018 Synopsys, Inc. All Rights Reserved.

Introductions

- Name
- Company
- Job Responsibilities
- Experience with Design Compiler
- Main Goal(s) and Expectations for this Course

i-7

©2018 Synopsys, Inc. All Rights Reserved.

Agenda

DAY 1

- i Introduction to Synthesis
- 1 Data Setup for DC Topographical Mode 
- 2 Accessing Design and Library Objects
- 3 Constraints: Reg-to-Reg and I/O Timing (Lecture)

i- 8

©2018 Synopsys, Inc. All Rights Reserved.

Agenda

DAY 2

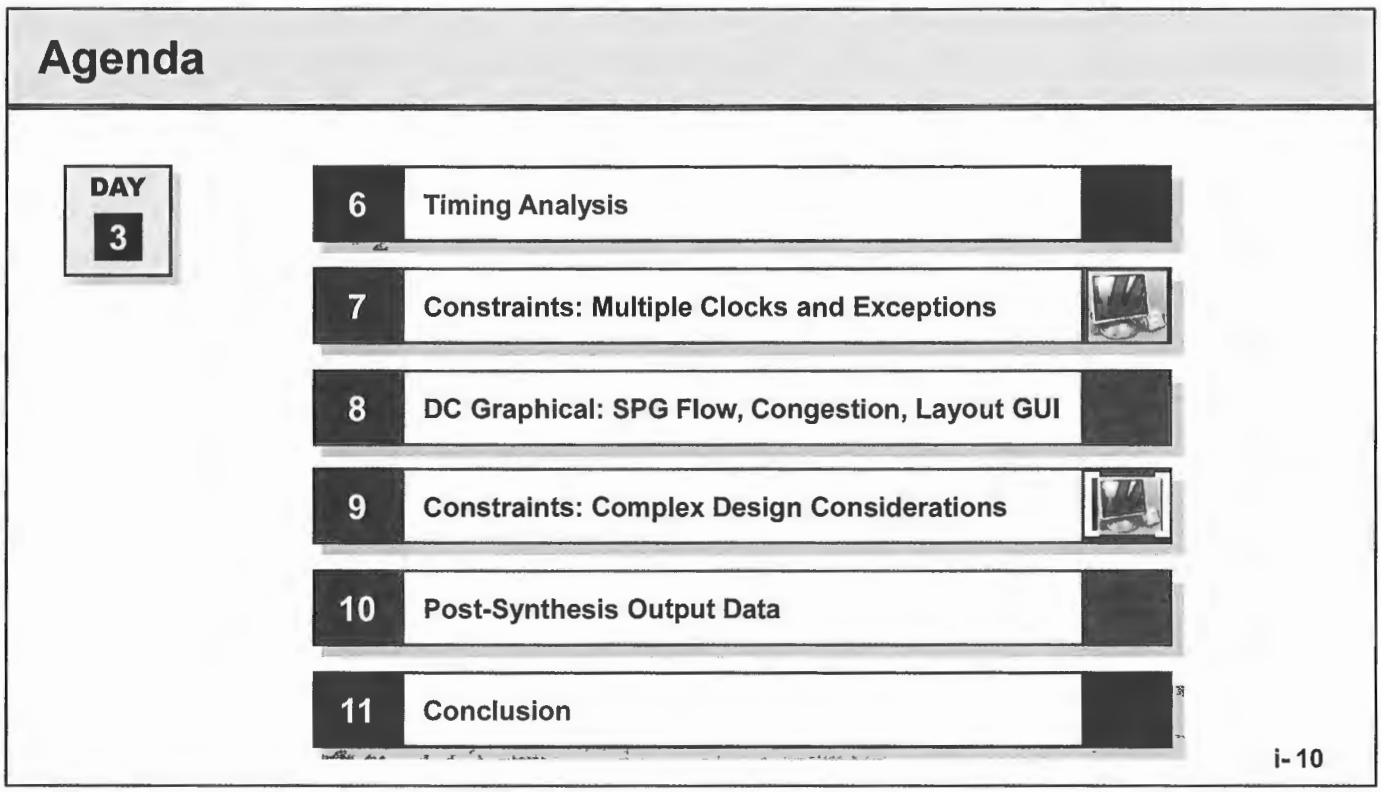
3 Constraints: Reg-to-Reg and I/O Timing (Lab) 

4 Constraints: Input Transition and Output Loading 

5 DC Ultra Synthesis Techniques 

i-9

©2018 Synopsys, Inc. All Rights Reserved.

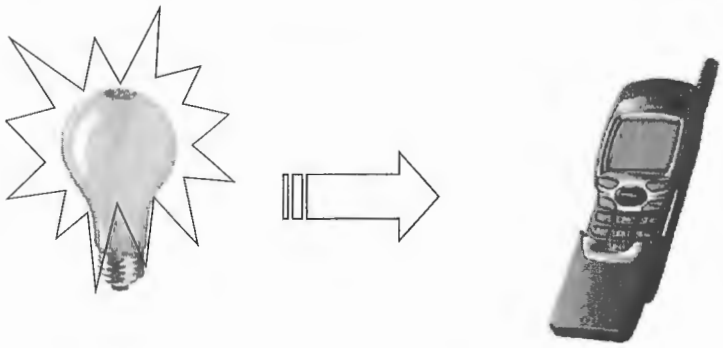


Lab 9: Optional lab that may be done after completing the workshop

©2018 Synopsys, Inc. All Rights Reserved.

What Does "Synthesis" Mean?

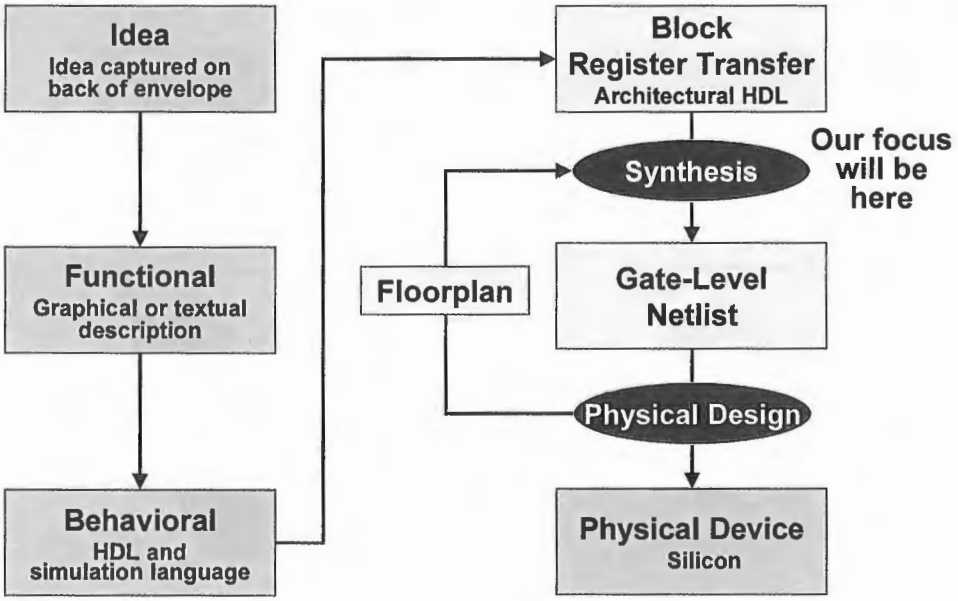
Synthesis is the *transformation* of an idea into a manufacturable device to carry out an intended function.



i- 11

©2018 Synopsys, Inc. All Rights Reserved.

What do *WE* Mean by "Synthesis"?

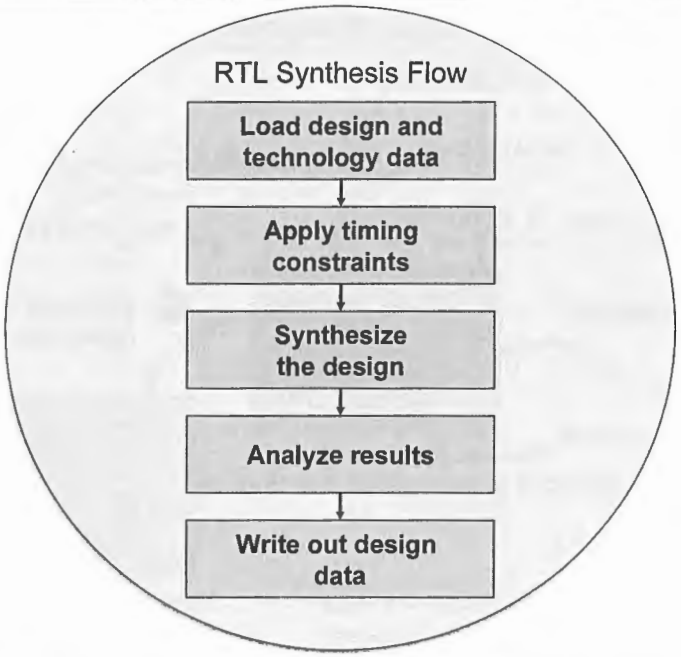


i- 12

The **functional** description serves to validate the intent of your new design. It might use stochastic processes to verify the performance.
The **behavioral** description uses HDL code to model the design at system-level. It allows you to capture key algorithms and alternative architectures.
The **register transfer level** completely models your design in detail. All clocks are defined and all registers declared. Use this HDL model as input for Design Compiler. You have enough details to synthesize trade-offs in area and timing.
The input for 'Place & Route' or physical design tools is a **gate-level** netlist. The physical design tools generate a GDSII file for the chip manufacturing.

©2018 Synopsys, Inc. All Rights Reserved.

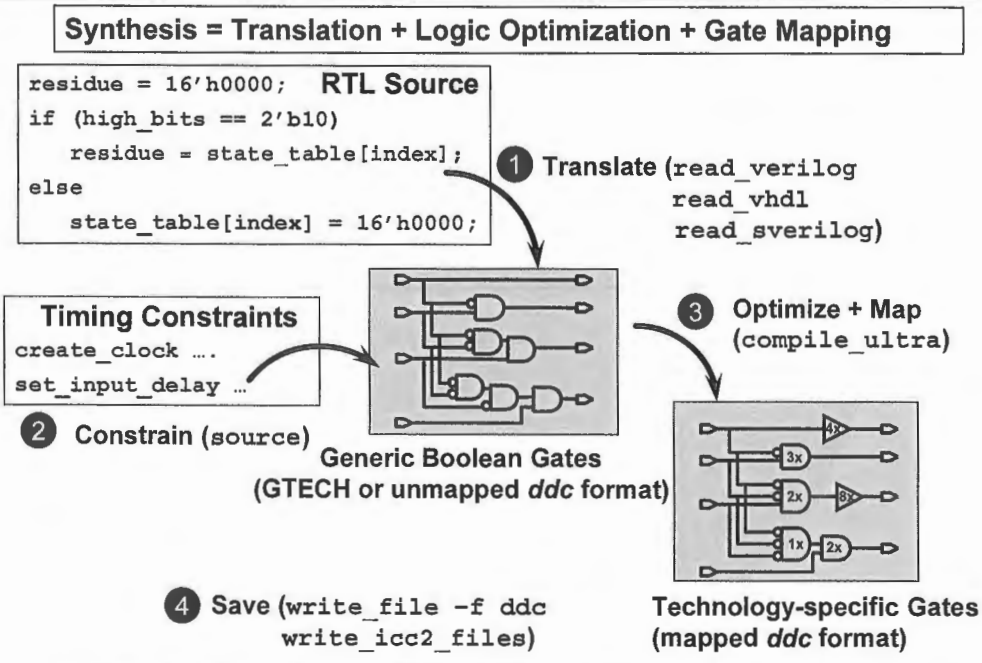
RTL Synthesis Flow



i- 13

©2018 Synopsys, Inc. All Rights Reserved.

Synthesis Transformations



The verb "to compile" is used synonymously with "to synthesize"

GTECH components have no timing or load characteristics, and are only used by *Design Compiler*.

The *ddc* format is a proprietary binary design database format which is used internally by *Design Compiler*. This format can be read in by other Synopsys tools, like *IC Compiler* and *PrimeTime*.

IC Compiler II does not read *ddc* format. Instead, the `write_icc2_files` command is used to write out all the required data (e.g. netlist, constraints, UPF, floorplan, placement, scan chains) in ASCII files.

Design and library files will be discussed in Unit 1: "Design and Technology Data".

The default Verilog language standard is 2005 and is controlled as follows:

```
set hdlin_vrlg_std 2005   (default)   [other accepted values are : 2001, 1995]
```

The default VHDL language standard is 2008 and is controlled as follows:

```
set hdlin_vhdl_std 2008   (default)   [other accepted values are : 1993, 1987]
```

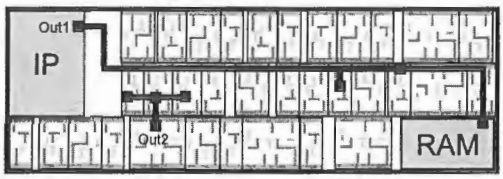
The default SystemVerilog language standard is 2012 controlled as follows:

```
set hdlin_sverilog_std 2012 (default) [other accepted values are : 2009, 2005]
```


©2018 Synopsys, Inc. All Rights Reserved.

Two Modes Available for Interconnect RCs

- **WLM mode calculates the exact same R and C for both nets below (based on fanout)**
  - The two net delays will likely be over- and under-estimated
  - The netlist will contain over- and under-buffered gates
  - May require additional iterations with physical design tools[1]
- **Topographical mode performs coarse placement with congestion-aware routing estimates during synthesis**
  - Resulting RCs correlate closely with post-layout results
  - Better post-layout QoR and TTR



Example: Two nets, each with a fanout of three, shown post Place & Route

i-15

In the above example the assumption is being made that the WLM R and C values are based on a wire length that's somewhere in between the short and long extremes shown above.

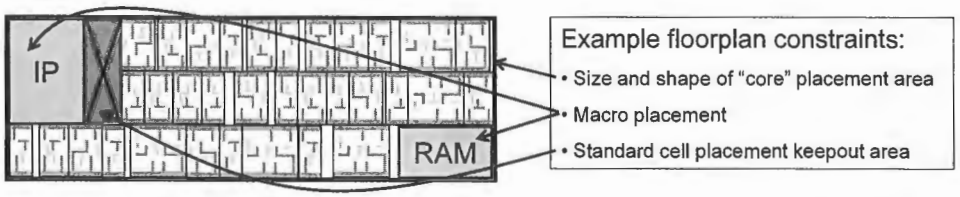
[1]The additional burden on the physical design tool to fix timing along the under-buffered path, and to reduce area and power on the over-buffered path, may require additional iterations

QoR = Quality of Results: Timing, area, power, testability, and congestion (routability)
TTR = Time to Results: The total time it takes to complete the entire design flow.

©2018 Synopsys, Inc. All Rights Reserved.

Workshop Based on *Topographical* Mode

- **This workshop is based on *topographical mode***
  - Requires physical library and technology data
  - Floorplan definition recommended for best results
  - No additional license required besides *DC-Ultra* and *DesignWare*
  - *DC-Extension* license enables additional *DC Graphical* features, including congestion analysis/optimization
- **A two-pass synthesis flow is recommended**
  - First netlist used by layout tool to design the floorplan
  - Second netlist (based on floorplan) is placed & routed

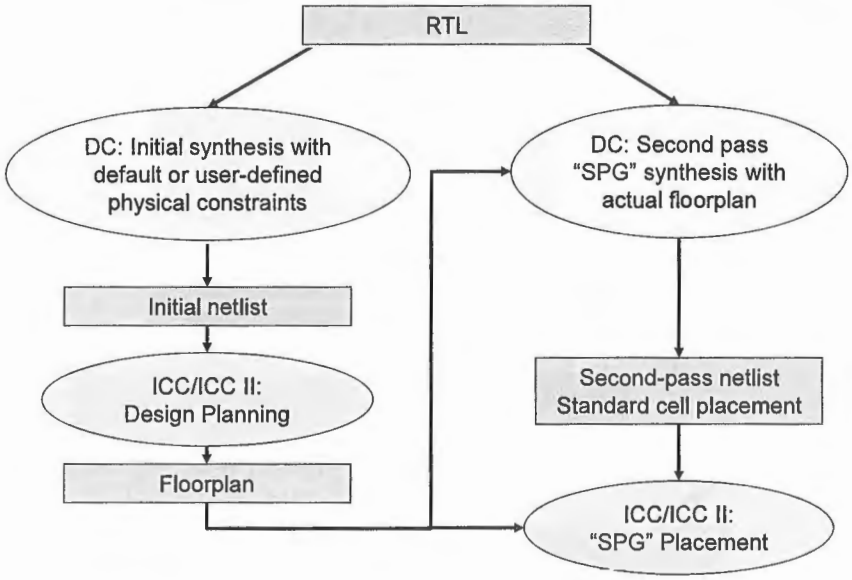


i- 16

If you are able to invoke the `compile_ultra` command, this means that you have the *DC-Ultra* and *DesignWare* licenses, which are the same licenses needed to invoke DC in *Topographical* mode. There is no additional license for *Topographical* mode.

©2018 Synopsys, Inc. All Rights Reserved.

Two-pass Synthesis Flow



i- 17

SPG = Synopsys physical guidance

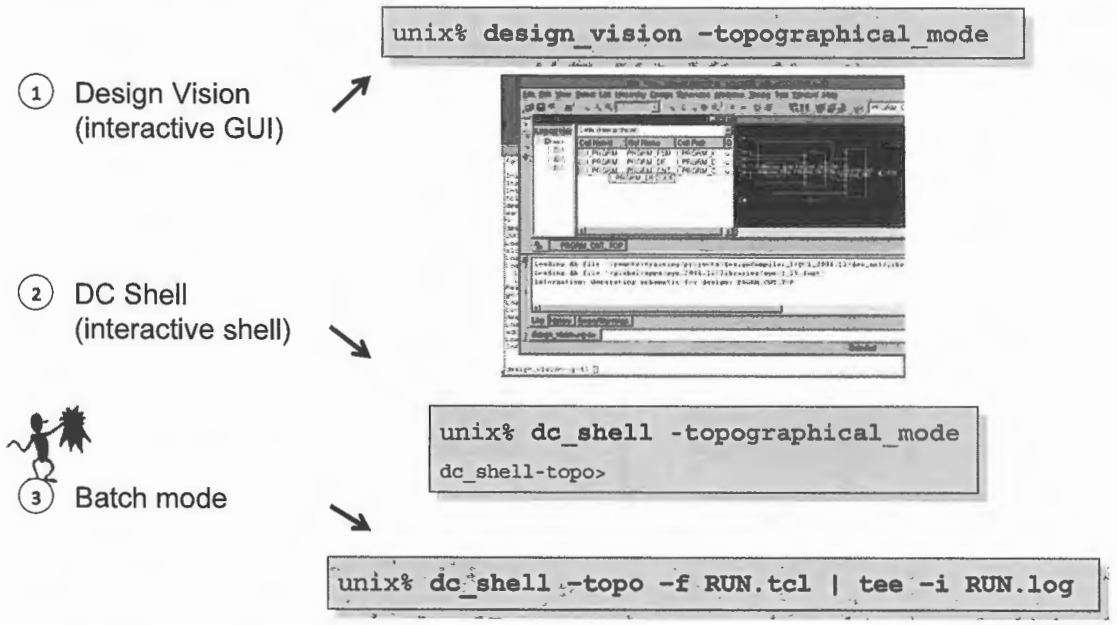
For the 2nd pass synthesis, it is recommend to perform *SPG* synthesis using `compile_ultra -spg` (discussed in later units). Using the `-spg` option, among many other features, allows the standard cell coarse placement from DC-T to be written out and passed on to ICC/ICC II. To be able to use the `-spg` option, a DC-Extension license is needed, which is referred to as "DC-Graphical" (DC-G). In ICC/ICC II, SPG placement uses the coarse placement from DC-G as a starting point to optimize and legalize the design – it skips coarse placement.

The above flow is recommended for designs with challenging "quality of results" (QoR) requirements. By re-synthesizing the design with DC-Graphical using an accurate floorplan, you are providing ICC/ICC II with a potentially better starting netlist, which may give better results. If QoR is not critical, the re-synthesis step can be skipped.

The above flow assumes that the RTL code is fairly complete when performing the initial synthesis. If the RTL contains a fair number of "black boxes" (i.e. undefined sections) you may need to perform additional iterations of design planning and synthesis, as the RTL code is solidified.

©2018 Synopsys, Inc. All Rights Reserved.

Three Interfaces to Design Compiler



i-18

As a beginner with Design Compiler the GUI interface provides a comfortable environment within which to learn the basic commands and flows in DC. Most commonly used commands are listed in pull-down menus so you do not need to memorize commands. There is also a DC-shell window at the bottom of the GUI in which the menu's underlying dc_shell commands are echoed, so you can familiarize yourself with the command syntax, and you can also type in the commands there. Once you are comfortable with the commands and analyzing reports without schematics, you can migrate to using the interactive DC-Shell environment. As you become very comfortable with DC you will want to create compile (or run) scripts (Example: dc.tcl) along with constraints file(s), and you'll be able to take advantage of executing your synthesis jobs in batch mode.

©2018 Synopsys, Inc. All Rights Reserved.

Helpful *UNIX-like* DC_Shell commands

Find the location and/or names of files[1]
```
dc_shell-topo> pwd; cd; ls
```

Show the history of commands entered:
```
dc_shell-topo> history
```

Repeat last command:
```
dc_shell-topo> !!
```

Execute command no. 7 from the history list:
```
dc_shell-topo> !7
```

Execute the last `report` command:
```
dc_shell-topo> !rep
```

Execute any UNIX command:
```
dc_shell-topo> sh <UNIX_command>
```

Get any UNIX variable value:
```
dc_shell-topo> get_unix_variable <UNIX_env_variable>
```
[2]

i-19

[1] Use `cd` very carefully, because you will change the relative starting point (`.`) for your directory `search_path`. By default ".", the current working directory, is set to the directory in which you invoked Design Compiler (shell or GUI).

[2] For example, use the following to determine if you are in a Sun or Linux environment:
`dc_shell-topo> get_unix_variable ARCH` → may return "*linux*" or "*sparcOS5*"

©2018 Synopsys, Inc. All Rights Reserved.

Summary: Exercise

Synthesis = _____ + _____ + _____

Circle the correct statements regarding *DC-Topographical mode*:

a. Enables better optimization algorithms
b. Enables coarse placement during compile
c. Allows more accurate net RC parasitic estimation
d. Requires physical library and technology files instead of logic library files

i- 20

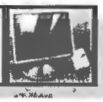
Translation + Logic Optimization + Gate Mapping

b. and c.

©2018 Synopsys, Inc. All Rights Reserved.

Agenda

DAY 1

- i Introduction to Synthesis
- 1 Data Setup for DC Topographical Mode 
- 2 Accessing Design and Library Objects
- 3 Constraints: Reg-to-Reg and I/O Timing (Lecture)

Synopsys 10-I-011-SSG-024

© 2018 Synopsys, Inc. All Rights Reserved

1-1

©2018 Synopsys, Inc. All Rights Reserved.

Unit Objectives



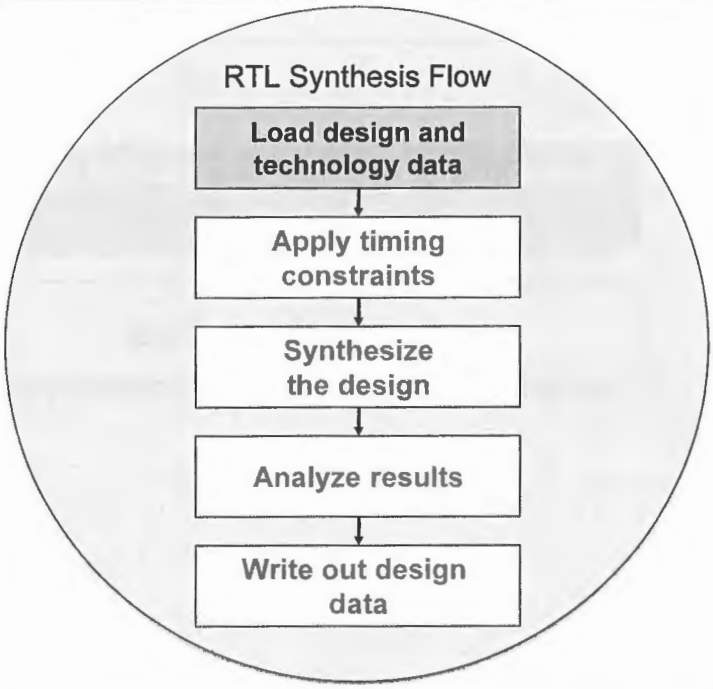
After completing this unit, you should be able to prepare a design for synthesis in *Topographical* mode, which includes :

- **Loading the RTL design and logic libraries**
  - Design to be synthesized (*Verilog/System Verilog/VHDL* RTL files)
  - Logic libraries (*db* files)
- **Loading physical technology and design data**
  - Physical or layout libraries (*Milkyway* directories)
  - Technology or routing layer definition file (*tf* file)
  - RC modeling files (*TLUPlus* & *map* files)
  - Floorplan data (*TCL physical constraints* or *DEF* file)

1-2

©2018 Synopsys, Inc. All Rights Reserved.

RTL Synthesis Flow



1-3

©2018 Synopsys, Inc. All Rights Reserved.

Unit Agenda

Loading Logic Libraries and RTL Designs

Loading Physical Data

1-4

©2018 Synopsys, Inc. All Rights Reserved.

Technology-Specific Logic Libraries

- **DC requires technology-specific logic libraries for synthesis**
  - Standard cell libraries
    - Used to implement and optimize RTL constructs (e.g. if/case, always@ or clk'event)
    - Contains logic function (e.g. AND, FF) and cell characterization data (area, timing, etc)
  - Hard macro or IP libraries
    - Instantiated in RTL (not optimized)
    - Contains only cell characterization data

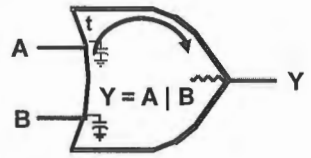
1-5

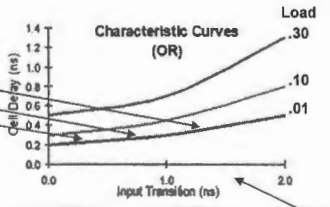
©2018 Synopsys, Inc. All Rights Reserved.

Example: Standard Cell Definition

Example of a cell description in *.lib* format

```
cell ( OR2_4x ) {                    <-- Cell name
    area : 8.000 ;                   <-- Cell Area
    pin ( Y ) {
        direction : output ;
        timing ( ) {
            related_pin : "A" ;
            timing_sense : positive_unate ;
            rise_propagation (drive_3_table_1) {
                values ("0.2616, 0.2711, 0.2831,..)
            }
            rise_transition (drive_3_table_2) {
              values ("0.0223, 0.0254, ...)
                . . . .
        function : "(A | B)";        <-- Pin Y Functionality
        max_capacitance :  1.14810 ; <-- Design Rule Constraints
        min_capacitance :  0.00220 ;     for Pin Y
    }
    pin ( A ) {
        direction : input ;
        capacitance : 0.012000;      <-- Electrical Characteristics
                . . . .                  of Pin A
```





Propagation and transition tables are "characterized" for a specific PVT corner, e.g. slowest or worst-case.

Note: During `compile_ultra` optimization, logic or buffering DRCs have higher priority than timing, by default.

Logic libraries are usually supplied as binary *.db* files

1-6

The technology library source is an *ASCII* file (known as `.lib` file), which is compiled by Synopsys **Library Compiler** to create a compiled binary version (known as `.db` file).

For a description of the library format, refer to the *Library Compiler User Guide*.

©2018 Synopsys, Inc. All Rights Reserved.

Generic Logic Libraries

- **DC also requires *generic* libraries (provided by DC) to represent RTL constructs prior to synthesis (functionality only – no timing, area, DRC)**
  - `gtech.db`: Generic gate-level logic functions (e.g. AND, XOR, FF)
  - `standard.sldb`: Generic arithmetic logic functions (e.g. +, *, >)
- **Generic libraries are used when RTL is read or loaded into the DC shell and translated into unmapped or gtech *ddc* format**
- **Three variables are used to specify these technology-specific and generic libraries:**
  - `target_library` **(technology-specific libraries)**
  - `link_library` **(technology-specific libraries)**
  - `search_path` **(paths to technology-specific and generic libraries)**

1-7

©2018 Synopsys, Inc. All Rights Reserved.

Target Library Used During Compile

- The *target* library is used during compile to create a technology-specific gate-level netlist
- DC optimization selects the smallest gates that meet the required DRCs, timing and logic functionality
- Default setting: `(printvar target_library)`

Non-existent default library name

```
target_library = your_library.db
```

- Specify the actual standard cell logic library file(s) provided by the silicon vendor or library group

```
set_app_var target_library libs/20nm_wc.db
```

Sets an application variable value: Typos generate an "Error" message - Safer than TCL "`set`" command	Reserved DC variable	Specify the library characterized for the appropriate PVT corner, for setup timing optimization

1-8

Use the following to list application variables that have been modified from their default setting:

```
get_app_var -list -only_changed_vars *
```


©2018 Synopsys, Inc. All Rights Reserved.

Link Library Used to Resolve Instantiated 'References'

- **Default:** `link_library = "* your_library.db"`



- **Used to *link* the design: 'Resolve' instantiated references in the RTL design or gate-level netlist**
  - First looks in DC memory for a matching design name (*module* or *entity* in RTL)
  - Then looks inside the listed libraries for a matching library cell name
- **Specify the standard cell and IP logic library names before the design is linked (the * should always be listed first)**

```
set_app_var link_library "* $target_library IP.db RAMS.db"
```

TCL: "Soft" quotes define a 'list' while allowing variable substitution ($var)

1-9

A 'reference' is any gate, block or sub-design that is *instantiated* in your design.
Designs and *Library cells* are key DC database "objects", to be discussed later in the Unit.

To display the value of the `link_library` variable: `printvar link_library`

TCL: Curly braces, {...}, are treated as "hard quotes": No variable substitutions or expression calculations are performed. The use of curly braces in the slide example would not work since the value of the `target_library` variable would not be substituted - `link_library` will be literally set to the character string: `* $target_library ...` instead of `* libs/20nm_wc.db ...`

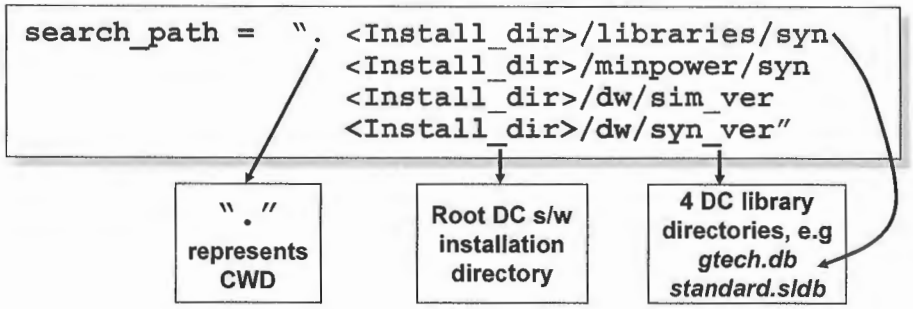
TCL: Variable substitution syntax: $*varName*. Variable name can be letters, digits, underscores: a-zA-Z0-9_. Variables do not need to be declared: All of type "*string*" of arbitrary length. Substitution may occur anywhere within a word:

Sample command	Result
`set b 66`	66
`set a b`	b
`set a $b`	66
`set a $b+$b+$b`	66+66+66
`set a $b.3`	66.3 (non-variable character "." delineates the variable)
`set a $b4`	"no such variable"
`set a ${b}4`	664

©2018 Synopsys, Inc. All Rights Reserved.

Search Path Used for *Generic* Libs and Search Directories

- **Default search directories:** `(printvar search_path)`



- **Default `search_path` includes location of required *generic* libraries**
- ***Add* search directories to the default list – do not over-write!**
  - DC looks for design and library files in the `search_path` directories

```
set_app_var search_path "$search_path \
                         mapped libs cons"
```

1- 10

Note: Whenever possible it is recommended that you define the UNIX `search_path` directories as 'relative' paths (e.g. ./mapped or mapped) rather than 'absolute' paths (e.g. /top/my_home/project/mapped). This makes your script files more portable to other UNIX environments.

©2018 Synopsys, Inc. All Rights Reserved.

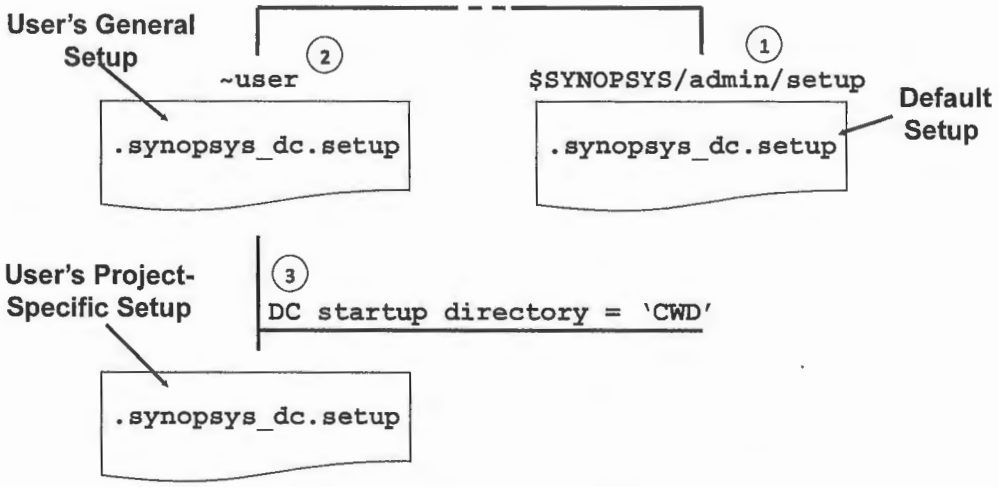
Application Variables Are Global and Non-Persistent

- **Most *application variables* in DC (and other Synopsys tools) are global and non-persistent**
  - They apply to all designs during the same tool session
  - Settings are not saved with the design
    - When the tool session ends and is later restarted, application variables revert back to their default values (if any)
- **Application variables must, therefore, be re-applied at tool start-up**
- **One convenient way do this is to include the variables in a reserved DC start-up file called `.synopsys_dc.setup`**

1-11

©2018 Synopsys, Inc. All Rights Reserved.

One Startup File Name – Up to Three File Locations



These files are automatically executed, in the order shown, upon startup of DC.

1- 12

CWD = current working directory. We will use this acronym throughout the workshop to represent the UNIX directory in which your DC session was invoked.

©2018 Synopsys, Inc. All Rights Reserved.

Default .../admin/setup/.synopsys_dc.setup

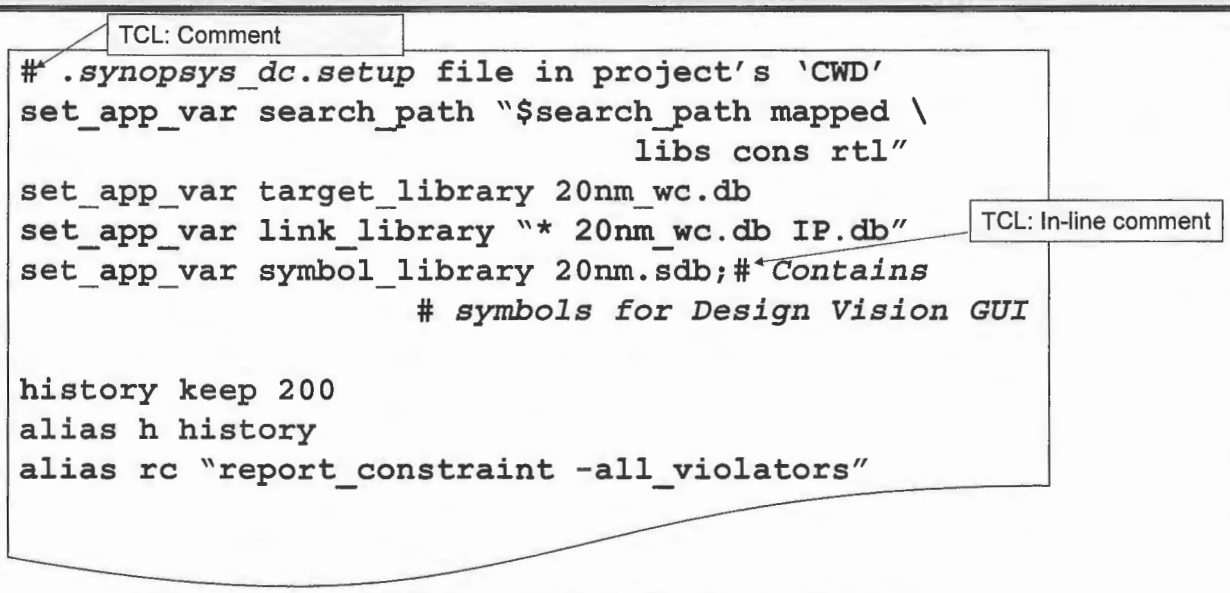
```
# .synopsys_dc.setup file in $SYNOPSYS/admin/setup
      .
      .
      .
set_app_var target_library your_library.db
set_app_var link_library {* your_library.db}
set_app_var symbol_library your_library.sdb
set_app_var search_path \
      ". <Install_dir>/libraries/syn ..."
      .
      .
```

This file is automatically executed first upon tool startup.

1-13

©2018 Synopsys, Inc. All Rights Reserved.

Example: Project-specific (CWD) .synopsys_dc.setup



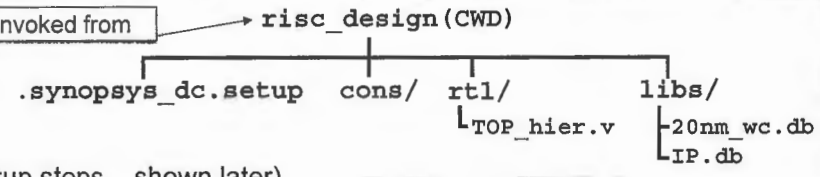
If present, this file is executed last upon tool startup and overrides previously set variables.

1-14

©2018 Synopsys, Inc. All Rights Reserved.

Invoking DC and Reading a Verilog RTL File

CWD or PWD: The directory that DC is invoked from




```
UNIX% cd risc_design
UNIX% dc_shell -topo
dc_shell-topo> ...(physical data setup steps – shown later)
dc_shell-topo> read_verilog TOP_hier.v
Loading db file '.../risc_design/libs/20nm_wc.db'
Loading db file '.../risc_design/libs/IP.db'
Loading db file '.../libraries/syn/gtech.db'
Loading db file '.../libraries/syn/standard.sldb'
Loading verilog file '.../risc_design/rtl/TOP_hier.v'
  ...
Loaded 3 designs.
Current design is 'MY_TOP'.
MY_TOP  MY_A  MY_B
```

.synopsys_dc.setup

```
set_app_var search_path     \
            "$search_path cons rtl libs"
set_app_var target_library \
            20nm_wc.db
set_app_var link_library    \
            "* $target_library IP.db"
```

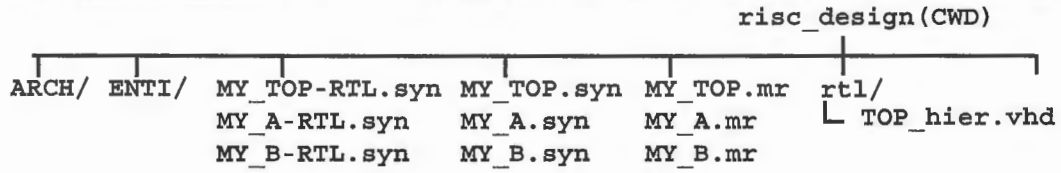
The *read* command:

- **Loads user-specified *link* libraries and default *gtech* libraries**
- **Reads *RTL* file(s) and translates into unmapped *ddc***
- **Loads designs (*Verilog* modules or *VHDL* entities) in DC memory**
- **Sets one of the designs in memory to the *current design***

1-15

©2018 Synopsys, Inc. All Rights Reserved.

Reading a VHDL RTL File



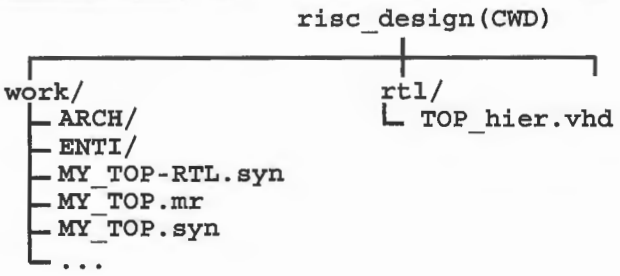

```
UNIX% cd risc_design
UNIX% dc_shell -topo
dc_shell-topo> ...
dc_shell-topo> read_vhdl TOP_hier.vhd
```

The `read_vhdl` command creates several intermediate files and directories which collectively form the "VHDL Design Library". Unless specified otherwise in your code, the VHDL design library is called 'WORK', by default. This is a VHDL-only concept. The 'WORK' files and directories are placed, by default, in the CWD.

1-16

©2018 Synopsys, Inc. All Rights Reserved.

Define a UNIX Path for the 'VHDL Library'




```
UNIX% cd risc_design
UNIX% dc_shell -topo
dc_shell-topo> ...
dc_shell-topo> define_design_lib WORK -path ./work
dc_shell-topo> read_vhdl TOP_hier.vhd
```

To keep your CWD structure relatively 'clean' you can redirect the 'VHDL Design Library' to be stored in a separate UNIX directory.

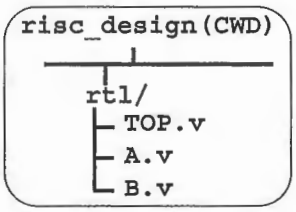
1-17

©2018 Synopsys, Inc. All Rights Reserved.

Good Practice: Explicitly Define the `current_design`

```
read_verilog TOP.v
read_verilog A.v
read_verilog B.v
   Current design is 'MY_B'
```

Last file read → current design

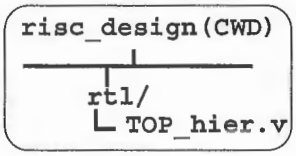



```
read_verilog {TOP.v A.v B.v}
   Current design is 'MY_TOP'
```

First file in list → current design

```
read_verilog TOP_hier.v
   Current design is 'MY_A'
```

First module in file → current design




```
read_verilog {A.v B.v TOP.v}
current_design MY_TOP
...
```

Good practice: Explicitly define the `current_design`

1-18

In the examples above it is assumed that the `search_path` includes the `rtl` directory.

TOP.v

```
module MY_TOP (IN1, IN2, ...
     input IN1, IN2, ...
     output [7:0] OUT1, ...
       MY_A U_a (.A1 ...
       MY_B U_b (.B1 ...
```

A.v

```
module MY_A (A1, A2, OUTA);
 input A1, A2;
     ...
```

B.v

```
module MY_B (B1, B2, OUTB);
      input B1, B2;
          ...
```

TOP_hier.v

```
module MY_A (A1, A2, OUTA);
 input A1, A2;
     ...

module MY_B (B1, B2, OUTB);
 input B1, B2;
     ...

module MY_TOP (IN1, IN2, ...
      input IN1, IN2, ...
      output [7:0] OUT1, ...
       MY_A U_a (.A1 ...
       MY_B U_b (.B1 ...
```


©2018 Synopsys, Inc. All Rights Reserved.

Good Practice: Explicitly `link` the Design

- ***Linking*** **locates all design and library components referenced in the current design and connects (*links*) them to the *current design***
- **Most `check_`, `report_` and `compile` commands perform an implicit *link***
  - Linking problems do not abort the command → good practice to explicitly link first

```
read_verilog {A.v B.v TOP.v}
current_design MY_TOP
link
...
```

Good practice: Explicitly `link` the design

```
Linking design 'MY_TOP'
Using the following designs and libraries:
--------------------------------------------------------------------------
* (3 designs)          .../risc_design/rtl/TOP_hier.db, etc
20nm_wc (library)      .../risc_design/libs/20nm_wc.db
hard_macros (library)  .../risc_design/libs/IP.db

1
```

Good News!!

1-19

The purpose of the `link` command is to locate all of the designs and library components referenced in the current design and connect (link) them to the current design.

The `link` command will return a 0 value if it is not able to completely link the design.

Most `check_`, `report_`, `compile` and *optimization* commands will perform an implicit link, so it is, strictly speaking, not necessary to execute an explicit link. It is, however, highly recommended to do so. If a design has a linking problem, you will be able to find the problem explicitly and correct it before executing a report or compile. On the other hand, a linking problem during an implicit link does not abort the command, so you may be compiling, or reporting on, an incomplete design.

There are several additional useful `report_` commands which will be discussed in later units.

©2018 Synopsys, Inc. All Rights Reserved.

Good Practice: Run `check_design`

- **`check_design` checks your current design for *connectivity* and *hierarchy* issues, for example:**
  - Missing ports or unconnected input pins
  - Recursive hierarchy or multiple instantiations
- **Issues warnings, errors or "No issues found"**
  - Any error returns a 0 value[1]

```
read_verilog {A.v B.v TOP.v}
current_design MY_TOP
link
check_design
...
```

Good practice: Run `check_design` after `link`

1-20

[1] The check_design command, just like the link command, returns a '1' or '0' value to indicate if the action was completed without, or with any serious problems, respectively.

If, instead of executing the above "run" commands interactively, you create a "run script" which is executed in batch mode, be aware that any errors reported by these commands do not abort the run. Your script will continue executing, and may invoke a long compile run on a design with serious problems. You can take advantage of this 1/0 return value to control your command flow, as shown below:

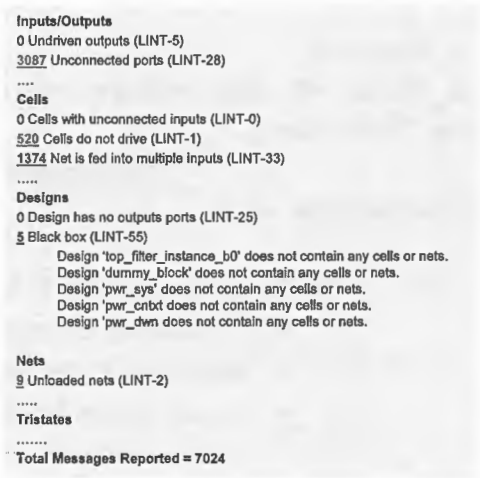
```
read_verilog {A.v B.v TOP.v}
current_design MY_TOP
if {[link] ==0} {
    echo "Linking Error"
    exit; # Exits DC if a serious linking problem is encountered
}
if {[check_design] ==0} {
    echo "Check Design Error"
    exit; # Exits DC if a check-design error is encountered
}       ; # Script continues to execute if NO problems encountered
source TOP.con
compile_ultra
  ...
```


©2018 Synopsys, Inc. All Rights Reserved.

HTML-based `check_design` report

The `check_design` report can be created in an HTML format for easier navigation and expansion

```
check_design -html check_design.html
sh firefox check_design.html
```



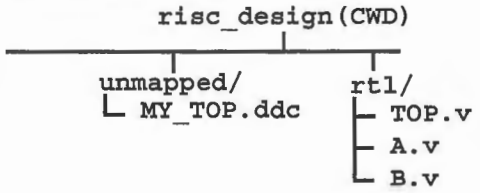
Inputs/Outputs
0 Undriven outputs (LINT-5)
3087 Unconnected ports (LINT-28)
....
Cells
0 Cells with unconnected inputs (LINT-0)
520 Cells do not drive (LINT-1)
1374 Net is fed into multiple inputs (LINT-33)
.....
Designs
0 Design has no outputs ports (LINT-25)
5 Black box (LINT-55)
Design 'top_filter_instance_b0' does not contain any cells or nets.
Design 'dummy_block' does not contain any cells or nets.
Design 'pwr_sys' does not contain any cells or nets.
Design 'pwr_cntxt does not contain any cells or nets.
Design 'pwr_dwn does not contain any cells or nets.

Nets
9 Unloaded nets (LINT-2)
.....
Tristates
.....
Total Messages Reported = 7024

1-21

©2018 Synopsys, Inc. All Rights Reserved.

Good Practice: Save the *ddc* Design Before `compile`




```
read_verilog {A.v B.v TOP.v}
current_design MY_TOP
link
check_design
write_file -f ddc -hier -output unmapped/MY_TOP.ddc
source TOP.con
...
compile_ultra
```

Discussed later

Good practice: Save *ddc* before constraining/compiling

`read_verilog/vhdl` translates RTL into unmapped *ddc* format

- Translation of large designs may take a long time
- May need to re-read the un-compiled design in the future
- `read_ddc` is faster → Save unmapped *ddc*

1-22

There are two methods for saving hierarchical designs, shown below. The first method is the more common approach. In both examples the current design is *MY_TOP*

One file containing the entire design hierarchy:

```
write_file -format ddc -hier -out unmapped/MY_TOP.ddc
```

Individual files containing parts of the design hierarchy:

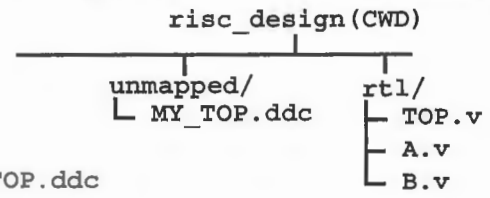
```
write_file -format ddc -out MY_TOP.ddc (writes the current_design by default)
write_file -format ddc MY_A -out MY_A.ddc
write_file -format ddc MY_B -out MY_B.ddc
```

If the original RTL code is updated you will need to re-read the RTL and re-save the *ddc* file.

©2018 Synopsys, Inc. All Rights Reserved.

Read *.ddc* Design File Instead of RTL File

```
read_verilog {A.v B.v TOP.v}
current_design MY_TOP
link
check_design
write_file -f ddc -hier -output unmapped/MY_TOP.ddc
source TOP.con
...
compile_ultra
```



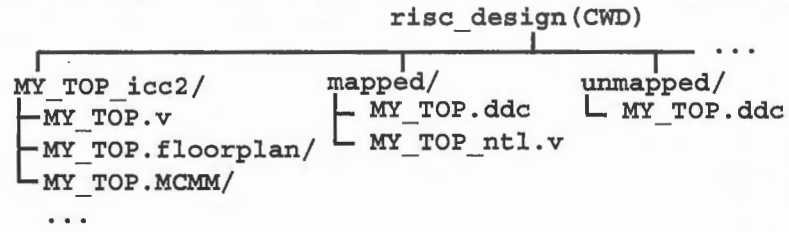

```
remove_design -designs
read_ddc MY_TOP.ddc
current_design MY_TOP
link
source TOP.con
...
compile_ultra
```

If constraints or other non-RTL design specifications change, read the *ddc* file instead of RTL – much faster loading!

1-23

©2018 Synopsys, Inc. All Rights Reserved.

Save the Design After `compile`




```
read_verilog {A.v B.v TOP.v}
current_design MY_TOP
link
check_design
write_file -format ddc -hier -output unmapped/MY_TOP.ddc
source TOP.con
...
compile_ultra
...
write_file -f verilog -hier -output mapped/MY_TOP_ntl.v[1]
write_file -f ddc -hier -output mapped/MY_TOP.ddc
write_icc2_files -out mapped/MY_TOP_icc2
```

The *ddc* format stores the design netlist, constraints, compile directives, attributes, std cell placement, and more. The *ddc* format can be read by DC or ICC, but not ICC II.

1-24

[1] The *Verilog* netlist output file is meant for 3rd party downstream tools, e.g Place&Route or simulator. These 3rd party tools may have netlist character restrictions. Because of this it is necessary to invoke the `change_names` command before writing out the netlist (discussed in a later Unit).

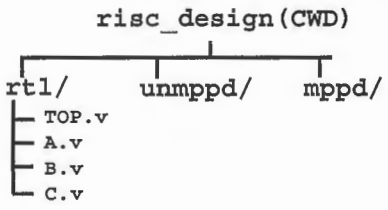
ICC = IC Compiler
ICC II = IC Compiler II

The `write_icc2_files` command writes ASCII-format design files and Tcl scripts for the current design to be loaded into IC Compiler II. Files are written into the directory specified by the `-output` option. The files include:

- o Netlist in Verilog format
- o Tcl floorplan and DEF files
- o Scan chain information in SCANDEF format
- o Constraints in SDC format
- o and more …

©2018 Synopsys, Inc. All Rights Reserved.

Reading Designs with `analyze` & `elaborate`



TOP.v

```
module MY_TOP (A, B, C, ... );
 input A, B, C, ... ;
 output [7:0] OUT1, ... ;
   MY_A U1 (.A1 (A), ...
   MY_B U2 (.B1 (B), ...
   MY_C U3 (.C1 (C), ...
```

```
analyze -format verilog {A.v B.v C.v TOP.v}
 Compiling source file .../risc_design/rtl/A.v
 Compiling source file .../risc_design/rtl/B.v
 Compiling source file .../risc_design/rtl/C.v
 Compiling source file .../risc_design/rtl/TOP.v
elaborate MY_TOP
  ...
 Current design is now 'MY_TOP'
```

No need to set the `current_design`!

1-25

analyze

- Reads source code files (Verilog or VHDL RTL)
- Checks syntax and issues errors/warnings
- Converts both Verilog and VHDL files into intermediate binary format files, placed in CWD
- Can use `define_design_lib` to redirect the files/directories to a sub-directory

elaborate

- Reads the intermediate *.pvl* files and builds the 'GTECH' design in DC memory (unmapped *ddc* format)
- Sets the current design to the specified design
- Links the specified design
- Allows specification of parameter values: *elaborate MY_TOP -parameters "N=8, M=3"*. This is not possible with the *read* command. See example on next page.
- Caution: If available, a .ddc file will be loaded and will over-write an RTL source code design that was explicitly analyzed, if the .ddc file is found in the *search_path* directories.

Note: The `read_vhdl` command ignores *configurations* while `analyze` + `elaborate` does not.

©2018 Synopsys, Inc. All Rights Reserved.

Modifying Parameters with `elaborate`

TOP.v

```
module MY_TOP (A, B, C, ... );
 parameter A_WIDTH 2;
 parameter B_WIDTH 4; ...
 input [A_WIDTH-1:0] A;
 input [B_WIDTH-1:0] B; ...
  MY_A U1 #(A_WIDTH) (.A1 (A), ...
  MY_B U2 #(B_WIDTH) (.B1 (B), ...
```

```
analyze -format verilog {A.v B.v C.v TOP.v}
elaborate MY_TOP -parameters "A_WIDTH=8, B_WIDTH=16"
```

These values override the default values defined in the RTL code

`elaborate` is the only way to change parameter values while reading in a design!

1-26

©2018 Synopsys, Inc. All Rights Reserved.

Example: Switching Technology Libraries

Old Technology Library	libs/old_45nm_wc.db
New Technology Library	libs/new_20nm_wc.db
Old Constraints	cons/old.con
New Constraints	cons/new.con

```
set_app_var target_library libs/new_20nm_wc.db
set_app_var link_library "* libs/new_20nm_wc.db"
read_verilog ORIGINAL_RTL.v
link
source cons/new.con
compile_ultra
```

When migrating to a new technology library you should ALWAYS start with the original RTL design

? **What if you only have an old *netlist* file?**

1-27

©2018 Synopsys, Inc. All Rights Reserved.

Exercise: Migrating a Netlist

Old Technology Library	libs/old_45nm_wc.db
New Technology Library	libs/new_20nm_wc.db
Old Constraints	cons/old.con
New Constraints	cons/new.con

```
set_app_var search_path "$search_path _______  _______"
set_app_var target_library _____________________
set_app_var link_library   _____________________
read_verilog OLD_NETL.v
link
source _____________________
compile_ultra -incremental
write_file -f verilog -out NEW_NTL.v
set_app_var link_library   _____________________ [1]
```

1-28

In the example above OLD_NETL.v is assumed to be a flat design with only one top-level module. It is further assumed that the original RTL code is not available, otherwise it would be best to re-compile from RTL, rather than from the gate-level netlist.

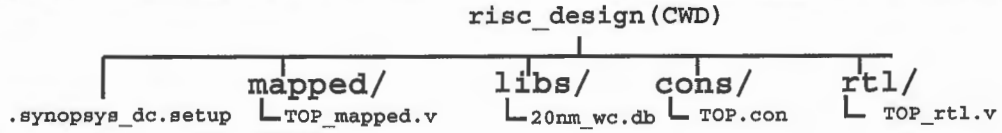
[1] Note: As long as you do not quit the DC session after *compile* it is not necessary to change the `link_library` variable. The mapped design in DC memory is already linked to the new technology library so any subsequent design analysis will use the new library characteristics. However, once you quit the DC session and subsequently re-invoke DC to read in and analyze the new netlist, you will need to first specify the new technology library as the `link_library`.

dc_shell-topo> set_app_var search_path "$search_path **libs cons**"
dc_shell-topo> set_app_var target_library **new_20nm_wc.db**
dc_shell-topo> set_app_var link_library "* **old_45nm_wc.db**"
dc_shell-topo> read_verilog OLD_NETL.v
dc_shell-topo> link
dc_shell-topo> source **new.con**
dc_shell-topo> check_timing
dc_shell-topo> compile_ultra -incremental
dc_shell-topo> write_file –f verilog –out NEW_NTL.v
dc_shell-topo> set_app_var link_library "* **$target_library**" (see "Note")

compile_ultra in the incremental (-incremental) mode runs faster and is suitable whenever a mapped design netlist is available as a starting point.

©2018 Synopsys, Inc. All Rights Reserved.

Exercise: Library Setup



.synopsys_dc.setup

```
set_app_var search_path "$search_path \
                                 mapped libs cons rtl"
set_app_var target_library 20nm_wc.db
set_app_var link_library "* $target_library"
```

```
UNIX% cd mapped
UNIX% dc_shell -topo
dc_shell-topo> ...
dc_shell-topo> read_verilog TOP_mapped.v
dc_shell-topo> link
 Warning: Can't read link_library file 'your_library.db'.
  ...
 Warning: Unable to resolve reference 'OR2_4x' in 'MYREG'.
  ...
```



What's wrong?

What's the solution?

1-29

Answers: DC was invoked from the *mapped* directory, so the `.synopsys_dc.setup` file was not loaded. To fix the problem, exit out of DC, change directories up one level to `risc_design` and then re-invoke DC. Alternatively, you can change directories to `risc_design` inside the `dc_shell`, and then `source` the `.synopsys_dc.setup` file. The only issue with this latter approach is that the log files will initially be created, and will remain in the incorrect directory.

©2018 Synopsys, Inc. All Rights Reserved.

Test for Understanding 1/2

1. **The `target_library` is used during optimization (`compile_ultra`) to select the smallest gates that meet DRCs and timing.**

   T or F?

2. **The `link_library` variable is used to resolve instantiated sub-designs and leaf cells by looking for matching cell names in the listed libraries.**

   T or F?

3. **The `link_library` and `target_library` variable settings must be re-applied within a DC session whenever a new design is read in (assuming the same libraries are used).**

   T or F?

1-30

Answer:

1. True
2. False. It first looks for matching designs in DC memory, before looking for a matching cell in the listed link libraries.
3. False. Variable settings remain unchanged when a new design is read in, so, assuming that the new design being read in uses the same libraries as the previously loaded design, the library variables do not need to be re-applied. However, when a new DC session is invoked, variable settings revert to their default values, and must therefore be re-applied (even when loading a design that was previously saved while the correct variable settings were applied – variables are not saved with the design).

©2018 Synopsys, Inc. All Rights Reserved.

Test for Understanding 2/2

5. **What is the advantage of using `set_app_var` over `set` when specifying DC variable values?**

6. **Which of the following is NOT contained in a *db* library cell definition of a NAND gate?**
   a. Timing models
   b. "Area" of the cell (in vendor-dependent units)
   c. Footprint (size and shape) of the cell
   d. Logic function definition
   e. Physical DRCs (e.g. minimum spacing, width, pitch)
   f. Logic or buffering DRCs (e.g. maximum transition, capacitance)
   g. Number of input/output pins
   h. Physical location of pins

1- 31

Answer:

5. If the user misspells an application variable, the set command does not issue any warning. It simply creates a new "user-defined" variable with the specified value. On the other hand, set_app_variable issues an Error message notifying the user that this is not an application variable, and then creates a user-defined variable.
6. C, E and H: C and H are defined in the corresponding "physical" cell library (*Milkyway* format), and E, "design rule checks" like minimum spacing, width, pitch, etcetera, are defined in the "technology file". This will be covered in the last section of this unit.

©2018 Synopsys, Inc. All Rights Reserved.

Unit Agenda

Loading Logic Libraries RTL Designs

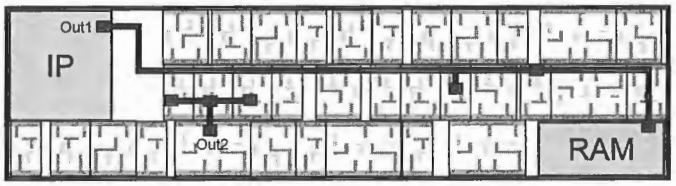
Loading Physical Data

1- 32

©2018 Synopsys, Inc. All Rights Reserved.

Recall: *Topographical Mode*

- ***Topographical mode* performs coarse placement with congestion-aware routing estimates during synthesis**
  - Used instead of WLMs to better predict parasitic net RCs
  - Requires physical library (*Milkyway* format) and technology data



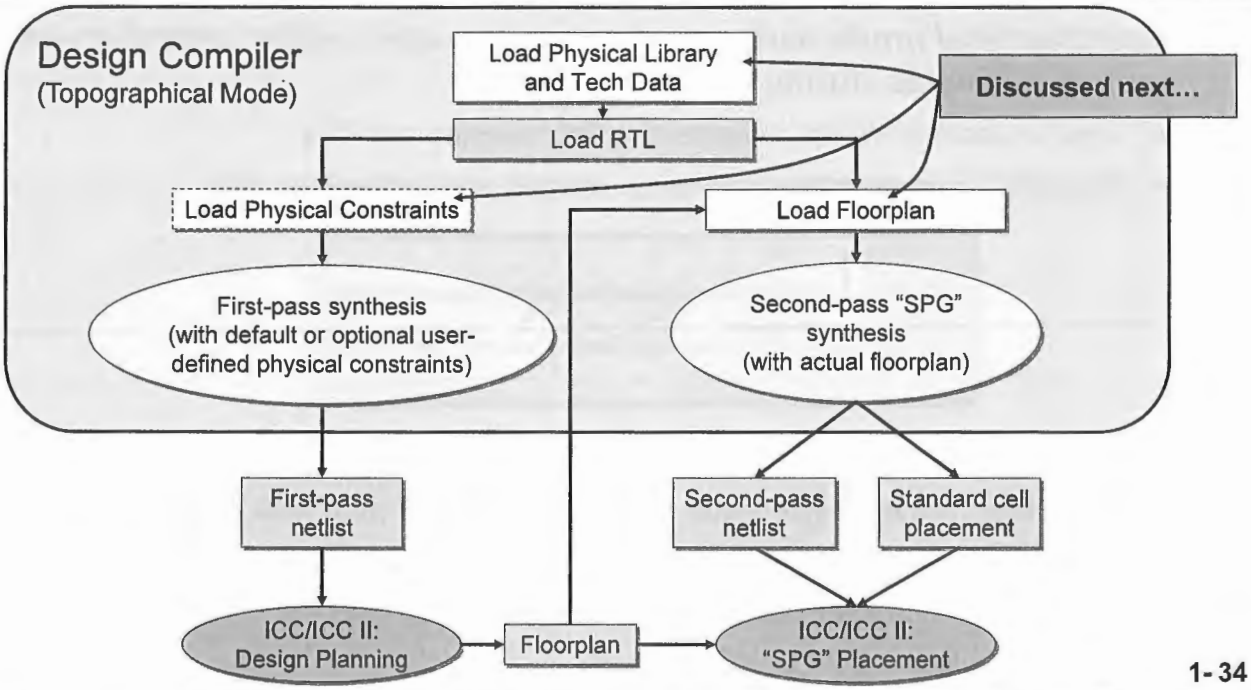
- **A two-pass synthesis flow is recommended**
  - First netlist is synthesized with default or early floorplan data
    - Used by *design planning* tool to design the actual floorplan
  - Second netlist is synthesized using the actual floorplan
    - Sent to *layout* tool for place&route

1-33

IC Compiler/IC Compiler II can be used to generate a floorplan (design planning) as well as a layout (placement, clock tree synthesis, and routing).

©2018 Synopsys, Inc. All Rights Reserved.

Two-pass Synthesis Flow



©2018 Synopsys, Inc. All Rights Reserved.

Physical Data Needed by DC-Topographical

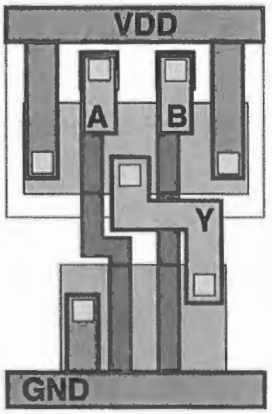
- **From the ASIC Vendor**
  - Physical cell libraries (*Milkyway*) ← Cell size and pin locations
    - Standard cells
    - IP or macro cells
  - Technology data
    - Technology File ← Definition of routing layers and vias (and more)
    - TLUPlus files ← Parasitic net RC models
    - Layer Mapping File ← Matches same layers with different names
- **From the layout team**
  - Floorplan information ← Constrains placement of std cells and macros
- **Created by DC during setup**
  - Design library (*Milkyway*) ← "Container" to hold the physical design and technology data

1-35

©2018 Synopsys, Inc. All Rights Reserved.

What is a Standard Cell Library?

- A *standard cell* is a predesigned layout of one specific basic logic gate
- Each cell usually has the same standard height
- A *standard cell library* contains a varied collection of standard cells
- Libraries are usually supplied by an ASIC vendor or library group and are unique to each foundry's technology or process parameters (for example: 22nm vs. 14 nm, 8 vs. 10 metal layers)
- DC-Topo uses physical cell libraries in *Milkyway* format



Layout View

2-Input NAND Gate

1- 36

Standard cell libraries describe the layout of basic combinational and sequential logic gates like:

- inverters
- buffers
- ands
- ors
- nands
- nors
- muxes
- latches
- flip-flops

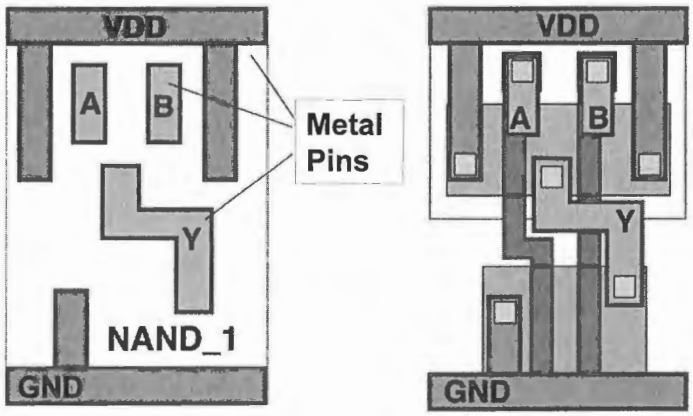
In addition to physical libraries for standard cells, physical libraries are needed for any macro cells or IP block used in the design, like RAMs, ROMs and other predesigned, standard, complex blocks.

DC-T uses the same *Milkyway* format as IC Compiler. On the other hand, ICC II uses a different physical library format called *NDM*.

©2018 Synopsys, Inc. All Rights Reserved.

"Layout" vs. "Abstract" Views

- **A standard cell library also contains a corresponding *abstract* view for each layout view**
- **Abstract views contain only the minimal data needed for Place & Route**



Abstract View (Milkyway FRAM view)

Layout View (Milkyway CEL view)

1-37

The use of abstract views instead of cell views enables smaller, more efficient design databases to be built and manipulated by P&R tools.

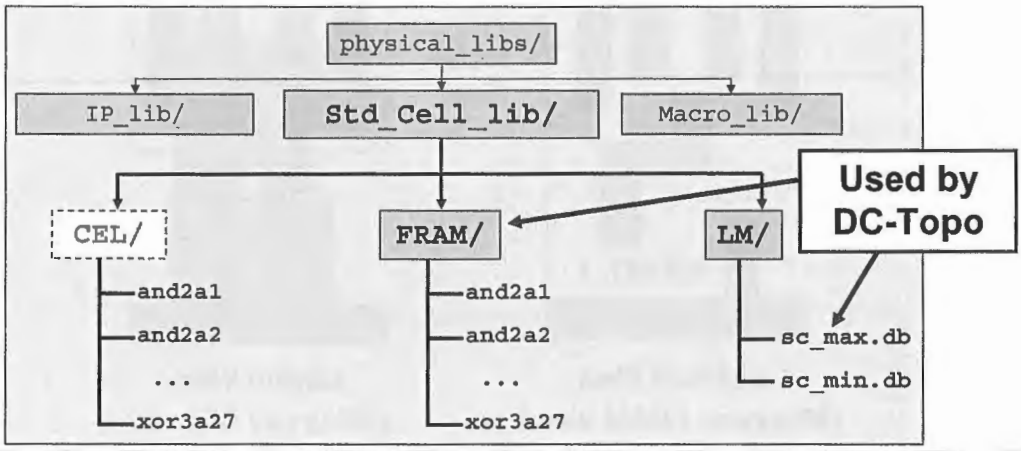
The abstract views do not contain underlying device details (poly, diffusion, n-wells, contacts). They contain only the following:
- **Outline of the cell** (The "placement" tool places each cell in available sites in the core of the chip. All the placer needs to know is the size and shape (outline) of the cell.)
- **Pin locations and layer** (Pins are usually metal connections to the underlying poly or diffusion areas of the devices. The "router" tool uses these pin locations to route metal wires or connections to each cell)
- **Metal blockages** (Areas in a cell where metal of a certain layer can not be routed because that layer is being used in the full layout of the cell, but does not show up as a pin in the abstract view. The example layout above is too simple to require blockages. These are more commonly needed in macro cells)

The layout and abstract views of all standard cells are built by a Library group. These cells are usually initially built in a standard format called GDSII and then converted into tool-specific formats (e.g. Milkyway database format for use by Synopsys tools).

©2018 Synopsys, Inc. All Rights Reserved.

UNIX Structure of a *Milkyway* Physical Cell Library

- ***Physical libraries* are *Milkyway* databases, unique to each technology, with a standard UNIX structure**
- ***Milkyway* libraries contain different *views:***
  - CEL: The full layout view (used by ICC when "taping out")
  - FRAM: The abstract view used by ICC for P&R, as well as by DC-Topo
  - LM: Logic libraries with Timing and Power info, used by ICC and DC-T



1-38

If the *LM* views contain the logic *db* libraries, DC-T will load them if the `search_path`, `target_` and `link_library` variables point to these files. DC-T will not automatically load the logic library files simply because they happen to be included in the physical library.

The full layout (CEL) view is used after Placement and Routing (P & R) are completed, and the design is ready to be "taped out". This means that the full layout representation of the design is written out in a standard format called "GDSII" – this file must contain ALL the physical data of the design, not just the "abstract" data used during P & R.

The *Milkyway* (MW) libraries are ideally provided by your library or ASIC vendor. Alternatively, they can be imported from *LEF* or *GDS* data using the Synopsys Milkyway utility (a license for Milkyway is provided with DC-Topographical). It is common for users to create their own MW libraries for memories or other IP.

For more information refer to the online documentation "Data Preparation User Guide" under "Milkyway Environment".

LEF: (Standard Cell) Library Exchange Format
GDS: Graphic Database System

©2018 Synopsys, Inc. All Rights Reserved.

Technology File (`.tf` file)

- **The *technology file* is unique to each technology**
- **Describes metal and via layer technology parameters**
  - Number and name designations for each layer/via
  - Dielectric constant for technology
  - Physical and electrical characteristics of each layer/via
  - Design rules for each layer/Via (minimum wire widths and wire-to-wire spacing, etc.)
  - Units and precision for electrical units

```
Technology  {
    name              = "cb1314"
    dielectric        = 3.73e-05
    unitTimeName      = "ns"
    timePrecision     = 1000
    unitLengthName    = "micron"
        ...
    }
Layer "METAL1"{
    layerNumber       = 14
    maskName          = "metal1"
    pitch             = 0.41
    defaultWidth      = 0.16
    minWidth          = 0.16
        ...
```

1-39

©2018 Synopsys, Inc. All Rights Reserved.

TLUPlus Files

- **Contain R and C look-up tables**
- **Model *UDSM* process effects**
  - Conformal Dielectric
  - Metal Fill
  - Shallow Trench Isolation
  - Copper Dishing:
    - Density Analysis
    - Width/Spacing
  - Trapezoid Conductor

```
# TECHNOLOGY=1314max
# CONDUCTOR cm4   { THICKNESS=0.90 WMIN=0.44 SMIN=0.46 RPSQ=0.060}
# CONDUCTOR cm3   { THICKNESS=0.35 WMIN=0.20 SMIN=0.21 RPSQ=0.081}
# CONDUCTOR cm2   { THICKNESS=0.35 WMIN=0.20 SMIN=0.21 RPSQ=0.081}
# CONDUCTOR cm1   { THICKNESS=0.26 WMIN=0.16 SMIN=0.18 RPSQ=0.109}
# CONDUCTOR poly { THICKNESS=0.18 WMIN=0.13 SMIN=0.20 RPSQ=10.965}
   ...
# DIELECTRIC cm4_extra1 { THICKNESS=0.23 ER=3.7 }
# DIELECTRIC cm4_extra2 { THICKNESS=0.03 ER=8.1 }
# DIELECTRIC cm4_extra3 { THICKNESS=0.09 ER=3.7 }
   ...
```

1-40

UDSM: Ultra Deep Submicron

The data shown above is an example of the top portion of a *TLUPlus* file. The actual *TLUPlus* data is binary, and is "compiled" from an *ASCII* "Interconnect Technology Format" (*ITF*) file. The commented constructs shown above are included from the *ITF* file.

The *ITF* file contains information about the physical characteristics of the technology (layer thickness, dielectrics, etc.). Synopsys' RC Extraction tool (*StarRC*) is used to "compile" ITF and convert the data into capacitance and resistance models, which are written out as *TLUPlus* files.

Ideally the *TLUPlus* data should be available from the foundry or ASIC vendor. Alternatively, the user is able to perform the "ITF-to-TLUPlus" conversion if the required tool licenses are available.

For more information on modeling UDSM process effects, consult the "Process Characterization Interface" section of the "StarRC User Guide" (online documentation).

©2018 Synopsys, Inc. All Rights Reserved.

Layer Mapping File

The *layer mapping* file matches the *technology* file layer/via names with the *TLUPlus* layer/via names

TLUPlus file

```
# TECHNOLOGY=1314max
# CONDUCTOR cm4   { THICKNESS=0.90 WMIN=0.44 SMIN=0.46 RPSQ=0.060}
# CONDUCTOR cm3   { THICKNESS=0.35 WMIN=0.20 SMIN=0.21 RPSQ=0.081}
# CONDUCTOR cm2   { THICKNESS=0.35 WMIN=0.20 SMIN=0.21 RPSQ=0.081}
# CONDUCTOR cm1   { THICKNESS=0.26 WMIN=0.16 SMIN=0.18 RPSQ=0.109}
# CONDUCTOR poly { THICKNESS=0.18 WMIN=0.13 SMIN=0.20 RPSQ=10.965}
   ...
```

Technology file

```
Technology  {
   name             = "cb1314"
   dielectric       = 3.73e-05
   unitTimeName     = "ns"
   timePrecision    = 1000
   unitLengthName = "micron"
      ...
   }
Layer  "METAL1" {
   layerNumber      = 14
   maskName         = "metal1"
   pitch            = 0.41
    ...
```

Layer Mapping file

```
cb13_4m.map
::::::::::::::
conducting_layers

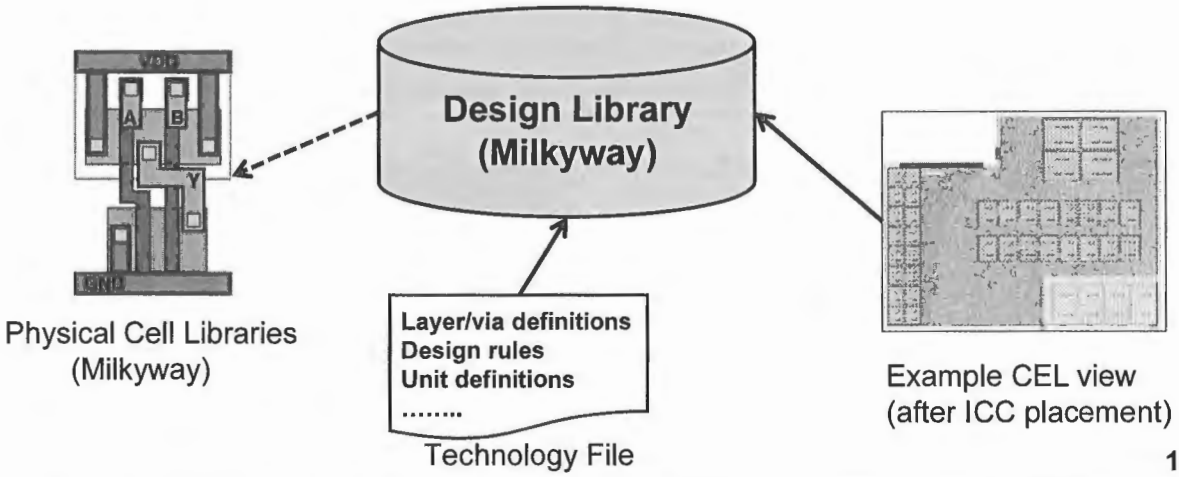
POLY            poly
METAL1          cm1
METAL2          cm2
METAL3          cm3
METAL4          cm4
```

1-41

©2018 Synopsys, Inc. All Rights Reserved.

Milkyway *Design Library*

- **The user must also create a *Milkyway design library***
  - Acts as a "container" to hold technology data and pointers to physical cell libraries
  - Later, ICC will add design layout or *CEL* views: Representations of the physical design after placement, clock tree synthesis, routing, etc



1-42

The physical cell libraries (standard cell and, if applicable, macro cell or IP libraries), along with the technology file are all provided by the back-end vendor or library group.

The arrow to the physical cell libraries is dashed to indicate that the physical libraries are not included in the design library – instead, the design library contains "pointers" to the libraries.

©2018 Synopsys, Inc. All Rights Reserved.

Loading Physical Technology Data 1st Time

First DC-T session

```
set_app_var mw_reference_library "./mw_libs/sc    \
                                  ./mw_libs/macros"
set_app_var mw_design_library ./ORCA_design_lib

create_mw_lib \
  -technology              ./mw_libs/tech/20nm.tf    \
  -mw_reference_library $mw_reference_library     \
                        $mw_design_library
open_mw_lib ./ORCA_design_lib
check_library
set_tlu_plus_files \
   -max_tluplus  ./mw_libs/tlup/20nm_max.tluplus \
   -tech2itf_map ./mw_libs/tlup/20nm.map
check_tlu_plus_files
```

Creates a design library with a user-defined name, which holds the physical design and technology data

Consistency check between logic and physical libraries

Consistency check between TLUPlus and technology files

1-43

Starting with version 2017.09-SP2, DC can read *nxtgrd* files as an alternative to *TLUPlus* files. The usage for *nxtgrd* files is the same as *TLUPlus* files. To use an *nxtgrd* file for extraction:
`set_tlu_plus_files -max_tluplus NXTGRD_file_max.nxtgrd -tech2itf_map design.map`

In this example, the command sets `NXTGRD_file_max.nxtgrd` as the file to be used for the maximum condition calculation of the resistance and capacitance. Supporting *nxtgrd* files aligns Design Compiler with the StarRC and PrimeTime tools so you no longer need to maintain a separate *TLUPlus* file. You must generate the *nxtgrd* file using the *grdgenxo* tool version N-2017.12 or later.

©2018 Synopsys, Inc. All Rights Reserved.

Re-Loading an Existing Design Library

Subsequent DC-T sessions

```
open_mw_lib $mw_design_library

set_tlu_plus_files                                \
   -max_tluplus  ./libs/tlup/20nm_max.tluplus \
   -tech2itf_map ./libs/tlup/20nm.map
```

1-44

In *IC Compiler*, the *TLU+* info is actually stored in the *Milkyway* CEL inside the design library. After setting the TLU+ files, when you SAVE your "CEL", the next time you open up the design library and the CEL, `report_tlu_plus` will load and then list the TLU+ files. If you were to subsequently close the CEL, `report_tlu_plus` still lists the TLU+ files – the loaded TLU+ files remain available for that session. Once you exit the session and reload the design library, `report_tlu_plus` files lists nothing, but if you then open the CEL, the TLU+ files are loaded and listed. So, in IC Compiler there is no need to re-apply TLU+ settings, since that info is saved and restored with the CEL.

In *Design Compiler*, however, since we only open the design library (`open_mw_lib`) but never open the CEL (`open_mw_cel`), the user must explicitly re-apply the TLU+ settings for that DC session.
Note: If you keep the session open, and then close the design library (`close_mw_lib`), `report_tlu_plus` will still list the TLU+ files.

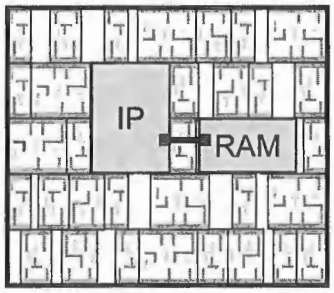
When the design library is first created with `create_mw_lib`, and the cell libraries are specified with the `-mw_reference_library` option, the full path to each library is stored in the design library. In a subsequent DC-T session, if the UNIX directory location of those cell libraries has changed, for whatever reason, the `open_mw_lib` command above will not be able to locate the cell libraries. You can then use the following command to specify the new cell library locations prior to opening the design library:

```
set_mw_lib_reference $mw_design_library \
    -mw_reference_library $new_mw_cell_library
```


©2018 Synopsys, Inc. All Rights Reserved.

Default Floorplan Constraints - 1st Synthesis

- **Topographical mode uses *default* floorplan constraints, if none are specified**
  - Square core placement area
  - Core size based on 60% utilization
  - Places IP or macro cells along with standard cells
  - Assigns arbitrary port locations to top-level design



Example DC-T interconnect estimate using default floorplan constraints

- **Acceptable for 1st pass synthesis**
  - This netlist will only be used to design the floorplan
  - Maximum netlist QoR is not crucial at this point

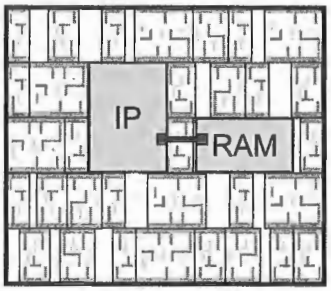
1-45

©2018 Synopsys, Inc. All Rights Reserved.

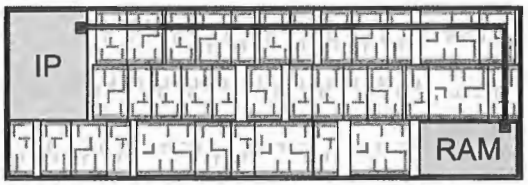
Modified FP Constraints – 1st Synthesis

Prior to having a finalized floorplan available, if any floorplan constraints are expected to be significantly different than the default
→ Specify them before the 1st synthesis run (after loading RTL design)

- May alter the design of the floorplan
- May be able to skip 2nd pass synthesis, if this "estimated" floorplan matches the actual floorplan



Example DC-T interconnect estimate using default floorplan constraints



Example DC-T interconnect estimate using modified floorplan constraints: Core size/shape, macro cell and pin placement

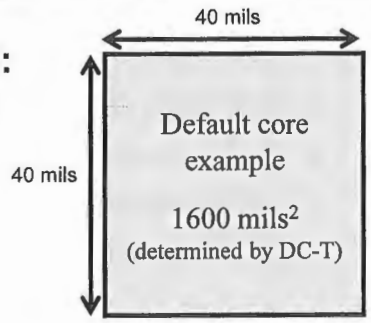
1-46

©2018 Synopsys, Inc. All Rights Reserved.

Default Core Size and Shape

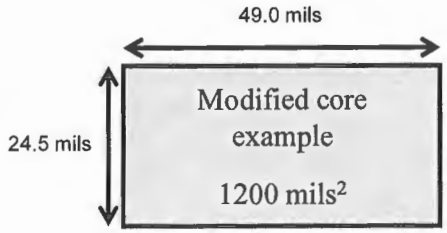
- **DC-T creates a default core placement area based on:**
  - Standard cell utilization area of 60% (see definition below)
  - Aspect ratio of 1 (height / width)

```
set_utilization  0.6
set_aspect_ratio 1
```



- **User can change the size and shape as needed:**

```
set_utilization  0.8
set_aspect_ratio 0.5
```



1-47

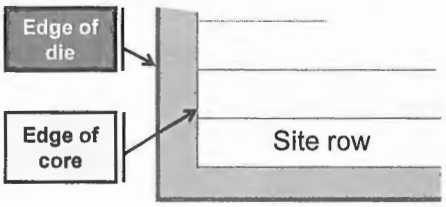
Standard cell utilization is defined as follows in the `set_utilization` command man page:

```
            Total_standard_cell_area
-------------------------------------------------
(Core_area - Blockage_area - Macro_cell_area)
```


©2018 Synopsys, Inc. All Rights Reserved.

Defining an Exact Rectangular/Rectilinear Die/Core Area

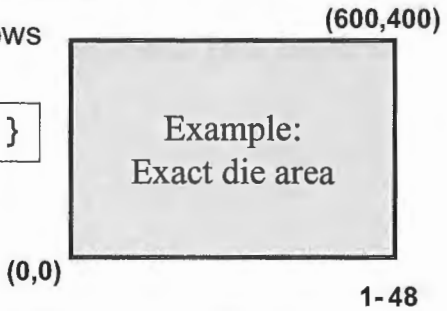
- **A rectangular or rectilinear die area can be specified with exact coordinates**
  - Overrides default utilization/aspect ratio constraints
  - Cell boundary or die area encompasses the core area:
    - Standard cells/macros are placed in core area
    - Pads and I/O pins are placed in die area



- **By default, DC-T assumes placement core area = die area**
  - Unless `create_site_row` is used to define core site rows

```
create_die_area -coordinate { {0 0} {600 400} }
```

Use `-polygon` *to define a rectilinear shape*



1-48

While `create_site_row` is the recommended constraint to define the placement site rows which make up the core area (and hence define the core area boundary), the following command, which defines a rectangular boundary of the core area (with no site rows) is still supported:

```
set_placement_area -coordinate {0 0 600 400}
```


©2018 Synopsys, Inc. All Rights Reserved.

Example: Physical Constraints File

MYDESIGN_phys_cons.tcl

```
create_die_area -coordinate {{0 0} {600 400}}

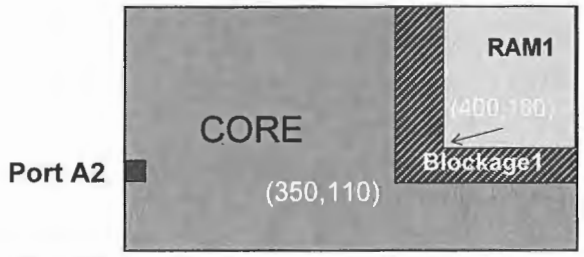
set_cell_location -coordinate {400 160} -fixed \
                  -orientation {N} RAM1

create_placement_blockage -coordinate {350 110 600 400}

create_terminal -bbox {0 40 2 42} -layer M1 \
                -direction {left} -port  A2

report_physical_constraints
```

Lists applied physical constraints



See complete list of physical constraints below

1-49

Physical constraints:

- set_aspect_ratio
- set_utilization
- create_die_area
- create_site_row
- set_port_side
- create_terminal
- set_cell_location
- create_placement_blockage
- create_voltage_area
- create_bounds
- create_route_guide
- create_net_shape
- create_user_shape
- set_pin_physical_constraints
- create_pin_guide
- create_via_master
- create_via
- create_track
- set_keepout_margin
- compute_polygons

©2018 Synopsys, Inc. All Rights Reserved.

Loading Actual Floorplan – 2nd Synthesis

- **After *design planning* in ICC/ICC II, floorplan data is captured either in:**
  - *DEF* file (with supplemental *floorplan Tcl*, as applicable)
  - *Floorplan Tcl* file
- **If floorplan data is captured in *DEF* format:**

```
extract_physical_constraints [-allow_physical_cells] \
                                        MYDESIGN.def; # Primary FP data
read_floorplan MYDESIGN_suppl_fp.tcl;                 # Suppl. FP data
```

- **If floorplan data is captured as *floorplan Tcl* commands:**

```
[read_floorplan MYDESIGN_fp.objects; # Physical-only cell creation ]
read_floorplan MYDESIGN_fp.tcl;      # Floorplan data
```

* The option/command in square brackets "[ ]" is needed if the physical-only cells exist in the floorplan

1-50

design planning = the iterative process of creating a floorplan; *DEF* = Design exchange format.
DC Graphical supports *DEF* versions through 5.8. The *2017.09-SP2* version of reference methodology recommends *DEF* version 5.7.
The `extract_physical_constraints` command ignores irrelevant information from the *DEF* file. The list of physical constraints imported can be found in the man page.

In ICC, *DEF* is created with `write_def`, but *DEF* may not contain all the required floorplanning constructs (see below), so a "supplemental" Tcl command file may be needed, along with *DEF*, to fully define the floorplan. In ICC II, the only way to write out floorplan data is with the `write_floorplan` command, which creates a primary *DEF* file, plus any required supplemental floorplan *Tcl* file(s) automatically. The following ICC/ICC II floorplanning commands, command options, and settings are not captured in *DEF*, so if used, must be included in a supplemental Tcl file, to be read in DC by `read_floorplan`. Note: DC understands ICC format commands; When ICC II syntax differs from ICC (marked below), ICC II commands must be converted to ICC format, which is the format that DC understands:

```
create_voltage_area
create_rp_groups
create_bounds -dimension
create_placement_blockage -no_register|-no_rp_group|-no_pin|-buffer_only|
                                  -no_hard_macro (ICC/DC)
create_placement_blockage -type (ICC II)
set_ignored_layers
set_app_var placer_max_cell_density_threshold (ICC/DC)
set_app_options -name place.coarse.max_density (ICC II)
```

(continued on next page)

©2018 Synopsys, Inc. All Rights Reserved.

Consistency of DC vs. ICC/ICC II Settings

For best correlation between DC and ICC/ICC II, it is important to ensure similar timing analysis and physical design settings between the tools:

- **Automatically change a key sub-set of DC timing variable settings to match default ICC or ICC II settings**

```
set_compile_spg_mode  icc | icc2
```

- **Run the "Consistency Checker" to compare actual physical design and timing settings between DC and ICC or ICC II**
  - Write out the environment data file from both DC and ICC/ICC II
  - Run the consistency checker from Unix to compare the files
    See SolvNet Doc ID 026366: DC-T versus ICC consistency
    See SolvNet Doc ID 2112936: DC-T/ICC versus ICC II consistency
  - Update any mismatched DC settings after running the checker

1-51

(continued from previous page)

In ICC, it is also possible to capture the floorplan using only floorplan *Tcl* commands (no *DEF*), generated by `write_floorplan`. However, this floorplan *Tcl* file from ICC does not contain `set_keepout_margin` commands: Either explicitly apply any keepout margin commands in DC (as a supplemental floorplan Tcl file), or a script can be executed in ICC to convert keepout margins into `create_placement_blockage` commands, which are saved in the floorplan Tcl file (see SolvNet doc ID 040831: "Creating Keepouts for Macros That are Saved by the `write_floorplan` Command").

If the floorplan contains *physical-only* cells (e.g. tap or end-cap cells), and the floorplan is defined in a floorplan *Tcl* file, the user will need to ensure that a separate file, containing a `create_cell` command for each physical-only cell, is loaded before loading the floorplan *Tcl* data file. On the example on the previous page, this is represented by the file called `MYDESIGN_fp.objects`.
If reading a *DEF* file as primary floorplan data, the *objects* file is not needed, because physical-only cells are included in *DEF*. *IC Compiler II's* `write_floorplan` includes physical-only cells in the primary *DEF* file. However, the `-allow_physical_cells` option must be used with `extract_physical_constraints`, to retain the physical-only cells.

The `set_compile_spg_mode` command automatically changes tool settings to improve timing correlation with IC Compiler II or IC Compiler. The command updates the settings before optimization starts during `compile_ultra`, `compile_ultra -incremental`, and `optimize_netlist`.

©2018 Synopsys, Inc. All Rights Reserved.

Typical DC-T Flow: Preparing to Compile

```
set_app_var mw_reference_library $MW_REF_LIBS
set_app_var mw_design_library $MW_DESIGN_LIB
create_mw_lib -technology $TECH_FILE \
              -mw_reference_library $mw_reference_library $mw_design_library
open_mw_lib $mw_design_library
check_library
set_tlu_plus_files -max_tluplus $TLU_MAX_FILE -tech2itf_map $MAP_FILE
check_tlu_plus_files
read_verilog MYDESIGN.v
link
# If some physical constraints are known before 1st compile
source  MYDESIGN_phys_cons.tcl ;
# Once an actual floorplan is available
extract_physical_constraints MYDESIGN.def; read_floorplan MYDESIGN.tcl
source timing.cons
compile_ultra
```

1-52

©2018 Synopsys, Inc. All Rights Reserved.

Handy Alternative Setup Files (1 of 2)

.synopsys_dc.setup

```
history keep 200
alias rc "report_constraint -all_violators"
...
source common_setup.tcl
source dc_setup.tcl
```

No edits!

common_setup.tcl

```
set ADDL_SEARCH_PATH "mapped rtl libs cons"
set TARGET_LIBS       20nm_wc.db
set ADDL_LINK_LIBS   "IP.db MEM.db"
set SYMBOL_LIBS       20nm.sdb
set TECH_FILE         ./libs/tech/20nm.tf
set MW_REF_LIBS      "./libs/mw/sc ./libs/mw/macros"
set MW_DESIGN_LIB     ORCA_design_lib
set TLU_MAX_FILE     ./libs/tlup/20nm_max.tluplus
set MAP_FILE         ./libs/tlup/20nm.map
```

User edits only this file!

1-53

©2018 Synopsys, Inc. All Rights Reserved.

Handy Alternative Setup Files (2 of 2)

No edits!

dc_setup.tcl

```
set_app_var search_path "$search_path $ADDL_SEARCH_PATH"
set_app_var target_library            $TARGET_LIBS
set_app_var link_library           "* $target_library \
                                      $ADDL_LINK_LIBS"
set_app_var symbol_library            $SYMBOL_LIBS
set_app_var mw_reference_library      $MW_REF_LIBS
set_app_var mw_design_library         $MW_DESIGN_LIB
create_mw_lib -technology             $TECH_FILE       \
        -mw_reference_library  $mw_reference_library \
                               $mw_design_library
open_mw_lib                    $mw_design_library
check_library
set_tlu_plus_files -max_tluplus       $TLU_MAX_FILE    \
                   -tech2itf_map      $MAP_FILE
check_tlu_plus_files
```

1-54

©2018 Synopsys, Inc. All Rights Reserved.

For Further Investigation

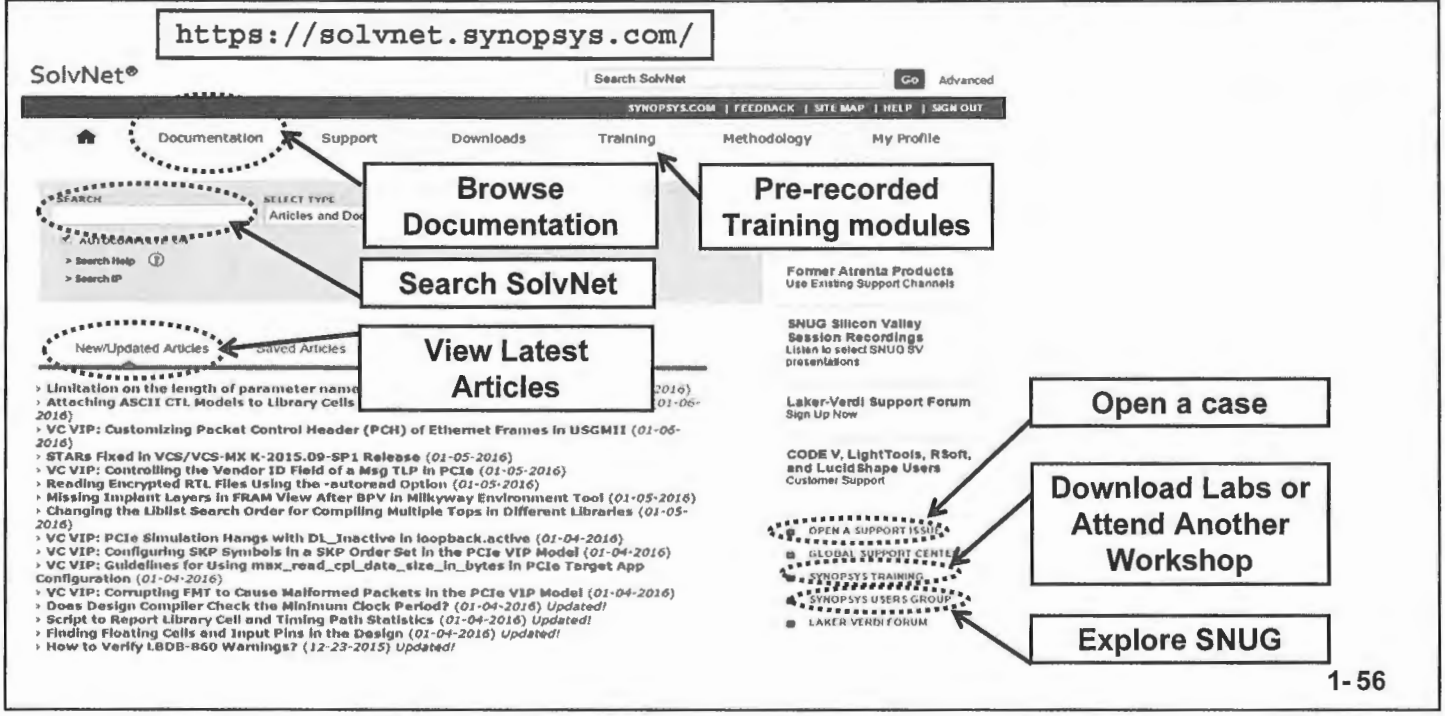


After the workshop, where will you go to find solutions for your Design Compiler problems?

1-55

©2018 Synopsys, Inc. All Rights Reserved.

SolvNet



SolvNet is your one stop shop for:

- Searching answers to questions
- Browsing the latest online documentation
- Opening a Support Center Case
- Link to Training Classes from Customer Education Services
- Exploring SNUG (alternative to the "snug-universal" link)
- and More …..

©2018 Synopsys, Inc. All Rights Reserved.

Test for Understanding 1/2

18. Circle all that apply: *DC-Topo* performs under-the-hood coarse placement to

a) Calculate accurate WLM RC's
b) Create accurate metal routes
c) Generate a netlist with better timing correlation to layout
d) Estimate net parasitics based on estimated routing
e) Create an accurate floorplan

19. What is the difference between a *Milkyway* cell and design library?

20. Circle all that apply: A *Milkyway* cell library *FRAM* view

a) Uses more machine memory than its CEL equivalent
b) Defines physical DRCs
c) Defines each cell's outline, as well as pin locations

1-57

Answers:

18. C and D.
19. Cell libraries contain physical layout descriptions of standard cells, macro and IO pad cells, which are used in a layout design. A design library is a container that holds the physical design or layout, and all the associated physical data.
20. C. FRAM views use less memory than CEL views, which is why they are used by DC-Topo. Physical DRCs (min spacing/width/pitch design rule checks) are defined in the technology file, not the FRAM view.

©2018 Synopsys, Inc. All Rights Reserved.

Test for Understanding 2/2

21. **Parasitic net RC modeling is supplied to DC-Topo with *TLUPlus* and *layer mapping* files. T or F?**

22. **A *technology file* defines the number of metal routing layers as well as logic DRCs for a specific technology. T or F?**

23. **In the DC-Topo two-pass synthesis flow, when are you likely to**

a) Apply physical constraints?

_____

a) Load a DEF/Floorplan-Tcl file?

_____

1-58

Answers:

21. True.
22. False. In addition to the number of metal layers, it defines physical DRCs: minimum spacing, width and pitch rule.
23. a: Prior to generating the 1st pass netlist, when an actual floorplan is not usually available, but some floorplan constraints are known, or can reasonably be guessed.
    b: Before 2nd pass synthesis, once an actual floorplan has been created by ICC/ICC II or a 3rd party layout tool.

©2018 Synopsys, Inc. All Rights Reserved.

Summary

You should now be able to prepare a design for synthesis in Topographical mode, which includes :

- **Loading the RTL and logic libraries**
  - Design to be synthesized (*Verilog/System Verilog/VHDL* RTL files)
  - Logic libraries (*db* files)
- **Loading physical technology and design data**
  - Physical or layout libraries (*Milkyway* directories)
  - Technology or routing layer definition file (*tf* file)
  - RC modeling files (*TLUPlus* & *map* files)
  - Floorplan data (*TCL physical constraints* or *DEF* file)

1- 59

©2018 Synopsys, Inc. All Rights Reserved.

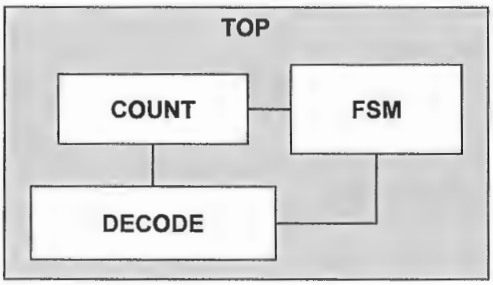
Lab 1: Data Setup for DC Topographical Mode

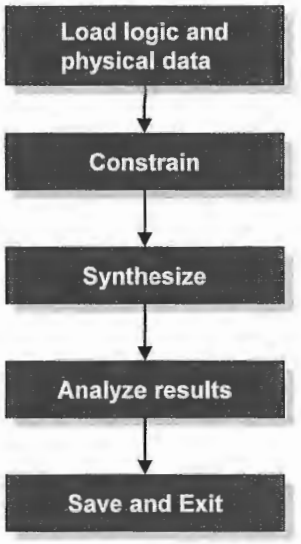


Lab 1: Data Setup for DC Topographical Mode

60 minutes

Perform setup steps to load logic and physical data.
Explore *Design Vision* and the synthesis flow.





1- 60

©2018 Synopsys, Inc. All Rights Reserved.

Agenda

DAY 1

- i Introduction to Synthesis
- 1 Data Setup for DC Topographical Mode 
- 2 Accessing Design and Library Objects
- 3 Constraints: Reg-to-Reg and I/O Timing (Lecture)

© 2018 Synopsys, Inc. All Rights Reserved

©2018 Synopsys, Inc. All Rights Reserved.

Unit Objectives



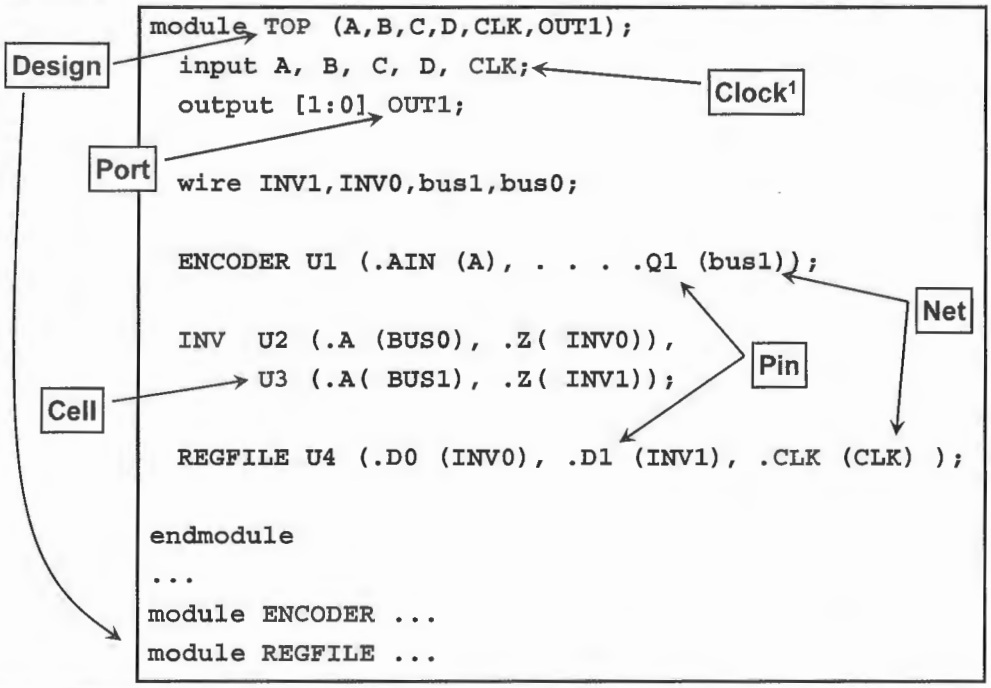
After completing this unit, you should be able to:

- **Access key *design* and *library objects* using `get_` and `all_` *collection* commands**

2-2

©2018 Synopsys, Inc. All Rights Reserved.

Design Objects: Verilog Perspective

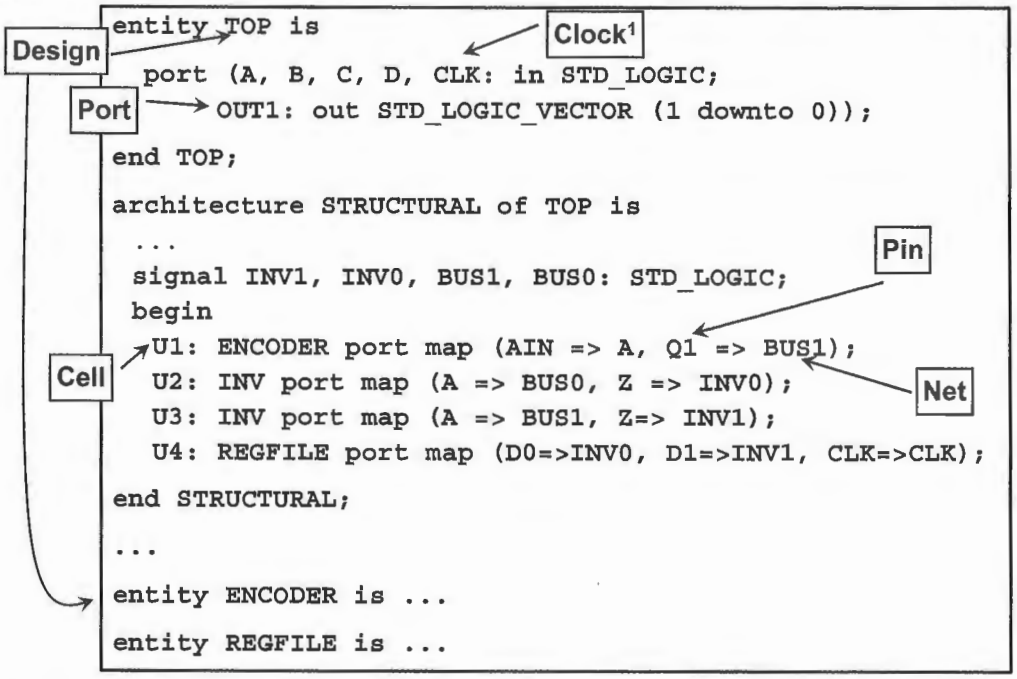


2-3

[1] Clock is a user-defined object within Design Compiler memory.

©2018 Synopsys, Inc. All Rights Reserved.

Design Objects: VHDL Perspective

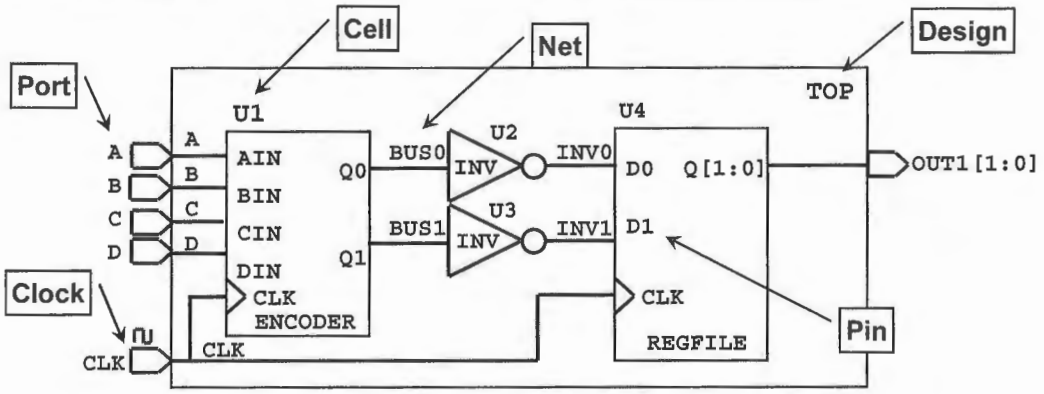


2-4

[1] Clock is a user-defined object within Design Compiler memory.

©2018 Synopsys, Inc. All Rights Reserved.

Design Objects: Schematic Perspective



Designs in DC memory: {TOP ENCODER REGFILE}
Cells in TOP: {U1 U2 U3 U4}
Pins in TOP: {U1/AIN U1/BIN ... U4/Q[0] U4/Q[1]}

? Why is INV not a design object like ENCODER or REGFILE?

2-5

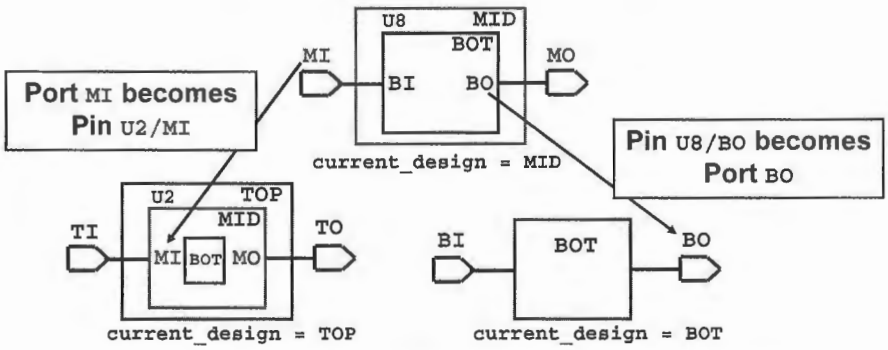
Pins are always associated with a specific instance or cell: cell_name/pin_name

ANSWER: There is no `module` or `entity` definition for `INV` (because it is a library cell).

©2018 Synopsys, Inc. All Rights Reserved.

Ports Versus Pins

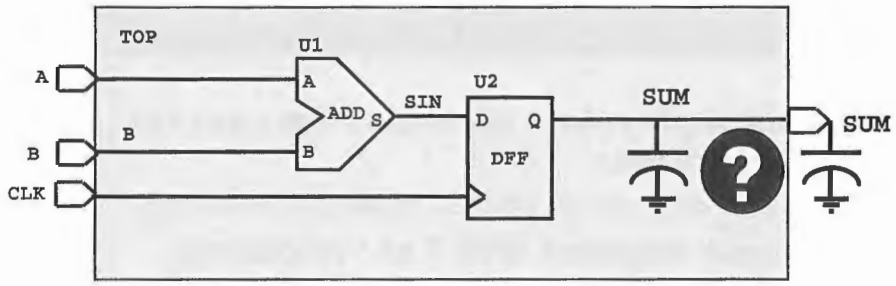
- **'Ports' are the inputs and outputs of the *current design***
  - They become 'Pins' if the current design moves up to a parent design
- **'Pins' are the inputs and outputs of any cell that is instantiated in the *current design***
  - They become ports if that instantiated design is made the current design



2-6

©2018 Synopsys, Inc. All Rights Reserved.

Multiple Objects with the Same Name




```
set_load 5 SUM
```

Does "SUM" refer to the port or the net object?

Does it matter onto which object DC places the load?

2-7

The SUM argument refers to BOTH the port and the net called SUM – it is ambiguous. If the load is applied to the NET, the user-specified load over-rides any internally-calculated net capacitance. If the load is applied to the PORT, the load is ADDED to any internally calculated net capacitance, so the total load on the register will be higher, and the delay will be larger.

In the example above DC decides which of the two objects it will apply the load to – it turns out that for this command DC chooses a port object. What if that is NOT what you had intended?

To avoid ambiguity issues it is highly recommended to always specify the object type.

©2018 Synopsys, Inc. All Rights Reserved.

The "get_*" Command

```
dc_shell-topo> set_load 5 [get_nets SUM]
```

- **The "get_*" commands return objects in the current_design, in DC memory, or in libraries:**
  - Can be used stand-alone or embedded in other commands
- **Objects may be used together with ? or * wildcards:**

```
set_load 5 [get_ports addr_bus*]
set_load 6 [get_ports "Y??M Z*"]
```

TCL: Embedded command

- **"get_*" commands return a collection of database objects that match the argument(s)**
  - If no matching objects are found, an empty collection is returned

2-8

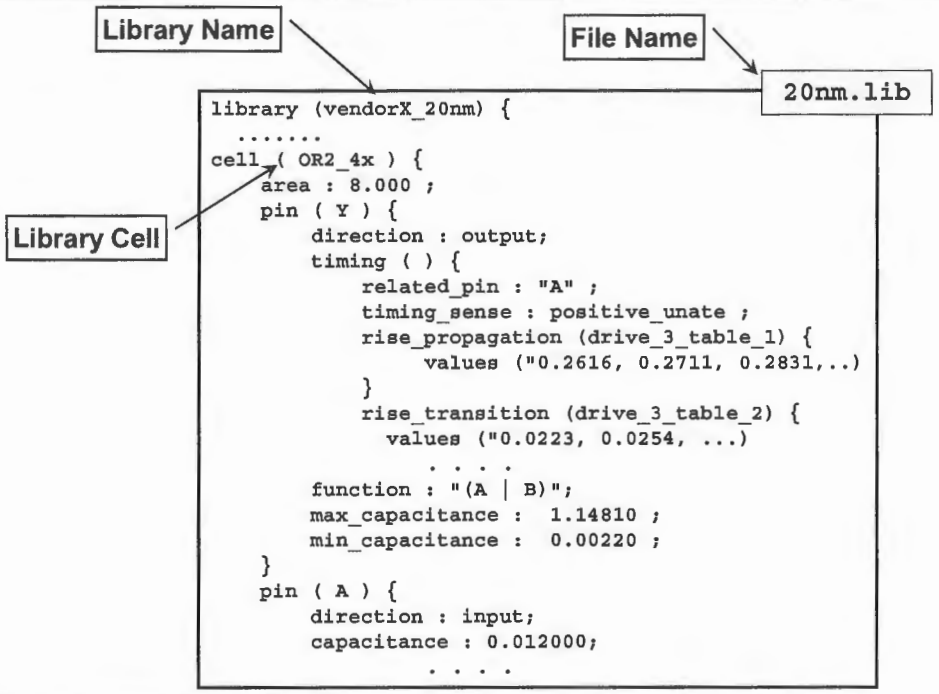
Some[1] useful get commands:	Returns a collection[2] containing the names of the following design or library objects:
get_cells	design *cells* (instances) in the current design, and in sub-blocks with -hier
get_clocks	*clocks* defined from the current design, or above, on pins/ports at the current design or sub-block level, defined by create_clock or create_generated_clock. Does not report clocks that were created from within a lower-level design
get_designs	*designs* loaded in DC memory (not just in the current design)
get_libs	*libraries* loaded in DC memory
get_lib_cells <libname/cellname>	*library cells* in the specified library
get_lib_pins <libname/cellname/pinname>	*pins* of specified *library cells*
get_nets	*nets* at the current design level, and sub-block level with -hier
get_pins <cell/pin>	input/output *pins* of cells (instances) at the current design level, and sub-block level with -hier; pins are always associated with a cell.
get_ports	primary input/output/bidir *ports* at the current design level, and sub-block level with -hier

To see the full list of get_ commands enter help get_*.

A *collection* is a set of objects which is accessible by its *collection handle*, using special collection commands. More details on collections are included in the appendix of this unit.

©2018 Synopsys, Inc. All Rights Reserved.

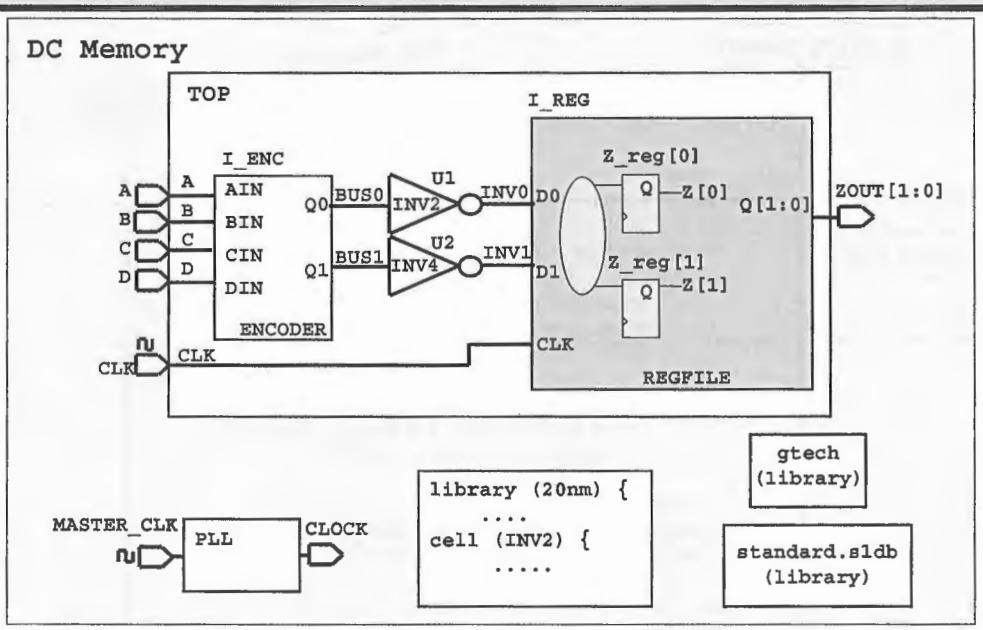
Library Objects



2-9

©2018 Synopsys, Inc. All Rights Reserved.

"get_*" Command Exercise 1/2



Use this picture and notes on the previous page to answer the questions on the next page. Assume the *current design* is *TOP*.

2-10

©2018 Synopsys, Inc. All Rights Reserved.

"get_*" Command Exercise 2/2

1. What does `get_designs *` return? _____
2. What does `get_ports {C? Z*}` return? _____
3. What does `get_libs` return? _____
4. How do you determine all the 20nm library cells that begin with INV?

   _____
5. How do you determine all the cells in the entire hierarchy with an underscore "_" in their name? What is returned?

   _____ _____
6. How do you determine all the Q* pins at the TOP level? What is returned?

   _____ _____
7. How do you determine all the Q pins inside REGFILE? What is returned?

   _____ _____

2-11

ANSWERS
{TOP ENCODER REGFILE PLL}
{ZOUT[0] ZOUT[1]}
{20nm gtech standard.sldb}
get_lib_cells 20nm/INV* returns {20nm/INV1 20nm/INV2 20nm/INV3 .....}
get_cells *_* -hier returns {I_ENC I_REG I_REG/Z_reg[0] I_REG/Z_reg[1]}
get_pins */Q* returns {I_ENC/Q0 I_ENC/Q1 I_REG/Q[0] I_REG/Q[1]}
get_pins I_REG/*/Q returns {I_REG/Z_reg[0]/Q I_REG/Z_reg[1]/Q}

©2018 Synopsys, Inc. All Rights Reserved.

Some Handy `all_*` Commands

- **Gets all input and inout ports of the current design:**
  `dc_shell-topo> all_inputs`
- **Gets all output and inout ports of the current design:**
  `dc_shell-topo> all_outputs`
- **Gets all clocks defined from the current design at the current design level, or below**
  `dc_shell-topo> all_clocks`
- **Gets all register cells in the entire current design's hierarchy:**
  `dc_shell-topo> all_registers`

2-12

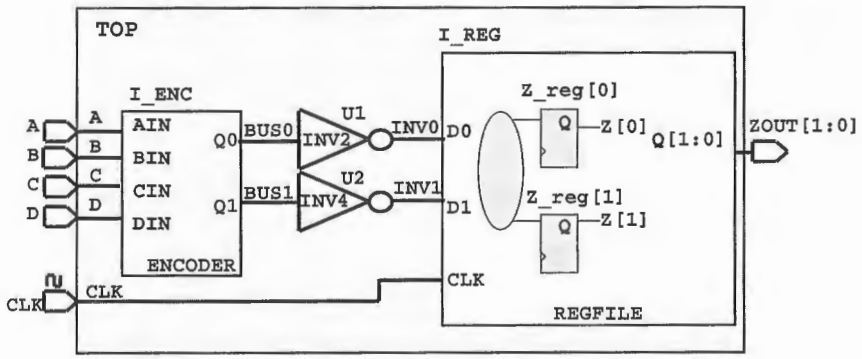
Some more useful `all_` commands:

```
all_ideal_nets
all_fanin
all_fanout
all_connected
all_dont_touch
all_high_fanout
```

To see the full list of `all_` commands enter `help all_*` .

©2018 Synopsys, Inc. All Rights Reserved.

"all_*" Command Exercise 1/3



Use this picture to answer the questions on the next page.
Assume that the current design is TOP.

2-13

©2018 Synopsys, Inc. All Rights Reserved.

"all_*" Command Exercise 2/3

1. What does `all_inputs` return?

   _____

2. What does `all_outputs` return?

   _____

3. What does `all_registers` return?

   _____

4. What does `all_inputs C*` return?

   _____

2-14

ANSWERS

1. {A B C D CLK}
2. {ZOUT[0] ZOUT[1]}
3. {I_REG/Z_reg[0] I_REG/Z-reg[1]}
4. "Error: extra positional option 'C*' (CMD-012)". (`all_` commands do not accept an object name as an argument)

©2018 Synopsys, Inc. All Rights Reserved.

"all_*" Command Exercise 3/3

5. Can you guess what the following returns?

```
remove_from_collection [all_inputs] [get_ports CLK]
```

_____



Homework:

Study this unit Appendix to learn more about:

- **Accessing and manipulating *collections***
- **TCL syntax**

2-15

ANSWERS

5. {A B C D}

©2018 Synopsys, Inc. All Rights Reserved.

Summary

You should now be able to:

- **Access key *design* and *library objects* using `get_` and `all_` *collection* commands**

2-16

©2018 Synopsys, Inc. All Rights Reserved.

Appendix: Unit-2 Design and Library Objects

Accessing and Manipulating Collections

2-17

©2018 Synopsys, Inc. All Rights Reserved.

Objects and Attributes

- **Recap: Designs can contain six different objects**
  - Designs, cells, ports, pins, nets and clocks
- **In order to keep track of circuit functionality and timing, DC attaches many attributes to each of these objects:**
  - Ports can have the following attributes

    `direction` `driving_cell_rise`
    `load` `max_capacitance` many others ...
  - Cells can have the following attributes

    `dont_touch` `is_hierarchical`
    `is_mapped` `is_sequential` many others ...

2-18

To see all DC-defined attributes for a particular object type:
`dc_shell-topo> list_attributes -application -class <object_type>`
where `<object_type>` = design, port, pin, cell, net, clock, lib, reference, cluster or bag.

Since the list of attributes can be quite long it may be easier to re-direct the output to a file, which can then be grepped for a specific string, for example:

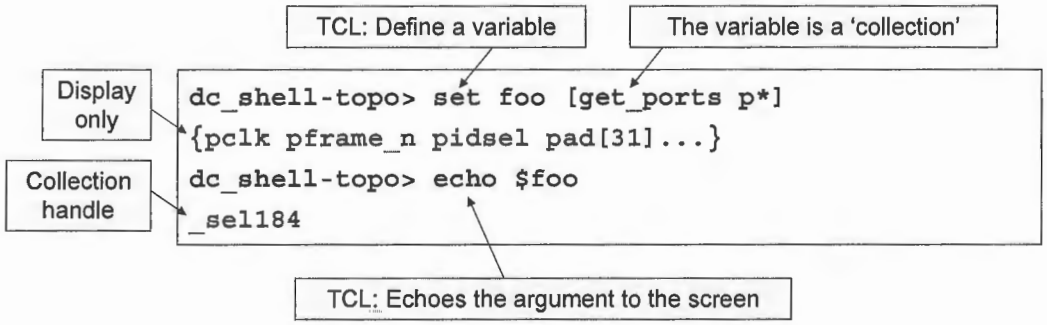
```
dc_shell-topo> redirect -file cell_attr \

{list_attributes -application -class cell}
UNIX% grep dont_ cell_attr | more
....
dont_duplicate_csa                    cell       boolean    a
dont_split_arithmetic                 cell       boolean    a
dont_touch                            cell       boolean    a
dont_touch_ref_design                 cell       boolean    u
dont_transform_csa                    cell       boolean    a
dont_uniquify                         cell       boolean    a
dont_use                              cell       boolean    a
....
```


©2018 Synopsys, Inc. All Rights Reserved.

Accessing the Synopsys Database

- **Access to DC *objects* in DC-Tcl is achieved through collections - a DC extension to standard Tcl**
- **When executed the `get_` or `all_` command:**
  - Creates a collection of objects, with their attributes
  - Echoes the contents of the collection to the screen
  - Returns a collection 'handle' - not a list of objects



2-19

©2018 Synopsys, Inc. All Rights Reserved.

Accessing and Manipulating Collections

```
dc_shell-topo> set foo [get_ports p*]
{pclk pframe_n pidsel pad[31]...}
dc_shell-topo> sizeof_collection $foo
50
dc_shell-topo> query_objects $foo
{pclk pframe_n pidsel pad[31]...}
```

```
dc_shell-topo> set pci_ports [get_ports DATA*]
{DATA[0] DATA[1] DATA[2] ...}
dc_shell-topo> set pci_ports [add_to_collection \
                    $pci_ports [get_ports CTRL*]]
{DATA[0] DATA[1] DATA[2] ... CTRL_A CTRL_B}
```

```
dc_shell-topo> set all_inputs_except_clk \
              [remove_from_collection [all_inputs] \
               [get_ports CLK]]
```

2-20

Collection commands return a collection handle, NOT a list! Standard Tcl list commands (`concat`, `llength`, `lappend`, etc) will not work with the output of a collection command!

```
dc_shell-topo> help *collection*
add_to_collection       # Add object(s)
compare_collections     # compares two collections
copy_collection         # Make a copy of a collection
filter_collection       # Filter a collection, resulting
                          in a new collection
foreach_in_collection   # Iterate over a collection
index_collection        # Extract object from collection
remove_from_collection  # Remove object(s) from a collection
sizeof_collection       # Number of objects in a collection
sort_collection         # Create a sorted copy of a collection

dc_shell-topo> help *object*
more collection related commands ...
```


©2018 Synopsys, Inc. All Rights Reserved.

Filtering Collections

- **Use the `filter_collection` command to get only objects you are interested in:**

```
filter_collection [get_cells *] "ref_name =~ AN*"
filter_collection [get_cells *] "is_mapped != true"
```

- **The `-filter` option is a nice short-cut:**

```
get_cells * -filter "dont_touch == true"
set fastclks [get_clocks * -filter "period < 10"]
```

- **Relational operators are:**

```
==, !=, >, <, >=, <=, =~, !~
```

2-21

Description of the examples above:
1. Returns all cells starting with the name "AN"
2. Returns all unmapped cells
3. Returns all cells with the "dont_touch" attribute
4. Returns all clocks with a period smaller than 10

`filter_collection` creates a new collection, or an empty string if no objects match the expression.
The `-filter` option is more efficient, because the collection does not have to be read twice.
Other examples:

```
get_cells -hier -filter "is_unmapped != true"
get_cells -hier -filter "is_hierarchical == true"
```


©2018 Synopsys, Inc. All Rights Reserved.

Iterating Over a Collection

- **Use the `foreach_in_collection` command to iterate over a collection:**

```
foreach_in_collection cell [get_cells -hier * -filter   \
                           "is_hierarchical == true"]   {
echo "Instance [get_object_name $cell] is hierarchical" }
```

```
Instance I_Ablock is hierarchical
Instance I_CONTROL is hierarchical
Instance I_Bblock is hierarchical
...
```

2- 22

`get_object_name` is more useful in this case than `query_objects:` the latter returns {I_Ablock} while the former returns I_Ablock.

©2018 Synopsys, Inc. All Rights Reserved.

Collection Versus Tcl List Commands



- ***Tcl lists* are structures to store *user-defined* data**
- ***Collections* are used to access *database* data**
  - Collections are more memory efficient for this purpose
- **List commands should not be used on collections and vise versa, for example:**
  - Use `foreach` to iterate through a *list*
  - Use `foreach_in_collection` to iterate through a *collection*



2-23

The above is a strong recommendation. DC does allow some mixing of lists and collections, but it is not recommended.
For example, the following is allowed:

```
set port_col [list [get_ports a*] [get_ports b*]]
```

`port_col:` is a list with two collections. This list may be passed to other collection manipulation commands.
It is better to convert the command to this:

```
set port_col [get_ports "a* b*"]
```


©2018 Synopsys, Inc. All Rights Reserved.

This page was intentionally left blank

©2018 Synopsys, Inc. All Rights Reserved.

Agenda

DAY 1

- i Introduction to Synthesis
- 1 Data Setup for DC Topographical Mode 
- 2 Accessing Design and Library Objects
- 3 Constraints: Reg-to-Reg and I/O Timing (Lecture)

© 2018 Synopsys, Inc. All Rights Reserved

©2018 Synopsys, Inc. All Rights Reserved.

Unit Objectives



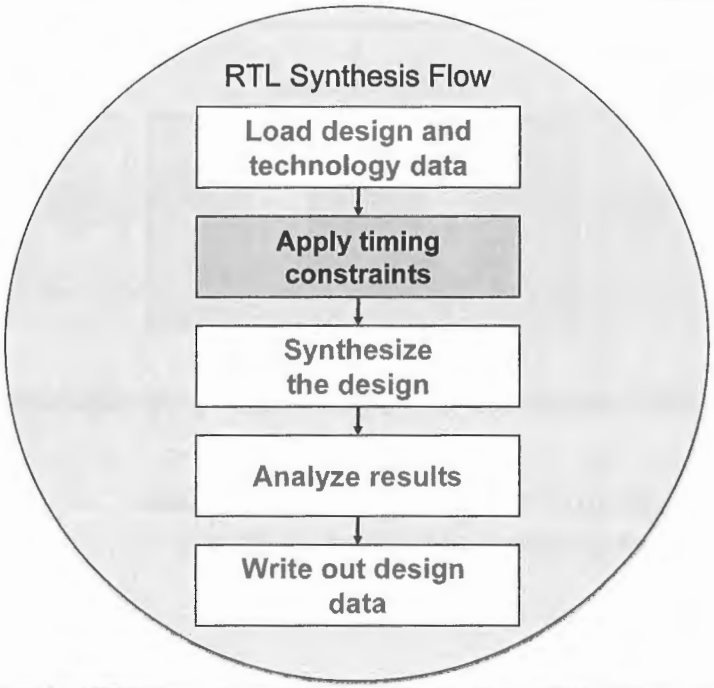
After completing this unit, you should be able to:

- **Constrain all reg-to-reg and I/O timing paths of a single clock design for *setup* timing**

3-2

©2018 Synopsys, Inc. All Rights Reserved.

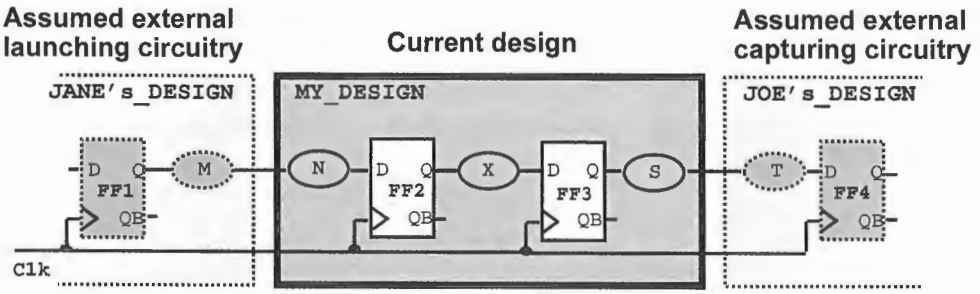
RTL Synthesis Flow



3-3

©2018 Synopsys, Inc. All Rights Reserved.

Default Design Scenario



- **Design Compiler assumes a "synchronously-clocked" environment.**

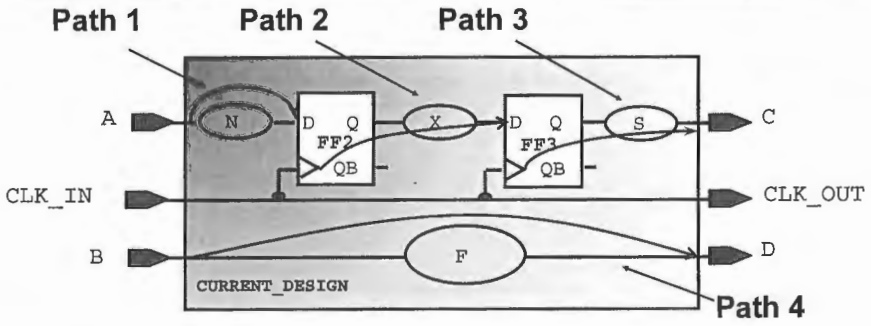
By default:

- Input data arrives from a pos-edge clocked device
- Output data goes to a pos-edge clocked device

3-4

©2018 Synopsys, Inc. All Rights Reserved.

Timing Path Definition



DC breaks designs into timing paths, each with a:

- **Startpoint**
  - Input port (other than a Clock port)
  - Clock pin of Flip-Flop or register
- **Endpoint**
  - Output port (other than a Clock port)
  - Any input pin of a sequential device, except clock pin[1]

3-5

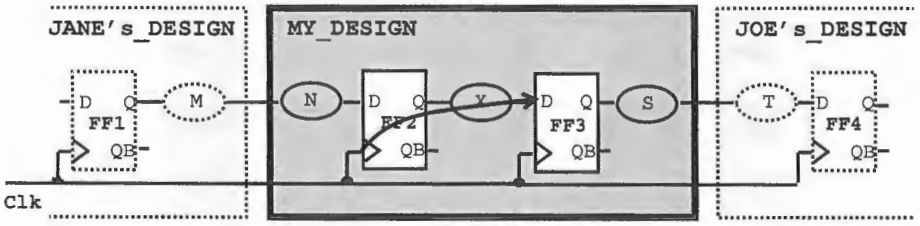
[1]Examples of non-clock input pins of a sequential device: D (Data), S (Set) or R (Reset), E (Enable).

Static Timing Analysis determines if a circuit meets timing constraints during and after synthesis
This involves three main steps:

1) The design is broken down into sets of timing paths
2) The delay of each path is calculated
3) Path delays are compared against expected arrival times to determine if constraints have been met or not

©2018 Synopsys, Inc. All Rights Reserved.

Constraining Register-to-Register Paths



What information must you provide to constrain all the register-to-register paths in MY_DESIGN for setup time?

3-6

The duty cycle matters only if your registers are clocked by both positive and negative edges of the clock. In the example above both registers are positive-edge-triggered so the duty cycle does not matter. The only information the user must supply is the clock period. DC retrieves the setup time of FF3 from the technology library.

©2018 Synopsys, Inc. All Rights Reserved.

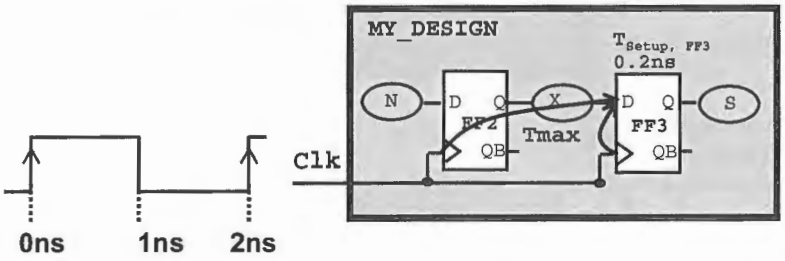
Constraining Reg-to-Reg Paths: Example

Spec:

Clock Period = 2ns

Unit of time is 1ns in this example.
Defined in the technology library.

```
create_clock -period 2 [get_ports Clk]
```



What is the maximum delay requirement Tmax for the register-to-register path through X in the MY_DESIGN? _____

3-7

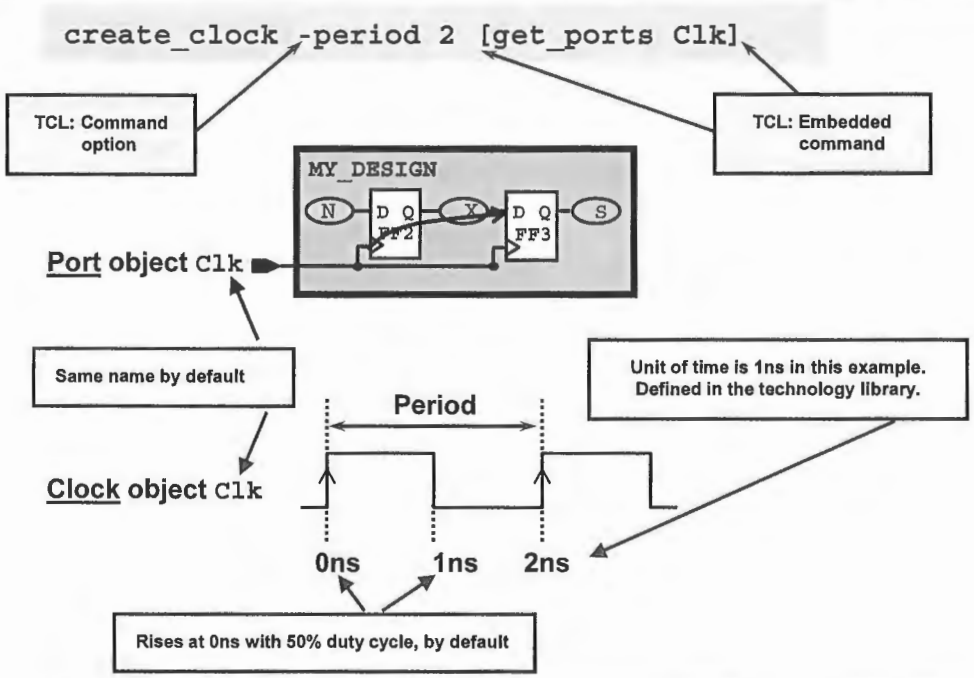
Unless explicitly defined with the `-waveform` option, the clock object that is created starts off going high at 0ns, comes down at 50% of the clock period, and repeats every clock period. The clock object's name is the same as the port/pin object to which it is assigned, unless explicitly defined by the `-name` option.

The unit of time is defined in the technology library. It is commonly 1 nano-second, as in the example above, but it may also be defined differently, for example: 0.1 nano-second, 10 pico-seconds, or 1 pico-second. You should verify the unit of time by looking at the top of the report generated by `report_lib <lib_name>` , or by `get_attribute <lib_name> time_unit_name`.

Reg-to-reg, max = P – Setup_Time (assuming 0 clock skew) = 2 – 0.2 = 1.8ns

©2018 Synopsys, Inc. All Rights Reserved.

create_clock Required Arguments



3-8

©2018 Synopsys, Inc. All Rights Reserved.

Ideal Clock Behavior

- **Defining the clock in a single-clock design constrains all timing paths between registers for single-cycle, setup time**
- **By default the clock rises at 0ns and has a 50% duty cycle**
- **DC will not "buffer up" the clock network, even when connected to many clock/enable pins of flip-flops/latches**
  - The clock network is treated as "ideal" - infinite drive capability
    - Zero rise/fall transition times
    - Zero skew
    - Zero insertion delay or latency
  - Estimated skew, latency and transition times can, and should be modeled for a more accurate representation of clock behavior

3-9

Design Compiler does not build the clock buffer tree. Clock tree buffering and skew balancing happens during clock tree synthesis (CTS). CTS is performed after synthesis, with a "layout" or "physical design" tool like IC Compiler (II).

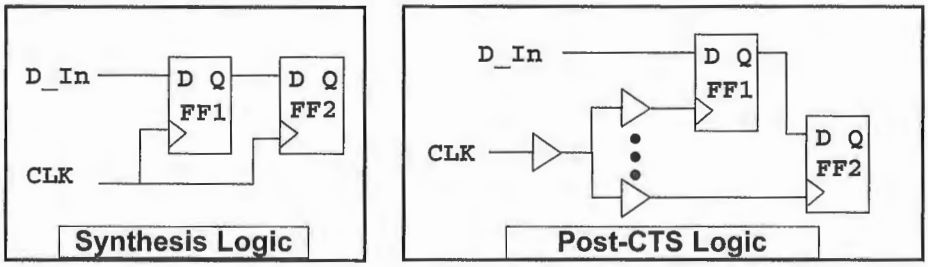
The recommended approach to dealing with clocks is to model estimated skew, latency and transition times during synthesis.

Clock nets in DC are also exempt from meeting design rule constraints (`max_transition`, `max_capacitance`).

©2018 Synopsys, Inc. All Rights Reserved.

Modeling Clock Trees

- Design Compiler is NOT used to synthesize clock buffer trees
- Clock tree synthesis is usually done by a physical implementation or layout tool, based on actual cell placement



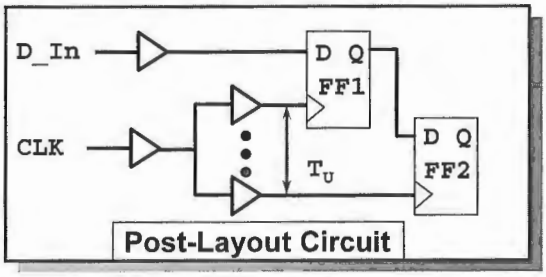
What clock tree effects need to be taken into account by the synthesis tool, prior to layout?

3-10

CTS = Clock Tree Synthesis

©2018 Synopsys, Inc. All Rights Reserved.

Modeling Clock Skew



Uncertainty models the maximum delay difference between the clock network branches, known as *clock skew*, but can also include *clock jitter* and *margin*:

```
set_clock_uncertainty -setup T_U [get_clocks CLK]
```

Pre-Layout: clock skew + jitter + margin

placed on clock objects

3-11

The default clock uncertainty is zero.

Uncertainty can also be defined between clock domains with as follows:

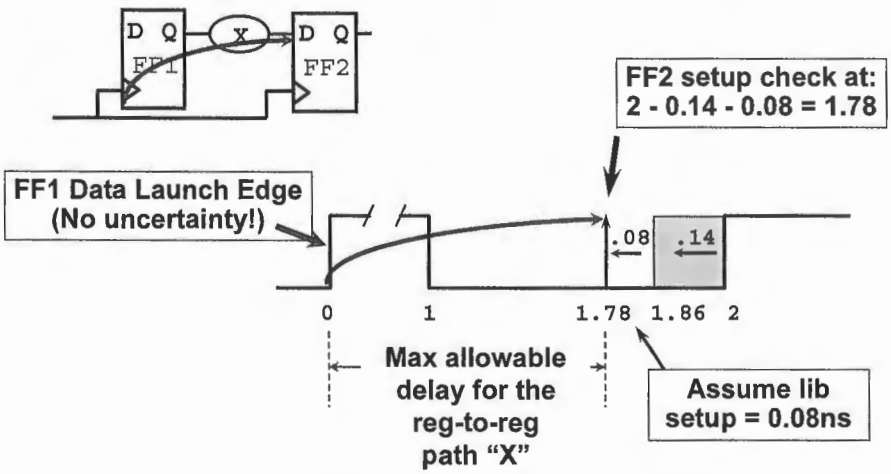
```
set_clock_uncertainty -setup T_U -from <Clk1> -to <Clk2>
```


©2018 Synopsys, Inc. All Rights Reserved.

set_clock_uncertainty and Setup Timing

Example:

```
create_clock -period 2 [get_ports CLK]
set_clock_uncertainty -setup 0.14 [get_clocks CLK]
```



3-12

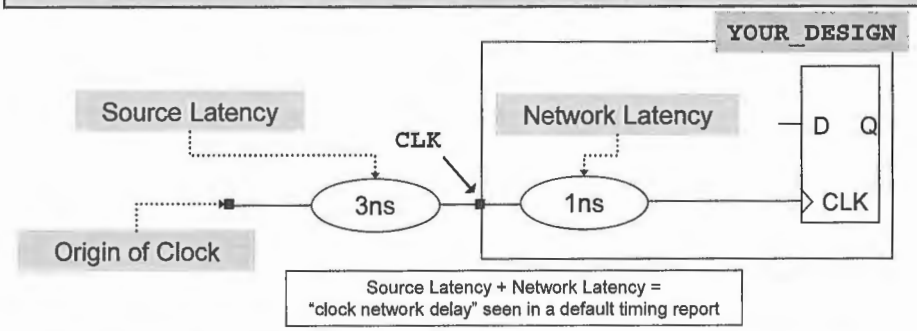
By default, the command "set_clock_uncertainty" assigns the same value to the setup and the hold uncertainty, unless either the –setup or the –hold switch are used. This allows the design to be synthesized with extra "clock period margin" for setup timing, which does not apply to hold timing.

©2018 Synopsys, Inc. All Rights Reserved.

Modeling Latency or Insertion Delay

- **Network latency models the average 'internal' delay from the `create_clock` port or pin to the register clock pins**
- **Source latency models the delay from the actual clock origin to the `create_clock` port or pin - used for either ideal or propagated clocks (post layout)**

```
create_clock -period 10 [get_ports CLK]
set_clock_latency -source -max 3 [get_clocks CLK]
set_clock_latency -max 1 [get_clocks CLK] ;# pre layout
# set_propagated_clock [get_ports CLK] ;# post layout
```

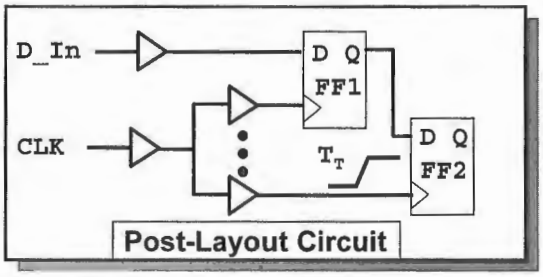


3-13

One of the benefits of this is for propagated clocks. Propagated clocks will calculate network latency. In addition you can now model source latency. The total latency to the register will be the source + network latency.
If you do not use the -source option with `set_clock_latency`, then it represents network latency.
You can model rise, fall, min and max numbers for both network and source latency.
Useful when clock generation circuitry is not part of your design, or for derived clocks.

©2018 Synopsys, Inc. All Rights Reserved.

Modeling Transition Time



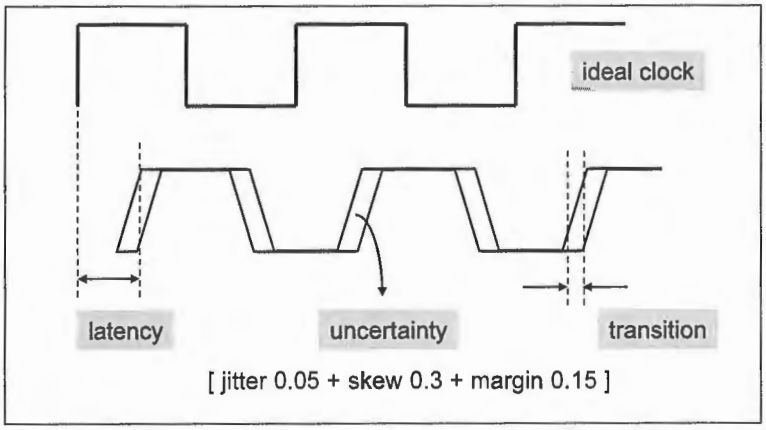
Transition models the rise and fall times of the clock waveform at the register clock pins:

```
set_clock_transition -max T_T [get_clocks CLK]
```

3- 14

©2018 Synopsys, Inc. All Rights Reserved.

Pre/Post Layout Clock



Synthesis (Pre-CTS) Clock Constraints

```
create_clock -period 5 [get_ports Clk]
set CLOCK [get_clocks Clk]
set_clock_latency -source -max 4 $CLOCK
set_clock_latency -max 2 $CLOCK
set_clock_uncertainty -setup 0.5 $CLOCK
set_clock_transition -max 0.08 $CLOCK
```

Post-CTS Clock Constraints

```
create_clock -period 5 [get_ports Clk]
set CLOCK [get_clocks Clk]
set_clock_latency -source -max 4 $CLOCK

set_clock_uncertainty -setup 0.2 $CLOCK
set_propagated_clock [get_ports Clk]
```

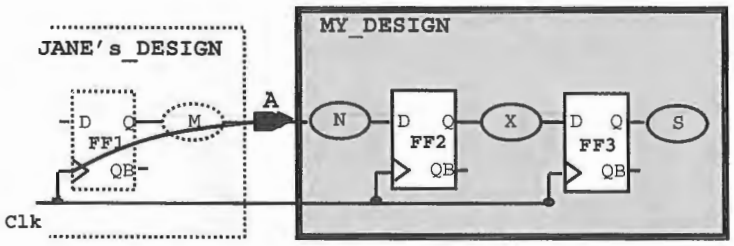
3-15

For post-CTS (Clock Tree Synthesis) static timing analysis `set_propagated_clock` forces the analyzer to calculate the ACTUAL clock tree skew, latency and transition times. The post-CTS constraints therefore do not include ideal transition and network latency commands. If the uncertainty number used during synthesis includes jitter and/or margin, these effects must still be included in the post-CTS analysis, along with the external source latency.

If the network latency and clock transition constraints are kept, along with the "propagated" clocks, these constraints are ignored during timing analysis. The actual propagated network latencies and transition times are calculated and applied. It is therefore not necessary to delete these constraints (as implied by the above post-CTS example).

©2018 Synopsys, Inc. All Rights Reserved.

Constraining Input Paths



What additional information must you provide to constrain all the input paths (N) in your design for setup time?

3-16

ANSWER: In addition to the clock definition, the user must supply the latest arrival time of the data at input port A, with respect to FF1's launching clock edge. In other words, an amount of time AFTER the external (Jane's) launching clock edge.

DC retrieves the setup time of FF2 from the technology library.

©2018 Synopsys, Inc. All Rights Reserved.

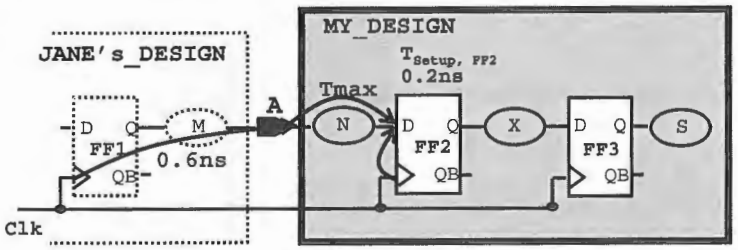
Constraining Input Paths: Example 1

Spec:

Latest Data Arrival Time at Port A, after Jane's launching clock edge = 0.6ns

mydesign.con

```
create_clock -period 2 [get_ports Clk]
set_clock_uncertainty -setup 0.3 [get_clocks Clk]
set_input_delay -max 0.6 -clock Clk [get_ports A]
```



What is the maximum delay Tmax for the input path N in MY_DESIGN? _____

3-17

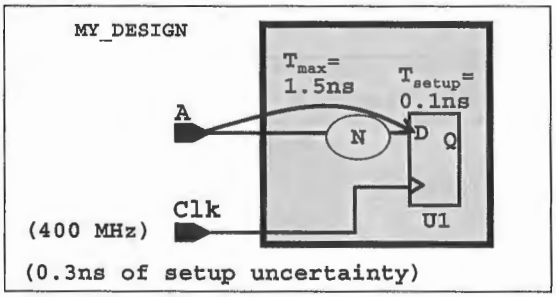
$$T_{input_logic,\ max} = P - Input_Delay\ (max) - Clock\ Uncertainty\ (setup) - Register\ Setup\ Time$$

$$Tmax = 2 - 0.6 - 0.3 - 0.2 = 0.9ns$$

©2018 Synopsys, Inc. All Rights Reserved.

Constraining Input Paths: Example 2

Spec: Clock frequency = 400MHz
Clock uncertainty for setup = 0.3ns
Maximum delay for path N = 1.5ns



How do you constrain MY_DESIGN for the indicated T_{max}?

```
create_clock _____________ [get_ports Clk]
set_clock_uncertainty _____________ [_________ Clk]
set_input_delay _____________________________________
```

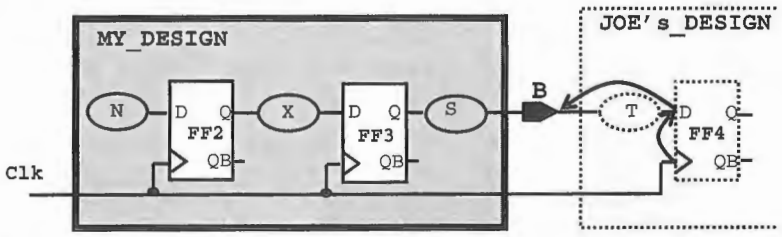
3-18

```
create_clock -period 2.5 [get_ports Clk]
set_clock_uncertainty -setup 0.3 [get_clocks Clk]
set_input_delay -max 0.6 -clock Clk [get_ports A]
```

(2.5 − 1.5 − 0.3 - 0.1 = 0.6ns)

©2018 Synopsys, Inc. All Rights Reserved.

Constraining Output Paths



What additional information must you provide to constrain all the output paths (S) in your design for setup time?

3-19

In addition to the clock definition, the user must supply the latest arrival time of the data at output port B, with respect to FF4's capturing clock edge, in other words, the maximum "setup time" at Joe's input, with respect to Joe's clock. This is an amount of time BEFORE the external capturing clock edge.

©2018 Synopsys, Inc. All Rights Reserved.

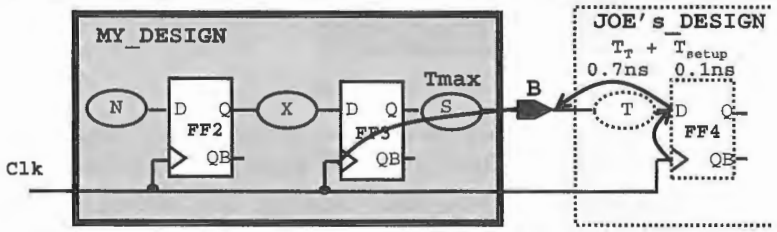
Constraining Output Paths : Example 1

Spec:

Latest Data Arrival Time at Port B, before Joe's capturing clock = 0.8ns

mydesign.con

```
create_clock -period 2 [get_ports Clk]
set_clock_uncertainty -setup 0.3 [get_clocks Clk]
set_input_delay -max 0.6 -clock Clk [get_ports A
set_output_delay -max 0.8 -clock Clk [get_ports B]
```



What is the maximum delay Tmax for the output path S in MY_DESIGN? _____

3-20

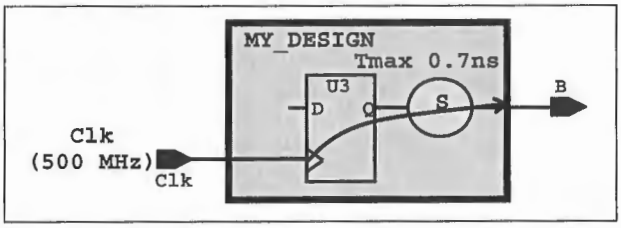
Tmax = 2 – 0.3 - 0.8 = 0.9 ns
Output_Logic, max = P – Clock Uncertainty (setup) - Output_Delay (max)

©2018 Synopsys, Inc. All Rights Reserved.

Constraining Output Paths : Example 2

Spec:

The maximum delay to Port B = 0.7ns



How do you constrain MY_DESIGN for the indicated T_{max}?

```
create_clock -period 2 [get_ports Clk]
set_clock_uncertainty -setup 0.3 [get_clocks Clk]
set_output_delay _____________________________________
```

3-21

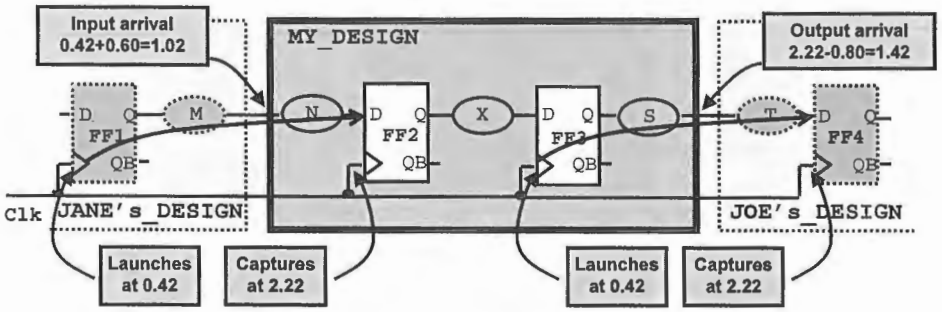
```
# Output delay = 2 - 0.3 - 0.7 = 1
set_output_delay -max 1 -clock Clk [get_ports B]

# Alternatively:
set_output_delay -max [expr {2 - 0.3 - 0.7}] -clock Clk [get_ports B]
```


©2018 Synopsys, Inc. All Rights Reserved.

Default I/O Clock Latency and Uncertainty

I/O clocks (`set_in/output_delay -clock`) inherit the same latencies and uncertainty as specified by the `set_clock_latency/uncertainty` commands, by default

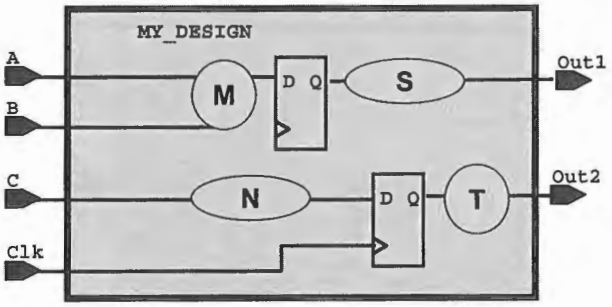



```
create_clock -period 2 [get_ports Clk]
set_clock_latency -source -max 0.3 [get_clocks Clk]
set_clock_latency -max 0.12 [get_clocks Clk]
set_clock_uncertainty -setup 0.2 [get_clocks Clk]
set_input_delay -max 0.6 -clock Clk [all_inputs]
set_output_delay -max 0.8 -clock Clk [all_outputs]
```

3-22

©2018 Synopsys, Inc. All Rights Reserved.

Multiple Inputs/Outputs - Same Constraints



To constrain all inputs the same, except for the clock port:

```
set_input_delay -max 0.5 -clock Clk \
  [remove_from_collection [all_inputs] [get_ports Clk]]
```

To constrain all outputs the same:

```
set_output_delay -max 1.1 -clock Clk [all_outputs]
```

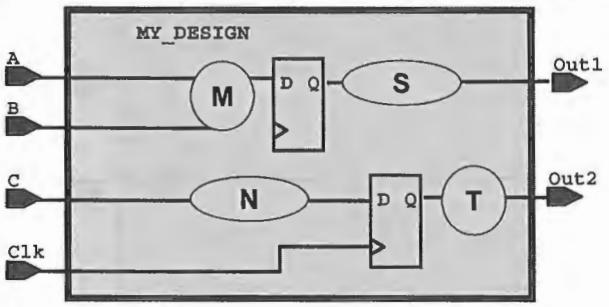
3-23

If you need to remove more than just one clock, use the following syntax:

```
remove_from_collection [all_inputs] [get_ports "Clk1 Clk2"]
```


©2018 Synopsys, Inc. All Rights Reserved.

Different Port Constraints



To constrain most ports the same, except for some:

Overrides

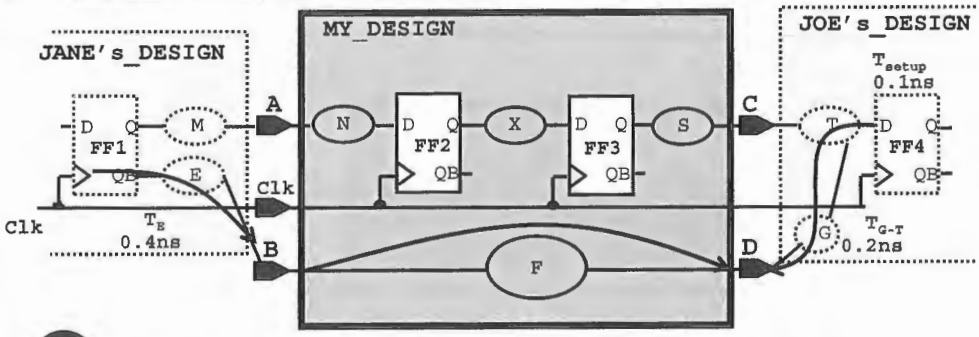
```
set_input_delay -max 0.5 -clock Clk [all_inputs]
set_input_delay -max 0.8 -clock Clk [get_ports C]
remove_input_delay [get_ports Clk]
```

Another way to remove the constraint from the Clk port

3- 24

©2018 Synopsys, Inc. All Rights Reserved.

Exercise: Combinational Path



How do you constrain the combinational path F?
What is the maximum delay through F?

```
create_clock -period 2 [get_ports Clk]
set_clock_uncertainty -setup 0.3 [get_clocks Clk]
set_input_delay _____
set_output_delay _____
```

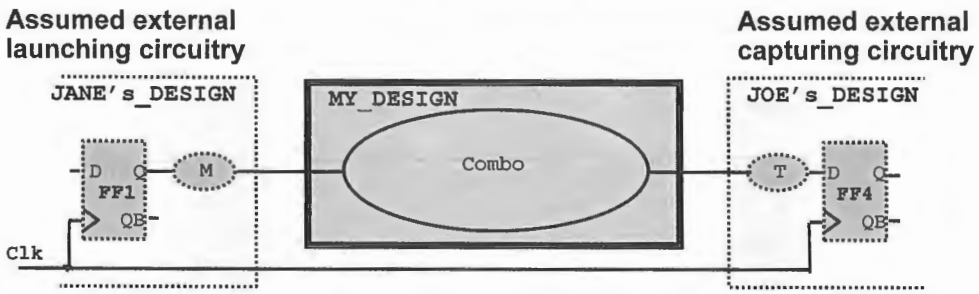
$T_{F,\ max}$ = _____

$T_{F,\max} = 2 - 0.3 - 0.4 - 0.3 = 1.0\text{ns}$

```
set_input_delay -clock Clk -max 0.4 [get_ports B]
set_output_delay -clock Clk -max 0.3 [get_ports D]
```


©2018 Synopsys, Inc. All Rights Reserved.

Constraining a Purely Combinational Design



? What is different about this design?

? How do we constrain such a design?

3-26

ANSWER: There is no clock input.

©2018 Synopsys, Inc. All Rights Reserved.

Answer: Use a Virtual Clock!

What is a virtual clock?

ANSWER:

- A clock that is *not connected* to any port or pin within the current design
- Serves as a *reference* for input or output delays
- Creates a *clock object* with a *user-specified name* within Design Compiler's memory

```
create_clock  -name VCLK  -period 2
```

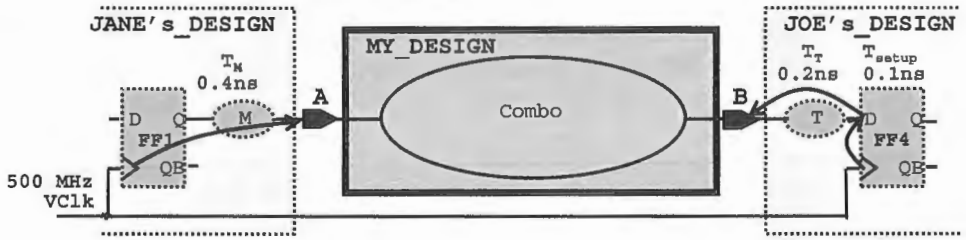
Must be named

No source pin or port!

3-27

©2018 Synopsys, Inc. All Rights Reserved.

Exercise: Combinational Design



How do you constrain the Combo path? What is the maximum delay through Combo?

```
create_clock -period 2 _____________
set_clock_uncertainty -setup 0.3 _____________________
set_input_delay  _____________________________________

set_output_delay _____________________________________
```

$T_{Combo,\ max}$ = _____

3-28

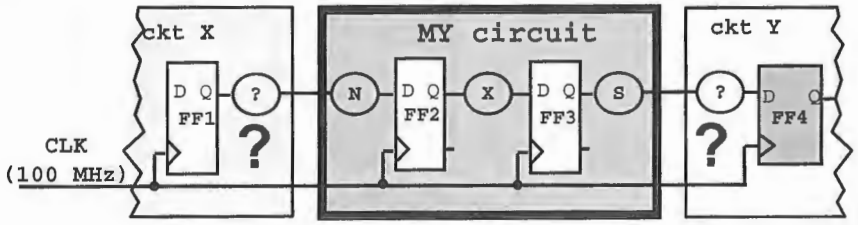
$T_{Combo,\ max} = 2 - 0.3 - 0.4 - 0.3 = 1.0ns$

```
create_clock -period 2 -name VClk
set_clock_uncertainty -setup 0.3 [get_clocks VClk]
set_input_delay -clock VClk -max 0.4 [get_ports A]
set_output_delay -clock VClk -max 0.3 [get_ports B]
```


©2018 Synopsys, Inc. All Rights Reserved.

Time Budgeting (1/2)

What if you do not know the delays on your inputs or the setup requirements of your outputs?

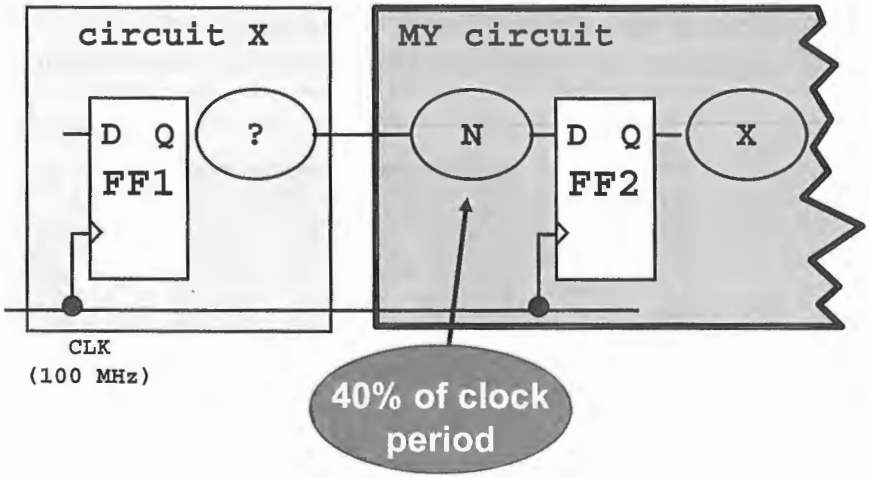


A: Create a Time Budget!

3-29

©2018 Synopsys, Inc. All Rights Reserved.

Time Budgeting (2/2)



Better to budget conservatively than to compile with unconstrained paths!

3-30

The goal of this example is to tell Design Compiler to only use 40% of the clock period for the input logic cloud. If the designer for Circuit X does the same for the output logic cloud, there will be a 20% margin, including the delay of FF1 and the setup time of FF2.

©2018 Synopsys, Inc. All Rights Reserved.

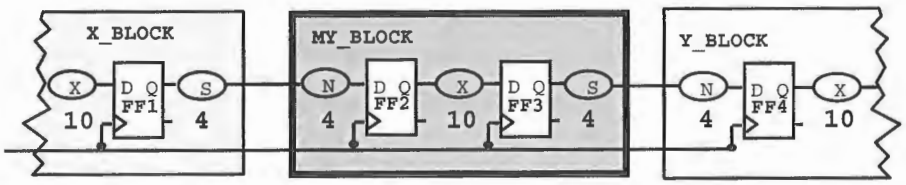
Time Budgeting Example

timing_budget.tcl

```
# A generic Time Budgeting script file
# for MY_BLOCK, X_BLOCK and Y_BLOCK

create_clock -period 10 [get_ports CLK]

set_input_delay -max 6 -clock CLK [all_inputs]
remove_input_delay [get_ports CLK]
set_output_delay -max 6 -clock CLK [all_outputs]
```

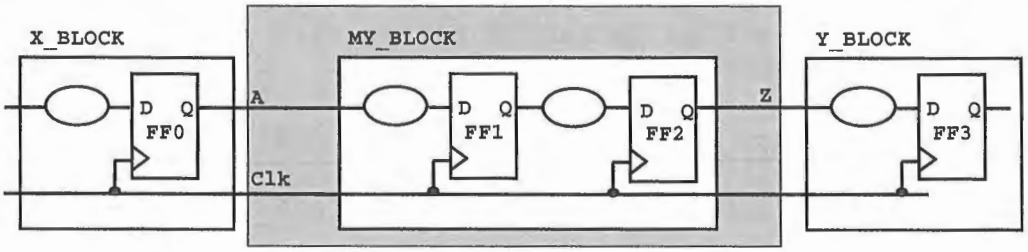


Would it be easier to specify a time budget if all outputs were registered?

3-31

©2018 Synopsys, Inc. All Rights Reserved.

Registered Outputs




```
# Assume every block has registered outputs, 10ns clock:
set CLK2Q_MAX 1.5; # Assume slowest register driving your input
set CLK2Q_MIN 0.2; # Assume fastest register driving your output
set ALL_INP_EXC_CLK [remove_from_collection [all_inputs] [get_ports Clk]]
set_input_delay  -max   $CLK2Q_MAX -clock CLK $ALL_INP_EXC_CLK
set_output_delay -max [expr {10 - $CLK2Q_MIN}] -clock CLK [all_outputs]
```

TCL: Arithmetic expression
with its arguments surrounded by curly braces {}

3-32

Additional examples using the TCL `expr` command:

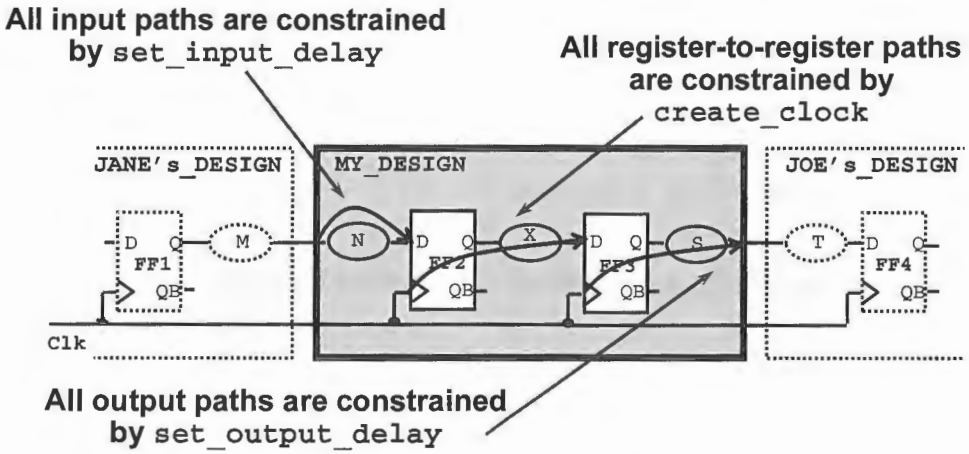
```
set PERIOD 10.0; # Returns "10.0"
set FREQ [expr  {1 / $PERIOD}]; # Returns "0.1"
echo "Freg = " [expr {$FREQ * 1000}] "MHz"; # Returns
                                        # "Freq = 100.0 MHz"

set_load [expr {[load_of ssc_core_slow/and2a0/A] * 5}] \
[all_outputs]; # Applies 5 and2a0 gate load to all outputs
```

Curly braces surrounding the arguments to an `expr` allows a more efficient evaluation of the arguments by the `expr` command by avoiding an additional unnecessary interpretation during command parsing. Additionally, this enhances the readability of the expr command arguments.

©2018 Synopsys, Inc. All Rights Reserved.

Timing Constraint Summary



You specify how much time is used by external logic...

DC calculates how much time is left for the internal logic.

3-33

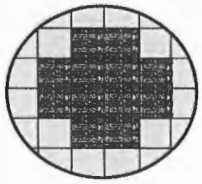
©2018 Synopsys, Inc. All Rights Reserved.

Area Constraint

- **The `compile_ultra` command automatically applies a maximum area constraint of zero to the current design**

```
.... (from the compile log)
Beginning Area-Recovery Phase  (max_area 0)
....
```

- **Area has the lowest priority compared to all other constraints, such as DRCs, timing and power**
  - The zero max area constraint will never impact timing or other design constraint results
- **Use `report_area` to list the design area after compile**



3- 34

©2018 Synopsys, Inc. All Rights Reserved.

Executing Commands Interactively

- **Commands can be typed interactively in DC-shell:**
  - OK for testing or debugging individual commands
  - Not efficient for "production work"

```
UNIX% dc_shell -topo
dc_shell-topo> read_verilog {A.v B.v TOP.v}
dc_shell-topo> current_design MY_TOP
dc_shell-topo> link
dc_shell-topo> check_design
dc_shell-topo> create_clock -period 2 [get_ports Clk]
dc_shell-topo> set_input_delay -max 0.6 -clock Clk [all_inputs]
               ...
```

3-35

©2018 Synopsys, Inc. All Rights Reserved.

Sourcing Constraints Files

- For better efficiency, capture the constraints in a "constraints file", which can be executed interactively in DC-shell:

```
dc_shell-topo> read_verilog {A.v B.v TOP.v}
dc_shell-topo> current_design MY_TOP
dc_shell-topo> link
dc_shell-topo> check_design
dc_shell-topo> source TOP.con
...
```

TOP.con

```
create_clock -period 2 [get_ports Clk]
set_input_delay -max 0.6 -clock Clk [all_inputs]
...
```

3-36

©2018 Synopsys, Inc. All Rights Reserved.

Executing *Run Scripts* in "Batch Mode"

- **For maximum efficiency capture 'run commands' in a 'run script' and execute in batch mode**

```
UNIX% dc_shell -topo -f RUN.tcl | tee -i dc.log
```

- Allows you to spend time on "other tasks" while DC is running
- Do this only once you are certain the run script and constraints file are complete and correct

```
read_verilog {A.v B.v TOP.v}
current_design MY_TOP
link
check_design
source TOP.con
...
```

RUN.tcl



Other tasks

3-37

The `tee` command displays the results on the screen while simultaneously writing them into the specified log file for later viewing.

The "-i" option tells the `tee` command to ignore interrupts. For example, if you enter <Ctrl-C> while in DC shell, the interrupt will be ignored by the `tee` command and passed on to the dc_shell command, which is what you intended. Otherwise, `tee` will process the <Ctrl-C> first.

©2018 Synopsys, Inc. All Rights Reserved.

Constraints File Recommendations (1 of 3)

Since constraints are saved in the *ddc* design format it is recommended to:

Erase all *constraints* from the current design before applying new constraints.

```
reset_design
create_clock -period 2 [get_ports Clk]
...
```



When applying *multiple* constraint scripts, there should only be ONE `reset_design` command.

3-38

©2018 Synopsys, Inc. All Rights Reserved.

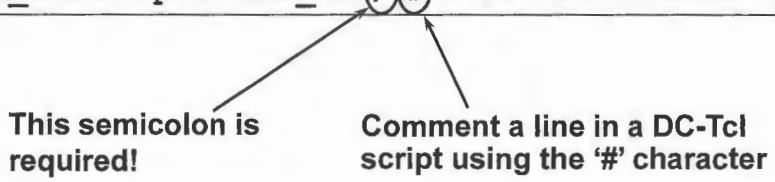
Constraints File Recommendations (2 of 3)

Include comments in your scripts

```
# Comments in Tcl

# If you want to comment on the same line, be sure
# to use a semicolon before the comment:

create_clock -p 5 -n V_Clk ; # This is a VIRTUAL clock
```



3-39

©2018 Synopsys, Inc. All Rights Reserved.

Constraints File Recommendations (3 of 3)

- **Use common extensions:**
  e.g. `dc.tcl` **or** `DESIGN.sdc`
- **Avoid using aliases and abbreviating commands**
- **Avoid abbreviating command options:**

```
create_clock -period 5 [get_ports clk]
```

- **Avoid "snake scripts"**

3-40

"Snake scripts" are scripts that call scripts, that call scripts: Very hard to debug.

Avoid sourcing scripts from your `.synopsys_dc.setup` file, since these scripts will be executed automatically every time you start the tool. This of course excludes scripts that only define procedures for later use.

©2018 Synopsys, Inc. All Rights Reserved.

Check the Syntax of Constraints

```
UNIX% dcprocheck TOP.con
```

```
UNIX% dcprocheck TOP.con
...

Unknown option 'create_clock -freq'
create_clock -freq 3.0 [get_ports clk]
             ^
```

- *dcprocheck* is a syntax-checking utility that is included with Design Compiler
- Available if you can launch DC – no additional user setup required

3-41

©2018 Synopsys, Inc. All Rights Reserved.

Check for Missing/Inconsistent Constraints

```
read_verilog {A.v B.v TOP.v}
current_design MY_TOP
link
check_design
source TOP.con
check_timing
 ...
```

Good practice:

`check_timing` after applying constraints

The `check_timing` command issues warnings for:

- **Missing endpoint constraints**
- **Missing, overlapping or multiple clocks**
- **Clock-gating signals that may interfere with the clock**
- **And more …**

3-42

©2018 Synopsys, Inc. All Rights Reserved.

Check Values/Options of Clock Constraints

```
dc_shell-topo> report_clock

Clock     Period   Waveform    Attrs     Sources
-------------------------------------------------
clk         2.00    {0 1}                {clk}
-------------------------------------------------
```

```
dc_shell-topo> report_clock -skew

                  Rise       Fall    Min Rise    Min fall      Uncertainty
Object           Delay      Delay      Delay       Delay      Plus       Minus
-------------------------------------------------------------------------------
clk                0.40       0.40       -           -          -          0.15

                  Max Source Latency                    Min Source Latency
            Early      Early     Late      Late      Early      Early     Late      Late
Object      Rise       Fall      Rise      Fall      Rise       Fall      Rise      Fall
-------------------------------------------------------------------------------------
clk         0.90       0.90      0.90      0.90      --         --        --        --

                  Max Transition      Min Transition
Object            Rise       Fall     Rise       Fall
-------------------------------------------------------
clk                0.22       0.22      --         --
```

```
report_clock -skew -attr; # Combines both outputs
```

3-43

©2018 Synopsys, Inc. All Rights Reserved.

Check Values/Options of Port Constraints

```
dc_shell-topo> report_port -verbose
...
 Output Delay
                    Min              Max        Related    Fanout
Output Port     Rise     Fall    Rise     Fall    Clock      Load
-------------------------------------------------------------------
EndOfInstrn     --       --      3.0      3.0      Clk       0.00
OUT_VALID       --       --      --       --       --        0.00
PSW[0]          --       --      2.4      2.4      Clk       0.00
...
```

Missing output constraint?

3-44

©2018 Synopsys, Inc. All Rights Reserved.

Redirect *Checks* and *Reports* to a File

It may be useful to redirect the output of run-script *checks* and *reports* to a separate file for later analysis

dc.tcl

```
read_verilog {A.v B.v TOP.v}
current_design MY_TOP
redirect -tee -file precompile.rpt {link
                                    check_design
                                    source TOP.con
                                    check_timing}

redirect -tee -file compile.rpt {compile_ultra}

redirect -tee -file post_compile.rpt {report_constraint -all
                                      report_timing}
```

3-45

The output of `redirect` is identical to ">" or ">>" (`-append`).

The following two commands give the same results:

```
redirect -append -file precompile.rpt {check_design}
check_design >> precompile.rpt
```

However, with `redirect`, you can also use the "`-tee`" option to send output to the screen.
There are additional `redirect` options (e.g. `-compress`), which are not available with ">".

©2018 Synopsys, Inc. All Rights Reserved.

Need Help with Commands and Variables?

- **For variables:**

```
printvar *_library;    # Lists all matching variables
                       # and corresponding values

man target_library;    # Complete 'man page'
```

- **For commands:**

```
help *clock;           # Lists all matching commands

man create_clock;      # Complete 'man page'

create_clock -help;    # Lists command options

# List command(s) containing the specified string,
# e.g. "period", in the command name or option(s):

apropos -symbols_only period;
```

3-46

```
dc_shell-topo> help *clock
 clock                    # Builtin
 create_clock             # create_clock
 create_test_clock        # create_test_clock
 remove_clock             # remove_clock
 remove_propagated_clock  # remove_propagated_clock
 report_clock             # report_clock
 set_propagated_clock     # set_propagated_clock

dc_shell-topo> help -verbose create_clock
 create_clock          # create_clock
   [-name clock_name]      (name for the clock)
   [-period period_value]  (period of the clock)
   [-waveform edge_list]   (alternating rise, fall times for
                            1 period)
   [port_pin_list]         (list of ports and/or pins)

dc_shell-topo> apropos -symbols_only  period
 create_clock          # create clock
   [-name clock_name]      (name for the clock)
   [-period period_value]  (period of the clock:
                            Value >= 0)
.....
```


©2018 Synopsys, Inc. All Rights Reserved.

Summary

You should now be able to:

- **Constrain a design for setup timing, under the following conditions:**
  - The design's specs are given or known
  - Single clock, single cycle environment
  - Default design scenario – minimal command options
- **Create, verify and execute a constraints file**

3-47

©2018 Synopsys, Inc. All Rights Reserved.

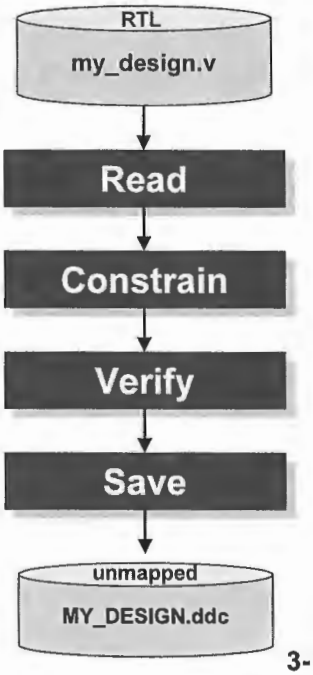
Lab 3: Constraints: Reg-to-Reg and I/O Timing



80 minutes

Lab 3: Constraints: Reg-to-Reg and I/O Timing

Create a Reg-to-Reg and I/O Timing Constraints File



3-48

©2018 Synopsys, Inc. All Rights Reserved.

Agenda

DAY 2

3	Constraints: Reg-to-Reg and I/O Timing (Lab) 
4	Constraints: Input Transition and Output Loading 
5	DC Ultra Synthesis Techniques 

© 2018 Synopsys, Inc. All Rights Reserved

©2018 Synopsys, Inc. All Rights Reserved.

Unit Objectives



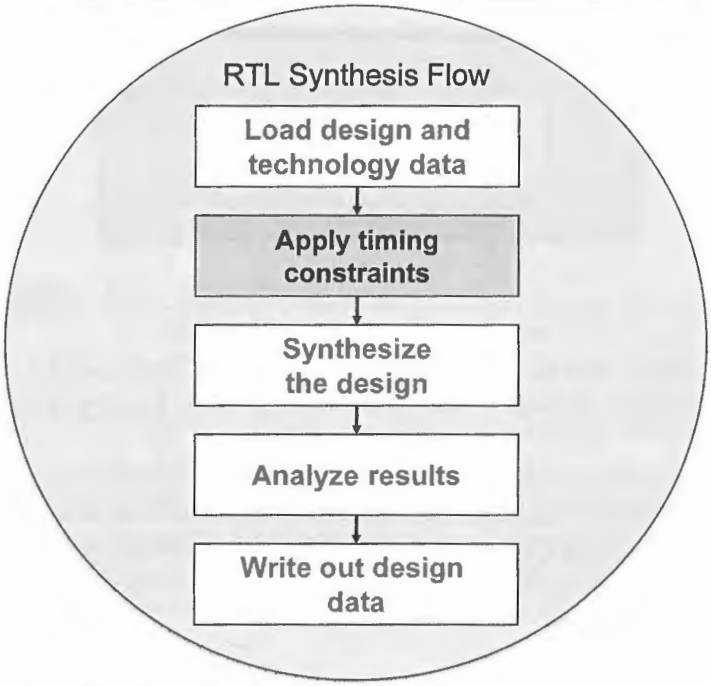
After completing this unit, you should be able to apply constraints to model I/O timing effects of:

- **Input transition times**
- **Capacitive output loads**

4-2

©2018 Synopsys, Inc. All Rights Reserved.

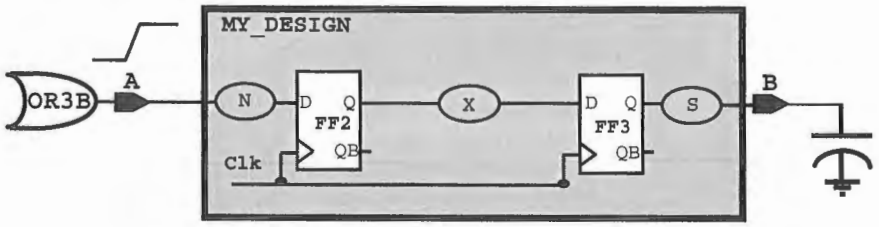
RTL Synthesis Flow



4- 3

©2018 Synopsys, Inc. All Rights Reserved.

Factors Affecting Timing



mydesign.con

```
create_clock -period 2 [get_ports Clk]
set_input_delay -max 0.6 -clock Clk [get_ports A]
set_output_delay -max 0.8 -clock Clk [get_ports B]
```

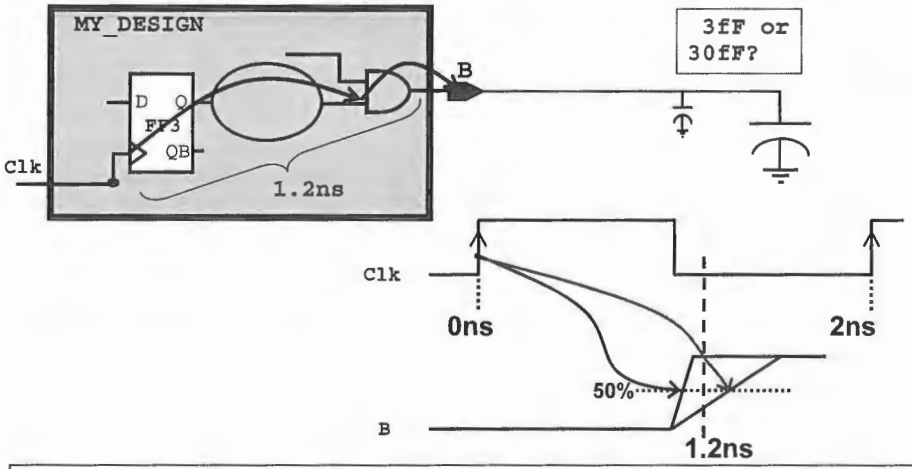
The above constraints are required, but not sufficient for accurate timing analysis and optimization of I/O logic paths.
Need to take into account input transition times and output loads.

4-4

The timing constraints determine how much time is available for reg-to-reg, input and output paths, but they do not describe under what conditions these delays must be met. Gate delays are a function of their input transition time and output capacitive loading. By default, DC assumes ideal (zero) loads on output ports, and ideal (zero) transition times on input ports. This would result in optimistic I/O path timing. For accurate I/O path timing, capacitive loading on outputs and transition times on inputs must be accurately modeled.

©2018 Synopsys, Inc. All Rights Reserved.

Effect of Output Capacitive Load



Capacitive loading on an output port affects the transition time, and thereby the cell delay, of the output driver.

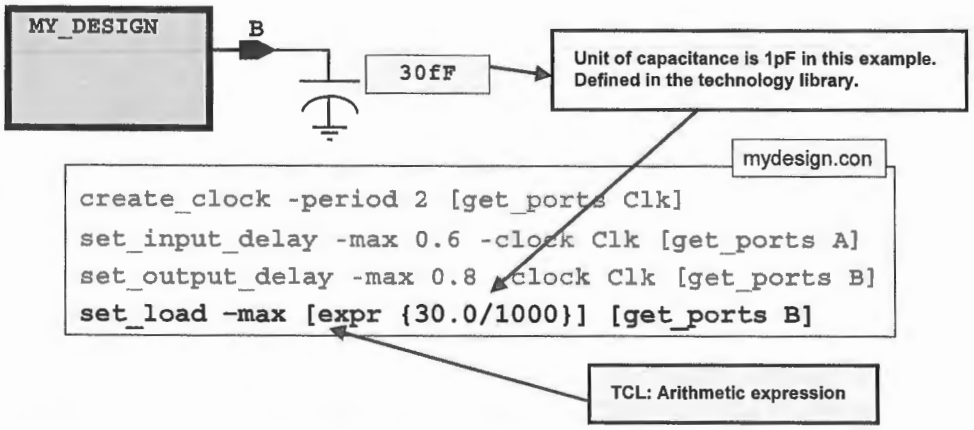
By default DC assumes zero capacitive loading on outputs. It is therefore important to accurately model capacitive loading on all outputs.

4-5

©2018 Synopsys, Inc. All Rights Reserved.

Modeling Output Capacitive Load: Example 1

Spec: Maximum capacitive load on output port B = 30fF



What if an absolute capacitive load value is not available?

4-6

©2018 Synopsys, Inc. All Rights Reserved.

Modeling Output Capacitive Load: Example 2

Spec: Maximum load on output port B = 1 "AN2" gate load, or
= 3 "inv1a0" gates

Use `load_of` *`lib/cell/pin`* to place the load of a gate from the technology library on the port:




```
set_load -max [load_of my_lib/AN2/A] [get_ports B]
```

```
set_load -max [expr {[load_of my_lib/inv1a0/A] * 3}] \
  [get_ports B]
```

[1]

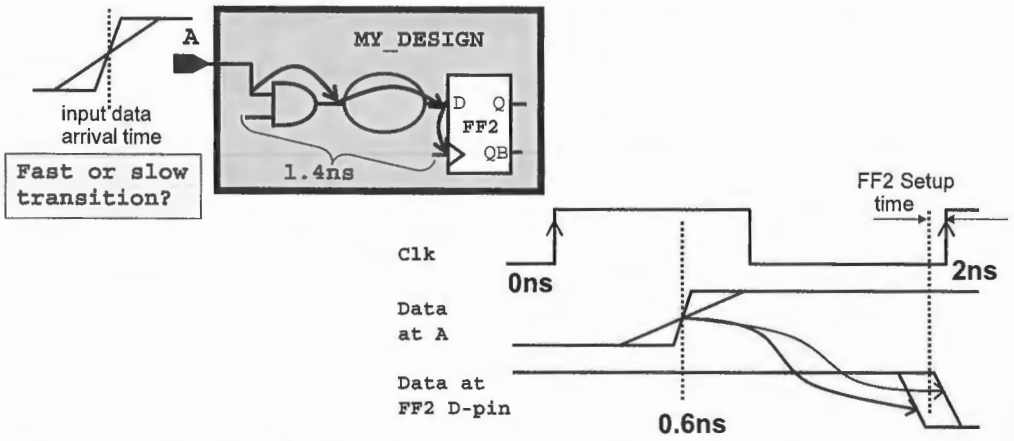
4-7

[1] While the curly braces in the `expr` argument can technically be omitted, when checking the syntax using `dcprocheck`, you will get the following syntax warning: " `... use curly braces to avoid double substitution` ". The `dcprocheck` utility is pickier about syntax than DC, in some case. You can also avoid this warning by using the curly braces just where they are expected as follows:

```
set_load -max [expr {[load_of my_lib/inv1a0/A]} * 3] [get_ports B]
```


©2018 Synopsys, Inc. All Rights Reserved.

Effect of Input Transition Time



Rise and fall transition times on an input port affect the cell delay of the input gate.

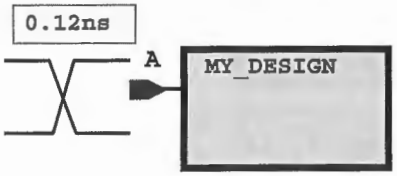
By default DC assumes zero transition times on inputs. It is therefore important to accurately model transition times on all inputs.

4-8

©2018 Synopsys, Inc. All Rights Reserved.

Modeling Input Transition: Example 1

Spec: Maximum rise/fall input transition on input port A = 0.12ns



mydesign.con

```
create_clock -period 2 [get_ports Clk]
set_input_delay -max 0.6 -clock Clk [get_ports A]
set_output_delay -max 0.8 -clock Clk [get_ports B]
set_load -max [expr {30.0/1000}] [get_ports B]
set_input_transition -max 0.12 [get_ports A]
```

What if a specific transition time value is not known?

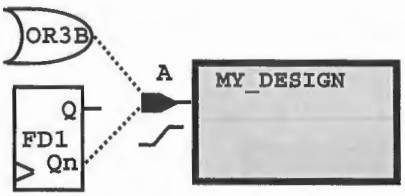
4-9

©2018 Synopsys, Inc. All Rights Reserved.

Modeling Input Transition: Example 2

Spec: Driving cell on input port A = OR3B gate, or

= Qn pin of FD1 flip-flop



mydesign.con

```
create_clock -period 10 [get_ports Clk]
set_input_delay -max 3 -clock Clk [get_ports A]
set_output_delay -max 4 -clock Clk [get_ports B]
set_load -max [expr {30.0/1000}] [get_ports B]
set_driving_cell -max -lib_cell OR3B [get_ports A]
                or
set_driving_cell -max -lib_cell FD1 -pin Qn [get_ports A]
```

If no pin is given, DC will use first output pin listed in the library cell definition!

Note: Port A will also inherit any logic DRCs (max_tran/cap) defined on the driving cell's output pin (in the library)

4-10

In addition to deriving the transition time of the driving cell and applying it to the input port, by default, design rule attributes (`max_fanout`, `max_capacitance`, `max_transition`, `min_fanout`, `min_capacitance`, `min_transition`) are copied from the output pin of the driving cell (from the library) and also applied to the input port. If the input port is actually expected to be driven by the specified driving cell, then this default behavior is the correct, acceptable behavior. A warning message about the DRCs is issued when the constraints are applied, which can be safely ignored.

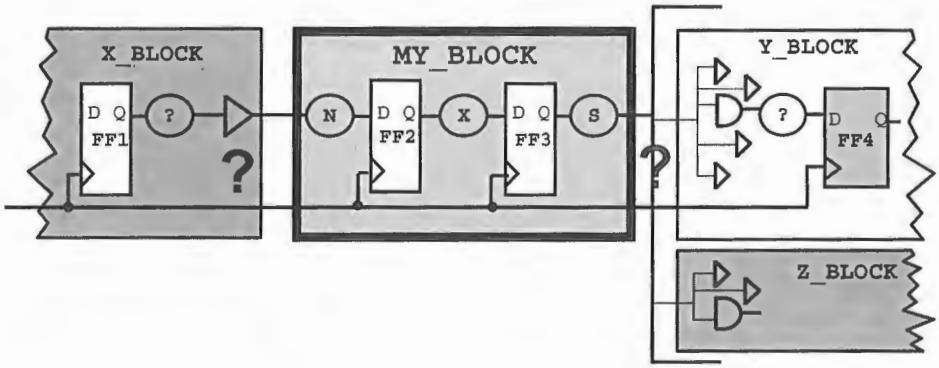
If, however, the driving cell being specified is purposely over- or under-constraining the input transition time, the default behavior may be applying over- or under-constrained DRCs, which may NOT be what is needed. When needed, the application of DRCs can be disabled as follows:

```
set_driving_cell -no_design_rule
```


©2018 Synopsys, Inc. All Rights Reserved.

Load Budgeting

What if, prior to compiling, the cells driving your inputs, and the loads on your outputs are not known?



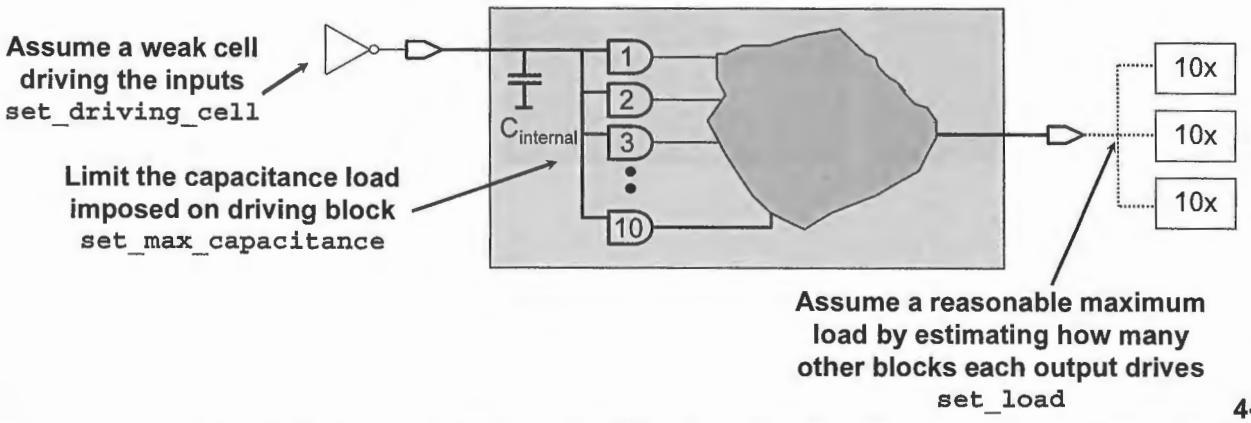
A: Create a Load Budget!

4-11

©2018 Synopsys, Inc. All Rights Reserved.

Load Budgeting Assumptions

- **Apply a weak driving cell on inputs (to be conservative): `inv1a1`**
- **Apply a reasonable maximum output load by:**
  - Everyone agreeing to limit their input capacitance fanout: 10 "`and2a1`" gates
  - Estimating the number of other major blocks your outputs may have to drive: 3



4-12

©2018 Synopsys, Inc. All Rights Reserved.

Load Budget Script

```
# A generic Time and Load Budgeting script file
# for MY_BLOCK, X_BLOCK and Y_BLOCK

reset_design
source timing_budget.tcl

set ALL_INP_EXC_CLK [remove_from_collection [all_inputs] [get_ports Clk]]


# Assume a weak driving buffer on the inputs[1]
set_driving_cell -max -no_design_rule -lib_cell invla1 $ALL_INP_EXC_CLK

# Limit the input load[2]
set MAX_INPUT_LOAD [expr {[load_of ssc_core_slow/and2a1/A] * 10}]
set_max_capacitance  $MAX_INPUT_LOAD  $ALL_INP_EXC_CLK

# Model the max possible load on the outputs, assuming
# outputs will only be tied to 3 subsequent blocks[2]
set_load -max [expr {$MAX_INPUT_LOAD * 3}]  [all_outputs]
```

Script from Unit 3

4-13

[1] `set_driving_cell`, by default, also applies the `max_capacitance` value of the driving cell to the input ports. DC applies the more restrictive of multiple design rules, so if the driving cell value is smaller than the `10 AND2` gate user-defined max cap rule (very likely), DC ignores the user-defined `set_max_capacitance` command! The `-no_design_rule` option prevents this from occurring.

The `set_max_capacitance` command is a "design rule constraint" (DRC) which limits the fanout capacitance (pin + wire) of a port. By default, DRCs have higher priority than timing and area constraints, so it is very important to not over-constrain user-defined DRCs.

[2] To determine the conservative load of your block's outputs, the above example requires that you limit the maximum fanout capacitance of all block's input ports to an arbitrarily large value. You must also estimate (arbitrarily) the maximum number of blocks that any of your output ports can drive. Alternatively, the following method is a way of applying a conservative output load which is not arbitrary, but is instead determined by your library cell characteristics. The method assumes that each output pin of your library cells has a vendor-defined `max_capacitance` attribute and value. It determines the largest `max_capacitance` value in the entire library. Since, by default, no output port is able to drive a larger load than this maximum value without violating a DRC, you set the load of each of your outputs to this maximum value.

```
set LIB_NAME ssc_core_slow
set MAX_CAP 0
set OUTPUT_PINS [get_lib_pins $LIB_NAME/*/* -filter "direction == 2"]
foreach_in_collection pin $OUTPUT_PINS {
   set NEW_CAP [get_attribute $pin max_capacitance]
   if {$NEW_CAP > $MAX_CAP} {
      set MAX_CAP $NEW_CAP      }
   }
set_load -max $MAX_CAP [all_outputs]
```


©2018 Synopsys, Inc. All Rights Reserved.

Summary

You should now be able to apply constraints to model I/O timing effects of:

- Input transition times
- Capacitive output loads

4-14

©2018 Synopsys, Inc. All Rights Reserved.

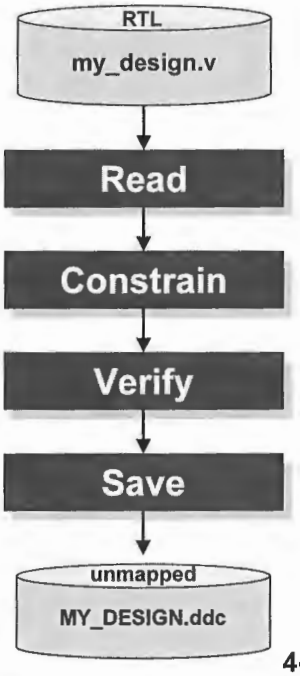
Lab 4: Constraints: Input Transition and Output Loading



45 minutes

Lab 4: Constraints: Input Transition and Output Loading

Apply Input Transition and Output Loading Constraints



4-15

©2018 Synopsys, Inc. All Rights Reserved.

This page was intentionally left blank

©2018 Synopsys, Inc. All Rights Reserved.

Agenda

DAY 2

3	Constraints: Reg-to-Reg and I/O Timing (Lab) 
4	Constraints: Input Transition and Output Loading 
5	DC Ultra Synthesis Techniques 

© 2018 Synopsys, Inc. All Rights Reserved

©2018 Synopsys, Inc. All Rights Reserved.

Unit Objectives

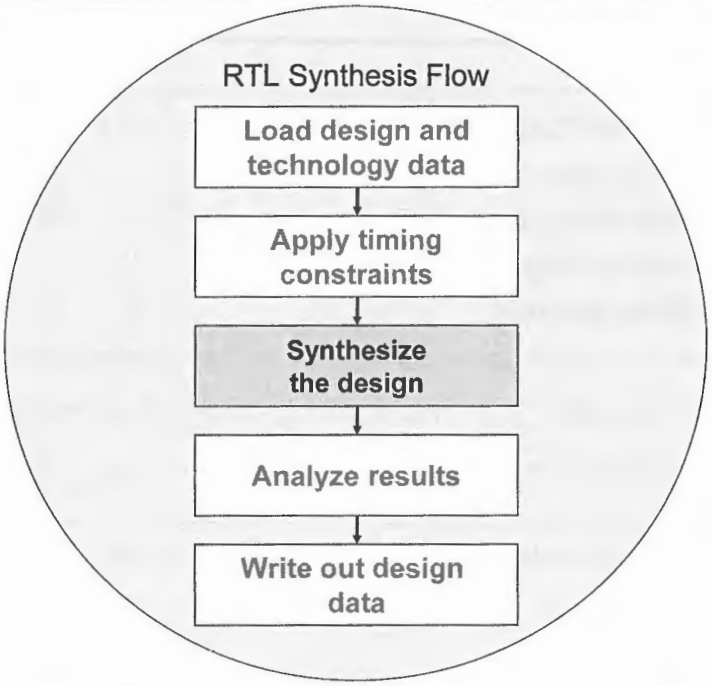
After completing this unit, you should be able to use DC Ultra techniques to:

- **Synthesize RTL to gates (top-down)**
- **Achieve timing goals with good post-layout correlation using applicable techniques:**
  - Auto ungrouping
  - Boundary optimization
  - Test-ready synthesis
  - Adaptive retiming
  - High effort timing
  - Prioritizing setup timing over DRCs
  - Disabling DRC fixing on clock network
  - User-defined path groups
  - TNS-driven placement
  - Pipeline or register retiming
  - Multi-core optimization
  - Disabling runtime intensive settings
  - Final area recovery

5-2

©2018 Synopsys, Inc. All Rights Reserved.

RTL Synthesis Flow



5- 3

©2018 Synopsys, Inc. All Rights Reserved.

Unit Agenda

- Some examples of default `compile_ultra` optimizations
  - Critical path re-synthesis
  - Arithmetic optimizations
  - Load splitting and logic duplication
  - Library analysis
- User controllable `compile_ultra` optimizations
- Post-compile analysis and incremental compile
- Final Area Recovery
- Complete flow overview

5-4

©2018 Synopsys, Inc. All Rights Reserved.

Requirements for Optimal Synthesis Results

- **The following are *crucial* for synthesis to meet aggressive timing goals with good post-layout correlation**
  - Good quality RTL code - optimum for DC
  - Complete and accurate constraints (Units 3, 4, 7, 9)
  - Compile using all applicable *DC Ultra* optimization techniques (this unit)
  - Take advantage of *DC Graphical* features (Unit 8)



See Appendix for a coding style example

- **To simplify the synthesis flow, compile as large a design (block) as possible "top-down"**
  - Fewer block constraint files, compiles and iterations needed
  - *Design Compiler* has no inherent design size limit - limited only by available machine memory and acceptable run times

In this unit the assumption is that you are starting with good quality RTL code and constraints, and are performing top-down compiles

5-5

Good quality RTL code - optimum for DC
- RTL follows DC's coding style guidelines and best practices
- RTL algorithms have been written in an efficient manner, with regards to the hardware that is inferred by the code

Complete and accurate constraints
- Realistic constraints and attributes, not over-constrained
- All false- or multi-cycle paths are identified

Compile "top-down"
- Read, constrain and compile the top-level design, when possible

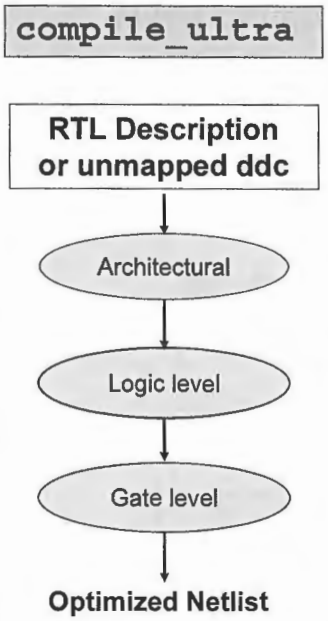
DC *Ultra*
- Requires *DC Ultra* and *DesignWare Foundation* licenses
- Enables use of `compile_ultra` command
- Invokes additional timing- and area-focused optimizations not available with `compile`

DC *Topographical*
- No additional license needed (beyond *DC Ultra* and *DesignWare Foundation*)
- Invoked with `dc_shell -topographical_mode`
- Performs coarse placement during `compile_ultra`
- Results in better interconnect RC parasitic estimates (compared to *WLM*)

DC *Graphical*
- Only available in *Topographical mode*; Requires a *DC-Extension* license
- Features: *Synopsys Physical Guidance* (SPG) flow; Layout visualization and congestion analysis; Floorplan exploration - invoking ICC (II)'s Design Planning tool within DC
- *SPG flow* provides: Congestion- and add'l timing-focused optimizations (`compile_ultra -spg`) and the ability to write out cell placement used as the start-point by *IC Compiler (II)* placement

©2018 Synopsys, Inc. All Rights Reserved.

Default `compile_ultra` Optimizations



- **Performs three levels of optimization:**
  - Architectural level or high level synthesis
    - Only performed when compiling RTL or unmapped *ddc*
  - Logic level or GTECH optimization
  - Gate-level or mapping optimization
- **Optimization priority: Logic DRCs and timing**
  - Minimizes area without impacting other constraints[1]
- **When used in *Topo* mode, performs under-the-hood placement and routing estimation to calculate net RCs**
- **Requires an *Ultra* as well as a *DesignWare Foundation* license**
- **Invokes additional high performance optimization algorithms (examples follow)**

5-6

[1] By default, `compile_ultra` uses a `max_area` constraint of zero. The area constraint always has the lowest priority during optimization, compared to timing, design rule or power constraints.

©2018 Synopsys, Inc. All Rights Reserved.

What is the *DesignWare* Library?

- **A collection of soft IP blocks and Datapath components:**
  - Technology independent, pre-verified, reusable, parameterizable, synthesizable
- **Accessing the Right Component:**
  - ***Operator inferencing* for arithmetic operators (architectural level)**
    - +, -, *, >, =, <
    - Operators greater than 4 bits wide infer a hierarchical sub-block
  - ***Instantiation* for a wide variety of standard IP**
    - DW_fifo_..., DW_shiftreg, DW_div_seq, DW_ram_...
- **Running `compile_ultra` requires *a DesignWare Foundation* license**
  - Enables arithmetic/datapath optimizations (shown next)
  - Allows access to the *DesignWare* IP
  - *DesignWare* library automatically included in the `synthetic_library` and `link_library` variables during `compile_ultra`

5-7

Arithmetic components are automatically *inferred* by using their corresponding operators (+, -, *, >, =, <) in the RTL code. It is therefore very straightforward to infer very complex arithmetic circuits. All the other functional components (which do not have a standard RTL *symbol* like the arithmetic operators) are instantiated in the code – also very straightforward.

For datasheets of *DesignWare* components refer to the *DesignWare Library Databook.*

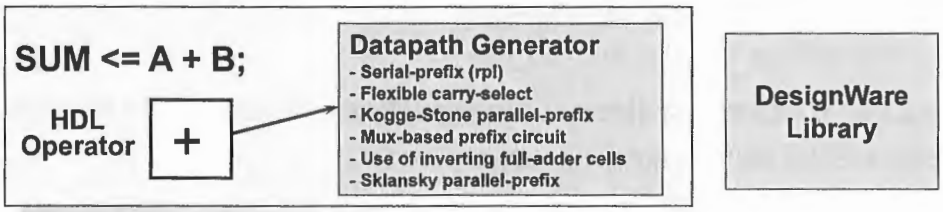
Basic architectures, or *implementations*, of +, -, *, >=, <, <= etc, are accessible by default from the 'standard' *DesignWare* library. To access the entire collection of basic and high-performance arithemetic implementations, plus the entire collection of additional standard IP, a DesignWare library license is required, and the `synthetic_library` and `link_library` variables need to be modified. The settings are applied automatically during `compile_ultra`.

The *DesignWare* library resides in the directory: *$SYNOPSYS/libraries/syn*

©2018 Synopsys, Inc. All Rights Reserved.

Singleton Arithmetic Optimization

- **The *DesignWare* library enables the use of a datapath generator to build many different implementations of arithmetic operators**
  - `+ , - , * , < , > , <= , >=`
- **Multiple implementations allow `compile_ultra` to choose the best implementation for each singleton operator in the design**
  - The smallest one that meets timing



- **Once selected, each implementation is then optimized for the target technology library**

5-8

The use of the *DesignWare library* requires a *DesignWare license*, which is required to run `compile_ultra`. The library is set up to be used automatically during `compile_ultra`.

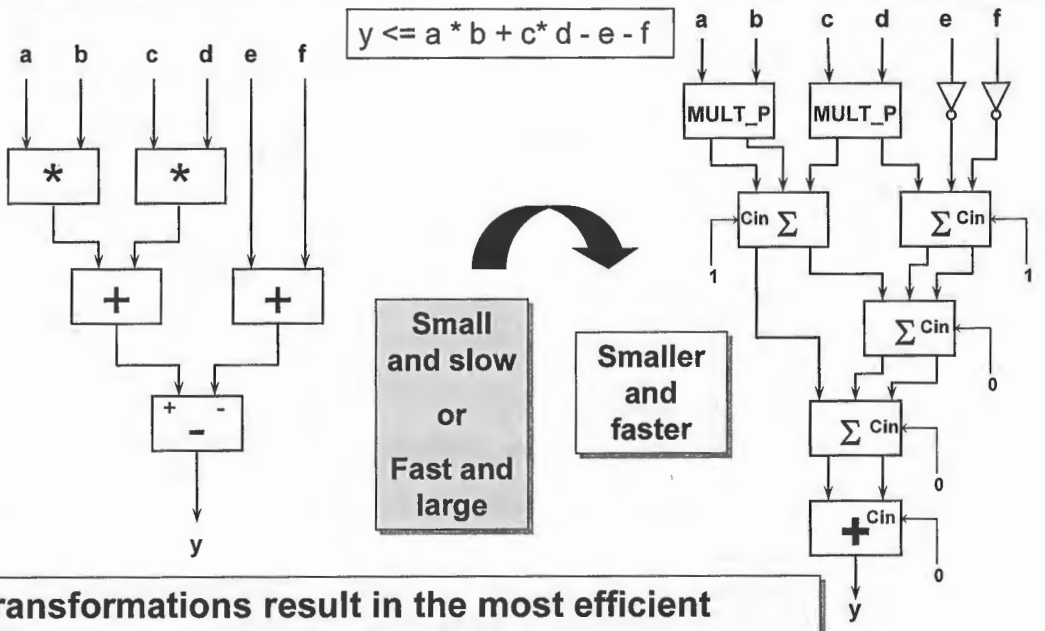
The datapath generator dynamically builds the appropriate implementation (the smallest that meets timing) for each arithmetic operator (+, -, *) , rather than selecting an implementation from a "static" set of pre-built architectures (e.g. *carry look-ahead* and *ripple carry* from the `standard.sldb` library). For example, for an adder it uses a constraint-driven variable prefix architecture (implementation architecture name: *pparch*), which includes:

- Serial-prefix (rpl)
- Flexible carry-select
- Kogge-Stone parallel-prefix (like "fastcla")
- Mux-based prefix circuit
- Use of inverting full-adder cells
- Sklansky parallel-prefix

©2018 Synopsys, Inc. All Rights Reserved.

Datapath Optimization: CSA Transformations

Arithmetic optimization



CSA transformations result in the most efficient arithmetic circuit – smaller *and* faster

5-9

Multipliers, adders and subtractors are transformed using Carry Save Adder (∑) trees: CSA adders are as small as ripple adders, but have no carry-ripple delay – they are small and fast.

©2018 Synopsys, Inc. All Rights Reserved.

Arithmetic Expression Optimization

- **Constant/operand folding**

```
A + 2*B - 2 + B - A + 7  →  3*B + 5
```

- **Common sub-expression sharing**

```
Z1 = A + B + C; Z2 = C + D + B;
→ T = B + C; Z1 = A + T; Z2 = T + D;
```

- **SOP → POS transformation**

```
A*C + B*C  →  (A+B) * C
```

- **Comparator sharing**

```
Z1 = A>B; Z2 = A<B; Z3 = A<=B;
```

→ Uses a single subtractor with multiple comparison outputs

- **Parallel constant multipliers optimization**

```
Temp=In1+In2; Out1=Temp*105;      Out2=Temp*621
                          (64+32+8+1)          (105+512+4)
→ Out1=(Temp<<6) + (Temp<<5) + (Temp<<3) + Temp
→ Out2=  Out1    + (Temp<<9) + (Temp<<2)
```

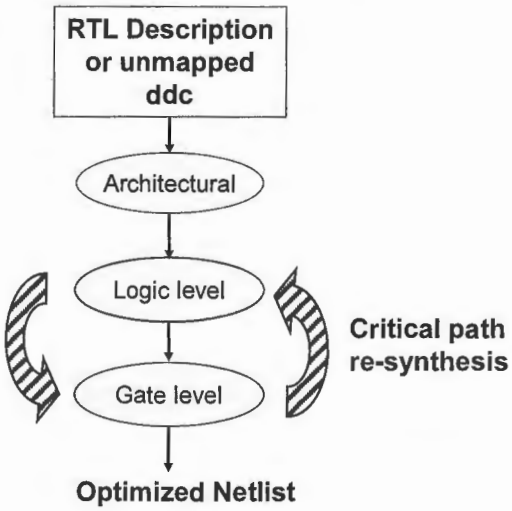
5-10

©2018 Synopsys, Inc. All Rights Reserved.

Critical Path Re-Synthesis

Critical path resynthesis

`compile_ultra`



Critical Path Re-synthesis
is automatically invoked as needed

- Cones of mapped logic containing stubborn, violating critical paths are returned to logic-level optimization and then re-mapped, iteratively

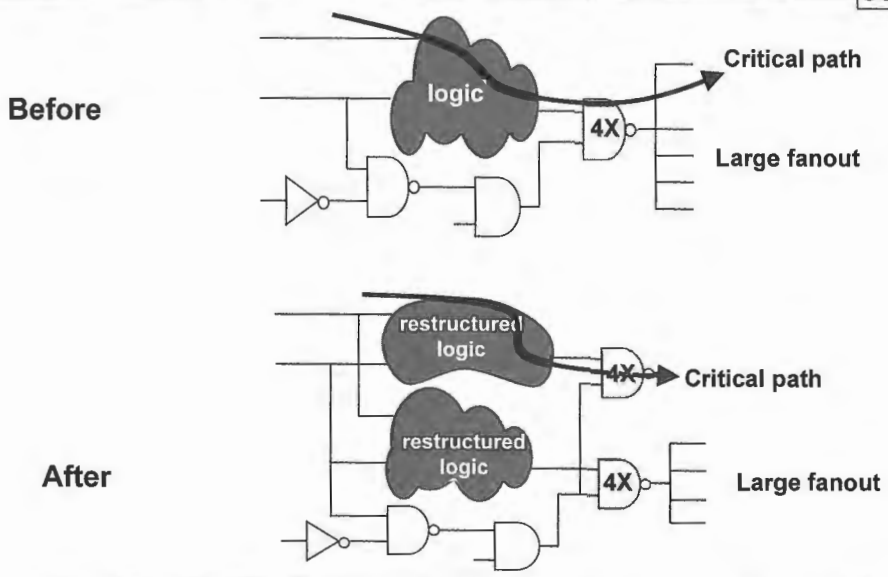
5-11

CPR is available with the `compile` command but must be explicitly invoked with the `-map_effort high` option.

©2018 Synopsys, Inc. All Rights Reserved.

Load Splitting and Combo Logic Duplication

Combinational logic duplication



Load-splitting and logic duplication reduces critical path delays – results in faster (but larger) circuit

Register replication is discussed in Unit 8

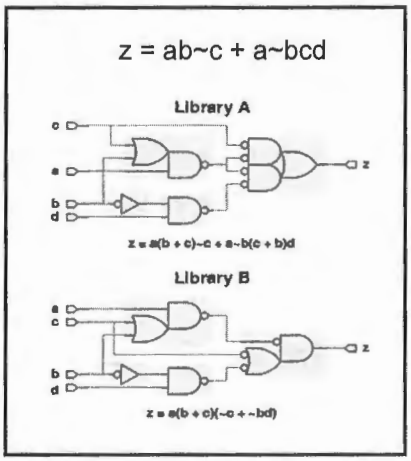
5-12

Combinational logic duplication is always enabled, and applied as needed to improve timing. There is no control to turn it off.
Register replication is also available, and is discussed in a later unit.

©2018 Synopsys, Inc. All Rights Reserved.

Library Analysis (ALIB)

- **Library analysis characterizes the target technology library**
  - Creates a cache (ALIB) of pre-built implementations of a wide range of complex Boolean functions
- **Results in better area optimization**
- **ALIB is generated once per technology library**
  - Happens during the first `compile_ultra`
  - Re-used by subsequent compiles and by other users



Example of one "complex Boolean function" and two of its many possible pre-built implementations, made up of available logic functions (yellow) in a specific target library

5-13

The library analysis computes size- and delay-independent implementations of more complex Boolean functions, based on simpler Boolean functions of the cells available in the library:

- Builds all functions up to 4 inputs
- Builds commonly used functions > 4 inputs
- About 15 implementations per function are generated

The `alib_analyze_libs` command can be used to explicitly generate ALIB. Otherwise, if it does not already exist, it is created automatically during the first `compile_ultra`.

The `alib_library_analysis_path` variable can be used to specify the UNIX directory where ALIB is to be created.

The ALIB cache is not re-created if the `dont_use` attribute is applied or removed from cells in an analyzed technology library.

©2018 Synopsys, Inc. All Rights Reserved.

Test for Understanding



1. **Circle the correct statement(s) regarding *DesignWare*:**
   a) Requires additional library variable settings prior to invoking `compile_ultra`
   b) Uses *operator inferencing* to synthesize a wide variety of arithmetic and relational operators
   c) Uses *operator inferencing* to synthesize a wide variety of standard IP, e.g. FIFOs, shift-registers, sequential dividers

2. **Circle the correct statement(s) regarding `compile_ultra`:**
   a) The `-cpr` option returns mapped gates to logic level optimization
   b) *CSA transformations* infer the smallest and fastest individual adders, multipliers and comparators (arithmetic *DesignWare* components)
   c) Arithmetic expression optimization can reduce the number of arithmetic operators needed
   d) Logic duplication can reduce the critical path delay and area

5-14

Answers:

1. B. `compile_ultra` automatically sets the library variables; non-arithmetic/relational IP can not be *inferred* by DC – they must be *instantiated* in the RTL code
2. C. *CPR* is invoked automatically with `compile_ultra`, as needed. There is no `-cpr` option. *CSA* transformations break down arithmetic operators into half-adder or *CSA* operators, which are then combined and optimized as a whole. It does not infer faster/smaller individual operators, which IS done by "implementation selection" on singleton operators. Logic duplication increases area – it does not reduce it.

©2018 Synopsys, Inc. All Rights Reserved.

Unit Agenda

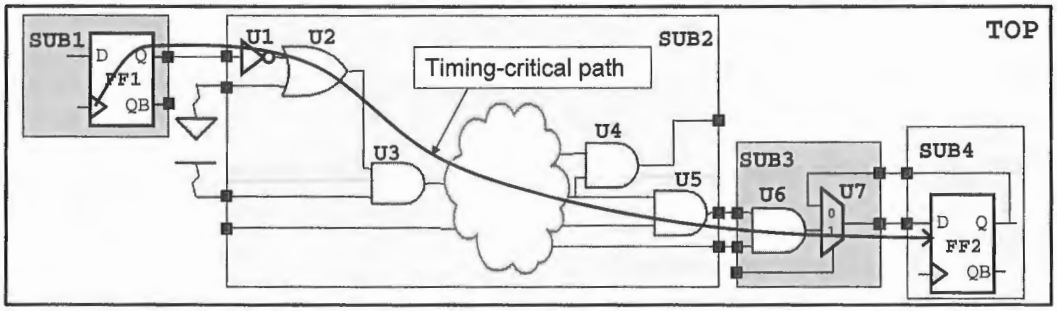
- Some examples of default `compile_ultra` optimizations
- User controllable `compile_ultra` optimizations
  - Auto-ungrouping
  - Boundary optimization
  - Test-ready synthesis
  - Adaptive retiming
- Additional User Controllable Optimizations
- Post-compile analysis and incremental compile
- Final Area Recovery
- Complete flow overview

5-15

The following optimization techniques can be controlled to influence timing, area and/or run time results. These techniques can be controlled with special commands applied prior to invoking, or with options when invoking `compile_ultra`.

©2018 Synopsys, Inc. All Rights Reserved.

Preservation of Sub-Block Pins



By default, synthesis must preserve pin definitions on sub-block hierarchies. As a result, if `SUB2`-`SUB4` hierarchies remain:

- `U1` can not be removed by reconnecting its input to `FF1/QB`
- `U2` and `U3` with tie low/high inputs can not be removed
- `U4` with unconnected output can not be removed
- `U5` and `U6` can not be combined into a *3-input AND* gate (combinational logic optimization)
- `U7` and `FF2` can not be combined into a single *Enable Flip-Flop* (sequential logic optimization)

? What does `compile_ultra` do?

5-16

©2018 Synopsys, Inc. All Rights Reserved.

Auto-Ungrouping - ON by Default

Auto-ungrouping

Auto-ungrouping **is ON, by default, with `compile_ultra`**

- **Automatically removes hierarchies of poorly partitioned sub-designs[1] with I/O paths that violate timing**
- **To disable auto-ungrouping altogether:**

```
compile_ultra -no_autoungroup
```

- **To disable auto-ungrouping on selective sub-designs apply the following before `compile_ultra`:**

```
set_ungroup <references_or_cells> false
```

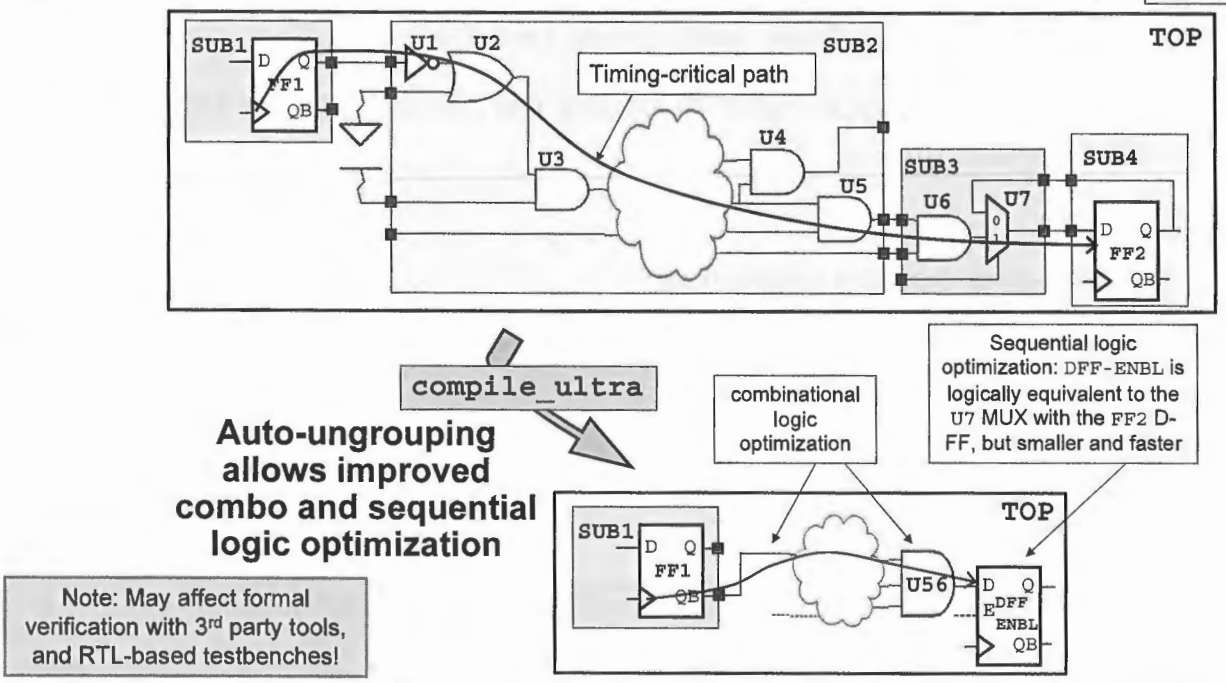
5-17

[1] A sub-design is considered "poorly partitioned" if all of its data inputs are not directly connected to register outputs, or if all of its data outputs are not registered. In other words, a well partitioned sub-design does not have hierarchy boundaries that "slice" through non-clock combinational logic or register inputs.

`set_ungroup false` can be used to disable auto-ungrouping on specified cells or references. By default, any sub-design that is poorly partitioned and has an input or output path that violates timing, can be auto-ungrouped, including top-level blocks. For example, if you want to avoid glue logic at the top level you can disable auto-ungrouping of all top-level cells. You can also use this attribute to maintain the hierarchy of specific designs for verification purposes, or "pipeline" sub-designs for register-retiming purposes.

©2018 Synopsys, Inc. All Rights Reserved.

Auto-Ungrouping Example



compile_ultra

Auto-ungrouping allows improved combo and sequential logic optimization

combinational logic optimization

Sequential logic optimization: DFF-ENBL is logically equivalent to the U7 MUX with the FF2 D-FF, but smaller and faster

Note: May affect formal verification with 3rd party tools, and RTL-based testbenches!

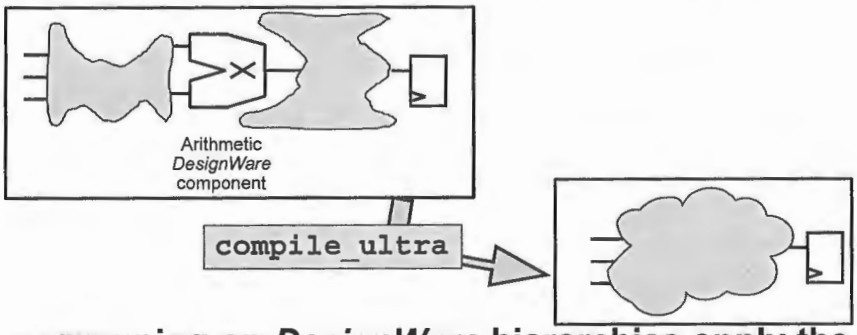
5-18

Note: A file called `default.svf` is created automatically during compile, which records the "changes" that *boundary optimization*, *register repositioning* and *ungrouping* make to the design. This file is readable by *Formality*, Synopsys' formal verification or equivalency checking tool. The *sfv* information helps *Formality* to verify the equivalency. To rename the file use `set_svf <My_name.svf>`; invoke this command before reading in RTL (can put it in your `.synopsys_dc.setup` file).

©2018 Synopsys, Inc. All Rights Reserved.

Auto-Ungrouping of *DesignWare* Hierarchies

- Auto-ungrouping also automatically removes hierarchies of *DesignWare* components



- To disable auto-ungrouping on *DesignWare* hierarchies apply the following before `compile_ultra`:

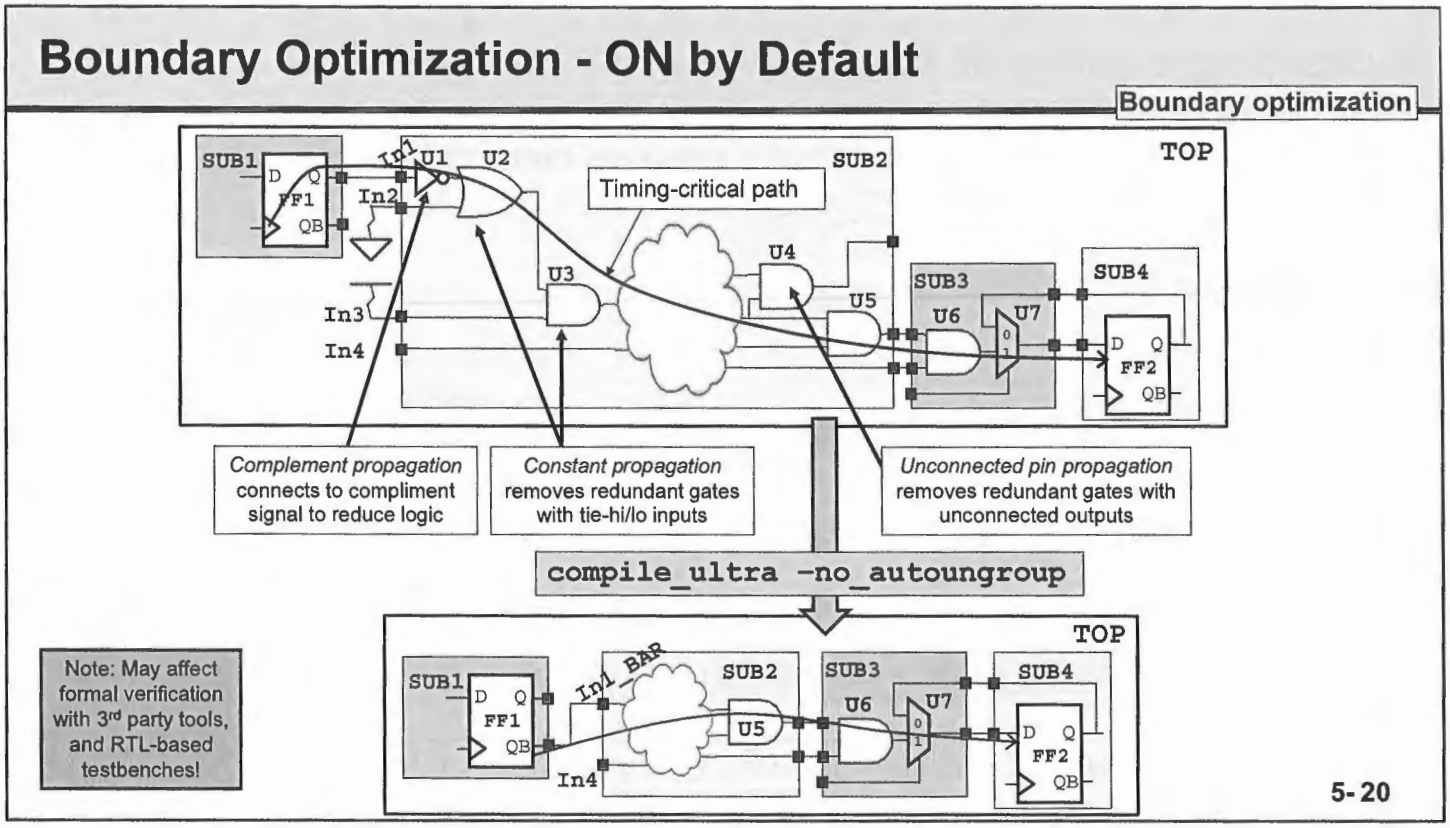
```
set_app_var compile_ultra_ungroup_dw false
```

? What if sub-design hierarchies must be maintained?

5-19

©2018 Synopsys, Inc. All Rights Reserved.

Boundary Optimization - ON by Default



The `-no_autoungroup` option is used here to prevent auto-ungrouping of the `SUB2`, `SUB3` and `SUB4` hierarchies, so that the effects of "boundary optimization" can be illustrated.

Boundary optimization propagates constants, unconnected pins, and complement information to reduce delay and/or area.

Since boundary optimization may eliminate output ports and invert input port signals, this can have an impact on verification: For example, if a verification test-bench was created for the RTL code, which probes or drives one of these block ports which is subsequently inverted or removed, the existing test-bench will no longer work. If you know in advance which blocks must be preserved for verification purposes, use `set_boundary_optimization` to enable boundary optimization only on selective blocks.

Note: A file called `default.svf` is created automatically during compile, which records the "changes" that *boundary optimization*, *register repositioning* and *ungrouping* make to the design. This file is readable by *Formality*, Synopsys' formal verification or equivalency checking tool. The *svf* information helps *Formality* to verify the equivalency. To rename the file use `set_svf <My_name.svf>`; invoke this command before reading in RTL (can put it in your `.synopsys_dc.setup` file).

Note: Any sub-design that is instantiated more than once (multiply-instantiated), is automatically "uniquified" during compile: The sub-designs are given new unique reference names (by default, `ref_name_0`, `ref_name_1`, etc). In the example above, if `SUB2` was instantiated twice, where the inputs `In2` and `In3` of instance `I1_SUB2` were tied high/low as shown above, but inputs `In2` and `In3` of instance `I2_SUB2` were connected to drivers, since the two instances are first uniquified, boundary optimization will then be able to perform constant propagation of `I1_SUB2`, while it is not needed and performed on `I2_SUB2`.

©2018 Synopsys, Inc. All Rights Reserved.

Controlling Boundary Optimization

Boundary optimization **is ON, by default, with** `compile_ultra`

- **To disable boundary optimization altogether:**

```
compile_ultra -no_boundary_optimization
```

Apply the following before `compile_ultra` **to:**

- **Disable boundary optimization on selective sub-designs**

```
set_boundary_optimization <cells_designs> false
```

- **Disable *constant propagation* (enabled, by default)**

```
set_app_var \
 compile_enable_constant_propagation_with_no_boundary_opt false
```

- **Disable *constant propagation* on specific sub-block pins**

```
set_compile_directives -constant_propagation false \
      [get_pins "SUB2/In2 SUB2/In3"]
```

5-21

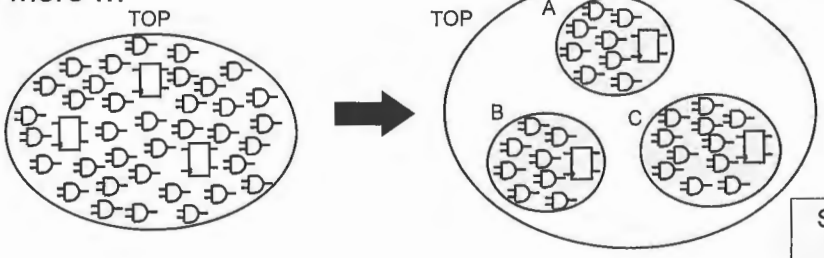
Complemented pins are automatically renamed with an "_BAR". This can be modified with `set_app_var port_complement_naming_style %s_SOMETHING_ELSE`

Note: If `set_boundary_optimization` is set to **`true`** on cells/designs, `compile_ultra` **`-no_bounday_optimization`** will apply boundary optimization to those cells/designs.

©2018 Synopsys, Inc. All Rights Reserved.

Design Partitioning

- **Partitioning into sub-designs or blocks is commonly required for large designs, driven by many needs:**
  - Separate distinct functions
  - Achieve workable size and complexity
  - Manage project in team environment
  - Design reuse
  - And more …



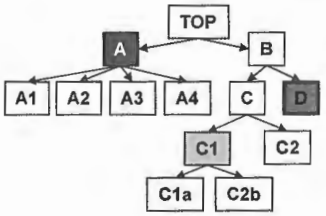
Sub-design logic hierarchy is defined in Verilog/VHDL by `module` or `entity` statements

5-22

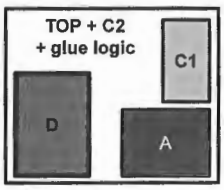
©2018 Synopsys, Inc. All Rights Reserved.

Partitioning Guidelines for Design Compiler

- **Create as large a block as possible, based on available *memory* resources and acceptable *run times***
- **Partition at register outputs**
  - Easier to constrain the blocks to be synthesized
  - Allows combinational and sequential logic optimization (which would otherwise be prevented by hierarchy boundaries)
- **Consider "hierarchical floorplanning" requirements**
  - Each physical hierarchy (macro) must correspond to a logic block



Logic hierarchy



Physical hierarchy

Physical hierarchy is created when individual blocks are "placed and routed" and converted into "macros"

Macros are, in turn, placed and routed (integrated) at a parent level

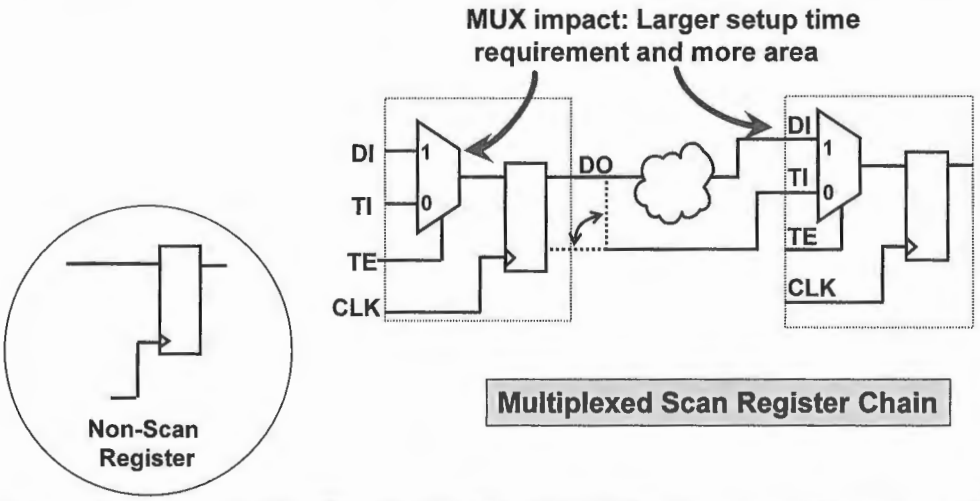
5-23

©2018 Synopsys, Inc. All Rights Reserved.

Scan Registers: The Problem

Test-ready synthesis

Since most designs will include scan chains, must account for the impact of scan registers during synthesis, to avoid negative surprises after scan insertion



Multiplexed Scan Register Chain

5-24

During scan chain stitching, *DFT Compiler* automatically selects the better (less loaded, less timing-critical) Q or Q-BAR pin to use for scan-out of each register.

©2018 Synopsys, Inc. All Rights Reserved.

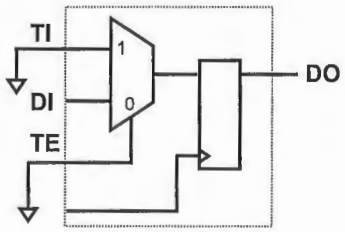
Test-Ready Synthesis - The Solution


```
set_scan_configuration -style \
<multiplexed_flip_flop | clocked_scan | lssd | aux_clock_lssd>
compile_ultra -scan ;# AND
compile_ultra -incremental -scan
```

Scan registers are added during compile, but not chained!

Benefits

- Accurate area and timing modeled up front.
- Easier synthesis flow -- scan cell insertion performed in one compilation step.



Test-ready synthesis requires a DFT Compiler license

5-25

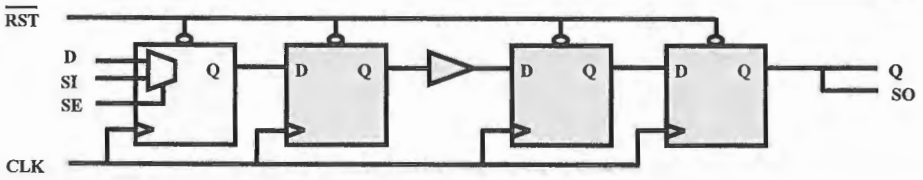
The `multiplexed_flip_flop` style is the default – if this is your scan style you do not have to apply this command.

After the scan cells are actually stitched together into a scan chain, the output of a scan register (DO) will be connected to the scan-in input (TI) of the next scan register in the chain. This additional fanout load is automatically taken into account during synthesis if the scan chain is not yet stitched up.

If you apply the `-scan` option during the first `compile_ultra`, you should continue to use the option for any subsequent incremental compiles, otherwise some of the scan registers may be removed.

©2018 Synopsys, Inc. All Rights Reserved.

Automatic Shift-Register Identification



- **Identifies shift registers automatically during scan insertion (`compile_ultra -scan ...`)**
  - Buffers or inverters between registers do not affect identification
- **Only the first register is replaced with its scan-equivalent**
  - Improved area
  - Reduced scan-enable fanout
- **This feature is ON by default – to disable:**

```
set_app_var compile_seqmap_identify_shift_registers false
```

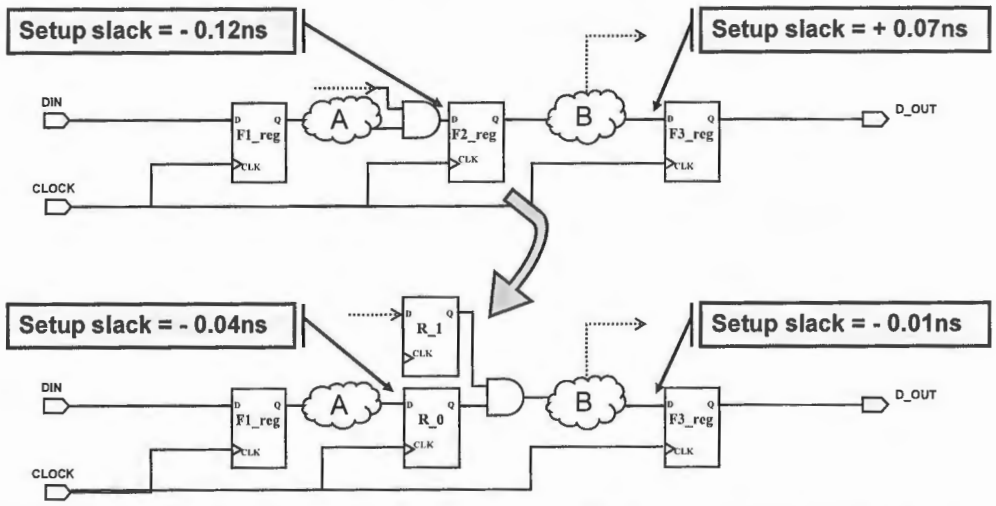
5-26

©2018 Synopsys, Inc. All Rights Reserved.

Adaptive Retiming

`compile_ultra -retime`

Invokes local 'adaptive retiming', which moves registers to reduce critical timing violations (WNS)



5- 27

In the example above, adaptive retiming is able to reduce the WNS from -0.12ns to -0.04ns. It introduces a new, but small violation of -0.01ns to accomplish this.

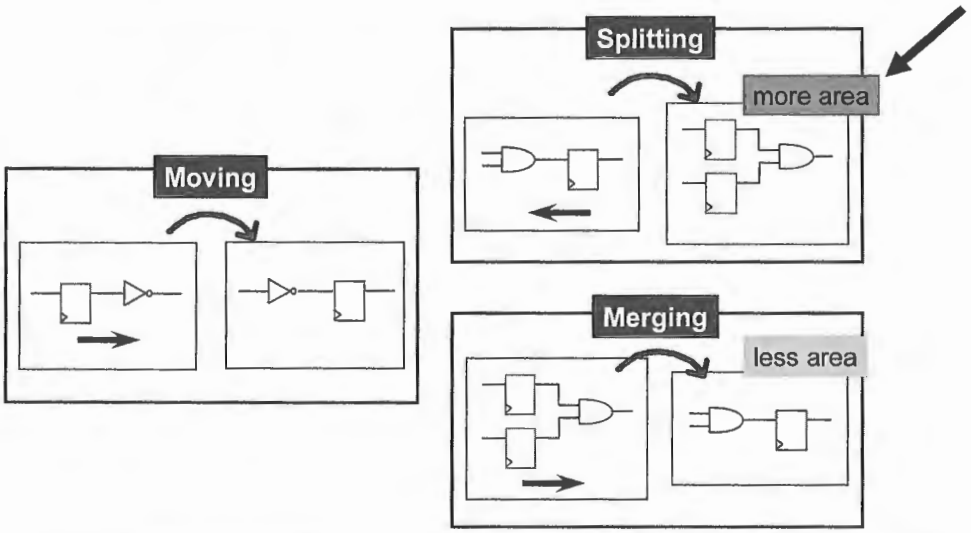
WNS = Worst negative slack

©2018 Synopsys, Inc. All Rights Reserved.

The Fundamentals of *Adaptive Retiming*

***Adaptive Retiming* works by moving, splitting, or merging registers through the cones of logic**

- "End-to-End" functionality of the circuit is unchanged



5- 28

Adaptive retiming makes intelligent choices as to whether a given register can be repositioned in order to reduce a path's WNS.
The overall register count may increase due to "splitting", or decrease (due to "merging"), but the combinational logic remains unchanged.

©2018 Synopsys, Inc. All Rights Reserved.

Adaptive Retiming Details

- **Performs "local" optimization - only on potential timing-critical paths**
  - No "global" delay balancing
  - No runtime overhead
- **Works with both `compile_ultra` and `compile_ultra -incremental`**
- **Moved registers are renamed "`R_##`"**
  - Renamed registers are automatically tracked in an `svf` file used by *Formality* (Synopsys' formal verification tool)
- **To prevent retiming on specific registers or sub-designs:**

Do not use `-retime` if your design requires fixed register positions and/or names!

```
set_dont_retime <cells or designs> true
```

5-29

©2018 Synopsys, Inc. All Rights Reserved.

Test for Understanding 1/2



3. **Circle the correct statement(s) regarding *boundary optimization* during `compile_ultra`:**
   a) Never modifies sub-design pin names
   b) Is ON by default – can be selectively or completely turned off
   c) May break a simulation testbench that worked pre-compile
   d) Can improve timing and area of a design
   e) Can make formal verification with *Formality* difficult
4. **Auto-ungrouping**
   a) Is invoked with `compile_ultra -auto_ungroup`
   b) Can only be applied to all or none of the sub-designs
   c) Does not ungroup *DesignWare* hierarchies, by default
   d) Ungroups all poorly partitioned sub-designs
   e) None of the above

5- 30

Answers:

3. B, C and D. Pin names may be changed if inverted input signals are used (`_bar` is appended, by default). As long as the DC-generated *svf* file is read in by *Formality*, there should be no problems verifying the logical equivalency of any logic modified by boundary optimization, or other optimizations to be discussed shortly.
4. E. Auto-ungrouping is enabled by default with `compile_ultra`. It can selectively be turned off on specific sub-designs with `set_ungroup <references_or_cells> false`. It ungroups *DesignWare* parts by default, which can be disabled with `set_app_var compile_ultra_ungroup_dw false`. It only ungroups poorly partitioned sub-designs with an IO path timing violation.

©2018 Synopsys, Inc. All Rights Reserved.

Test for Understanding 2/2



5. **Test-ready synthesis with `compile_ultra -scan`:**
   a) Enables the additional delay/area impact of scan registers to be considered during compile
   b) Performs scan register insertion and scan chain stitching
   c) Speeds up the scan-in to scan-out path delay
   d) All of the above

6. **Adaptive retiming with `compile_ultra -retime`:**
   a) May improve *WNS* while increasing the number of violating paths
   b) May increase the total number of registers in the design
   c) May decrease the total number of registers in the design
   d) All of the above

5-31

Answers:

5. A. Test-ready synthesis is a technique that allows the effect of scan registers (additional area and data path delay) to be taken into account during optimization – it does not speed up scan-in to scan-out path delays. No scan chain stitching is performed.
6. D. Register merging and splitting affect the overall register count, which may result in more or fewer registers.

©2018 Synopsys, Inc. All Rights Reserved.

Unit Agenda

- Some examples of default `compile_ultra` optimizations
- User controllable `compile_ultra` optimizations
  - Optimizations controlled by `compile_ultra` options
    - High-effort timing optimization
    - Prioritizing setup timing over DRCs
    - Disabling DRC fixing on clock network
    - User-defined path groups
    - TNS-driven placement
    - Pipeline or register retiming
    - Multi-core optimization
    - Disabling runtime-intensive settings
- Post-compile analysis and incremental compile
- Final Area Recovery
- Complete flow overview

5- 32

©2018 Synopsys, Inc. All Rights Reserved.

High-Effort Timing Optimization

- **High-effort timing optimization enables additional optimization strategies to improve timing QoR**
  - Can increase runtime - not intended for 1st pass compile
  - Enable the setting before *compile*

```
set_app_var compile_timing_high_effort true
# default is false
```

5- 33

©2018 Synopsys, Inc. All Rights Reserved.

Prioritizing Setup Timing over DRCs

- **By default, `max_transition`, `max_fanout` and `max_capacitance` design rule constraints (DRCs) have higher priority than timing during optimization**
  - If optimization can only fix a DRC violation or a timing violation, but not both, the DRC is fixed at the expense of timing
- **It may be more important for synthesis to prioritize fixing setup timing over DRC violations**
  - See examples below
- **To increase the priority of setup timing over max DRCs:**

```
set_cost_priority -delay
```

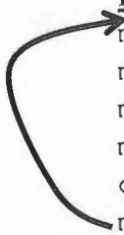
  - DRC fixing is still performed, but will not hurt setup timing

5-34

In many technology libraries, the only significant DRC violations that cannot be fixed without hurting delay are over-constrained nets, such as input ports with large external loads, or around logic marked `dont_touch`. Placing `max_delay` ahead of DRCs in priority allows these DRC violations to be fixed in a way that does not hurt delay. *Design Compiler* might, for example, resize the drivers in another module.
When synthesizing a small block of logic, such as an extracted critical region of a larger design, the possibility of over-constraints at the block boundaries is high. In this case, design rule fixing might better be postponed until the small block has been regrouped into the larger design.

Default optimization cost priorities:

`set_cost_priority -delay` moves "max_delay" (setup timing) up in priority, above the max DRCs.



Priority (descending order)	Notes
`min_capacitance`	Design rule constraint
`max_transition`	Design rule constraint
`max_fanout`	Design rule constraint
`max_capacitance`	Design rule constraint
`cell_degradation`	Design rule constraint
`max_delay`	Optimization constraint
`min_delay`	Optimization constraint
`power`	Optimization constraint
`area`	Optimization constraint

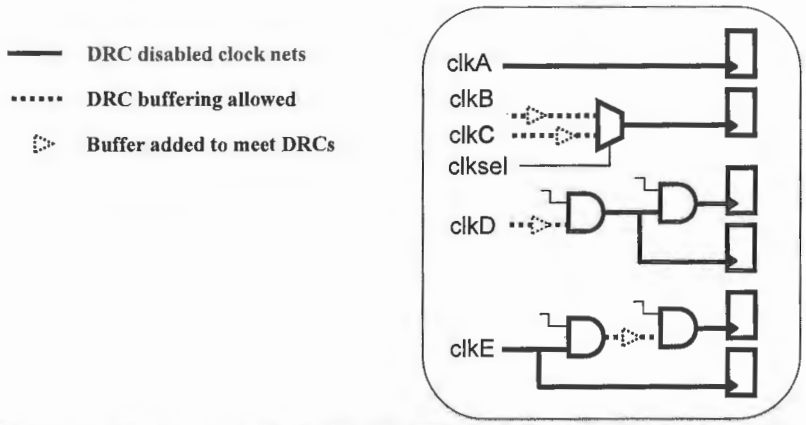
The `-min_delay` option specifies that `min_delay` has higher priority than the `max_delay`, but lower priority than the maximum design rules. Alternatively, specify a list of costs in decreasing order of priority, selected from the following: `max_delay`, `min_delay`, `max_transition`, `max_fanout`, `max_capacitance`, `cell_degradation`, and `max_design_rules`. The following example sets the priority so that `max_fanout` and `max_capacitance` have higher priority than `max_delay`, but `max_transition` and `cell_degradation` have lower priority:

```
set_cost_priority {max_fanout max_capacitance max_delay}
```


©2018 Synopsys, Inc. All Rights Reserved.

Clock Network: Default DRC-Disabling

- **By default, DRC fixing is automatically disabled on clock nets directly connected to register clock pins**
- **Clock nets that are separated from clock pins by logic are not DRC disabled - may be buffered to meet DRCs!**
  - Clock buffering prior to CTS is generally not desired



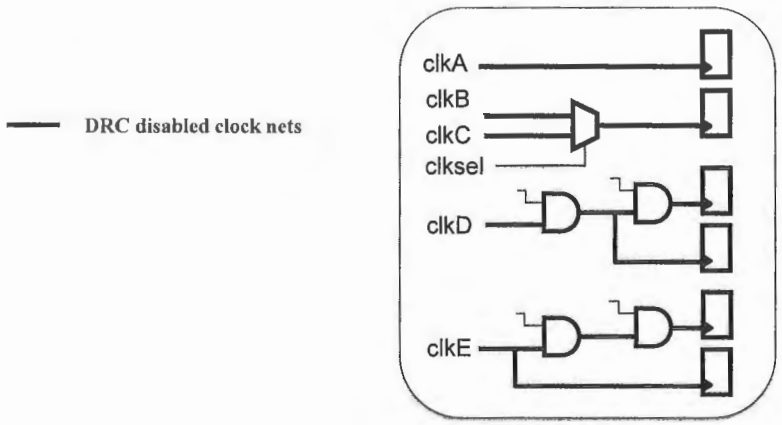
5- 35

©2018 Synopsys, Inc. All Rights Reserved.

Disabling DRCs on Entire Clock Network

Disable DRC fixing on the entire clock network, as follows:

```
set_auto_disable_drc_nets -on_clock_network true
```



5-36

There are several ways to disable DRC buffering in the clock network. The method shown above is the recommended/preferred method. Three methods are described below, with their pros/cons:

`set_auto_disable_drc_nets -on_clock_network true`:

- Disables DRC rules on the entire clock network, which prevents buffering – the desired effect
- Does not affect clock nets that are used as data – these nets are buffered for DRC as well as timing, which is desired, since clock tree synthesis does not buffer these clock-as-data nets

`set_dont_touch_network [get_ports "clkA ... clkE"]`:

- Applies the "`dont_touch`" attribute on the entire transitive fanout from the clock ports on, up to but not including the registers. This prevents any addition, removal, or sizing of logic, which prevents buffering on the entire clock network – the desired effect
- Does not disable DRC rules, which means DRC violations may be reported on the clock network – not desired
- Applies to clock nets that are used as data – prevents these nets from being buffered for DRC as well as timing – not a desired effect

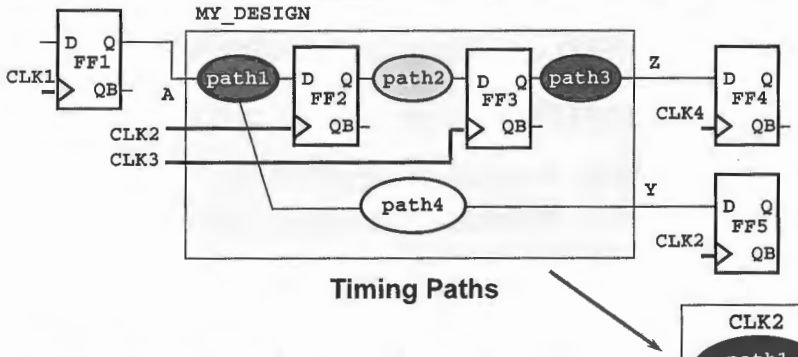
`set_ideal_network [get_ports "clkA ... clkE"]`:

- Applies the "`ideal`" and the "`dont_touch`" attributes on the entire transitive fanout from the clock ports on, up to but not including the registers (with some restrictions); Also disables DRC rules.
- This prevents buffering on the entire clock network – the desired effect
- Applies to clock nets that are used as data – prevents these nets from being buffered for DRC as well as timing – not a desired effect
- Cannot use the `set_auto_disable_drc_nets` command to override the settings specified by the `set_ideal_network` command

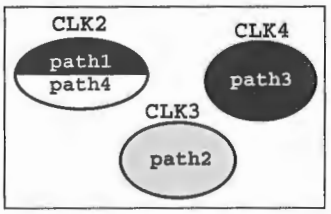
©2018 Synopsys, Inc. All Rights Reserved.

Default *Path Groups*

Path groups



Timing Paths



Path Groups

- During compile each timing path is placed into a path group associated with that path's capturing clock
- DC then optimizes each path group in turn, starting with the critical path in each group

`report_path_group` lists the path groups which are defined in the current design

5- 37

Path groups are named the same as their related (end-point) clock object names, by default.

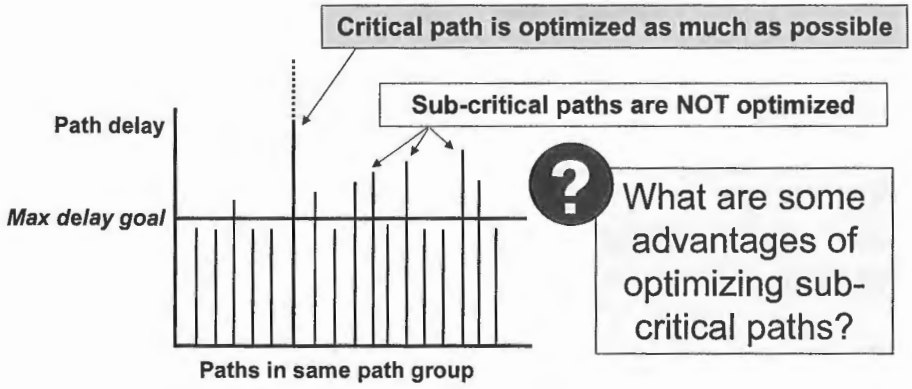
©2018 Synopsys, Inc. All Rights Reserved.

Problem: Sub-Critical Paths Ignored

Path groups

By default, optimization within a path group stops when:

- **All paths in the group meet timing, or**
- **The critical path can not be further improved → sub-critical paths are not optimized! (to save run time)**



5-38

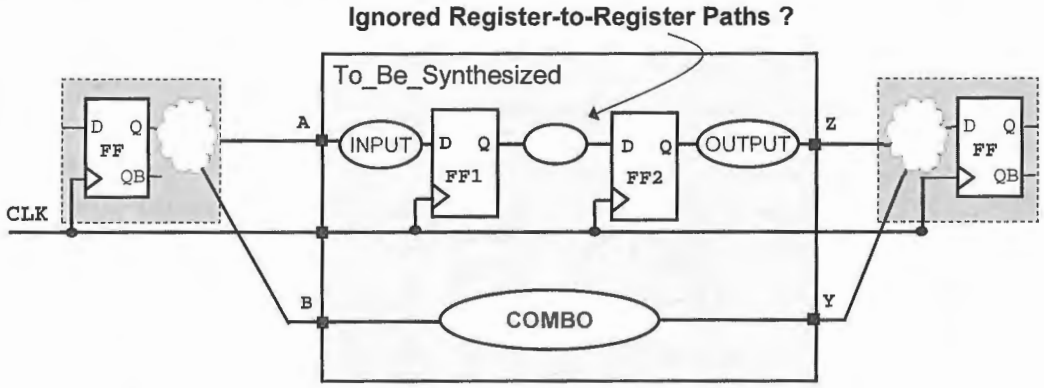
ANSWER:
If the sub-critical paths are intertwined with the critical path (they share logic), it is possible that by improving one or more sub-critical nets this actually helps the critical path as well.
Another advantage is that you will end up with fewer violations to contend with after synthesis, an advantage if you will be using a timing-driven physical design or Placement tool: It is much easier for the physical design tool to fix a small number of violations through savvy placement, compared to having to handle a large number of violations.
Another issue is discussed on the next slide …

©2018 Synopsys, Inc. All Rights Reserved.

Problem: Reg-to-Reg Paths Ignored

Path groups

Assume that, maybe due to poor partitioning, the input and output constraints are purposely pessimistic...



What happens if the "critical path" is an I/O path and DC gives up on it?

5-39

In the example above "To_Be_Synthesized" is a sub-design of a larger design. If the sub-designs are poorly partitioned (e.g. slicing through combinational logic), and the sub-designs on the input and output of "To_Be_Synthesized" have not been synthesized yet, the designer of the "To_Be_Synthesized" block may decide to conservatively or pessimistically constrain the input and output ports.

ANSWER: Since this is a single-clock design, all paths are in the same path group. If DC gives up on the critical path, no optimization is performed on the less critical paths, including the reg-to-reg paths. The reg-to-reg path are very accurately constrained (since they are only dependent on the clock waveform, which is usually well defined), yet they will be totally ignored due to the poor I/O constraints. This is not good.

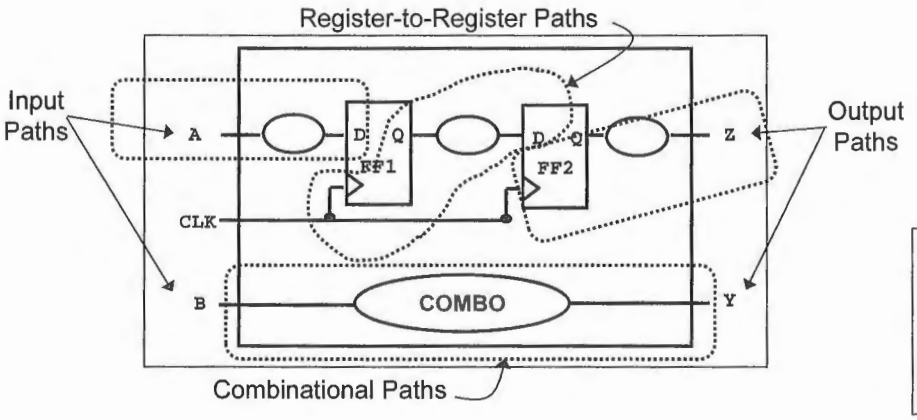
©2018 Synopsys, Inc. All Rights Reserved.

Solution: User-Defined Path Groups

Path groups

Custom path groups allow more control over optimization

- Each path group is optimized independently
- Worst violator in one path group does not prevent optimization in another group



User-defined path group creation will be discussed to explain the concepts; Automatic path group creation is available – see notes below.

5-40

Creating user-defined path groups can also facilitate a divide-and-conquer timing analysis strategy, since `report_timing` reports each path group's timing separately. This can help you isolate or analyze problems in a certain region of your design.

As an alternative to manual path group creation, you can direct the tool to automatically create path groups. You can control whether the tool creates the path groups before optimization or only for blocks that fail timing after optimization. To create path groups before optimization:

```
create_auto_path_groups -mode rtl
compile_ultra
remove_auto_path_groups
report_qor
```

This creates one path group per hierarchy within the design. If there are multiple levels of hierarchy, the tool creates path groups for each level. The advantage of creating path groups at this stage is that the tool can perform all optimizations during the initial compile. However, the number of hierarchical levels within the design can be large, leading to a large number of path groups and causing excessive runtime. To create path groups after the design has completed initial compile:

```
compile_ultra
create_auto_path_groups -mode mapped
compile_ultra -incremental
remove_auto_path_groups
report_qor
```

This creates one path group per hierarchy only for those hierarchical blocks in the design that do not meet timing. If multiple levels of hierarchy do not meet timing, the tool creates path groups for each of those levels. Use the `remove_auto_path_groups` command after the initial or incremental compile to get a QoR report with only the user-created path groups, or after the initial compile to remove the path groups created with the `create_auto_path_groups -mode rtl` command before creating new path groups with the `create_auto_path_groups -mode mapped` command.

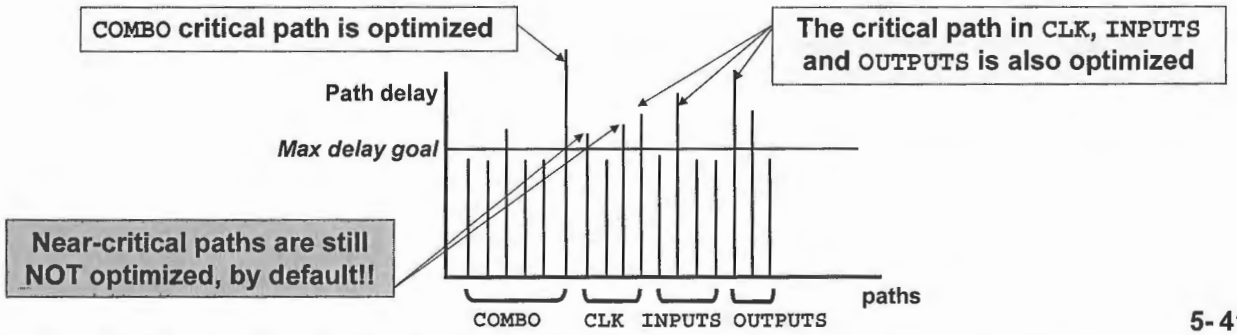
©2018 Synopsys, Inc. All Rights Reserved.

Creating User-defined Path Groups

Path groups

```
# Ensure that the reg-reg paths get optimized
group_path -name INPUTS -from [all_inputs]
group_path -name OUTPUTS -to [all_outputs]
group_path -name COMBO -from [all_inputs] -to [all_outputs]
```

Where are the reg-to-reg paths?
Are the COMBO paths in three path groups?



5-41

The reg-to-reg paths remain in the CLK path group, which is created by default by DC during synthesis. The group_path command can be used for CLK to assign a non-default *weight* or *critical range* (to be discussed) to the group, but is not needed not to define the reg-to-reg paths.

A path can only be in one path group, but according to the path group arguments, the combo paths can be part of the INPUTS, OUTPUTS or COMBO group – so where are they?

The COMBO paths wind up in the COMBO group. Assuming the commands are executed in the order listed in the slide, they are first moved from CLK to INPUTS, because they match the startpoint argument -from [all_inputs]. They will not be moved to the OUTPUTS group, even though their endpoints match -to[all_outputs], because "-from" has priority over "-to". They finally end up in the COMBO group because their startpoints **and** endpoints match the -from AND -to arguments. DC works this way to prevent having different results if the command sequence changes!

Use report_path_group to get a summary of the path groups in the design.

Question: Why is it not necessary to remove the clock input ports from the all_inputs collection?

Answer: The group_path command identifies paths, and a clock input port is not a startpoint of a path (only a register clock pin), so the INPUTS path group will not contain any paths starting at the clock ports.

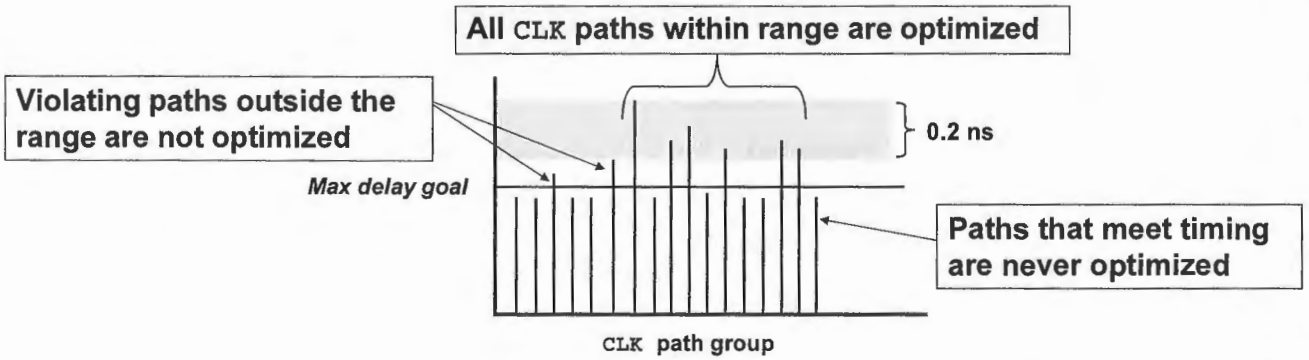
©2018 Synopsys, Inc. All Rights Reserved.

Enabling Near-Critical Path Optimization

Path groups

- The critical range is with respect to the worst or critical delay
- Recommendation: Do not exceed 10% of the clock period

```
group_path -name CLK -critical_range 0.2
```



5-42

By default the critical range of all paths is zero.

Fixing sub-critical paths that share logic with the critical path may help the critical path. Since the critical range is with respect to the critical path delay, if the critical path delay is improved, the critical range band moves lower, along with the improved critical path.

Once a sub-critical path that is being optimized falls outside the critical range, it is no longer optimized. Critical range optimization will not improve a sub-critical path if the improvements make the critical path worse. With a non-zero critical range, DC will reduce TNS (Total Negative Slack) in the design even if it cannot reduce the WNS (Worst Negative Slack).

The recommendation to not exceed 10% of the effective clock period is simply to control run time. If you exceed this limit your run time can become prohibitively long.

Question: Why is it not recommended to use a weight much larger than 5, let's say 100 (the maximum allowable number), for example?

Answer: Theoretically this would allow optimization to reduce the reg-to-reg violation from -0.1ns to -0.09 ns (10ps improvement) while increasing the INPUT path violation from -0.4ns up to -1.4ns (1000ps worse)! Probably not a worthwhile trade-off. The non-default weight numbers of 5 and 2 have been empirically found to be reasonable numbers to use, in general. Of course users may have a very specific situation where they may want to go above and beyond the "normal" recommended settings, as long as they are aware of the potential consequences (like drastically increasing the WNS of the design).

©2018 Synopsys, Inc. All Rights Reserved.

Prioritizing Path Groups: `-weight`

Path groups

- **Path groups can be given a relative optimization priority, or weight**
- **Allows path improvements in a given group that may degrade another group's worst violator, if the overall cost function (∑neg_slack X weight) is improved**
- **Recommendations:**
  - Apply a weight of 5 to the most critical paths (reg-to-reg)
  - Apply a weight of 2 to less critical paths
  - A default weight of 1 is assigned to all other paths (e.g. inaccurate I/O paths)

```
group_path -name INPUTS -from [all_inputs]
group_path -name OUTPUTS -to [all_outputs]
group_path -name COMBO -from [all_inputs] \
                       -to   [all_outputs]
group_path -name CLK -critical 0.2 -weight 5
```

5-43

It is possible, and recommended, to assign *weights* to different path groups by using the `-weight` option. This is a way to control the relative priority of violations. Priority is given to the path with the highest "Cost" = Negative slack X Weight. For example: A path group with a critical violation of –2ns and a *weight* of 5 has a *cost* of 10 and will therefore have higher priority than another path group with a critical violation of –3ns but a default *weight* of only 1 - *cost* of 3. If these two paths are related such that improving one hurts the other, DC will favor improving the higher weight path, even though its slack is less than the lower weight path.

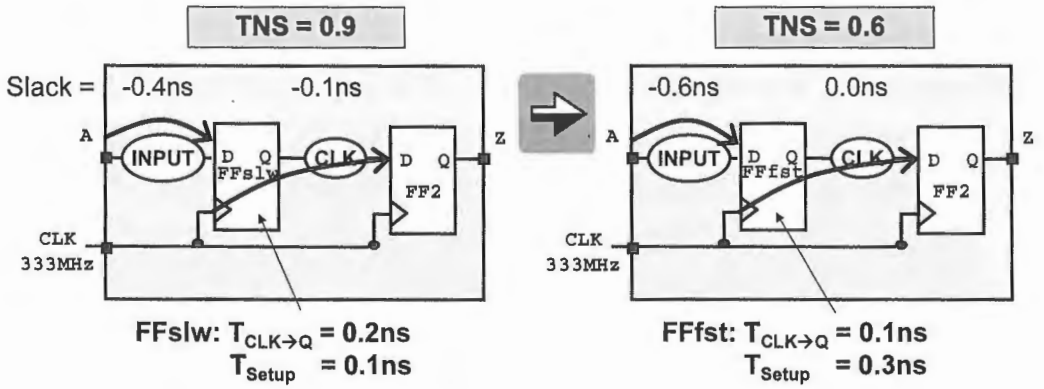
The `group_path` command also has a `-priority` option, which can be used to over-ride default command priority or precedence (as opposed to optimization priority): The specified `priority_level` assigns a priority to the command to resolve conflicting path selections between different `group_path` commands. By default, different `group_path` commands follow ordinary rules of priority to resolve conflicting path selections. For example, a command using `-from` and `-to` to select paths has priority over a conflicting command that selects a subset of those paths using `-through`, so the latter command is ignored for the paths that could be in either. By setting a higher priority in the command using the `-through` option, its path selection is honored, over-riding the usual order of priority on paths in the intersection of the two groups. If unspecified, the priority is `0`. You can set the priority for a `group_path` command to any integer `0` or greater.

©2018 Synopsys, Inc. All Rights Reserved.

Example: `-weight`

Path groups

```
group_path -name INPUTS -from [all_inputs]
group_path -name OUTPUTS -to [all_outputs]
group_path -name COMBO -from [all_inputs] -to [all_outputs]
group_path -name CLK -critical 0.2 -weight 5
```



5-44

By adding a path group weight of `5` to the `CLK` group, the reg-to-reg timing is improved, but at the expense of the input timing path (the -0.4ns violation increases to -0.6ns). This happened because swapping `FFslw` to `FFfst` reduced the overall cost function or *total negative slack* (TNS), which is calculated as the sum of the weighted negative slacks. The *TNS* for the design before the swap to `FFfst` is [(0.4 x 1) + (0.1 x 5)] = 0.9. After swapping `FFslw` to `FFfst`, the TNS is 0.6. DC was able to improve the reg-to-reg timing path by switching to a register with a faster CLK→Q delay, while giving up some setup time (which hurt the `INPUTS` path with a default weight of `1`).

Without the `-weight` option applied to the `CLK` group, both paths have a default weight of 1 and DC would not have considered this register swap, because this would have *increased* the TNS from 0.5 to 0.6.

©2018 Synopsys, Inc. All Rights Reserved.

Multi-Clock Example

Path groups

In summary: If I/O port constraints are known to be pessimistic, focus optimization on all reg-to-reg paths

```
group_path -name CLK1 -critical 0.3 -weight 5
group_path -name CLK2 -critical 0.1 -weight 5
group_path -name CLK3 -critical 0.2 -weight 2
group_path -name INPUTS  -from [all_inputs]
group_path -name OUTPUTS -to   [all_outputs]
group_path -name COMBO   -from [all_inputs] -to   [all_outputs]
report_path_group
```

Group Name	Weight	Critical Range
default	1.00	0.00
CLK1	5.00	0.30
CLK2	5.00	0.10
CLK3	2.00	0.20
COMBO	1.00	0.00
INPUTS	1.00	0.00
OUTPUTS	1.00	0.00

If I/O port constraints are accurate, what `group_path` modification is still useful?

5-45

In the above example the *CLK1* and *CLK2* groups are considered the most critical groups, and are therefore given a weight of 5. The *CLK3* group is still critical, but less so than *CLK1* and *CLK2*, and is therefore given a weight o 2. Each clock group is assigned a different critical range representing 10% of their respective clock periods (0.3, 0.1 and 0.2, resp.) The `INPUTS`, `OUTPUTS` and `COMBO` groups are not considered critical, since the input and output delay constraints are estimated on the conservative side. If the I/O paths ARE critical you may apply the appropriate weight and critical range values.

The above solution is a very CPU, and memory-intensive option, but it is worth considering. Because DC by default works only on the most critical path in each path group, it will not go very far if it gets stuck on one path that it cannot get to meet timing. By opening up the "window" to allow more paths to be optimized, DC can do a better job on the rest of the design, even at the cost of that one path. You can limit the number of non-critical paths that are optimized, and thereby keep the run time reasonable, by setting their critical range to zero.

ANSWER: If the I/O port constraints are accurate (not pessimistic), I/O path violations can not be considered less important than the reg-to-reg violations, so applying higher weights to reg-to-reg paths is not applicable. You may, however, still want to apply a critical range to all paths, to allow sub-critical paths to be optimized. Since the I/O paths will have the same critical range (and weight) as the reg-to-reg paths, it is not necessary to create separate I/O path groups. You would apply the following commands:

```
group_path -name CLK1 -critical 0.3
group_path -name CLK2 -critical 0.1
group_path -name CLK3 -critical 0.2
```

Constrained paths that are not part of clock, user-defined, `**async**` or `**clock_gating**` path groups, for example `set_max/min_delay paths`, go into the `**default**` path group.

©2018 Synopsys, Inc. All Rights Reserved.

TNS-Driven Placement

- **By default, timing-driven placement focuses on reducing *worst negative slack* (WNS)**
- **Enable the following variable before initial compile to focus placement on *total negative slack* (TNS) reduction instead:**

```
set_app_var placer_tns_driven true
compile_ultra
```

5-46

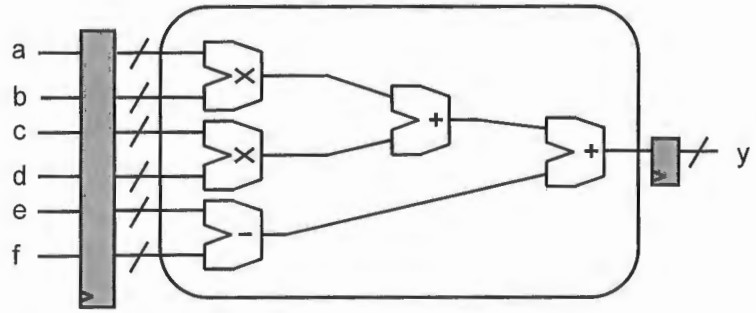
During incremental compile (`compile_ultra -incremental`) timing-driven placement, by default, optimizes the TNS in the design instead of the WNS. To optimize WNS during incremental compile, set the `placer_tns_driven_in_incremental_compile` variable to `false` before running the `compile_ultra -incremental` command.

©2018 Synopsys, Inc. All Rights Reserved.

Reducing Large Datapath Delay

Register retiming

```
y <= a*b + c*d + e-f;
```



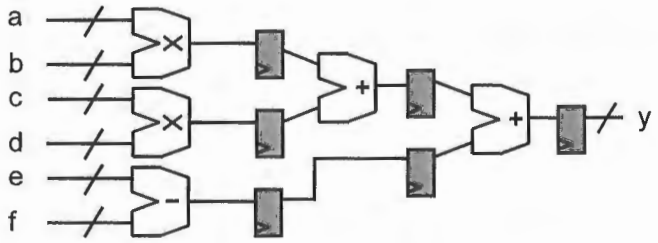
How can we speed up the clock frequency that this datapath can run at?

5-47

©2018 Synopsys, Inc. All Rights Reserved.

Some Things to Try ...

- Add pipeline stages
- Use the fastest available arithmetic *implementations* (*DesignWare*)




```
if (clk'event and clk='1') then
begin
    prodAB <= a * b;
    prodCD <= c * d;
    diffEF <= e - f;

    p2_1   <= prodAB + prodCD;
    p2_2   <= diffEF;

    y      <= p2_1 + p2_2 ;
```

```
always @ (posedge clk)
begin
    prodAB <= a * b;
    prodCD <= c * d;
    diffEF <= e - f;

    p2_1   <= prodAB + prodCD;
    p2_2   <= diffEF;

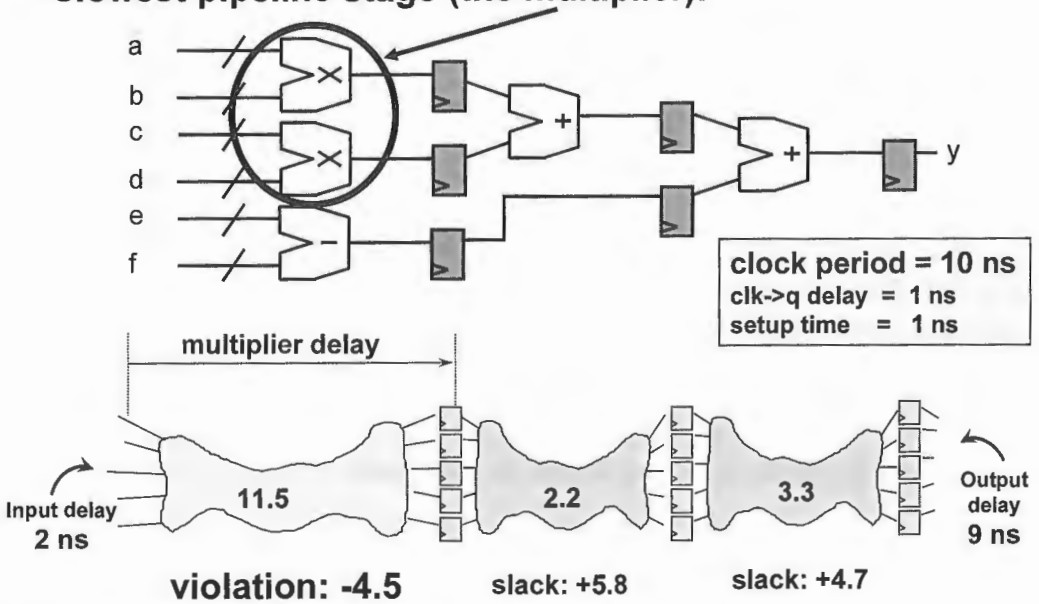
    y      <= p2_1 + p2_2 ;
```

5-48

©2018 Synopsys, Inc. All Rights Reserved.

What if the Pipeline is Still Violating Timing?

The clock frequency is limited by the total delay in the slowest pipeline stage (the multiplier):



5- 49

The above circuit will only work with clock period >= 14.5ns instead of the required 10ns.

©2018 Synopsys, Inc. All Rights Reserved.

The Solution: *Pipeline* or *Register Retiming*[1]

Register retiming

```
set_optimize_registers true -design Pipeline
compile_ultra ...
```

- ***Register retiming*** **is an alternative to "adaptive retiming" (`compile_ultra -retime`)**
- **Both techniques move, merge and split registers**
- ***Register retiming*** **is targeted at improving timing and/or area of pure pipelined designs**
- ***Adaptive retiming*** **is targeted at reducing the WNS between non-pipelined registers**
- **Both techniques can be applied in parallel**

5- 50

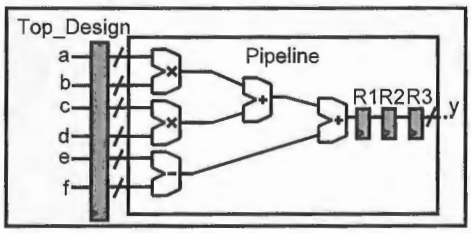
1) Register Retiming is also referred to as:
Register Repositioning, or
Pipeline Retiming, or
Behavioral Retiming (BRT)

©2018 Synopsys, Inc. All Rights Reserved.

RTL Recommendations for Pipelines

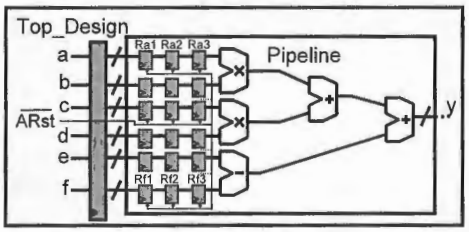
Register retiming

- **Place each pipeline in its own sub-block**
  - Allows targeted application of *register retiming*
- **Concentrate registers at the inputs or outputs**
  - Allows all datapath optimization algorithms to be applied



Group simple D-FF or D-Latch registers at the outputs

Easier to code in RTL



Group registers with control pins (e.g. set, reset) at the inputs

Same QoR, but reduces run time if the pipeline is large

If your RTL can NOT be defined as recommended use "adaptive retiming" instead of "register retiming"

5-51

Recommended RTL coding style for pipelined designs with D-type Flip-Flops:

```
data <= a*b + c*d + e-f;

if (clk'event and
clk='1') then
begin
    R1 <= data;
    R2 <= R1;
    R3 <= R2;
```

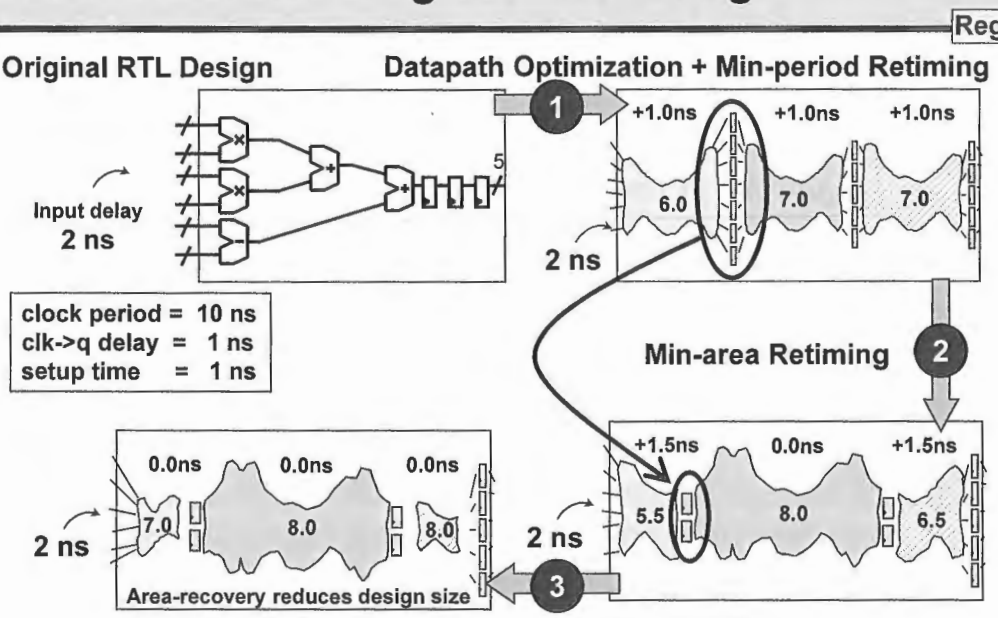
```
data <= a*b + c*d + e-f;

always @ (posedge clk)
begin
    R1 <= data;
    R2 <= R1;
    R3 <= R2;
```

Alternatively, you can instantiate the *DesignWare* part **DW_pl_reg** to define a pipeline register of any bit width and any number of stages. You can furthermore define if the pipeline will have input or output registers, and synchronous or asynchronous resets.

©2018 Synopsys, Inc. All Rights Reserved.

Optimization Phases of *Register Retiming*



When `compile_ultra` is invoked, the datapath logic is first optimized using the full suite of datapath optimization techniques in Ultra. What started out as discreet arithmetic components is now one big cloud of combinational logic. Now register retiming kicks in, which initially balances the slacks between register stages (min-period retiming), then takes advantage of any positive slack to reduce the number of registers per stage (min-area retiming). Now that a new grouping of combinational logic is formed, some more logic optimization is performed. This may introduce some timing violations – for example, by fixing high-fanout DRC violations. Adaptive retiming is then invoked to further reduce or eliminate the timing violations.

Use `set_svf <My_name.svf>` to record register position changes. This is helpful to ensure successful formal verification with Synopys' *Formality* tool. Invoke this command before reading in RTL (can put it in your *.synopsys_dc.setup* file).

©2018 Synopsys, Inc. All Rights Reserved.

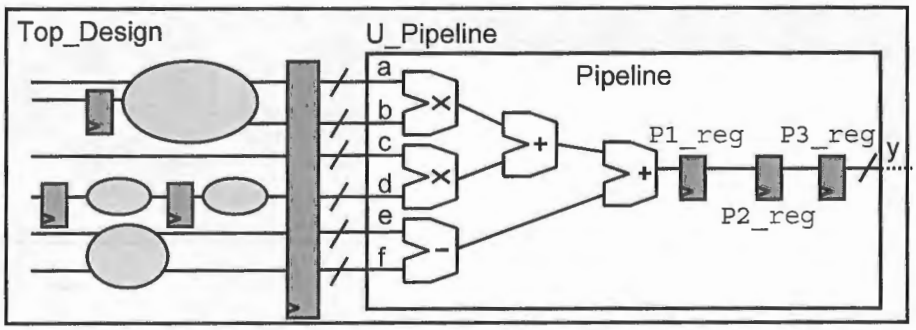
Adaptive + Register Retiming Flow Example

Register retiming

```
current_design Top_Design
set_optimize_registers true -design Pipeline
compile_ultra -retime ...
```

Turns off auto-ungrouping

Adaptive retiming (`-retime`) is applied to non-pipelined logic while register retiming is applied to the pipeline during the same optimization run



5-53

Adaptive retiming due to the `-retime` option has no effect on designs with `set_optimize_registers` set to `true`.

However, register retiming includes an adaptive retiming "post-processing" step, which is automatically applied to the pipeline registers if needed. It is not controlled by the `-retime` option – it happens with or without that option.

Auto-ungrouping during `compile_ultra` is automatically disabled for any design with `set_optimize_registers true` applied. It is therefore not necessary for the user to explicitly apply a `set_ungroup false` to prevent auto-ungrouping from removing the hierarchy before register retiming is able to optimize the pipelined design.

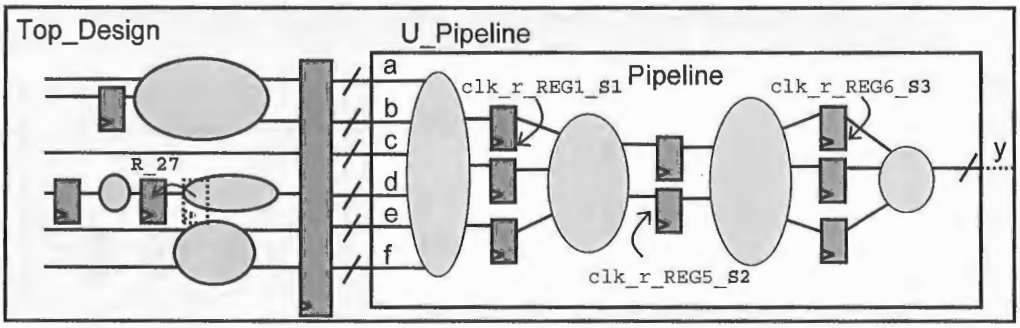
©2018 Synopsys, Inc. All Rights Reserved.

Resulting Pipeline

- **All pipeline registers, including the output registers, can be repositioned**
- **Repositioned pipeline register cells are renamed:**

  `XYZ_reg[#]` → `clockname_r_REG#_S#`

  where the ending `S#` represents the register stage number

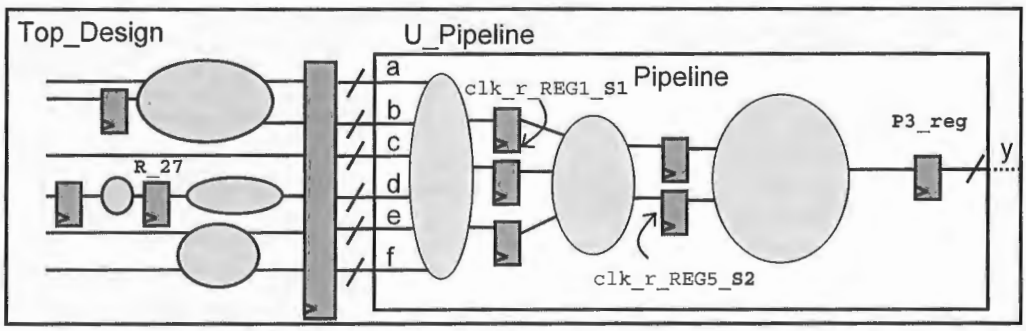


5- 54

©2018 Synopsys, Inc. All Rights Reserved.

Maintaining Registered Outputs


```
current_design Top_Design
set_dont_retime [get_cells U_Pipeline/P3_reg*] true
set_optimize_registers true -design Pipeline
compile_ultra -retime ...
```



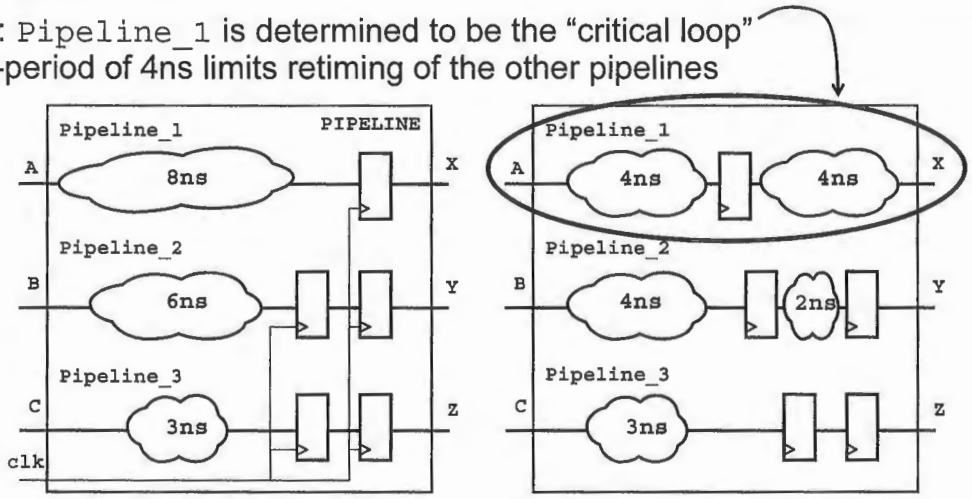
5-55

©2018 Synopsys, Inc. All Rights Reserved.

Critical Loop Limits Min Period Retiming

- During min-period retiming, the *critical loop* limits the minimum clock period that can be achieved by retiming, by default
  - Critical loop: The sequence of pipeline cells containing the critical path that cannot be further improved by moving registers

Example: `Pipeline_1` is determined to be the "critical loop" – its min-period of 4ns limits retiming of the other pipelines



Before min-period retiming **Clock Period: 1ns** After min-period retiming

In the example above, the design called `PIPELINE` contains three independent pipelines, labeled `Pipeline_1` through `Pipeline_3`. Register retiming is enabled on the entire `PIPELINE` block (`set_optimize_registers true -design PIPELINE`).

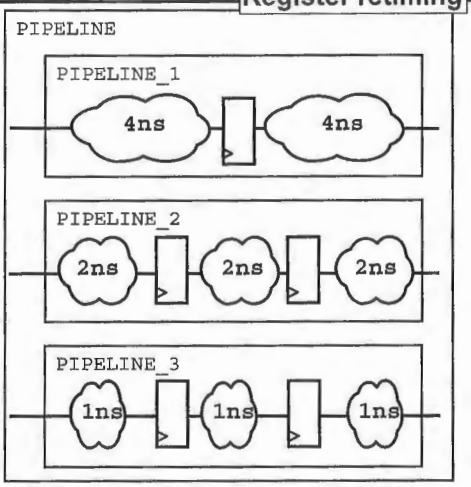
©2018 Synopsys, Inc. All Rights Reserved.

Solution #1: Separate Pipeline Hierarchies

Register retiming

Recommended solution:

- Place each independent pipeline in its own hierarchical sub-design
- Enable register retiming on each pipeline sub-design




```
create_clock -period 1.0 clk
set_optimize_registers true \
     -design "PIPELINE_1 PIPELINE_2 PIPELINE_3"
compile_ultra ...
```

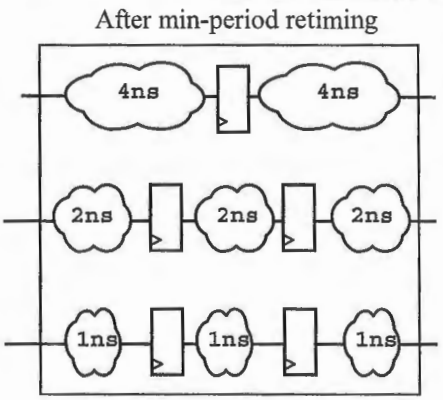
5-57

©2018 Synopsys, Inc. All Rights Reserved.

Solution #2: Threshold Optimization

Register retiming

- *Threshold optimization* allows retiming to work on subcritical paths
  - The *delay threshold* value becomes the "effective clock period" target
  - Use a threshold value of 0.97-1.00x clock period
  - Path group's *critical range* allows further logic optimization of sub-critical paths during logic optimization




```
create_clock -period 1.0 clk
set_optimize_registers true -design Pipeline \
            -clock clk -delay_threshold 0.97
group_path -name clk -critical_range 0.10
compile_ultra ...
```

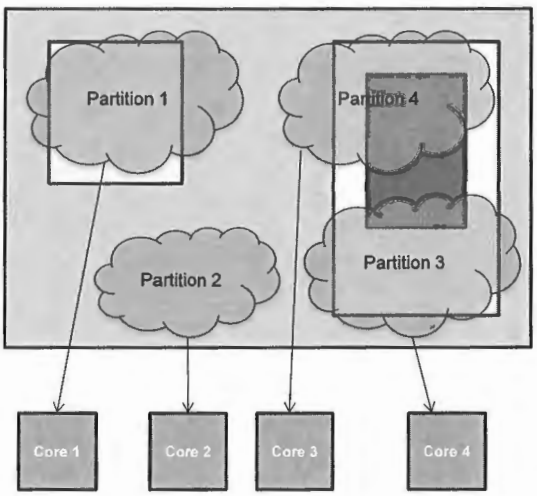
5-58

If the different paths within the same pipeline have widely varying total combinational logic delays, then the "critical loop" is determined by the longest path, which can also limit retiming of the sub-critical paths within the pipeline. In this case the only solution is to enable threshold optimization (since it is not really practical to put each path of a pipeline in its own hierarchy!).

©2018 Synopsys, Inc. All Rights Reserved.

Multi-Core Optimization

Multi-core



- **Threading and parallel processing is supported during `compile_ultra`**
- **Can specify 1 (the default) to 16 cores**
  - Most effective speedup seen with four cores
  - Incremental speedup with more cores
  - Measure speedup using wall-clock time, not CPU time
- **Recommended memory and swap space:**
  - 4 cores requires 2X memory requirement of 1 core
  - 8 cores requires 3X memory requirement of 1 core
  - Swap space = 2X memory

```
set_host_options -max_cores 4
compile_ultra ...
```

5-59

If hyper-threading is enabled and if, for example, you have 4 cores and 8 threads available, you can specify a `set_host_options -max cores` number of up to `8`.

©2018 Synopsys, Inc. All Rights Reserved.

Multi-Core License Requirements

Multi-core

- **Multi-core will require 1 license per 8 cores for:**
  - DC-Expert
  - DC-Ultra
  - DesignWare
  - DC-Extension (if applicable)
  - Power-Compiler (if applicable)
  - Test-Compiler (if applicable)
- **Report the current options with:**

```
report_host_options
```

- **Multi-core optimization, if enabled, can be turned off with:**

```
remove_host_options
# OR
set_host_options -max_cores 1
```

5-60

As of *Design Compiler* version 2016.03-SP3, one license can be used for up to 8 cores (increased from 4 in prior versions).

©2018 Synopsys, Inc. All Rights Reserved.

Disabling Runtime-intensive Settings

Runtime

- **The `compile_prefer_runtime` command disables runtime-intensive optimization settings, such as *multiple clocks per registers* (see notes)**
  - Also uses lower effort for reports (e.g. `report_congestion`)
  - Changed settings are captured in the *logfile*
- **Affects the following commands:**
  - `compile_ultra`
  - `compile_ultra -incr`
  - `optimize_netlist -area`
- **May impact QoR**
  - Intended to be used with 1st pass `compile_ultra`

```
compile_prefer_runtime
compile_ultra
```

5-61

Settings affected by **`compile_prefer_runtime`** :

Command, variable, or setting	Default	With **`compile_prefer_runtime`**
`timing_enable_multiple_clocks_per_reg`	`true`	`false if > 20 clocks per pin`
`set_ahfs_options -global_route`	`false`	`false`
`placer_enable_enhanced_router`	`false`	`false`
`placer_congestion_effort`	`auto`	`auto`
`placer_reduce_high_density_regions`	`false`	`false`
`placer_channel_detect_mode`	`false`	`false`
`spg_congestion_placement_in_incremental_compile`	`false`	`false`
`placer_max_cell_density_threshold`	`-1`	`0.7 (only if utilization < 50 %)`
`spg_enhanced_timing_model`	`false`	`false`
`spg_enable_via_resistance_support`	`false`	`false`
`compile_final_drc_fix`	`none`	`none`
`compile_enable_register_merging`	`true`	`false`

©2018 Synopsys, Inc. All Rights Reserved.

Test for Understanding 1/2



7. **Circle the correct statement(s) regarding *Register Retiming*:**
   a) Splits/merges registers – does not optimize combinational logic
   b) May result in smaller (area) pipelined designs
   c) May increase the number of register stages in the pipeline
   d) May increase delay violations to reduce the register count
8. **What determines the *path group* to which an output path belongs, by default, without `group_path`?**
9. **By default, optimization within a path group stops when DC is not able to improve the delay of the critical and near-critical paths – True or False?**
10. **Why is it recommended to optimize *near-critical* paths?**

5- 62

Answers:

7. B
8. The clock defined in `set_output_delay -clock XYZ ...`
9. False. It stops once it gets stuck on the critical path. The near-critical paths are not optimized at all.
10. By optimizing near critical paths: DC may be able to improve the critical path if the paths are "related"; Also, the design ends up with fewer violating paths, which are easier for down-stream layout or physical design tools to fix; Finally, well-constrained reg-to-reg paths may be ignored due to over-constrained I/O paths – not good.

©2018 Synopsys, Inc. All Rights Reserved.

Test for Understanding 2/2



11. **Near critical paths that would be ignored by default can be optimized by:**
    a) Placing them in their own *path group*
    b) Applying a *critical range* to `create_clock`
    c) Applying a *weight* to their *path group*
    d) All of the above

12. **By applying a `-weight` option to a *path group* it is possible to worsen the worst violator in another path group. True or False?**

13. **Increasing `set_host_options -max_cores` from `4` to `16` will result in the same speedup factor as increasing it from `1` to `4`. True or False?**

5- 63

Answers:

11. A. The *critical range* is applied to *path groups*, not the clock constraint; Applying a *weight* does not direct DC to optimize otherwise-ignored paths - it only applies more "cost" weight to paths that are already being considered for optimization
12. True, as long as the total weighted negative slack is maintained or reduced.
13. False. Increasing by 4x from 1 to 4 will typically result in a larger speedup factor, compared to a 4x increase from 4 to 16 cores.

©2018 Synopsys, Inc. All Rights Reserved.

Unit Agenda

- Some examples of default compile_ultra optimizations
- User controllable compile_ultra optimizations
- Post-compile analysis and incremental compile
- Final Area Recovery
- Complete flow overview

5- 64

©2018 Synopsys, Inc. All Rights Reserved.

Generate a Constraint Report After Compile

```
compile_ultra ...
report_constraint -all_violators
```

```
max_delay/setup ('Clk' group)
                          Required        Actual
  Endpoint                Path Delay      Path Delay    Slack
  ---------------------------------------------------------------
I_COUNT/PCint_reg[4]/D       1.71            1.75 r      -0.03  (VIOLATED)
I_COUNT/PCint_reg[6]/D       1.71            1.75 r      -0.03  (VIOLATED)
I_COUNT/PCint_reg[2]/D       1.71            1.75 r      -0.03  (VIOLATED)
I_COUNT/PCint_reg[3]/D       1.71            1.74 r      -0.03  (VIOLATED)
I_COUNT/PCint_reg[0]/D       1.71            1.73 f      -0.02  (VIOLATED)
I_COUNT/PCint_reg[7]/D       1.71            1.73 r      -0.02  (VIOLATED)
I_COUNT/PCint_reg[1]/D       1.71            1.72 r      -0.01  (VIOLATED)

max_capacitance
                         Required      Actual
Net                      Capacitance   Capacitance      Slack
-----------------------------------------------------------------
CurrentState[0]             0.20          0.24          -0.04  (VIOLATED)
```

Summarizes of all violating constraints. If no violations are reported, no further analysis or optimization is needed. Use `report_timing` for detailed timing path information.

5-65

©2018 Synopsys, Inc. All Rights Reserved.

Generate Timing Reports for More Detail

`report_timing`

```
Point                                    Incr       Path
-------------------------------------------------------------
clock Clk (rise edge)                    0.00       0.00
clock network delay (ideal)              0.00       0.00
input external delay                     1.20       1.20 r
Neg_Flag (in)                            0.06       1.26 r
U102/ZN (nd02d0)                         0.13 *     1.38 f
U97/ZN (nd02d2)                          0.09 *     1.48 r
U98/ZN (invbd4)                          0.06 *     1.54 f
U159/ZN (nd02d0)                         0.07 *     1.61 r
U50/Z (an03d1)                           0.14 *     1.75 r
I_COUNT/PCint_reg[4]/D (dfnrn4)          0.00 *     1.75 r
data arrival time                                   1.75

clock Clk (rise edge)                    2.00       2.00
clock network delay (ideal)              0.00       2.00
clock uncertainty                       -0.20       1.80
I_COUNT/PCint_reg[4]/CP (dfnrn4)         0.00       1.80 r
library setup time                      -0.09       1.71
data required time                                  1.71
-------------------------------------------------------------
data required time                                  1.71
data arrival time                                  -1.75
-------------------------------------------------------------
slack (VIOLATED)                                   -0.03
```

Timing reports will be discussed in the next unit

5- 66

©2018 Synopsys, Inc. All Rights Reserved.

Parallel Command Execution

- **Speeds up design `check_` and `report_` commands**
  - Each command is assigned to one core
  - Up to a maximum of four cores will be used
  - Execute `update_timing` before parallel reporting
- **List up to 8 commands to be run in parallel**

```
set_host_options -max_cores 8
...
update_timing
parallel_execute [list \
 "report_constraint -all > rc.rpt" \
 "report_timing          > rt.rpt" \
 "report_area            > ra.rpt" \
 "report_qor             > rq.rpt" \
  ...                              ]
```

5-67

Supported commands to be executed by `parallel_execute` can be listed using:
`parallel_execute -list_all`

©2018 Synopsys, Inc. All Rights Reserved.

Subsequent Incremental Compile

```
compile_ultra |-no_autoungroup| |-no_boundary| \
              |-scan| |-retime| |-spg|
report_timing
# Apply focus to critical path(s):
group_path -critical -weight -from .. -to ..
insert_dft; # Create scan chains
# Execute an incremental compile:
compile_ultra |-scan| |-retime| |-spg| -incremental
```

After the first compile and scan chain stitching (if applicable), follow with an *incremental* compile to improve results, if needed

- If timing analysis identifies critical timing paths that were not previously part of a focused path group, use `group_path` to focus optimization on the critical paths, if applicable

5-68

©2018 Synopsys, Inc. All Rights Reserved.

What if Still Violating Constraints?

If timing or DRC constraint violations are still unacceptably large after an incremental compile:

- **Modify the timing constraints → Re-synthesize the original RTL**

  or

- **Modify the RTL code and start over**

5-69

©2018 Synopsys, Inc. All Rights Reserved.

Final Area Recovery

After the last *compile*, perform *final area recovery*

- Improves area without degrading timing or leakage power

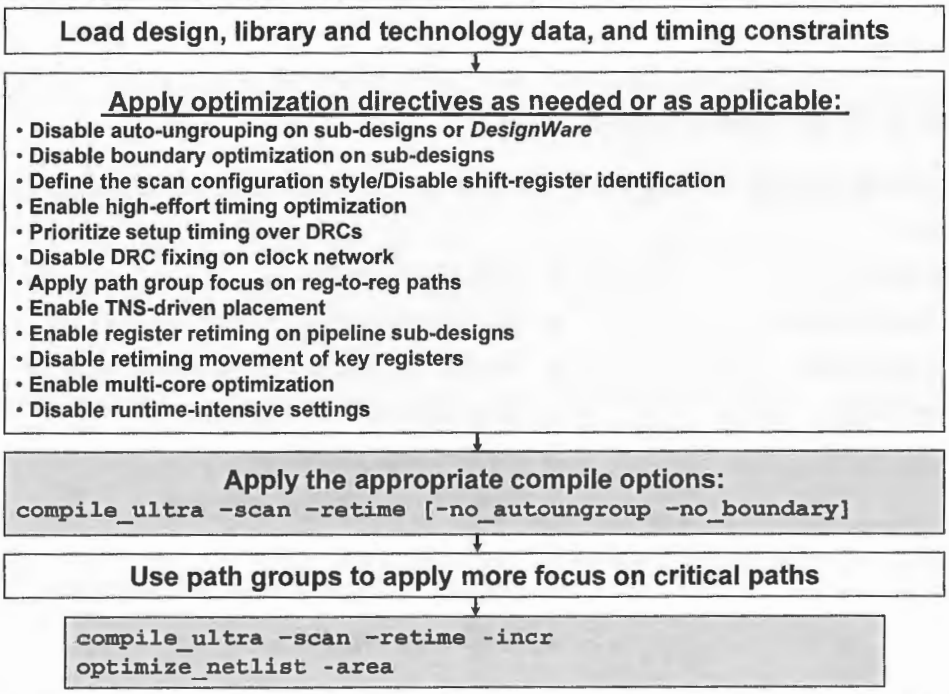
```
compile_ultra |-no_autoungroup| |-no_boundary| \
              |-scan| |-retime| |-spg|
report_timing
# Apply focus to critical path(s):
group_path -critical -weight -from .. -to ..
insert_dft; # Create scan chains
# Execute an incremental compile:
compile_ultra |-scan| |-retime| |-spg| -incremental
# Final Area Recovery:
optimize_netlist -area
```

5-70

You can ignore the weights set on path groups during area optimization by setting the `optimize_area_ignore_path_group_weights` variable to `true`. The default is `false`. When you enable the variable, the weights of all path groups are set to 1 when the `optimize_netlist -area` command performs area optimization. After area optimization, the weight on each path group is restored to the original user-specified value.

©2018 Synopsys, Inc. All Rights Reserved.

Synthesis Optimization Overview



5-71

©2018 Synopsys, Inc. All Rights Reserved.

Summary: Unit Objectives

You should now be able to use DC Ultra techniques to:

- **Synthesize RTL to gates (top-down)**
- **Achieve timing goals with good post-layout correlation using applicable techniques:**
  - Auto ungrouping
  - Boundary optimization
  - Test-ready synthesis
  - Adaptive retiming
  - High effort timing
  - Prioritizing setup timing over DRCs
  - Disabling DRC fixing on clock network
  - User-defined path groups
  - TNS-driven placement
  - Pipeline or register retiming
  - Multi-core optimization
  - Disabling runtime intensive settings
  - Final area recovery

5-72

©2018 Synopsys, Inc. All Rights Reserved.

Lab 5: DC Ultra Synthesis Techniques



75 minutes

Lab 5: DC Ultra Synthesis Techniques

Create and verify a complete *compile* run script.

Synthesize a design using the appropriate Ultra synthesis techniques.

Analyze the results after each key step.

Verify the functional equivalence of synthesized design to RTL.

5-73

©2018 Synopsys, Inc. All Rights Reserved.

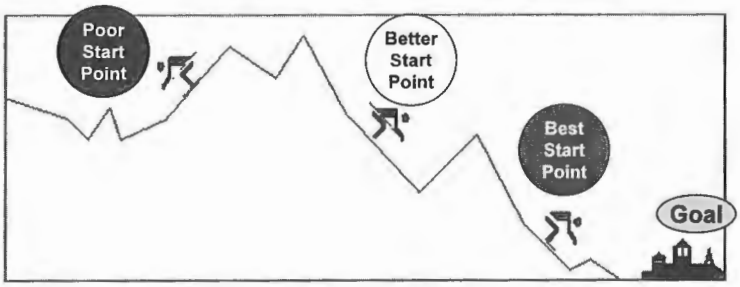
Appendix

RTL Coding Style Examples

5- 74

©2018 Synopsys, Inc. All Rights Reserved.

The Importance of Quality Source Code



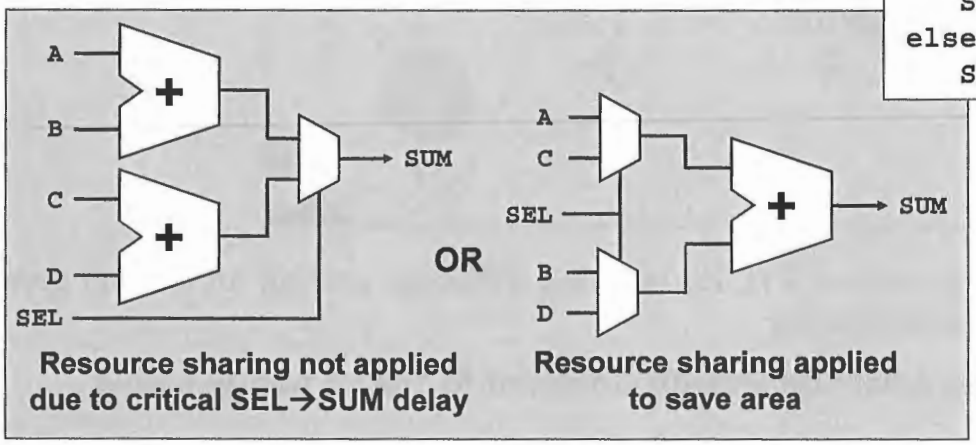
- Functionally equivalent RTL code using different *coding styles* will give different synthesis results
- You cannot rely solely on Design Compiler to "fix" a poorly coded design!
- Understanding how DC interprets RTL coding style will enable you to get the best synthesis results → See following examples …

5-75

©2018 Synopsys, Inc. All Rights Reserved.

Example: Coding to Allow *Resource Sharing*

```
if (SEL)
   SUM = A + B;
else
   SUM = C + D;
```



Resource sharing not applied due to critical SEL→SUM delay

Resource sharing applied to save area

DC coding style: Arithmetic "resources" within an *if* or *case* statement will be considered for resource sharing. This allows DC to select the smallest architecture that meets timing

5-76

Resource sharing decisions are made during the architectural level optimization phase of compile. The decision to share or not to share is based on two factors: 1) Coding style – only resources within the same *if* or *case* statement will be considered for sharing; 2) Constraints – sharing saves area but occurs only if it does not introduce or increase any path delay violations.

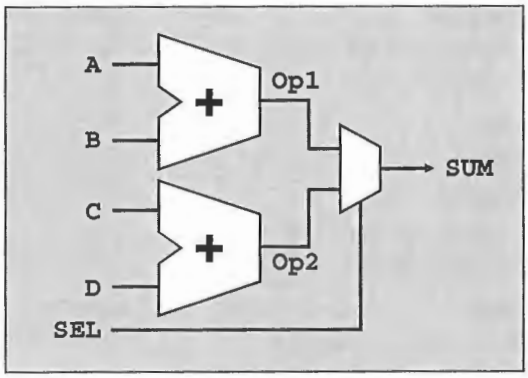
Actual circuit implementation depends on cells available in the target library and the constraints.

©2018 Synopsys, Inc. All Rights Reserved.

Example: Preventing Resource Sharing

```
Op1 = A + B;
Op2 = C + D;

if (SEL)
  SUM = Op1;
else
  SUM = Op2;
```



Since the arithmetic "resources" are outside the *if* statement no resource sharing occurs – the design may be much larger than necessary

5-77

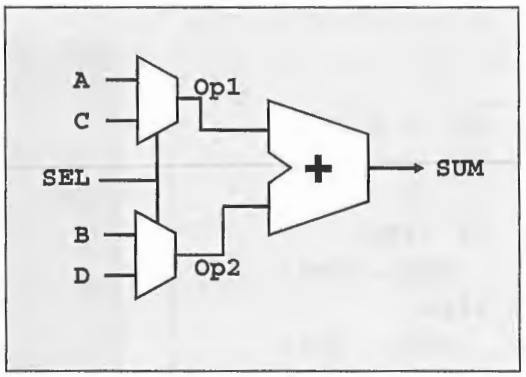
If you are not aware of the coding style which allows resource sharing to be considered by DC, you may code your design as above, which does NOT result in an area-efficient design.

A benefit of compile_ultra is that without having to recode, the adder can be shared, but, DC expert requires recoding

©2018 Synopsys, Inc. All Rights Reserved.

Example: 'Forced' Resource Sharing

```
if (SEL)
  begin
    Op1 = A;
    Op2 = B;
  end
else
  begin
    Op1 = C;
    Op2 = D;
  end
SUM = Op1 + Op2;
```



Since the arithmetic "resource" is being shared outside the *if* statement DC can not "un-share" – Poor architecture if SEL→SUM is timing critical

Recommendation: Write code that lets DC decide if *resource-sharing* is warranted or not!

5-78

By unknowingly coding as above you are in essence "forcing" a resource-shared architecture, which can not be un-done. If SEL→SUM is a timing-critical path, this design is NOT the best choice!

©2018 Synopsys, Inc. All Rights Reserved.

For More Information - Documentation



Online Documentation available from SolvNet

- HDL Compiler for VHDL User Guide
- HDL Compiler for Presto Verilog User Guide
- HDL Compiler for SystemVerilog user Guide
- SolvNet Doc Id: 015771 “Coding Guidelines for Datapath Synthesis”

5-79

©2018 Synopsys, Inc. All Rights Reserved.

This page was intentionally left blank

©2018 Synopsys, Inc. All Rights Reserved.

Agenda

DAY 3

6 Timing Analysis

7 Constraints: Multiple Clocks and Exceptions 

8 DC Graphical: SPG Flow, Congestion, Layout GUI

9 Constraints: Complex Design Considerations 

10 Post-Synthesis Output Data

11 Conclusion

© 2018 Synopsys, Inc. All Rights Reserved

©2018 Synopsys, Inc. All Rights Reserved.

Unit Objectives



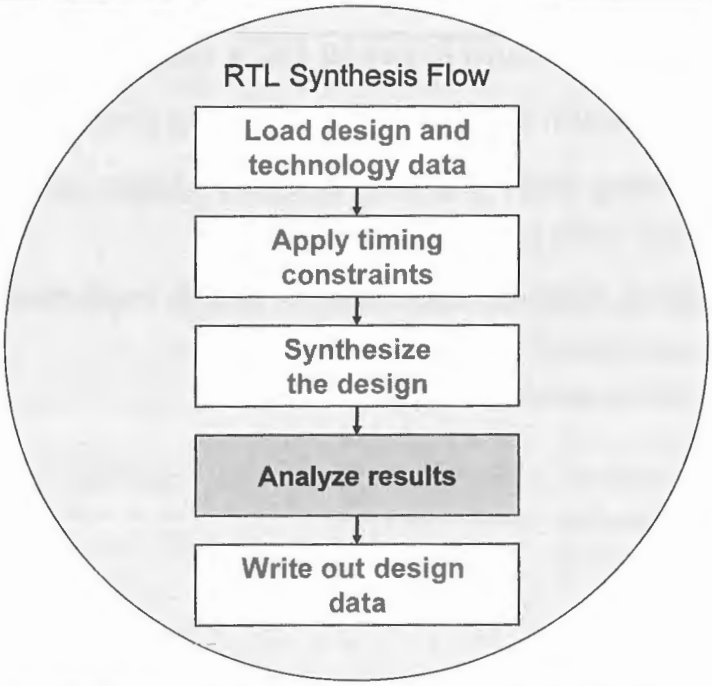
After completing this unit you should be able to:

Generate timing reports, with additional command options as needed, to diagnose timing constraint violations

6-2

©2018 Synopsys, Inc. All Rights Reserved.

RTL Synthesis Flow



6-3

©2018 Synopsys, Inc. All Rights Reserved.

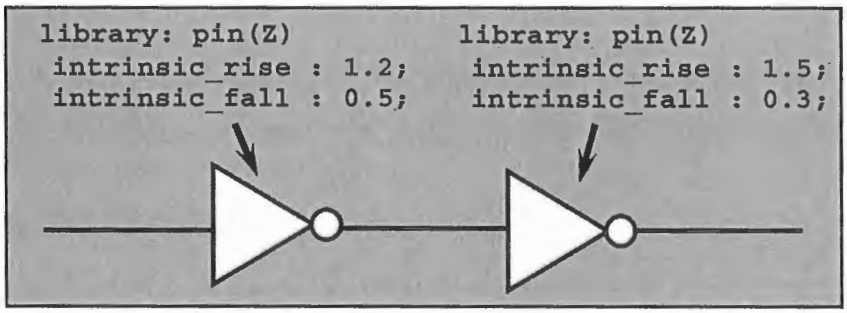
Timing Reports

The `report_timing` command invokes DC's *Static Timing Analyzer*, which:

- **Breaks the design down into individual timing paths**
- **Analyzes each timing path at least twice for single-cycle max-delay timing**
  - Rising endpoint and falling endpoint[1]
- **Generates a default, four-section report which includes:**
  - One path, the worst violator[2], per path group
  - Maximum delay or *setup* timing only
    - No *hold* timing (`report_timing -delay min`)
    - No DRC (`report_constraint -all -max_tran -max_cap`)
    - No area (`report_area -physical`)

6-4

[1] What is the longest path for this circuit?



Answer: 0.5 + 1.5 = 2.0 ns.

There is an "edge sensitivity" (called unateness) in a cell's timing arc: Design Compiler keeps track of unateness in each timing path. At a minimum, timing analysis is performed twice for each path: Once for rising, and once for falling input data. If the path contains non-unate gates, e.g. XOR, XNOR, or the Select-to-output arc of a MUX, then the path is actually analyzed four times: The "other" input of the non-unate gate, the input that is NOT part of the timing path being analyzed, is held to logic 0 while the path is analyzed for rise- and fall-delay, and then repeated again while that "other" input is now set to logic 1.

[2] The worst violator is defined as the path with the largest or *worst negative slack* (WNS). If a clock group has no violating paths then the path with the smallest positive slack is reported.

©2018 Synopsys, Inc. All Rights Reserved.

Timing Report: Path Information Section

```
****************************************
Report : timing
         -path full
         -delay max
         -max_paths 1
Design : MY_DESIGN
Version: M-2017.09-SP4
Date   : Mon Mar 26 09:46:48 2018
****************************************

 * Some/all delay information is back-annotated.

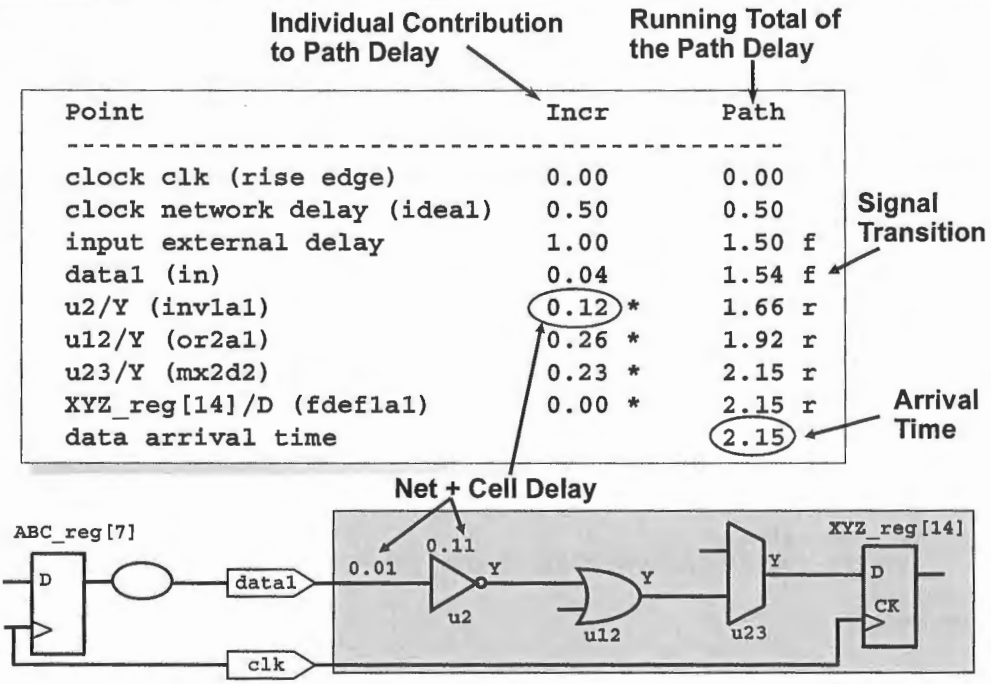
Operating Conditions: slow_125_1.62 Library: ssc_core_slow
Wire Load Model Mode: Inactive.

  Startpoint: data1 (input port clocked by clk)
  Endpoint: XYZ_reg[14]
            (rising edge-triggered flip-flop clocked by clk)
  Path Group: clk
  Path Type: max
```

6-5

©2018 Synopsys, Inc. All Rights Reserved.

Timing Report: Path Delay Section



6-6

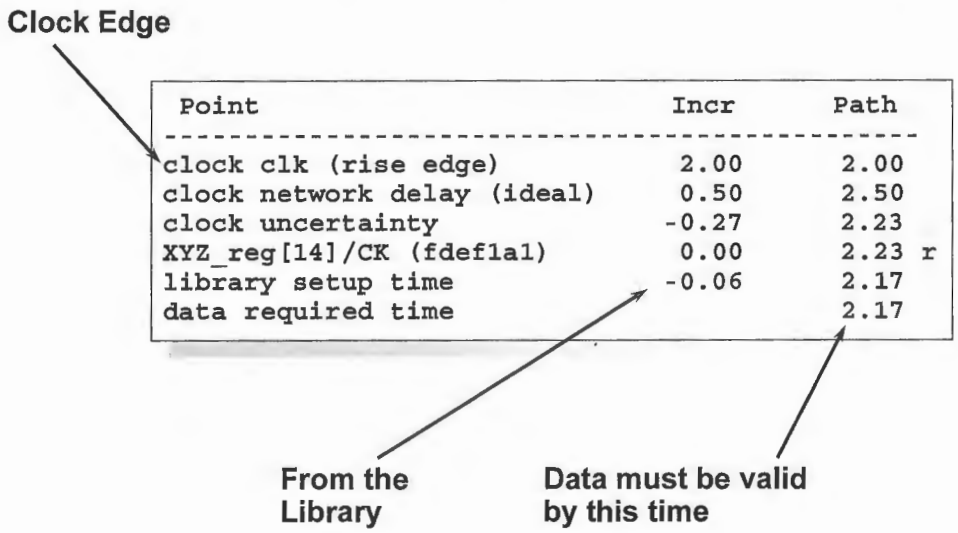
By default, the data arrival time at an input port is equal to the `set_clock_latency` (network + source latency) of the input launching clock (clock network delay), plus the `set_input_delay` (input external delay), plus the delay associated with the input transition or `set_driving_cell` (reported as an incremental delay on the input port itself.

If the `set_input_delay` includes the `-network_latency_included` and/or `-source_latency_included` options then DC assumes that the launching clock's latency is already included on the `set_input_delay` number, so no additional latency is added on to the input data arrival time.

The incremental number shown in the timing report on the output pin of a cell is the sum of the cell delay and the time-of-flight of the net on the input pin of the cell. To see the net delay separately from the cell delay, include `-input_pins`, and increase the significant digits in the timing report with `-significant_digits <#>`.

©2018 Synopsys, Inc. All Rights Reserved.

Timing Report: Path Required Section



6-7

©2018 Synopsys, Inc. All Rights Reserved.

Timing Report: Summary Section

```
--------------------------------------------------
data required time                          2.17
data arrival time                          -2.15
--------------------------------------------------
slack (MET)                                 0.02
```

Either `(MET)` **or** `(VIOLATED)`

Timing margin or *slack*:
(positive number = met
negative number = violation)

6-8

©2018 Synopsys, Inc. All Rights Reserved.

Timing Report: Options

The default behavior of `report_timing` is to report the path with the worst slack within each path group.

```
report_timing
              [ -delay max/min ]
              [ -to pin_port_clock_list ]
              [ -from pin_port_clock_list ]
              [ -through pin_port_list ]
              [ -group]
              [ -input_pins ]
              [ -max_paths path_count ]
              [ -nworst paths_per_endpoint_count ]
              [ -nets ]
              [ -transition ]
              [ -capacitance ]
              [ -significant_digits number ]
              ...
```

Can you guess which option reports cell and net delays separately?

6-9

ANSWER: **`-input_pins`**! By default, the delays are reported to cell output pins; This option includes delays to cell input pins in addition to output pins. The timing arc from a cell's output pin to the next cell's input pin represents the net delay.

`report_timing` **`-input_pins`** `-significant_digits 6`

This option shows the delay of the net connected to the input pin.

```
   Point                                                    Incr       Path
-------------------------------------------------------------------------------
 I_PRGRM_DECODE/Crnt_Instrn[25] (PRGRM_DECODE)           0.000000   2.040000 f
 I_PRGRM_DECODE/U362/A (buf1a2)                          0.000246 * 2.040246 f
 I_PRGRM_DECODE/U362/Y (buf1a2)                          0.480000   2.530246 f
```

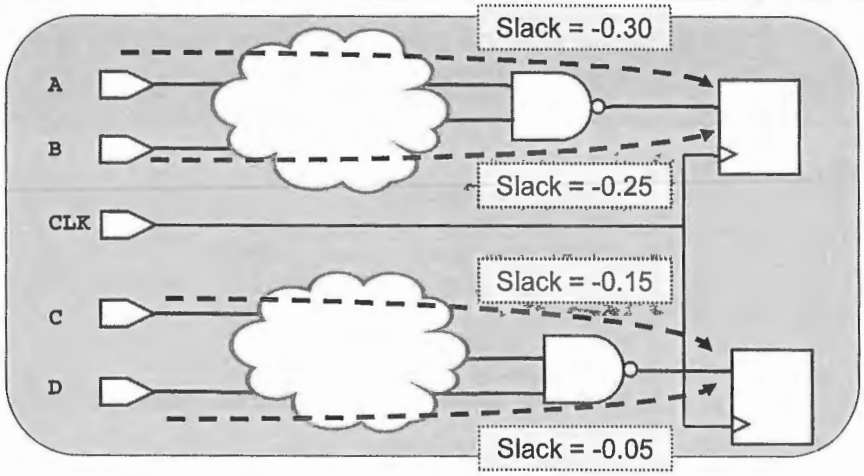
`report_timing` **`-nets`**

The option shows the net name and the fanout, it does not report the delay.

```
   Point                                            Fanout   Incr       Path
-------------------------------------------------------------------------------
 I_PRGRM_DECODE/Crnt_Instrn[25] (PRGRM_DECODE)               0.00       2.04 f
 I_PRGRM_DECODE/Crnt_Instrn[25] (net)                        0.00       2.04 f
 I_PRGRM_DECODE/U362/Y (buf1a2)                              0.48 *     2.53 f
 I_PRGRM_DECODE/n984 (net)                          4        0.00       2.53 f
 I_PRGRM_DECODE/U371/Y (xor2a0)                              0.56 *     3.09 f
```


©2018 Synopsys, Inc. All Rights Reserved.

Example -nworst vs. -max_paths



What are the reported slack(s)

```
report_timing
report_timing -max_paths 2
report_timing -max_paths 2 -nworst 2
```

6-10

Answers: Since all 4 paths are part of the same path group, only one path, the path with the worst negative slack (WNS) of -0.3 is reported with a default `report_timing`.

`report_timing -max_paths 2` will generate two worst slack reports, but only allowing one path per endpoint. In this example it will give you the paths with Slack = -0.3 and Slack = -0.15.

`report_timing -max_paths 2 -nworst 2` will generate two worst slack reports, but this time allowing up to two paths per endpoint. In this example it will give you the paths with Slack = -0.3 and Slack = -0.25.

©2018 Synopsys, Inc. All Rights Reserved.

Timing Analysis Exercise (1 of 3)

```
report_constraint -all_violators
```

```
max_delay/setup ('clk' group)

                  Required     Actual
Endpoint          Path Delay   Path Delay   Slack
-----------------------------------------------------
coeff_reg[14]/D      5.52        6.98 f    -1.46  (VIOLATED)
coeff_reg[15]/D      5.52        6.91 f    -1.39  (VIOLATED)
 ....                ....        ....       ....
coeff_reg[0]/D       5.53        6.55 r    -1.02  (VIOLATED)
mul_reg[31]/D        5.54        5.87 f    -0.33  (VIOLATED)
mul_reg[30]/D        5.53        5.85 r    -0.32  (VIOLATED)
   ....              ....        ....       ....
mul_reg[0]/D         5.50        5.69 f    -0.19  (VIOLATED)
```

1) What are some important things to notice about the violations?
2) What could be a possible cause of the above violation pattern?
3) How can you confirm your suspicion?

6-11

1) All violations are in the same path group called "`clk`". The WNS violation is over 26% (1.46/5.52), which is very big. There are many sub-critical violations ending at "`coeff_reg`" as well as "`mul_reg`". There is a large difference in slack between the violations ending at "`coeff_reg`" compared to the ones ending at "`mul_reg`".
2) The large WNS violation (1.46 ns) may be a result of over-constrained input ports. The remaining violations could be a result of "near critical paths" that are being ignored during optimization. The "`mul_reg`" violations are under 6% of the required path delay, which are more reasonable. These could possibly be reg-to-reg violations which are being ignored due to over-constrained inputs!
3) First, find out what kind of paths the "`coeff_reg`" and the "`mul_reg`" violations correspond to - input or reg-to-reg paths (they can not be output paths because the endpoints are register input pins. If they were output paths the endpoints would be output ports). This can be done with appropriate `report_timing` commands. Next, check to see what the "critical range" is of this path group using `report_path_group`.

©2018 Synopsys, Inc. All Rights Reserved.

Timing Analysis Exercise (2 of 3)

report_timing

```
Path Group: clk
Path Type: max

Point                                    Incr       Path
-------------------------------------------------------
clock clk (rise edge)                    0.00       0.00
clock network delay (ideal)              0.80       0.80
input external delay                     3.00       3.80 f
coeff[0] (in)                            0.00       3.80 f
U2405/ZN (nd02d1)                        0.11 *     3.81 r
U3575/ZN (invbd2)                        0.12 *     4.03 f
U2353/ZN (nd02d1)                        0.12 *     4.15 r
   .....                                 ....       ....
U3830/Z (aor21d1)                        0.19 *     6.98 f
coeff_reg[14]/D (dfcrq2)                 0.00 *     6.98 f
data arrival time                                   6.98

clock clk (rise edge)                    5.00       5.00
clock network delay (ideal)              0.80       5.80
clock uncertainty                       -0.20       5.60
coeff_reg[14]/CP (dfcrq2)                0.00       5.60 r
library setup time                      -0.08       5.52
data required time                                  5.52
-------------------------------------------------------
data required time                                  5.52
data arrival time                                  -6.98
-------------------------------------------------------
slack (VIOLATED)                                   -1.46
```

report_timing -to mul_reg[31]/D

```
Path Group: clk
Path Type: max

Point                                    Incr       Path
-------------------------------------------------------
clock clk (rise edge)                    0.00       0.00
clock network delay (ideal)              0.80       0.80
coeff_reg[14]/CP (dfcrq2)                0.00       0.80 r
coeff_q_reg[14]/Q (dfcrq2)               0.46       1.26 f
U2265/ZN (nd02d1)                        0.15 *     1.41 r
U3940/ZN (nr03d0)                        0.14 *     1.55 f
U2595/ZN (oan211d1)                      0.25 *     1.80 r
   .....                                 ....       ....
U2568/ZN (xn02d1)                        0.24 *     5.86 f
mul_reg[31]/D (dfnrq1)                   0.01 *     5.87 f
data arrival time                                   5.87

clock clk (rise edge)                    5.00       5.00
clock network delay (ideal)              0.80       5.80
clock uncertainty                       -0.20       5.60
mul_reg[31]/CP (dfnrq1)                  0.00       5.60 r
library setup time                      -0.06       5.54
data required time                                  5.54
-------------------------------------------------------
data required time                                  5.54
data arrival time                                 - 5.87
-------------------------------------------------------
slack (VIOLATED)                                   -0.33
```

What conclusions can you draw from the above two reports?
What could you do next to try to improve critical path timing?

6-12

1) The critical path is an "input" path, with a relatively large "input delay" of 3 ns, which is 60% of the 5 ns clock period. The resulting large violation could be due to an over-constrained input. The "`mul_reg`" path is a reg-to-reg path. Since this path is in the same path group as the critical path, and it's worst slack of -0.33 ns is far smaller than the 1.46 ns WNS of the path group, it is possible that the reg-to-reg paths are being ignored during optimization.
2) You should first check if there are already any non-default path group setting applied (for example a non-zero "critical range" value). If the "`clk`" path group already has a critical range of 1.5 ns, for example, then creating separate path groups for the input versus reg-to-reg paths may not result in significant improvements (a non-default "weight" may help). Next, create a separate path group for the input paths, and apply the recommended critical range and weight options to the `clk` group. Since there are no "output" or "combo" path violations, it is not necessary to create path groups for these paths. Finally, execute an incremental compile.

©2018 Synopsys, Inc. All Rights Reserved.

Timing Analysis Exercise (3 of 3)

```
report_path_group
```

```
                                Critical
Group Name      Weight          Range
------------------------------------
**default**     1.00            0.00
clk             1.00            0.00
```

```
group_path -name INPUTS_COEFF -to coeff_reg*/D
group_path -name clk -critical 0.33 -weight 5
report_path_group
```

```
                                Critical
Group Name      Weight          Range
------------------------------------
**default**     1.00            0.00
INPUTS_COEFF    1.00            0.00
clk             5.00            0.33
```

```
compile_ultra -scan -spg -retime -incremental
report_constraint -all_violators
```

6-13

©2018 Synopsys, Inc. All Rights Reserved.

Analysis Recommendations

After each compile or optimization step

- **Use `report_constraint -all` to determine all the constraint violations in your design**
- **Use `report_timing`, with appropriate options, to analyze violating timing paths in more detail**



6-14

©2018 Synopsys, Inc. All Rights Reserved.

Summary

You should now be able to:

Generate timing reports, with additional command options as needed, to diagnose timing constraint violations

6-15

©2018 Synopsys, Inc. All Rights Reserved.

This page was intentionally left blank

©2018 Synopsys, Inc. All Rights Reserved.

Agenda

DAY 3

6 Timing Analysis

7 Constraints: Multiple Clocks and Exceptions 

8 DC Graphical: SPG Flow, Congestion, Layout GUI

9 Constraints: Complex Design Considerations 

10 Post-Synthesis Output Data

11 Conclusion

© 2018 Synopsys, Inc. All Rights Reserved

©2018 Synopsys, Inc. All Rights Reserved.

Unit Objectives



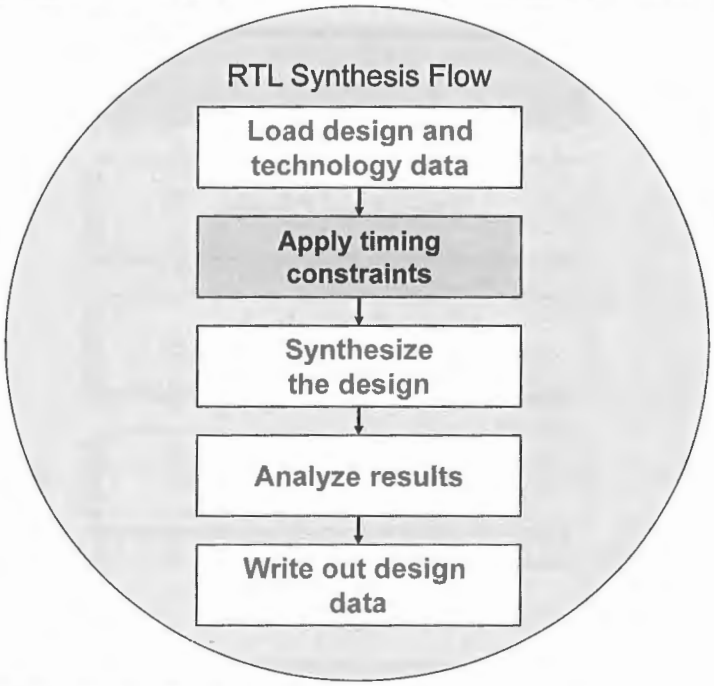
After completing this unit, you should be able to constrain a design with:

- Multiple synchronous clocks
- Clocks derived from sequential logic
- Mutually exclusive synchronous clocks
- Asynchronous clocks
- Multi-cycle timing

7-2

©2018 Synopsys, Inc. All Rights Reserved.

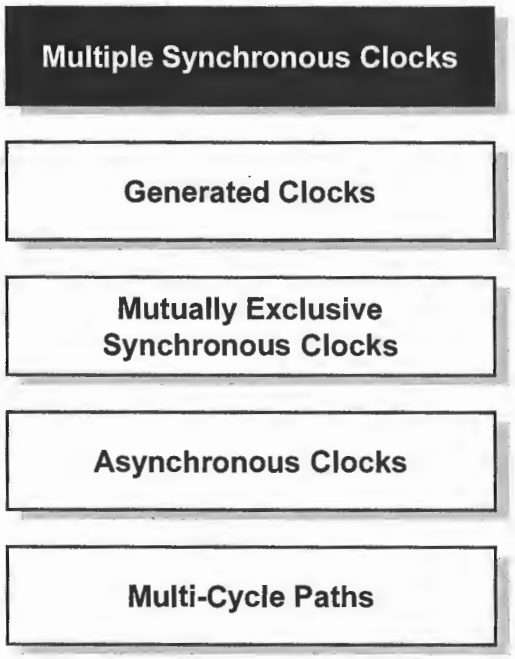
RTL Synthesis Flow



7-3

©2018 Synopsys, Inc. All Rights Reserved.

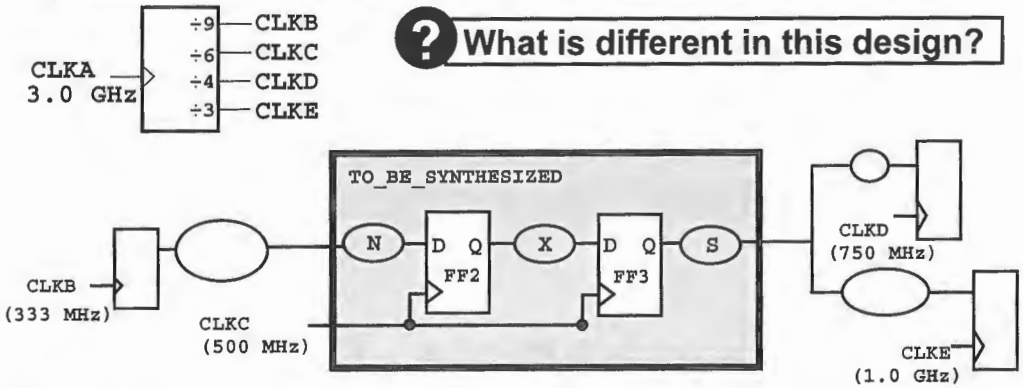
Multiple Synchronous Clocks



7-4

©2018 Synopsys, Inc. All Rights Reserved.

Multiple Synchronous Clock Designs



- Multiple clocks
- All derived from the same clock source
- Some clocks do not have a corresponding clock port on our design
- Multiple constraints on a single port

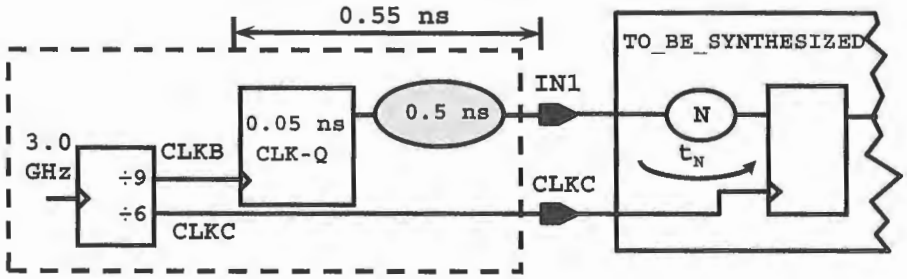
7-5

©2018 Synopsys, Inc. All Rights Reserved.

Multiple Clock Input Delay

```
create_clock -period 2.0 [get_ports CLKC]
create_clock -period 3.0 -name CLKB

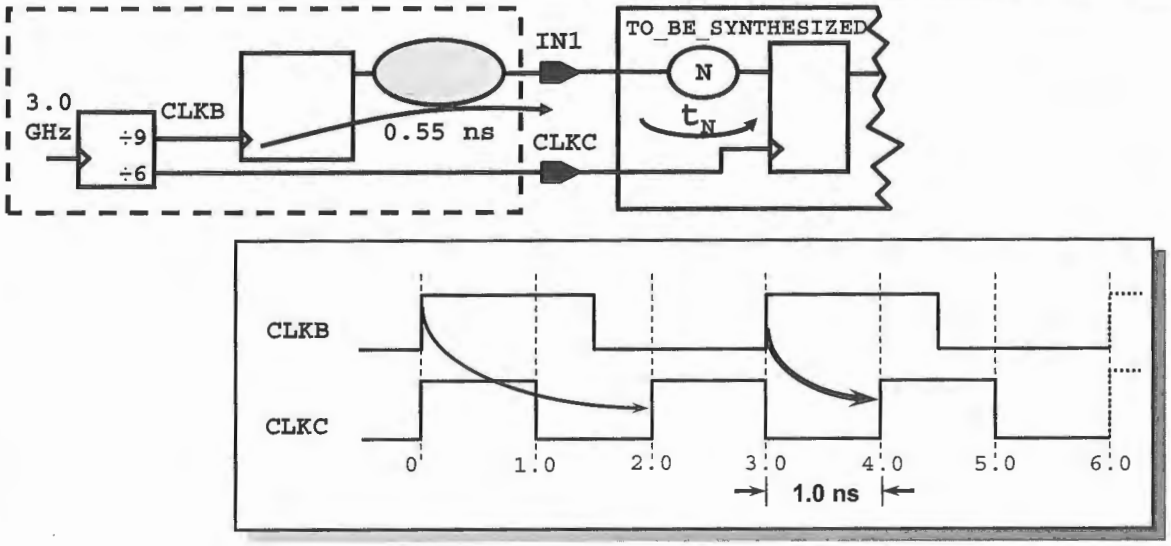
set_input_delay -max 0.55 -clock CLKB [get_ports IN1]
```



7-6

©2018 Synopsys, Inc. All Rights Reserved.

Maximum Internal Input Delay Calculation



For the example shown, input logic cloud *N* must meet:

$$t_N < 1.0 - 0.55 - t_{setup}$$

7-7

To get "common" or base period information: `report_interclock_relation`

The following example shows the output when there are two clocks: `CLKB` has a period of 3ns and `CLKC` has a period of 2ns.

```
dc_shell-topo> report_interclock_relation

*****************************
Report : Interclock Relation
*****************************
From    Period1  Count1    To     Period2    Count2   Common
-------------------------------------------------------------
CLKB     3.00       2      CLKC     2.00        3        6.00
CLKC     2.00       3      CLKB     3.00        2        6.00
```

If there is a `set_false_path -from CLKC -to CLKB`, then:

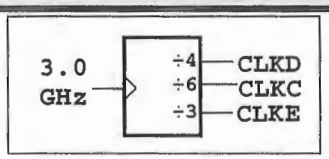
```
From    Period1  Count1    To     Period2    Count2   Common
-------------------------------------------------------------
CLKB     3.00       2      CLKC     2.00        3        6.00
```

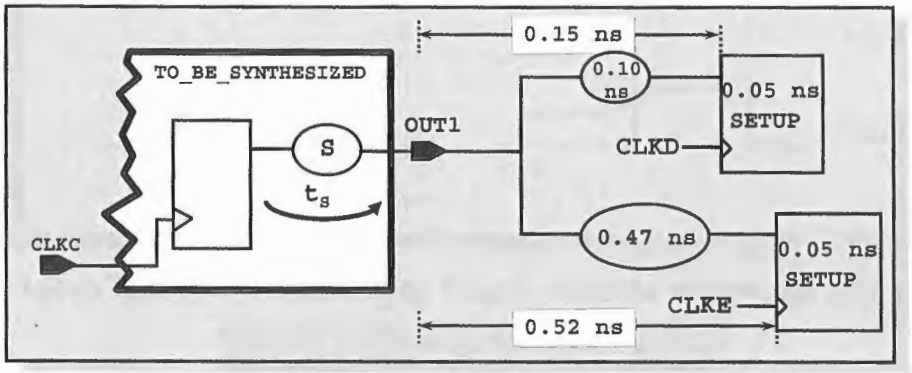

©2018 Synopsys, Inc. All Rights Reserved.

Multiple Clock Output Delay: Example

```
create_clock -period 2.0 [get_ports CLKC]
create_clock -period 1.0 -name CLKE
create_clock -period [expr {1000/750.0}] -name CLKD

set_output_delay -max 0.15 -clock CLKD [get_ports OUT1]
set_output_delay -max 0.52 -clock CLKE -add_delay [get_ports
OUT1]
```





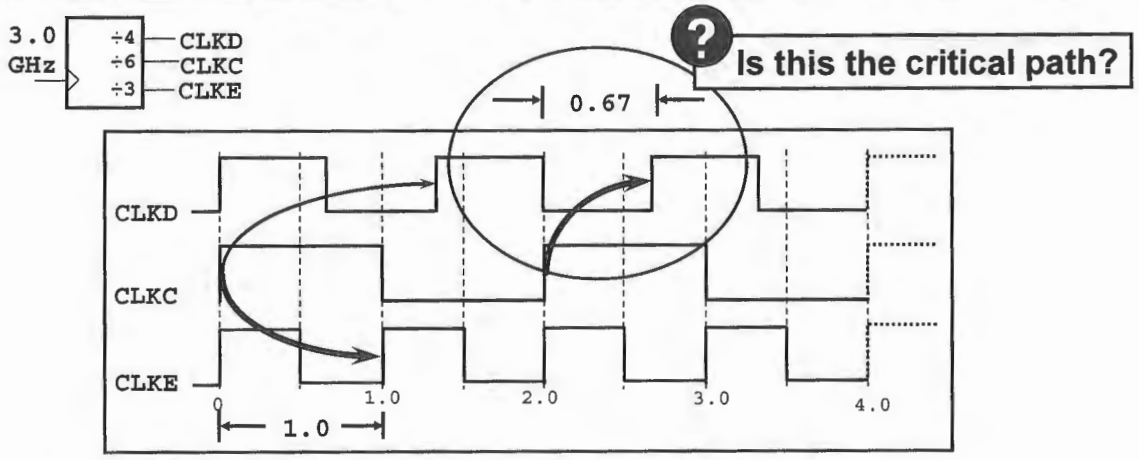
7-8

The frequency for clock CLKD is 750MHz (3 GHz/4). For calculating the clock period, make sure at least one of the operands is a real number so that the answer is also a real number for the clock period. The result of [expr {1000/75}] would be 13, instead of 13.333.

Without the `-add_delay` option, the second `set_output_delay` would over-write the first one. This is not what we want. We want DC to consider both paths and to constrain the output logic path S to meet both end-point constraints.

©2018 Synopsys, Inc. All Rights Reserved.

Maximum Internal Output Delay Calculation



The path through output cloud *S* must meet the smaller of:

$t_s < 1.0 - 0.52$ (0.48 ns) **AND** $t_s < 0.67 - 0.15$ (0.52 ns)

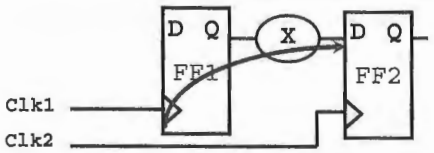
7-9

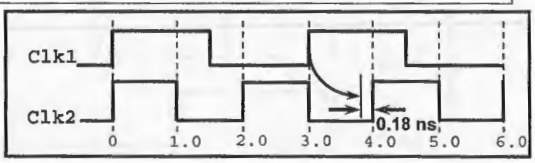
©2018 Synopsys, Inc. All Rights Reserved.

Inter Clock Uncertainty

Example:

```
create_clock -period 3 [get_ports Clk1]
create_clock -period 2 [get_ports Clk2]
set_clock_uncertainty -setup 0.10 [get_clocks Clk1]
set_clock_uncertainty -setup 0.18 [get_clocks Clk2]
```

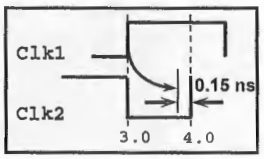




Problem: Only the capture clock uncertainty is applied (0.18)

SOLUTION - Assuming the combined effects of skew, jitter and margin are estimated to reduce the "effective clock period" from `Clk1`→`Clk2` by `0.15`:

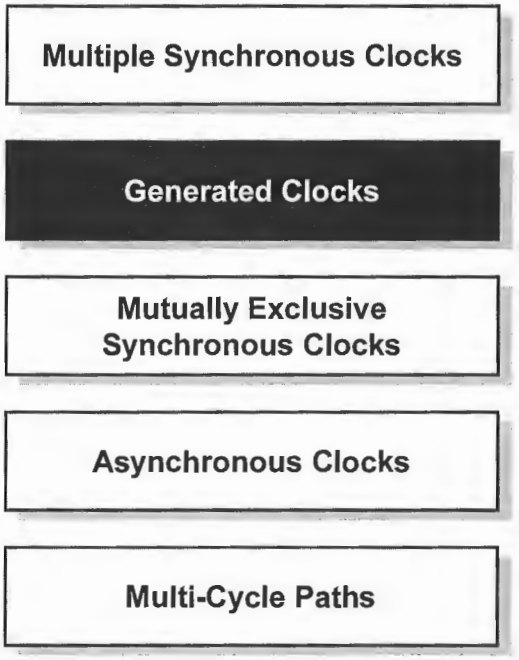
```
create_clock -period 3 [get_ports Clk1]
create_clock -period 2 [get_ports Clk2]
set_clock_uncertainty -setup 0.10 [get_clocks Clk1]
set_clock_uncertainty -setup 0.18 [get_clocks Clk2]
set_clock_uncertainty -setup 0.15  \
  -from [get_clocks Clk1] -to [get_clocks Clk2]
```



7-10

©2018 Synopsys, Inc. All Rights Reserved.

Generated Clocks

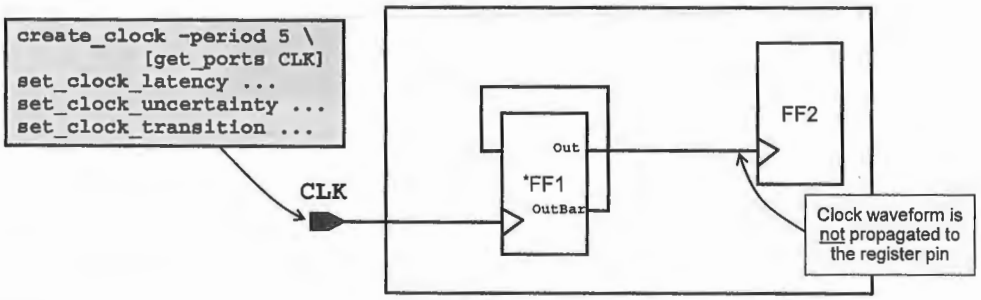


7-11

©2018 Synopsys, Inc. All Rights Reserved.

Clock Propagation Through Sequential Logic

Clock definitions (`create_clock`) do not propagate through sequential logic



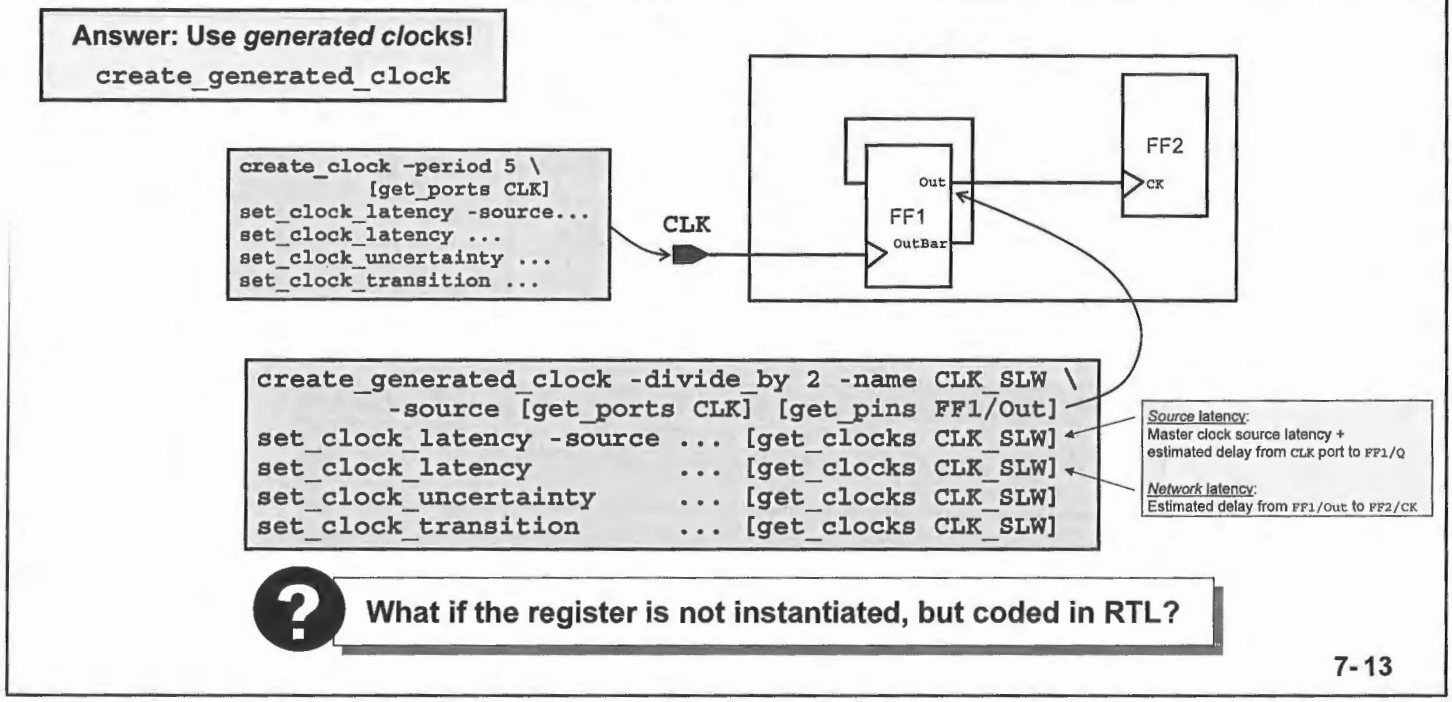
How do we constrain clocks that are derived from sequential logic?

*Note: In this example we are assuming that the divide-by-two register is instantiated, so its cell and pin names are known prior to compile.

7-12

©2018 Synopsys, Inc. All Rights Reserved.

Generated Clocks – Instantiated Register




```
create_generated_clock  -source master_pin   source_objects
                              [-name clock_name] [-add]
                              [-divide_by divide_factor]
                              |-multiply_by multiply_factor]
                              [-duty_cycle percent] [-invert]
                              [-preinvert]
                              [-edges edge_list]
                              [-edge_shift edge_shift_list]
                              [-combinational]
```

The "`-source`" of the generated clock is the pin or port name of the "master" clock from which the generated clock is derived. The "`source_objects`"are the pins or ports where the generated clock is being applied.

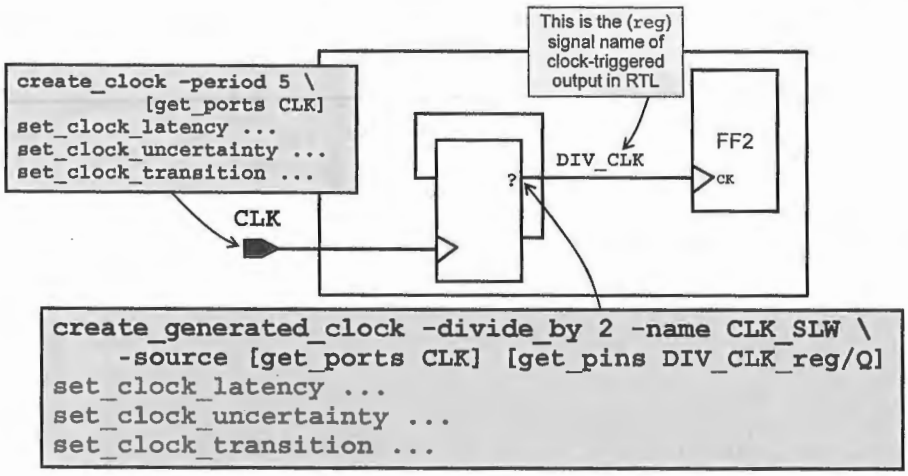
The *ideal* source and network latencies of the generated clock are zero, by default – they do not inherit the master clock's ideal latencies. Therefore, in the above example, the generated clock's *source* latency should model the master clock's source latency plus the estimated clock delay from the clock port `CLK` to `FF1/Q`. The *network* latency should model the estimated delay from `FF1/Out` to `FF2/CK`. After clock tree synthesis, when the clocks change from *ideal* to *propagated*, the generated clock's ideal source/network latency constraints are ignored. The propagated generated clock's *source* latency is calculated as the master clock's source latency plus the actual propagated delay to `FF1/Out` . It's network latency will be the actual propagated delay from `FF1/Out` to `FF2/CK`.

If `create_clock` is used instead of a generated clock, its propagated *source* latency ignores the actual propagated delay up to `FF1/Out` (zero delay, by default). This delay would need to be estimated by the user as a source latency constraint. This is why it is preferred to use generated clocks over "master" clocks for defining derived clocks.

©2018 Synopsys, Inc. All Rights Reserved.

Generated Clocks – RTL Register

Answer: Use the *gtech* cell/pin name!




```
create_generated_clock -divide_by 2 -name CLK_SLW \
    -source [get_ports CLK] [get_pins DIV_CLK_reg/Q]
set_clock_latency ...
set_clock_uncertainty ...
set_clock_transition ...
```

The cell and pin names of any RTL-coded register is known prior to compile, and can be used with constraints – see notes below

7- 14

Register cell name: `OUTPUT_SIGNAL_NAME_reg`
Clock pin: `clocked_on`
Data pin: `next_state`
Q output pin: `Q`
Q-bar output pin: `QN`
Asynch reset: `clear`
Asynch set: `preset`

©2018 Synopsys, Inc. All Rights Reserved.

Mutually Exclusive Synchronous Clocks

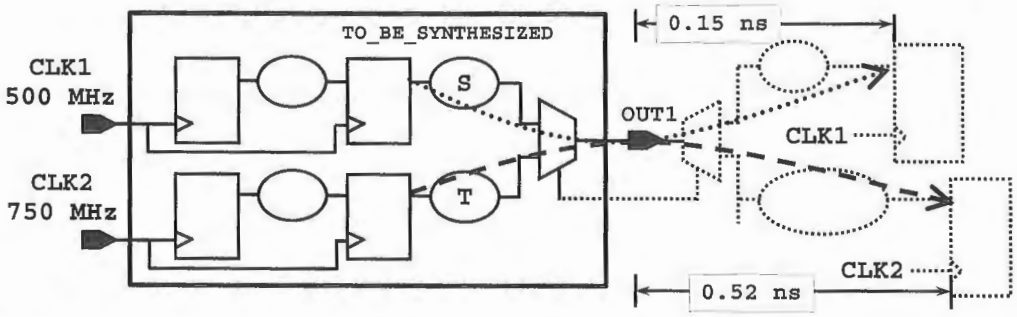
- Multiple Synchronous Clocks
- Generated Clocks
- **Mutually Exclusive Synchronous Clocks**
- Asynchronous Clocks
- Multi-Cycle Paths

7-15

©2018 Synopsys, Inc. All Rights Reserved.

Logically Exclusive Clocks: Problem #1

CLK1 and CLK2 are synchronous and logically mutually exclusive




```
create_clock -period 2.0 [get_ports CLK1]
create_clock -period [expr {1000/750.0}] [get_ports CLK2]
set_output_delay -max .15 -clock CLK1 [get_ports OUT1]
set_output_delay -max .52 -clock CLK2 -add [get_ports OUT1]
```

? **Is this enough?**

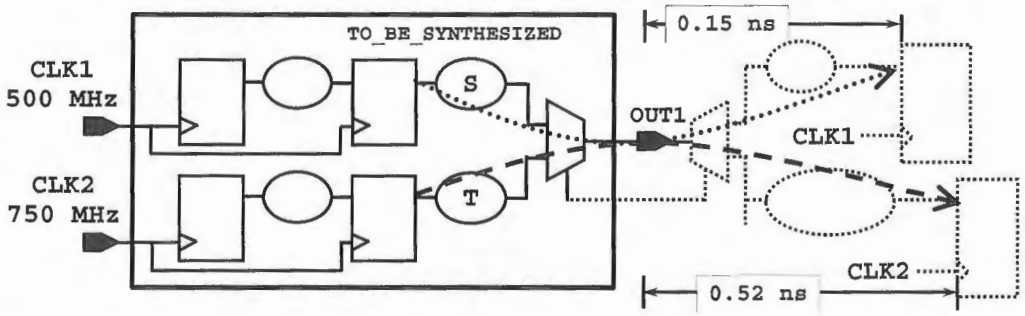
7-16

ANSWER: No! The paths through "S" and "T" may be incorrectly over-constrained because Design Compiler will pick the worst of the two output delay constraints and apply that to each of the paths.

©2018 Synopsys, Inc. All Rights Reserved.

Logically Exclusive Clocks: Solution #1

CLK1 and CLK2 are synchronous and logically mutually exclusive




```
create_clock -period 2.0 [get_ports CLK1]
create_clock -period [expr {1000/750.0}] [get_ports CLK2]
set_output_delay -max .15 -clock CLK1 [get_ports OUT1]
set_output_delay -max .52 -clock CLK2 -add_delay [get_ports OUT1]

set_false_path -from [get_clocks CLK1] -to [get_clocks CLK2]
set_false_path -from [get_clocks CLK2] -to [get_clocks CLK1]
# OR
set_clock_groups -logically_exclusive -group CLK1 -group CLK2
```

7-17

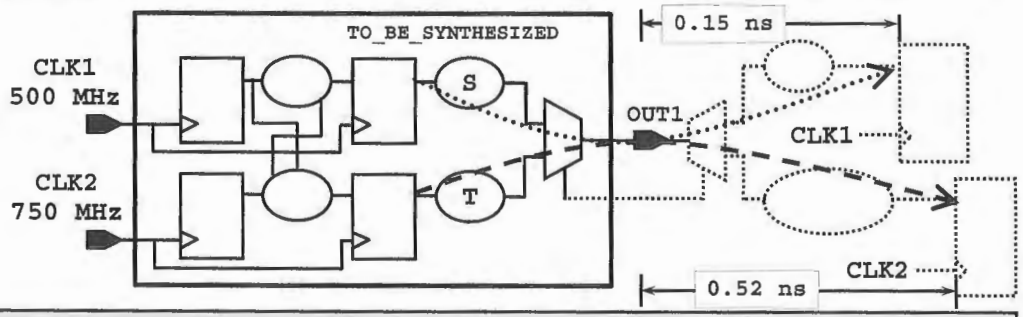
Two "logically exclusive" clocks do not have any timing paths between them. In other words, no timing analysis is performed (by *Design Compiler*, *IC Compile (II)*, or *PrimeTime*) on paths that are launched by one of the clocks and captured by the other clock.
However, crosstalk analysis in *IC Compile (II)* or *PrimeTime SI* (*signal integrity*) still checks for possible crosstalk affects on timing paths of one clock domain from nearby metal traces of paths from the other clock domain.

In the above example, the two inputs of the MUX may be long parallel metal traces routed on adjacent routing tracks. The coupling capacitance between the two traces may cause switching activity on the `CLK1` input of the MUX to affect (slow down or speed up) the delay through the `CLK2` input of the MUX. SI analysis in other tools will correctly consider these crosstalk affects on *setup* and *hold* timing.

©2018 Synopsys, Inc. All Rights Reserved.

Logically Exclusive Clocks: Problem #2

CLK1 and CLK2 are synchronous and "partially" mutually exclusive




```
create_clock -period 2.0 [get_ports CLK1]
create_clock -period [expr {1000/750.0}] [get_ports CLK2]
set_output_delay -max .15 -clock CLK1 [get_ports OUT1]
set_output_delay -max .52 -clock CLK2 -add_delay [get_ports OUT1]

set_false_path -from [get_clocks CLK1] -to [get_clocks CLK2]
set_false_path -from [get_clocks CLK2] -to [get_clocks CLK1]
```

What's wrong with this?

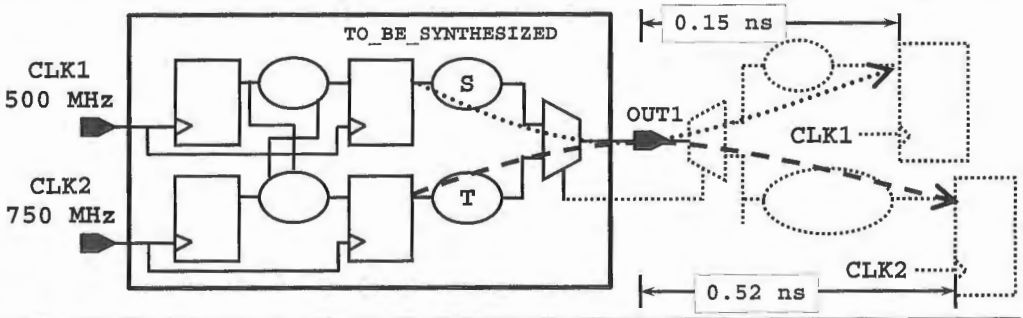
7-18

ANSWER: This will disable optimization along "legal" timing paths between CLK1 and CLK2 – paths connected by the "red" nets above.

©2018 Synopsys, Inc. All Rights Reserved.

Logically Exclusive Clocks: Solution #2

CLK1 and CLK2 are synchronous and "partially" mutually exclusive



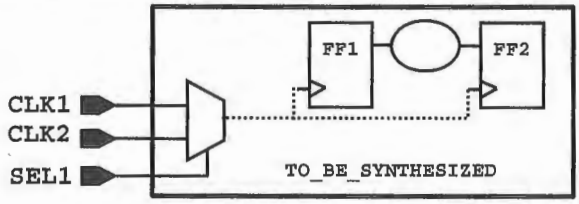

```
create_clock -period 2.0 [get_ports CLK1]
create_clock -period [expr {1000/750.0}] [get_ports CLK2]
set_output_delay -max .15 -clock CLK1 [get_ports OUT1]
set_output_delay -max .52 -clock CLK2 -add_delay [get_ports OUT1]

set_false_path -from [get_clocks CLK1] \
               -through [get_ports OUT1] -to [get_clocks CLK2]
set_false_path -from [get_clocks CLK2] \
               -through [get_ports OUT1] -to [get_clocks CLK1]
```

7-19

©2018 Synopsys, Inc. All Rights Reserved.

Multiple Clocks per Register: Problem #3




```
create_clock -period 5 [get_ports CLK1]
create_clock -period 8 [get_ports CLK2]
```

- **Multiple clocks are allowed to reach register clock pins, and are all analyzed for worst case timing**
  - Controlled by the variable `timing_enable_multiple_clocks_per_reg` (`true` by default)
- **Muxed clocks are not inferred as exclusive clocks!**

As a result, DC optimization will consider all of the following four cases:
CLK1 → CLK1
CLK1 → CLK2
CLK2 → CLK1
CLK2 → CLK2

How do you direct DC to consider only CLK1→CLK1 and CLK2→CLK2, and to optimize for the worst of the two?

7-20

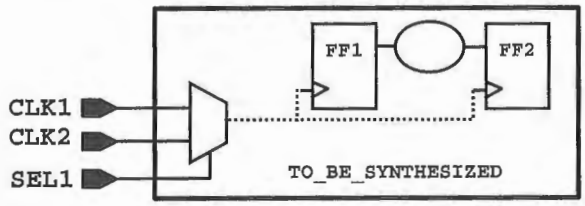
The above variable is also **`true`** by default in *PrimeTime* and *IC Compiler*.
In *IC Compiler II*, this "multiple clock per reg" behavior is innate – not user controllable.

CAUTION: It is very important that the variable value is not set to **`false`**, which would allow only a single clock to reach register clock pins: Clock assignment is performed independently for each register by tracing backwards from endpoint to available clock source(s); Clock assignment becomes random – no worst-case analysis is performed. As a result, DC will optimize the path between FF1 to FF2 for only one of the following four cases, and there is no way to predict which of the four:

CLK1 → CLK1
CLK1 → CLK2
CLK2 → CLK1
CLK2 → CLK2

©2018 Synopsys, Inc. All Rights Reserved.

Multiple Clocks per Register: Solution #3




```
create_clock -period ... [get_ports CLK1]
create_clock -period ... [get_ports CLK2]
set_clock_groups -logically_exclusive -group CLK1 -group CLK2
```

The delay between the registers will be optimized for the worst of CLK1→CLK1 or CLK2→CLK2 timing

You can alternatively apply two false paths

Note: If `set_case_analysis` is applied to `SEL1`, timing analysis automatically uses only the "selected" clock

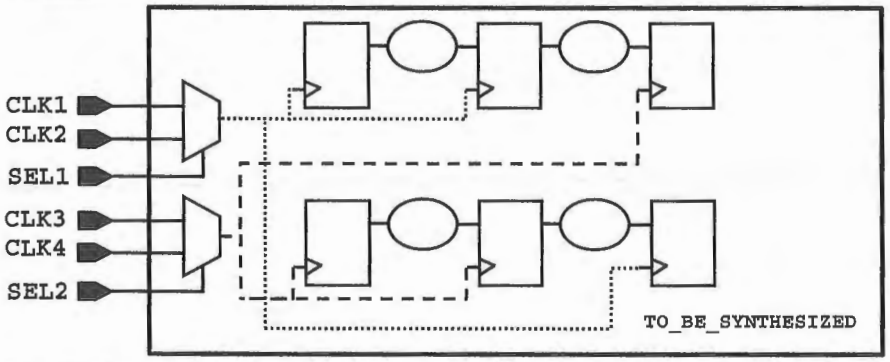
7-21

The above command is equivalent to the following "false path" commands – using "clock groups" is more convenient (fewer commands):

```
set_false_path -from [get_clocks CLK1] -to [get_clocks CLK2]
set_false_path -from [get_clocks CLK2] -to [get_clocks CLK1]
```


©2018 Synopsys, Inc. All Rights Reserved.

Multiple Clocks per Register: Example #4




```
create_clock -period ... [get_ports CLK1]
create_clock -period ... [get_ports CLK2]
create_clock -period ... [get_ports CLK3]
create_clock -period ... [get_ports CLK4]
set_clock_groups -logically_exclusive -group CLK1 -group CLK2
set_clock_groups -logically_exclusive -group CLK3 -group CLK4
```

You can alternatively apply four false paths!

The delay of each path will be optimized for the worst of the logically possible timing conditions

7-22

The above commands are equivalent to the following "false path" commands – using "clock groups" is more convenient (fewer commands):

```
set_false_path -from [get_clocks CLK1] -to [get_clocks CLK2]
set_false_path -from [get_clocks CLK2] -to [get_clocks CLK1]
set_false_path -from [get_clocks CLK3] -to [get_clocks CLK4]
set_false_path -from [get_clocks CLK4] -to [get_clocks CLK3]
```

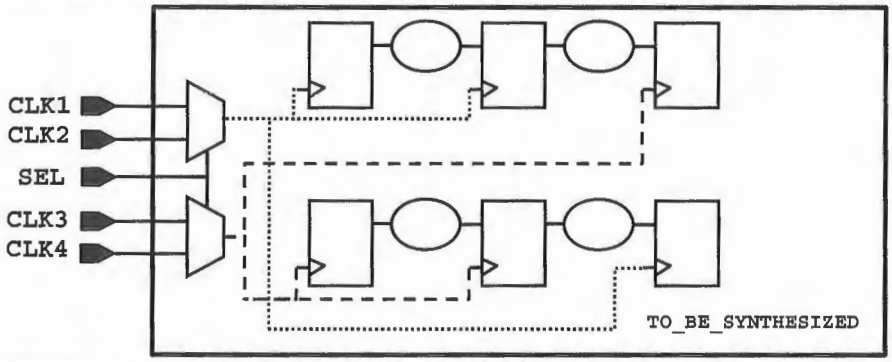
The user can also apply `set_case_analysis`, but this limits logic optimization to the specified condition only. The responsibility is now on the user to correctly define the "worst of the logically possible timing conditions". The following example limits optimization to the following set of clocks: CLK2 → CLK2, CLK2 → CLK3, CLK3 → CLK2, CLK3 → CLK3:

```
set_case_analysis 1 [get_ports SEL1]
set_case_analysis 0 [get_ports SEL2]
```

What if in the above case analysis example, the "allowed" CLK2 ←→ CLK3 relationships are NOT the worst possible timing conditions for all paths? What if CLK3 → CLK4 is worse than CLK3 → CLK2? You would not be able to use "case analysis" to simultaneously optimize the design for these conditions in the same run. Using "clock groups" or "false paths" DOES allow optimization to determine the worst case for each path.

©2018 Synopsys, Inc. All Rights Reserved.

Multiple Clocks per Register: Example #5




```
create_clock -period ... [get_ports CLK1]
create_clock -period ... [get_ports CLK2]
create_clock -period ... [get_ports CLK3]
create_clock -period ... [get_ports CLK4]
set_clock_groups -logically_exclusive \
                 -group "CLK1 CLK3" -group "CLK2 CLK4"
```

You can alternatively apply eight false paths!

The delay of each path will be optimized for the worst of the logically possible timing conditions

7-23

The above single command is equivalent to the following set of eight "false path" commands – clearly using "clock groups" is more convenient:

```
set_false_path -from [get_clocks CLK1] -to [get_clocks CLK2]
set_false_path -from [get_clocks CLK1] -to [get_clocks CLK4]
set_false_path -from [get_clocks CLK3] -to [get_clocks CLK2]
set_false_path -from [get_clocks CLK3] -to [get_clocks CLK4]
set_false_path -from [get_clocks CLK2] -to [get_clocks CLK1]
set_false_path -from [get_clocks CLK2] -to [get_clocks CLK3]
set_false_path -from [get_clocks CLK4] -to [get_clocks CLK1]
set_false_path -from [get_clocks CLK4] -to [get_clocks CLK3]
```

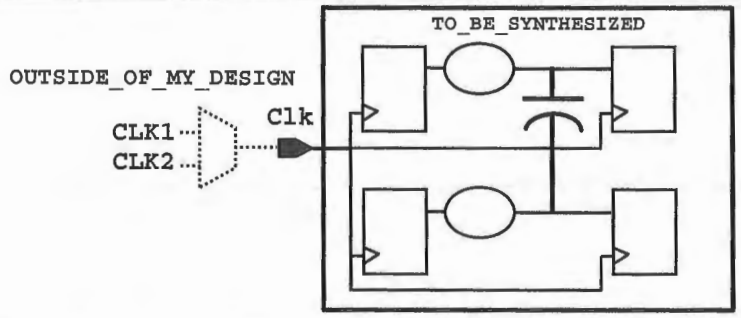
The user can also apply `set_case_analysis`, but this limits logic optimization to the specified condition only. The responsibility is now on the user to correctly define the "worst of the logically possible timing conditions". The following example limits optimization to the following set of clocks: CLK2 → CLK2, CLK2 → CLK4, CLK4 → CLK2, CLK4 → CLK4:

```
set_case_analysis 1 [get_ports SEL]
```

What if, between CLK1 and CLK2, CLK2 → CLK2 is in fact the worst timing condition, but between CLK3 and CLK4, CLK3 → CLK3 the worst? You would not be able to use "case analysis" to simultaneously optimize the design for these conditions in the same run. Using "clock groups" or "false paths" DOES allow optimization to determine the worst case for each path.

©2018 Synopsys, Inc. All Rights Reserved.

Crosstalk Analysis in ICC (II) and PT-SI: Problem #6




```
create_clock -period 3 -name CLK1 [get_ports Clk]
create_clock -period 4 -name CLK2 -add [get_ports Clk]
set_clock_groups -logically_exclusive -group CLK1 -group CLK2
```

Design Compiler **delay optimization does the right thing**

Crosstalk analysis (ICC (II), PT-SI) assumes both clocks are available simultaneously → Pessimistic crosstalk-induced delta delays

Can we write constraints that will result in correct/accurate behavior in both DC and PT-SI?

7-24

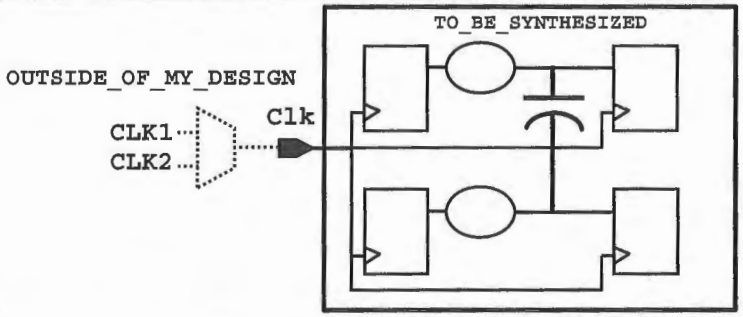
Notice the need for the "-name" and "-add" options when creating multiple clocks on a single port.

Design Compiler delay optimization correctly optimizes each path for the worst of CLK1→CLK1 or CLK2→CLK2.

After synthesis and layout, when analyzing cross talk effects during SI-based timing analysis (in *IC Compiler (II)* and *PrimeTime-SI*), clocks that are defined as "false paths" or "logically exclusive" may result in slightly pessimistic crosstalk-induced delta delays. This is because, for crosstalk analysis purposes, *SI analysis* assumes that aggressor and victim nets can be driven by DIFFERENT clock waveforms, even if they are "logically exclusive", which can result in larger timing window overlaps, and thereby, pessimistic crosstalk-induced delta delays.

©2018 Synopsys, Inc. All Rights Reserved.

Crosstalk Analysis in ICC (II) and PT-SI: Solution #6




```
create_clock -period 3 -name CLK1 [get_ports Clk]
create_clock -period 4 -name CLK2 -add [get_ports Clk]
set_clock_groups -physically_exclusive -group CLK1 -group CLK2
```

Delay optimization in *Design Compiler* treats *physically exclusive* clocks exactly the same as *logically exclusive* clocks (or *false paths*)

***SI* analysis treats clocks as exclusive as well**
→ Accurate crosstalk-induced delay analysis

Why not ALWAYS use `-physically_exclusive` instead of `-logically_exclusive`?
Ans: See page 7-17.

7-25

In *Design Compiler*, the "physically exclusive clocks" will yield the same synthesis results as the following traditional "false path" commands, or the "logically exclusive" clock group command:

```
set_false_path -from [get_clocks CLK1] -to [get_clocks CLK2]
set_false_path -from [get_clocks CLK2] -to [get_clocks CLK1]
```

OR

```
set_clock_groups -logically_exclusive -group CLK1 -group CLK2
```

However, after synthesis and layout, when analyzing cross talk effects during "signal integrity" (SI) timing analysis, there is a difference between using "physically exclusive" clock groups, and "logically exclusive" clock groups or "false paths":

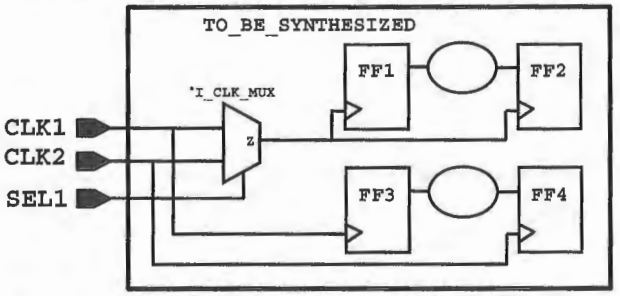
Two clocks defined to be physically exclusive of each other have no timing paths between them, and furthermore, are considered physically isolated from each other: No timing analysis is performed (by *Design Compiler*, *IC Compile (II)*, or *PrimeTime*) on paths that are launched by one of the clocks and captured by the other clock, and *SI-based* timing analysis assumes no possible crosstalk interaction between them.

A physically exclusive clock group correctly considers that the SAME clock waveform drives the aggressor nets and the victim nets at any given time, which results in accurate timing window overlaps, and thereby accurate crosstalk-induced delta delay analysis.

©2018 Synopsys, Inc. All Rights Reserved.

Challenge Exercise

* We are assuming that the MUX is instantiated, so its cell and pin names are known prior to compile.




```
create_clock -period 5 -name FST_CLK [get_ports CLK1]
create_clock -period 8 -name SLW_CLK [get_ports CLK2]
```

We would like to constrain this design for correct and accurate timing analysis in both DC and PT-SI

Why can we not define `CLK1` and `CLK2` as *exclusive* clock groups?

What trick can we use to define <u>only</u> the `FF1` and `FF2` clocks as *physically exclusive*?

7-26

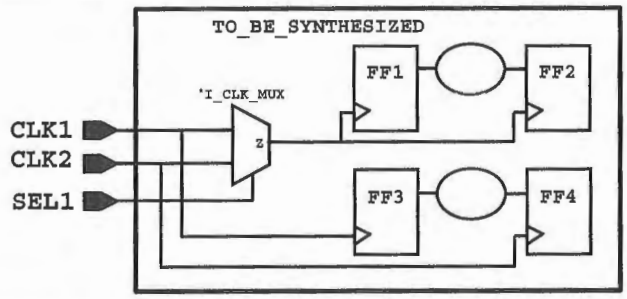
Answer:

`CLK1` and `CLK2` are mutually exclusive for `FF1` and `FF2`, but not for `FF3` and `FF4`.

Create generated clocks at the MUX output, and define the generated clocks as physically exclusive.

©2018 Synopsys, Inc. All Rights Reserved.

Challenge Exercise: Solution



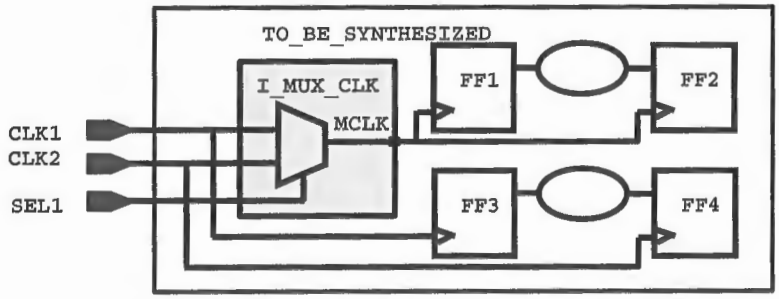

```
create_clock -period 5 -name FST_CLK [get_ports CLK1]
create_clock -period 8 -name SLW_CLK [get_ports CLK2]
create_generated_clock -divide_by 1 -name MCLK1 \
            -source CLK1 [get_pins I_CLK_MUX/Z]
create_generated_clock -divide_by 1 -name MCLK2 -add \
            -source CLK2 [get_pins I_CLK_MUX/Z]
set_clock_groups -physically_exclusive -group MCLK1 -group MCLK2
set_clock_latency ...
set_clock_uncertainty ...
set_clock_transition ...
```

7-27

Since there are two generated clocks applied at the MUX output, you must use the `-add` option.

What if the MUX is not instantiated, but is coded in RTL? The simplest solution is to place the MUX (or any clock generator logic) in its own hierarchical sub-design (RTL *module* or *entity*). In *Design Com*piler you could then apply the generated clock to the output pin name of the sub-block. However, this is not recommended since it will cause issues in *PrimeTime* and *IC Compiler (II)* timing analysis (see *Solvnet* article "How Can I Avoid UITE-130 and UITE-136 Warnings in my Design?" **Doc Id:** 011059). The better solution is to take advantage of the sub-design pin name to derive the pin name of the clock-driver cell, as follows:

```
proc get_driving_pin {pin} {
 return [get_pins -leaf -of [all_connected [get_pins $pin]] \
  -filter {pin_direction =~ *out}]
}
create_generated_clock -divide_by 1 -name MCLK1 \
            -source CLK1 [get_driving_pin I_MUX_CLK/MCLK]
```



©2018 Synopsys, Inc. All Rights Reserved.

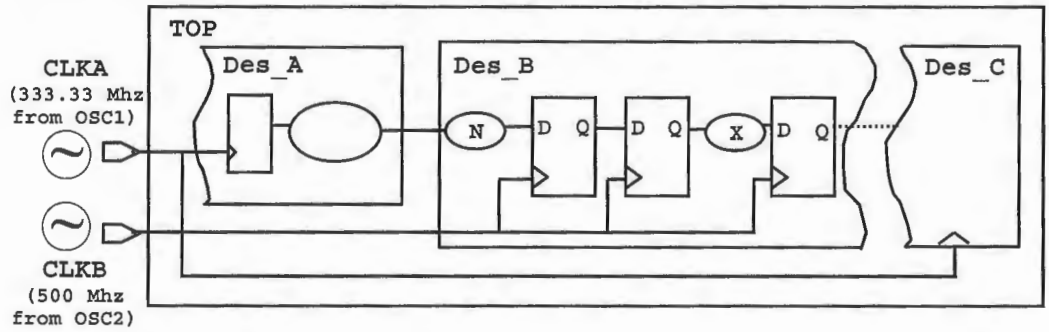
Asynchronous Clocks

- Multiple Synchronous Clocks
- Generated Clocks
- Mutually Exclusive Synchronous Clocks
- **Asynchronous Clocks**
- Multi-Cycle Paths

7-28

©2018 Synopsys, Inc. All Rights Reserved.

Asynchronous Clock Designs



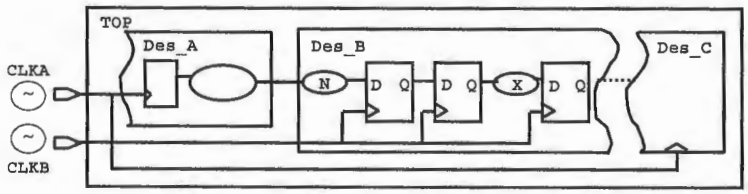
What do you do if the design has *asynchronous* clock sources?

7-29

©2018 Synopsys, Inc. All Rights Reserved.

Synthesizing with Asynchronous Clocks

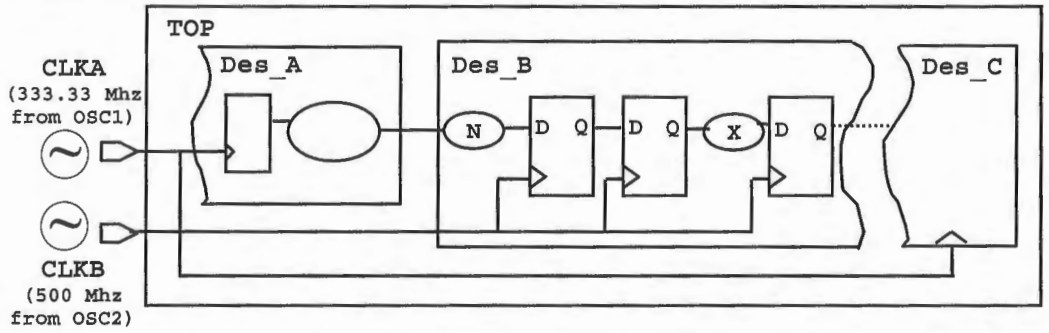
- **It is your responsibility to design the logic in order to prevent metastability:**
  - Instantiate double-clocking, metastable-hard Flip-Flops
  - Dual-port FIFO, etc
- **Create clocks to constrain the paths within each clock domain**
- **Disable timing optimization along paths between asynchronous clock domains - prevents DC from:**
  - Wasting run time on a non-critical timing paths
  - **Ignoring timing optimization of actual timing critical paths**



7-30

©2018 Synopsys, Inc. All Rights Reserved.

Example: Asynchronous Design Constraints




```
current_design TOP
# Constrain register-register paths within each clock domain
create_clock -period 3 [get_ports CLKA]
create_clock -period 2 [get_ports CLKB]
...
# Disable timing optimization across asynchronous clock domains
set_clock_groups -asynchronous -group CLKA -group CLKB
...
compile_ultra ...
```

7-31

Use of the `set_clock_groups` command with the `-async` option is required to perform accurate crosstalk analysis in *IC Compile (II)* and *PrimeTime-SI*. Asynchronous clocks are treated the same as *false paths* or *logically exclusive* paths from a timing perspective in DC, ICC (II) or PT – no timing paths are considered between asynchronous clock domains. However, SI-based timing analysis treats neighboring metal traces from two asynchronous clock domains as having an "infinite arrival window" overlap, which accounts for realistic worst-case crosstalk delay effects.

If using `set_false_path` instead (see below), the results will be the same in *Design Compiler*, however, post-layout SI-based timing analysis may be optimistic. The arrival window is calculated based on the clocks being synchronous, which results in a smaller overlapping timing window, and possibly optimistic crosstalk delay effects:

```
set_false_path -from [get_clocks CLKA] -to [get_clocks CLKB]
set_false_path -from [get_clocks CLKB] -to [get_clocks CLKA]
```


©2018 Synopsys, Inc. All Rights Reserved.

Multi-Cycle Paths

Multiple Synchronous Clocks

Generated Clocks

Mutually Exclusive Synchronous Clocks

Asynchronous Clocks

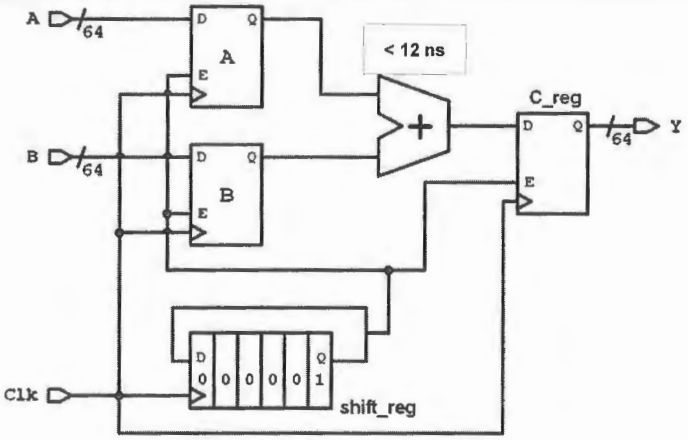
Multi-Cycle Paths

7-32

©2018 Synopsys, Inc. All Rights Reserved.

Example Multi-Cycle Design

Clock period is 2 ns. The adder takes almost 6 clock cycles. What happens when you apply `create_clock -period 2 Clk`?



7-33

By specifying a `create_clock` with a period of 2ns the non-multi-cycle paths are constrained correctly. However, since DC does not perform logic analysis it does not consider the fact that the registers above are enabled every sixth cycle only. DC therefore assumes that the paths through the adder must also meet the 1ns constraint!

©2018 Synopsys, Inc. All Rights Reserved.

Default Single-Cycle Timing

- **All timing paths are constrained by single-cycle timing, by default, by the following implicit `set_multicycle_path` multiplier values:**

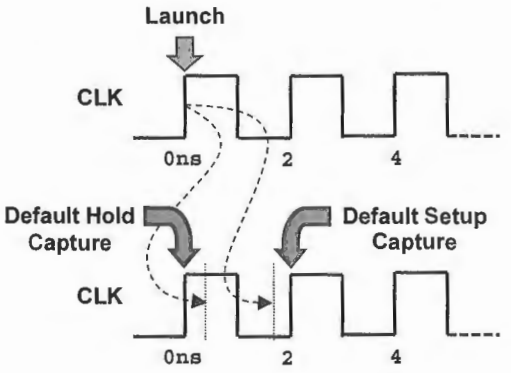
  `-setup` = 1

  `-hold` = 0

  - Setup capture edge occurs 1 cycle after launch edge (for single clock)
  - Hold capture edge is relative to setup capture edge - occurs 1 edge before setup capture edge

- **Increasing the *setup* multiplier without changing the *hold* multiplier moves both the setup and hold capture edges!**

```
create_clock -period 2 [get_ports CLK]
set_multicycle_path -setup 1 <all paths>
set_multicycle_path -hold  0 <all paths>
```



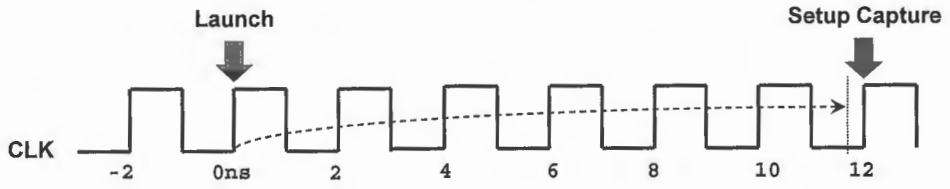
How do we constrain for 6 cycle timing?

7-34

©2018 Synopsys, Inc. All Rights Reserved.

Multi-Cycle Timing Constraint

```
create_clock -period 2 [get_ports CLK]
set_multicycle_path -setup 6 -from {A_reg[*] B_reg[*]} -to C_reg[*]
```



Where does the hold check occur?

7-35

The default setup multiplier value is 1 – this is why DC performs single-cycle optimization and analysis by default. By changing the setup multiplier to 6, DC will perform the setup analysis on edge 6, i.e. at 12ns. This will allow the adder's logic to have a delay of (12ns – setup_time – uncertainty).

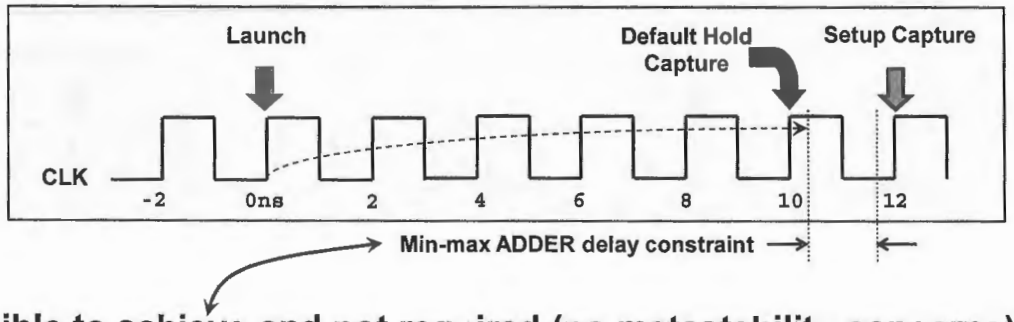
Since multi-cycle path commands should be applied to the design prior to the first compile, you can not use library cell pin names as your start or endpoints. For registers, it is generally possible to use the register's *cell name* as a start or end-point – DC will automatically find the correct *–from* and *–to* pins. If this is too ambiguous (e.g. multiple startpoint pins: *clk* and *enable*, or multiple endpoint pins: *data*, *set* and *reset*), use the following for *–from* or *-to*:
`[get_pins XYX_reg/gtech_pin_name]`, where `gtech_pin_name` is equal to `clocked_on` for the clock pin or `next_state` for the data input pin. Note that after synthesis the *gtech* pin names are replaced by the actual *library cell pin* names.

©2018 Synopsys, Inc. All Rights Reserved.

Default Hold Check

```
set_multicycle_path -setup 6 -from {A_reg[*] B_reg[*]} -to C_reg[*]
```

- **The hold multiplier remains 0 → Hold capture edge is 10 ns!**



- **Impossible to achieve and not required (no metastability concerns)!**
- **Hold check can safely occur at 0ns**

? How do we move the hold check back 5 edges?

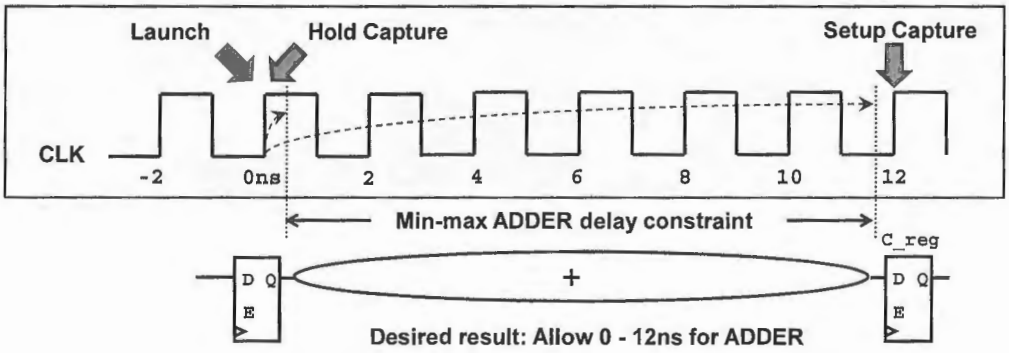
7-36

The default HOLD multiplier is set to 0. It is not intuitively obvious, but this means that the hold check is always performed in the same cycle as the setup check, namely on the clock edge before the setup check, which in this case is at 10ns.
Synthesizing a path that has a setup requirement of <12ns and a hold requirement >10ns is virtually impossible (consider worst case/best case PVT corner variations plus LSB versus MSB delay differences)! Furthermore, since the registers are disabled at the 2ns, 4ns, 6ns, 8ns and 10ns clock edges, there is no concern for metastability if the data were to arrive right at one of these edges between 0ns and 12ns. Therefore, we can safely allow the hold check to be done at the 0ns edge.

©2018 Synopsys, Inc. All Rights Reserved.

Setting the Proper Hold Constraint

```
set_multicycle_path -setup 6 -from {A_reg[*] B_reg[*]} -to C_reg[*]
set_multicycle_path -hold  5 -from {A_reg[*] B_reg[*]} -to C_reg[*]
```



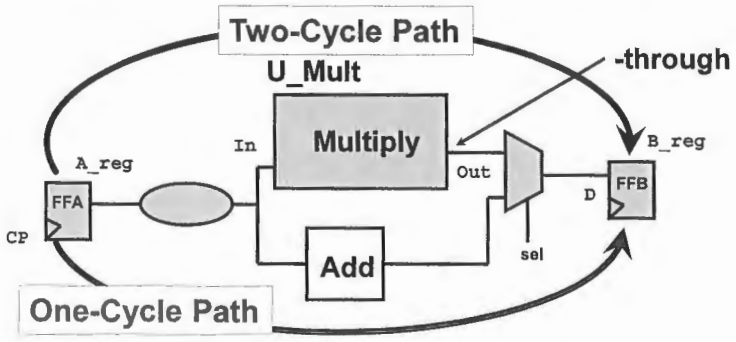
7-37

The default hold-check edge is one clock edge before the setup-check edge. By increasing the *setup* multiplier from the default of 1, the clock edges at which the setup check as well as the hold check are performed are increased by that many clock cycles. Conversely, by increasing the *hold* multiplier from the default value of 0, the edge at which the hold check is performed is *decreased* by that many clock cycles. So, if you wish to move ONLY the setup-check edge, while keeping the hold-check edge in its original position, always set the *hold* multiplier to one less than the *setup* multiplier.

©2018 Synopsys, Inc. All Rights Reserved.

Another Example

```
set_multicycle_path -setup 2 -through U_Mult/Out
set_multicycle_path -hold 1 -through U_Mult/Out
```



7-38

For situations when there are multiple paths that go through at a given point, you may need to specify a specific start and/or endpoint to isolate the path, for example:

```
set_multicycle_path -setup 2 -from A_reg/clocked_on \
   -through U_Mult/Out -to B_reg/next_state
set_multicycle_path -hold 1 -from A_reg/clocked_on \
   -through U_Mult/Out -to B_reg/next_state
```

Note: If the "Multiply" block is auto-ungrouped during optimization, the "timing exception" (multi-cycle or false-path) constraints are automatically "pushed down" to the appropriate elements inside the block, so the path remains correctly constrained.

©2018 Synopsys, Inc. All Rights Reserved.

Always Check for Invalid Exceptions

No warnings are issued if an invalid exception is applied to a design, so it is recommended to explicitly check for invalid exceptions:

```
report_timing_requirements -ignored
```

```
Description                                  Setup          Hold
-------------------------------------------------------------------
NONEXISTENT PATH                             FALSE          FALSE
        -from     { CLK1_B}
        -to       { CLK2_A\
                    CLK2_D}
INVALID FROM OBJECT                            6              5
        -from     FF1_reg/Q
```

To remove and correct invalid exceptions:

```
reset_path -from FF1_reg/Q; # OR
set_multicycle_path -reset_path -from FF1_reg/Q
set_multicycle_path -from FF1_reg/clocked_on
```

7-39

There is no **report_timing_requirements -valid**! **report_timing_requirements** reports ALL exceptions, valid and invalid ones.

```
dc_shell-topo> report_timing_requirements  -help
Usage: report_timing_requirements    # report timing_requirements
        [-attributes]           (path timing attributes)
        [-ignored]              (ignored path timing attributes)
        [-from <from_list>]     (from clocks, cells, pins, or ports)
        [-through <through_list>]
                                (list of path through points)
        [-to <to_list>]         (to clocks, cells, pins, or ports)
        [-expanded]             (report exceptions in expanded format)
        [-nosplit]              (do not split lines when column fields overflow)
```

NONEXISTENT PATH could have been the result of issuing:

```
set_false_path -from [get_clocks CLK1_*] -to [get_clocks CLK2_*]
```


©2018 Synopsys, Inc. All Rights Reserved.

Summary

You should now be able to constrain a design with:

- Multiple synchronous clocks
- Clocks derived from sequential logic
- Mutually exclusive synchronous clocks
- Asynchronous clocks
- Multi-cycle timing

7-40

©2018 Synopsys, Inc. All Rights Reserved.

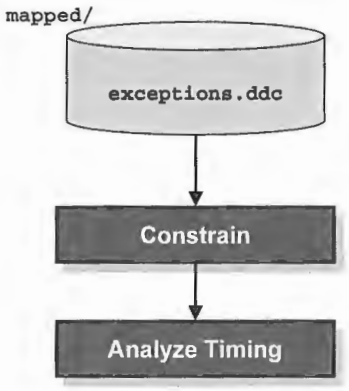
Lab 7: Constraints: Multiple Clocks and Exceptions



Lab 7: Constraints: Multiple Clocks and Exceptions

60 minutes

Constrain a multi-clock design using virtual clocks and timing exceptions



7-41

©2018 Synopsys, Inc. All Rights Reserved.

This page was intentionally left blank

©2018 Synopsys, Inc. All Rights Reserved.

Agenda

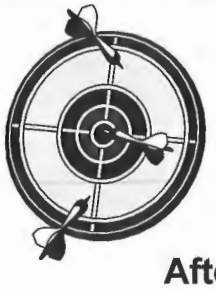
DAY 3

6	Timing Analysis
7	Constraints: Multiple Clocks and Exceptions
8	DC Graphical: SPG Flow, Congestion, Layout GUI
9	Constraints: Complex Design Considerations
10	Post-Synthesis Output Data
11	Conclusion

Synopsys 10-I-011-SSG-024 © 2018 Synopsys, Inc. All Rights Reserved

©2018 Synopsys, Inc. All Rights Reserved.

Unit Objectives



After completing this unit, you should be able to:

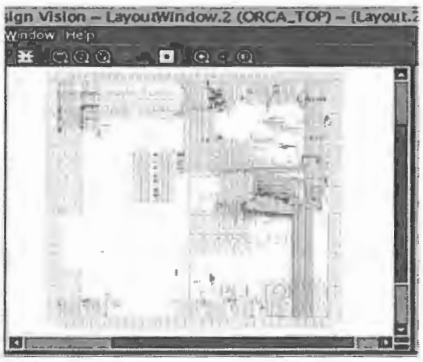
- Describe the features of *DC Graphical*
- Execute the *Synopsys Physical Guidance* flow
- Control *SPG* optimization features
- Analyze congestion

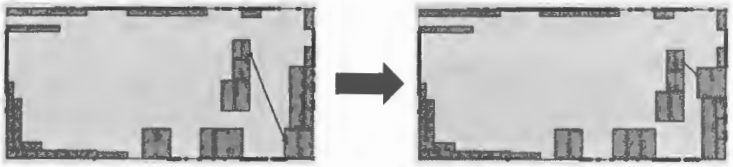
8-2

©2018 Synopsys, Inc. All Rights Reserved.

What is DC-Graphical?

- **DC-Graphical (DC-G) is a "package" of features:**
  - *Synopsys Physical Guidance* (*SPG*) flow
  - Layout visualization and congestion analysis
  - *Floorplan Exploration*
    - Create/modify a floorplan in ICC (II)





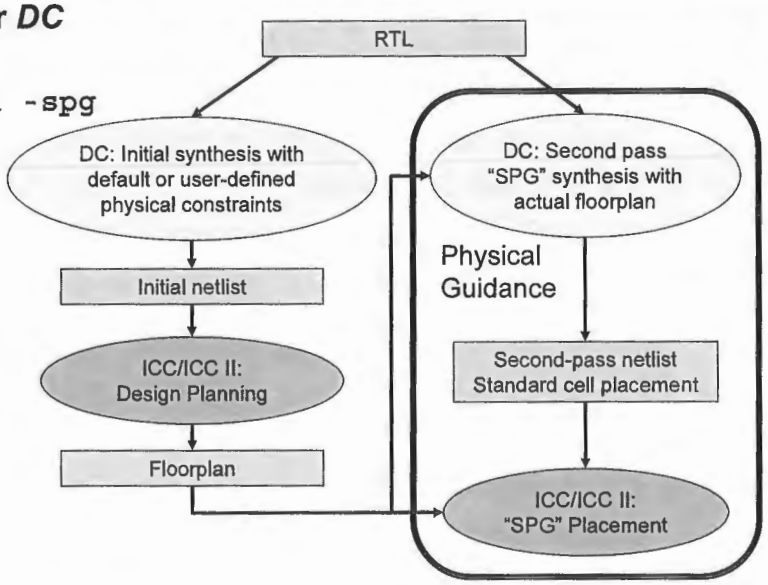
- **Can improve timing, congestion and post-placement timing correlation**
- **DC-G is enabled through the DC-Extension license**

8-3

©2018 Synopsys, Inc. All Rights Reserved.

Synopsys Physical Guidance (SPG) Flow

- **The "SPG Flow" is recommended for *DC* and *ICC/ICC II***
  - Enabled in DC with `compile_ultra -spg`
  - Used during 2nd pass synthesis
- **The `-spg` option enables:**
  - Automatic register duplication
    - Improved timing
  - Net layer optimization
    - Improved timing
  - Netlist topology optimization
    - Improved congestion
  - Physical guidance to ICC (II)
    - Improved timing correlation



8-4

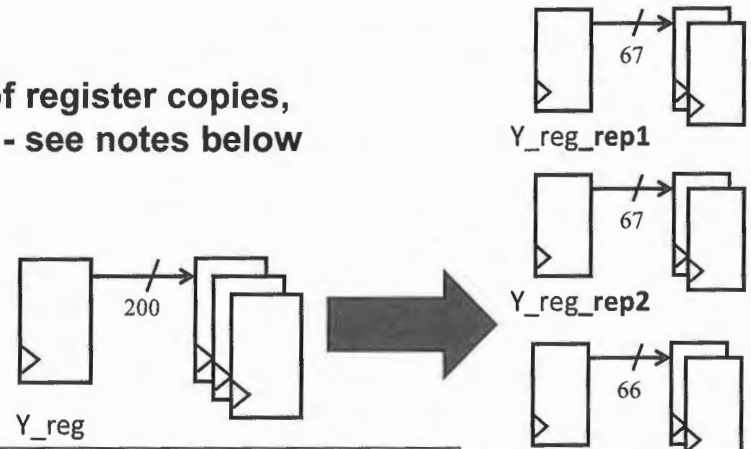
Using the `-spg` option with `compile_ultra` requires *DC-Graphical* (DC-G) capability, which is enabled with a *DC-Extension* license.

©2018 Synopsys, Inc. All Rights Reserved.

Automatic Register Replication

- ***Register replication*** **is used to improve critical path timing**
- **Automatically enabled with `compile_ultra -spg`**
- **User can control the number of register copies, or limit the fanout per register - see notes below**

Replicated register names, by default, are appended with `_rep#` -- See Notes



Register replication does not occur with scan-stitched designs, constant registers, retimed registers, multi-bit registers, registers with `dont_touch` or `size_only` attribute

8-5

Register replication is enabled by default with `compile_ultra -spg`. This is because of the `compile_register_replication` variable, which is set to a value of `default`, by default. To turn off register replication in the "SPG flow" apply `set_app_var compile_register_replication false`. In a non-SPG flow, the user can manually enable register replication with:

```
set_app_var compile_register_replication true
```

The replication is driven solely by timing. The user can control selected registers, to define either the number of registers to be duplicated, or to limit the maximum fanout per register. Using the example above:

Make 4 copies of `Y_reg` (each new register ends up driving a fanout of 50):

```
set_register_replication -num_copies 3 [get_cells Y_reg*]
```

Ensure each register does not fan out to more than 40 (you therefore get 5 copies of `Y_reg`):

```
set_register_replication -max_fanout 40 [get_cells Y_reg*]
```

Find out what register replication setting is applied to `Y_reg`:

```
get_attribute [get_cells Y_reg*] register_replication
```

Control the naming convention of duplicated registers:

```
set_app_var register_replication_naming_style "%s_rep%d"
```

Remove any manually applied register replication settings:

```
remove_attribute [get_cells Y_reg*] register_replication
```

If `set_register_replication` is applied without `-num_copies` or `-max_fanout` options, a default `-num_copies` of 2 is used. If both options are applied simultaneously, `-num_copies` has higher priority over `-max_fanout`.

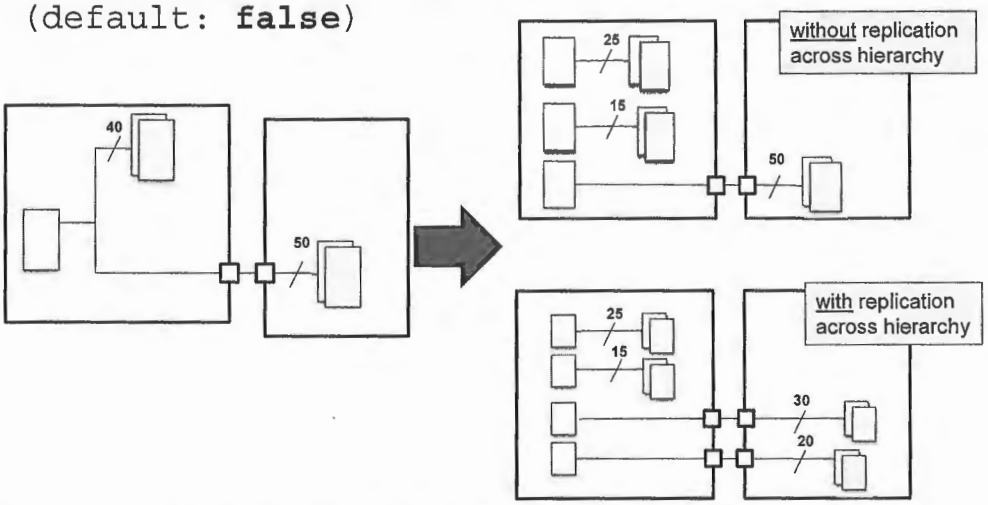
©2018 Synopsys, Inc. All Rights Reserved.

Register Replication Across Hierarchy

Register replication **considers loads across hierarchies, if enabled:**

```
set_app_var compile_register_replication_across_hierarchy true
```

(default: **false**)



8-6

In the example on this, and the previous page, it is assumed that there is combinational logic between the registers, which will also be duplicated along with the split registers.

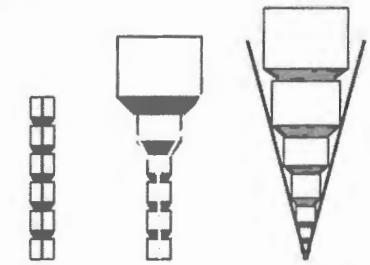
Automatic register replication (during `compile_ultra -spg`) is timing-driven, by default. In the example above, notice that the fanouts of the split registers are not necessarily equally balanced. This is because the number of split registers and the fanout of each split register are governed by critical timing (taking placement into consideration). This means that more timing-critical end-point registers may be grouped into a smaller fanout branch than less timing-critical endpoint registers.

©2018 Synopsys, Inc. All Rights Reserved.

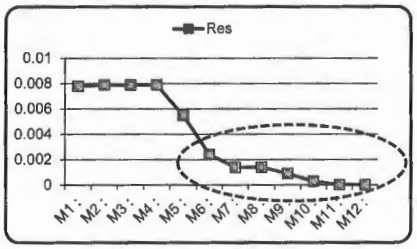
Problem: Resistance Variation Across Metal Layers

Metal layer resistance varies substantially in smaller (sub 45nm) geometries

- 500-1000X variation possible between lowest and highest metal layers
- ICC/ICC II *detail route* tends to route long nets on upper layers (low R)
- By default, DC-T estimates net delays using an average R of all metal layers
- Can lead to increased buffer count and sub-optimal timing during synthesis



Metal stack trends



Smaller resistance across layers

8-7

Metal layer resistance varies substantially in smaller technology nodes, impacting pre-route R estimation, especially for long nets. This can lead to: increased timing violations and buffer count, and reduced correlation with post-route results.

```
****************************************************************
Information: TLUPlus based RC computation is enabled. (RCEX-141)
****************************************************************
Information: The distance unit in Cap and Res is 1 micron. (RCEX-007)
Information: The RC model used is TLU+. (RCEX-015)
Information: Library Derived Res for layer M1 : 420 420 (RCEX-011)
...
Information: Library Derived Res for layer M7 : 2 2 (RCEX-011)
Information: Library Derived Res for layer M8 : 1 1 (RCEX-011)
...
Information: Library Derived Horizontal Res : 167.5 167.5 (RCEX-011)
Information: Library Derived Vertical Res : 245.2 245.2 (RCEX-011)
```


©2018 Synopsys, Inc. All Rights Reserved.

Solution: Net Layer Optimization (NLO)

Three types of layer optimization are available to address metal stack trends in sub-45 nm:

1. **Automatic NLO**
2. **Pattern-based NLO**
3. **Manual NLO**

8-8

©2018 Synopsys, Inc. All Rights Reserved.

1. Automatic Net Layer Optimization

```
compile_ultra -spg
```

- **Long, timing-critical nets are automatically promoted to the two upper available layers**
  - RC average based only on promoted layers
  - Honors existing `set_ignored_layers` settings
  - Only supported when using the `-spg` option
  - ON by default – can not be turned off
- **Can enable promotion of nets to multiple layers:**
  - *Zroute global routing* identifies the set of nets to promote, based on net length and timing, and determines the exact range of layers for each net (runtime intensive)

```
set_app_var spg_enable_zroute_layer_promotion true
```

8-9

©2018 Synopsys, Inc. All Rights Reserved.

2. Pattern-Based NLO

Allows more control on nets to be assigned to specific layers, if identified by "physical design" team:

- Identify known problematic nets by creating "net search patterns" based on:
  - Fanout, Net length, Timing violations, Connection to I/O ports or macros, ...
- Assign minimum or maximum layer to those nets
- Run layer optimization during compile

```
create_net_search_pattern ...
set_net_search_pattern_delay_estimation_options...
compile_ultra -spg
```

8-10

```
create_net_search_pattern
        [-fanout_lower_limit number]
        [-fanout_upper_limit number]
        [-bbox_half_perimeter_lower_limit length]
        [-bbox_half_perimeter_upper_limit length]
        [-net_length_lower_limit length]
        [-net_length_upper_limit length]
        [-blocked_area_ratio_lower_limit percentage]
        [-blocked_area_ratio_upper_limit percentage]
        [-centered_within {llx lly urx ury}]
        [-aspect_ratio_lower_limit ratio]
        [-aspect_ratio_upper_limit ratio]
        [-connect_to_port]
        [-connect_to_macro]

report_net_search_pattern [-pattern | -all]
remove_net_search_pattern [-pattern | -all]
```


©2018 Synopsys, Inc. All Rights Reserved.

2. Pattern-Based NLO Examples

```
remove_net_search_pattern -all

set PATTERN_FO [create_net_search_pattern -fanout_lower_limit 150]
set_net_search_pattern_delay_estimation_options \
       -min_layer_name M6 -max_layer_name M8 -pattern $PATTERN_FO

set PATTERN_NL [create_net_search_pattern -net_length_lower_limit 100]
set_net_search_pattern_delay_estimation_options \
       -min_layer_name M7 -max_layer_name M8 -pattern $PATTERN_NL

set PATTERN_CRIT_IO [create_net_search_pattern \
       -setup_slack_upper_limit -1.0 -connect_to_port]
set_net_search_pattern_delay_estimation_options \
       -min_layer M7 -max_layer M10 -pattern $PATTERN_CRIT_IO

compile_ultra -spg
```

8-11

Net length units are microns.

©2018 Synopsys, Inc. All Rights Reserved.

3. Manual NLO

- **Automatic and pattern based layer assignments can be refined manually**
  - Assign minimum and maximum layers to specific nets
- **Recommended on a mapped netlist for incremental optimization or RC estimation**

```
set_net_routing_layer_constraints \
    -min_layer_name M5 \
    -max_layer_name M6 [get_nets top/sub/net1]

compile_ultra -incremental -spg
  OR
extract_rc -estimate
```

8-12

```
set_net_routing_layer_constraints
   -min_layer_name min_layer_name (Minimum routing layer name)
   -max_layer_name max_layer_name (Maximum routing layer name)
    object_list                   (nets)
```


©2018 Synopsys, Inc. All Rights Reserved.

NLO Results Passed on to ICC/ICC II

- **Net layer assignments are automatically passed on to ICC with a *ddc* netlist handoff**
  - To see the net layer assignments from DC:

```
report_net_routing_layer_constraints \
    [get_nets -hier] -output NLO.tcl
```

- **The `write_icc2_files` command generates a file containing net layer assignments for ICC II**
- **These routing rules are followed by ICC/ICC II from the placement through the routing phases**

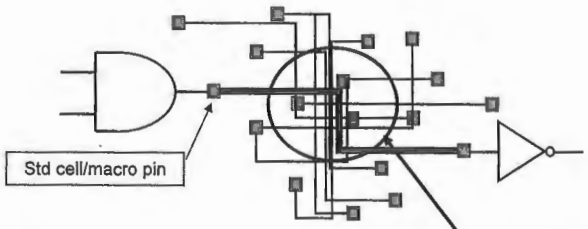
8-13

See application note on SolvNet: "How to Implement Net Layer Optimization with Design Compiler Graphical" https://solvnet.synopsys.com/retrieve/037946.html

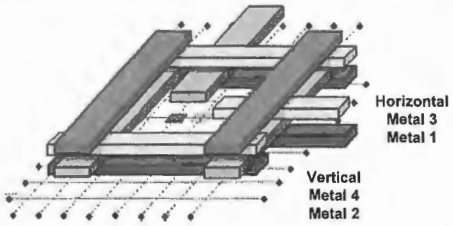
©2018 Synopsys, Inc. All Rights Reserved.

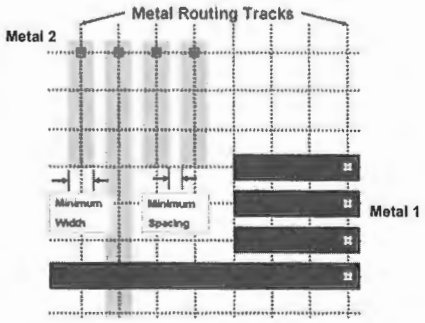
What is Congestion?

It is a measure of a design's "routability", and is analyzed prior to actual routing



There is a limit to the number of nets that can be routed through a given small area





When you approach or exceed this limit, this area is said to be congested

8-14

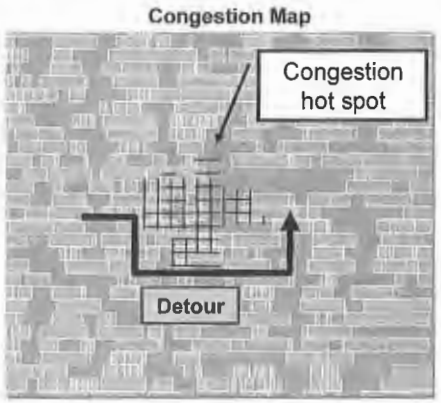
Each foundry's "technology" dictates the number of available routing metal layers, and the minimum width and minimum spacing rules per layer, which results in limited routing resource. Congestion arises when a router tries to use a routing track in an area where all routing tracks have been used.

By default metal 1, 3, 5, etc. are used to route in the horizontal direction, while metal 2, 4, 6, … are used to route vertically. These are called the "preferred routing directions.

©2018 Synopsys, Inc. All Rights Reserved.

Detouring Around Localized Congestion

- **If congestion is localized and not too severe, the actual route can be detoured around the congested area**
- **Detoured nets will have slightly increased RC delays**
  - Should not cause any problems for non timing-critical paths



8-15

©2018 Synopsys, Inc. All Rights Reserved.

The Problem with Severe Congestion

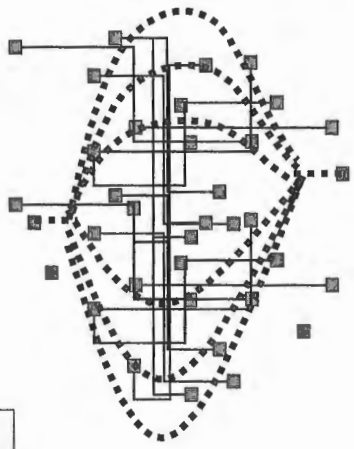
Severe congestion can cause timing violations, or cause a design to be un-routable

Requires design iterations, which increase the design cycle:

- Either floorplan modifications, then new Place&Route
- Or worse - RTL re-coding, then re-synthesis + Place&Route



The goal for synthesis: Eliminate serious congestion prior to placement to prevent iterations

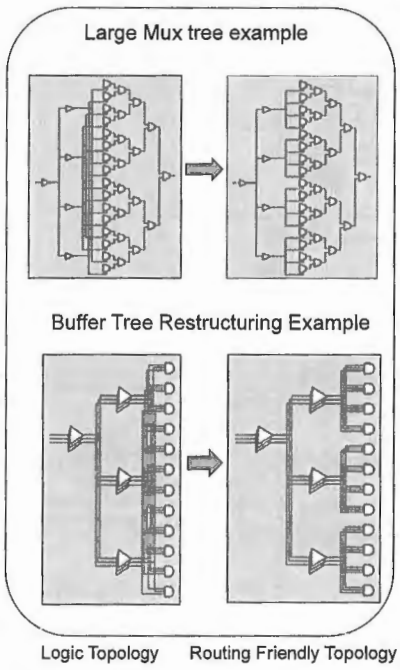


8-16

In severely congested areas the router may not be able to find a detour. This will require a floorplan change, followed by another "place and route" (increased layout design cycle - bad), or worse, a change in the RTL code, which requires re-synthesizing the design, possibly re-floorplanning, and then place& route (increased synthesis + layout design cycle – really bad).

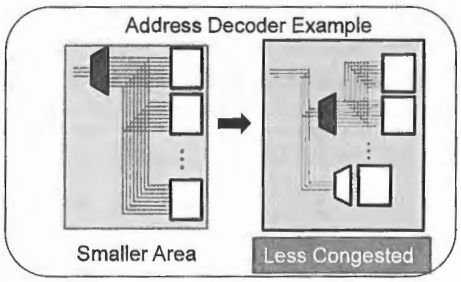
©2018 Synopsys, Inc. All Rights Reserved.

Netlist Topology Congestion Optimization 1/3




```
compile_ultra ... -spg
```

- **Makes routing-friendly netlist "topology" or "structural" changes**
  - Minimizes highly connected structures
  - Minimizes wires and wire crossing
  - Shares and un-shares logic as needed
- **Concurrent optimization with timing, area, power and scan chains**



8-17

Congestion-based optimization in Design Compiler Graphical (enabled with `-spg`) primarily addresses scenarios where the design's RTL logic structures result in congestion. This standard cell congestion is often caused by the topology of the netlist. Minimizing the resulting congestion for such designs often require significant netlist topology changes that can not be performed during the place and route stage, and is best done early in the flow .i.e. during synthesis.
Congestion optimization performs congestion-aware logic-based netlist optimizations during synthesis with minimal impact on timing, area, power and scan chains. There is no need to change user constraints or restrict cell selection: This feature is tuned to work with all available target library cells to identify the optimal structure and mapping and generate a netlist topology with reduced congestion.

Example on the left: It tries to minimize the number of wires and wire crossings.

Example on the right: Some signals such as address lines of a decoder are used in many places in a chip. If the logic involving these signals is shared, many more signals may need to be routed around the chip which can lead to congestion. Congestion optimization can identify this type of shared logic and un-share it to alleviate congestion.

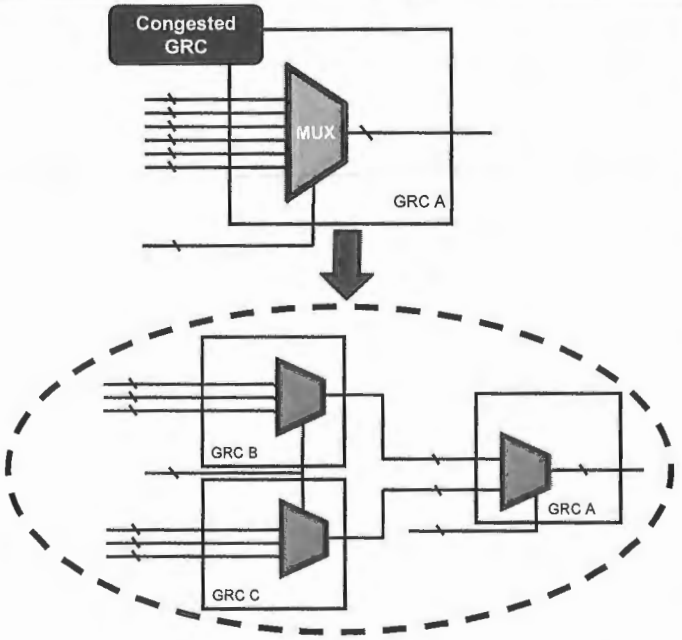
©2018 Synopsys, Inc. All Rights Reserved.

Netlist Topology Optimization Examples 2/3

- **Netlist topology optimization restructures existing high-input MUXes in the netlist into MUX trees**

- **Additionally, apply the following before compile to favor the use of MUXes over AOIs for multiplexers, to further improve congestion:**

```
set_app_var compile_prefer_mux true
```



8-18

Lookup table logic (coded as case statements) can create highly connected sub graphs in the netlist with its many-to-many mapping topology.
Large MUX structures (many ways to select, large busses) cause many high-fanout nets.

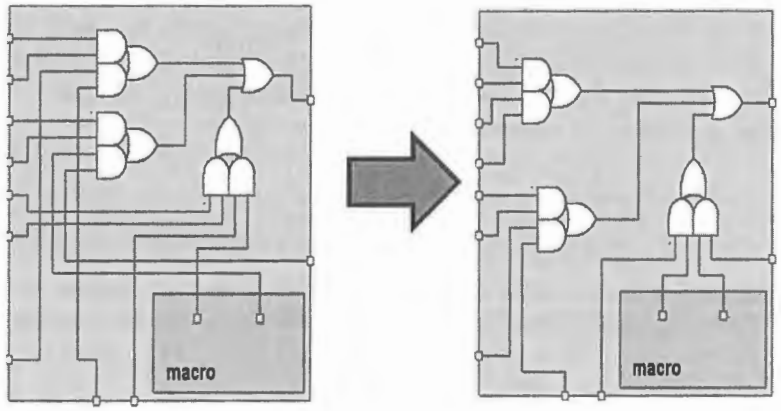
When you set the `compile_prefer_mux` variable to `true`, the `compile_ultra` command runs a strategy intended to improve the congestion of designs that contain multiplexing logic, possibly at the cost of additional runtime and some area. The default flow typically maps most multiplexers to and-or-invert (AOI) logic in order to minimize area, but in some cases this can result in congestion hotspots. With `compile_prefer_mux` enabled, multiplexing logic that is likely to cause congestion is converted to MUX trees where possible.

©2018 Synopsys, Inc. All Rights Reserved.

Netlist Topology Optimization Examples 3/3

Associative-Commutative Tree Decomposition

Improved congestion by decomposing and restructuring AND/OR trees based on port or macro pin locations



8-19

©2018 Synopsys, Inc. All Rights Reserved.

Targeting/Limiting Congestion Optimization

- **`compile_ultra -spg` targets the entire current design for structural congestion optimization, by default**
- **The `set_congestion_optimization` command can be used to limit or focus which sub-blocks are targeted**

Perform congestion optimization on all sub-blocks except the specified design:

```
set_congestion_optimization [get_designs A] false
compile_ultra ... -spg
```

Perform congestion optimization only on the specified designs/cells:

```
set_congestion_optimization [get_designs *] false
set_congestion_optimization [get_designs "A D"] true
set_congestion_optimization [get_cells I_X4] true
compile_ultra ... -spg
```

8-20

The **`set_congestion_optimization`** command applies **`congestion_optimization`** attribute on the specified designs or cells in current design. By default, this attribute does not exist on designs and cells. Any sub-block without this attribute, or with a **`congestion_optimization`** value of **`true`**, will undergo structural congestion optimization. Any sub-block with this attribute value set to **`false`** will not undergo structural congestion optimization.

To determine if the attribute exists on a design or cell, and what its value is:

```
get_attribute [get_designs ABC] congestion_optimization
get_attribute [get_cells XYX] congestion_optimization
```


©2018 Synopsys, Inc. All Rights Reserved.

Settings that Affect Placement/Congestion

- **The following command and variable settings affect coarse placement during compile and incremental compile (if enabled), which can directly or indirectly affect congestion**
- **Experiment with these settings to improve congestion** (use `man` to learn more)

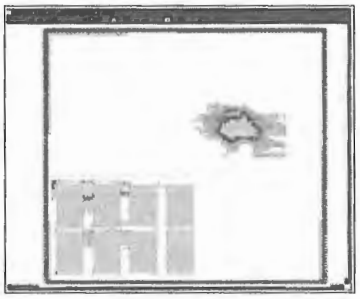
```
set_app_var placer_channel_detect_mode                          false | true
set_app_var placer_congestion_effort                           auto | medium
set_app_var placer_enable_enhanced_router                       false | true
set_app_var placer_enable_enhanced_soft_blockages               false | true
set_app_var placer_enable_redefined_blockage_behavior           false | true
set_app_var placer_max_cell_density_threshold          -1 | (0<=value<=1.0)
set_app_var placer_reduce_high_density_regions                  false | true
set_app_var placer_tns_driven                                   false | true
set_app_var placer_tns_driven_in_incremental_compile            false | true
set_app_var spg_congestion_placement_in_incremental_compile     false | true
set_app_var spg_high_effort_mux_area_structuring                false | true
set_app_var spg_place_enable_precluster                         false | true
set_congestion_options [-max_util] [-layer] [-availability] [-coordinate]
compile_ultra ... [incremental]
```

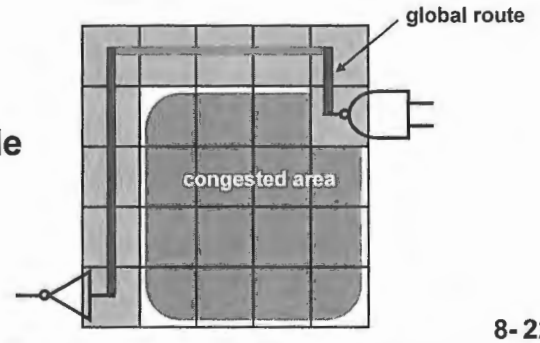
8-21

©2018 Synopsys, Inc. All Rights Reserved.

Congestion Analysis After Synthesis

- **After synthesis, congestion can be visually displayed as a "heat map" from the layout window**
- **Congestion analysis is based on *global routing***
  - Core placement area is divided into "global routing cells" (GRCs)
  - Each GRC has a limited number of horizontal and vertical routing tracks along its edges
  - Global routing goes around congested GRCs
- **Congestion exists along a GRC edge when more tracks are needed than available**





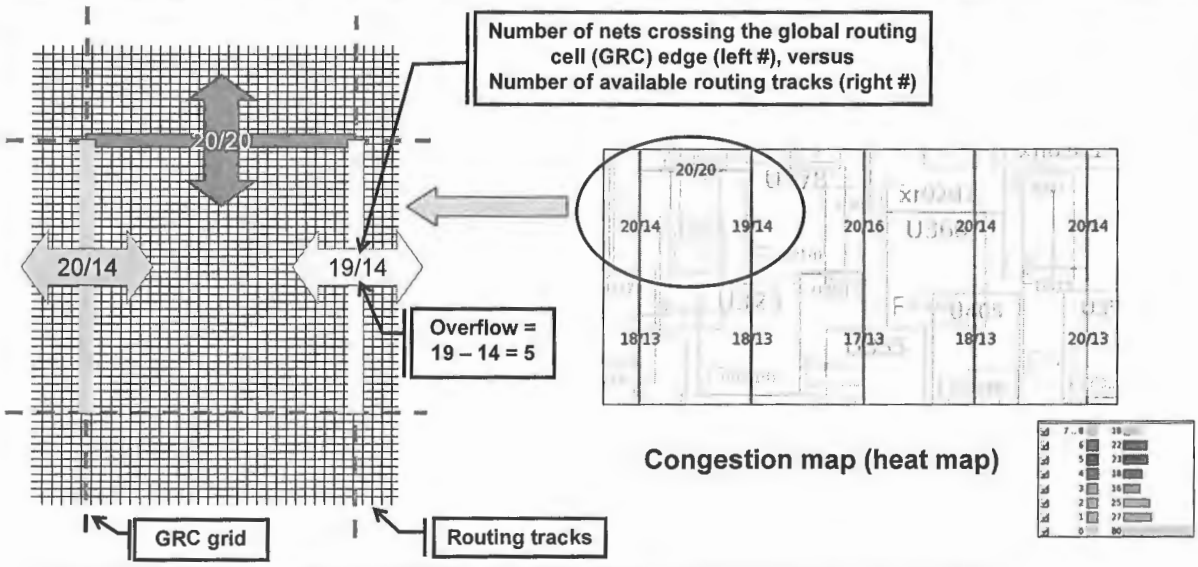
8-22

The coarse placement and global routing technology used by Design Compiler Topographical is the same technology that is used by Synopsys' physical design tool, IC Compiler / IC Compiler II.

Congestion reporting uses *Zroute* global routing for improved correlation with *IC Compiler / IC Compiler II* for complex floorplans and rectilinear cores.

©2018 Synopsys, Inc. All Rights Reserved.

Understanding the Congestion "Heat Map"



The size of the 'overflow' determines the highlighting color – larger overflows are brighter or 'hotter'

8- 23

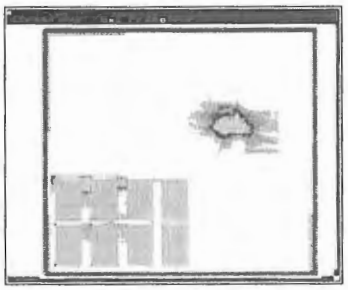
©2018 Synopsys, Inc. All Rights Reserved.

Textual Congestion Analysis

A textual congestion report can also be generated with `report_congestion`

Overall Congestion	Worst Hotspot	Violating GRCs
Both Dirs: Overflow = 3621	Max = 13 (1 GRCs)	GRCs = 2247 (2.73%)
H routing: Overflow = 1724	Max = 13 (1 GRCs)	GRCs = 1139 (1.38%)
V routing: Overflow = 1897	Max = 8 (1 GRCs)	GRCs = 1108 (1.34%)

Congestion Direction	Overflow: Total # wires for which a GRC routing resource was not available	Maximum overflows for a single GRC	# GRCs having maximum overflow	# GRCs with *any* overflow	Percentage Violating GRCs to Total GRCs



Congestion can be serious if one or more of the following are true:

- **Large or many "hotspots" in the congestion map**
- **Any GRC "overflow" larger than around 10**
- **About ~2%, or more, GRC edges have an overflow**

8-24

In the example below we see the textual output of `report_congestion`. In this example the "`Max`" overflow is 13. This means that there is at least one GRC edge with an overflow of 13, for example "19/6". This means that there are 13 routes that want to route through a specific GRC area which do not have any available routing resources. In practice, any maximum GRC overflow of around 10, or larger, has a very high probability of resulting in unroutable (shorts or opens) nets in that area. The total number of GRC overflows is 2.73%. This means that 2.73% of the total number of GRC edges in the design have an overflow of 1 or more. In practice, if this number is around 2% or more (this is a very rough guideline and is very design-dependent), this can result in poor routing overall, which could manifest itself in signal integrity (e.g. crosstalk) problems or poor timing. The number of violating GRCs (`2247`) can be misleading as a measure of design congestion. A design with 100 GRCs, each with an overflow of 1, would have the same "GRC" value as a design with 100 GRCs, each with an overflow of 50. A revised design may result in increased or decreased overflows, better or worse hotspots. A better metric to compare revised synthesis results using the textual congestion report is to compare the average violation per GRC: `Overflow/#GRCs` or `3621/2247=1.61` in this example.

```
****************************************
Report : congestion
****************************************
Both Dirs: Overflow = 3621 Max = 13 (1 GRCs) GRCs = 2247 (2.73%)
H routing: Overflow = 1724 Max = 13 (1 GRCs) GRCs = 1139 (1.38%)
V routing: Overflow = 1897 Max =  8 (1 GRCs) GRCs = 1108 (1.34%)
```


©2018 Synopsys, Inc. All Rights Reserved.

Opening the Layout Window

After synthesis is completed open the GUI and follow these steps:

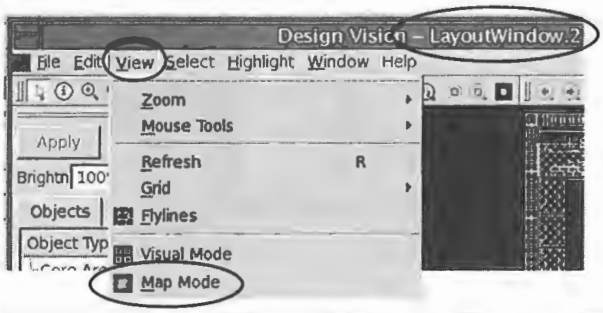
1



2



3



8-25

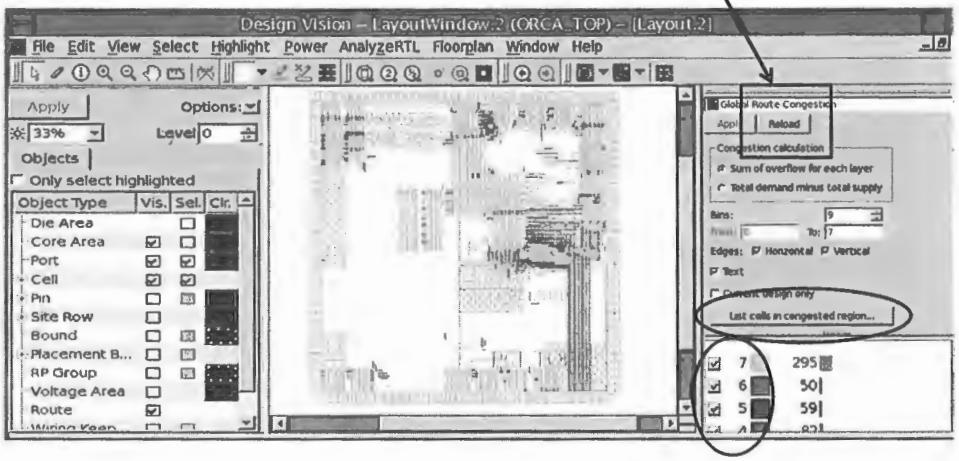
This requires DC-Graphical licensing (DC-Extension feature)

©2018 Synopsys, Inc. All Rights Reserved.

Generating a Congestion Heat Map

4

Select the *Reload* button to generate a congestion map

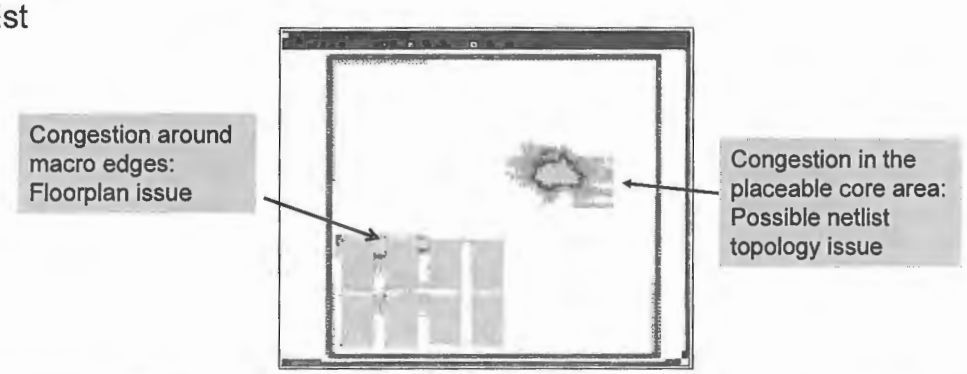


8- 26

©2018 Synopsys, Inc. All Rights Reserved.

Analyzing Congestion in the GUI

- **Use the congestion heat map to determine:**
  - If your design has serious congestion that needs to be addressed (see previous "Congestion Guidelines")
  - What is causing the various congestion areas and how best to address them
- **Congestion can stem from two main sources:**
  - The floorplan
  - The netlist



8-27

©2018 Synopsys, Inc. All Rights Reserved.

Improved Congestion Correlation

Congestion reporting uses *global routing* for improved correlation with *ICC (II)*, especially for rectilinear cores and complex floorplans

For best correlation, do not use any switches other than **`-effort`** with **`report_congestion`** in *ICC* (*ICC II* has no *effort* option):

`report_congestion [-effort minimum | low | medium | high ]` (default: **`medium`**)

For best correlation, use same global and common route settings in *DC-G* as in *ICC* (see notes below for *ICC II*):

`set_route_zrt_common_options`
`set_route_zrt_global_options`

8-28

The following commands are supported for common route options:

```
set_route_zrt_common_options
get_route_zrt_common_options
report_route_zrt_common_options
```

The following commands are supported for global route options:

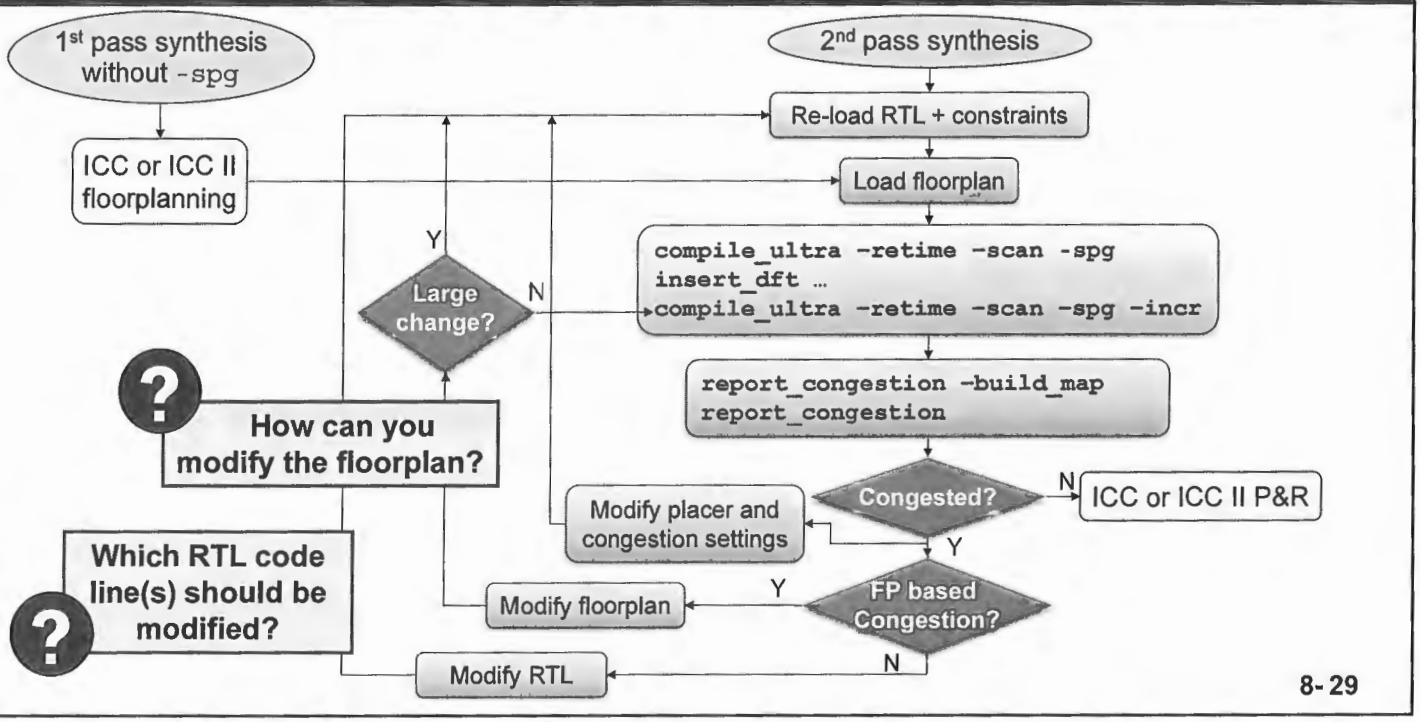
```
set_route_zrt_global_options
get_route_zrt_global_options
report_route_zrt_global_options
```

In ICC II, common and global route settings are controlled by specifying the following *application options*. To ensure best correlation, apply the equivalent `set_route_common/global_options` commands in DC:

```
route.common.*
route.global.*
```


©2018 Synopsys, Inc. All Rights Reserved.

Congestion Alleviation Flow

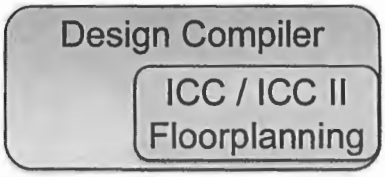


8-29

©2018 Synopsys, Inc. All Rights Reserved.

Floorplan Exploration in DC-G

- **Design Compiler Graphical allows access to IC Compiler (II) Design Planning**
  - Called "Floorplan Exploration Flow" or "DCG-DP Link"
- **RTL designers can**
  - Create new floorplan
  - Modify existing floorplan



- **Benefits**
  - What-if analysis for physical exploration
  - Enables optimal placement of macros and ports

8-30

©2018 Synopsys, Inc. All Rights Reserved.

Example: Modifying Floorplan in DC-G

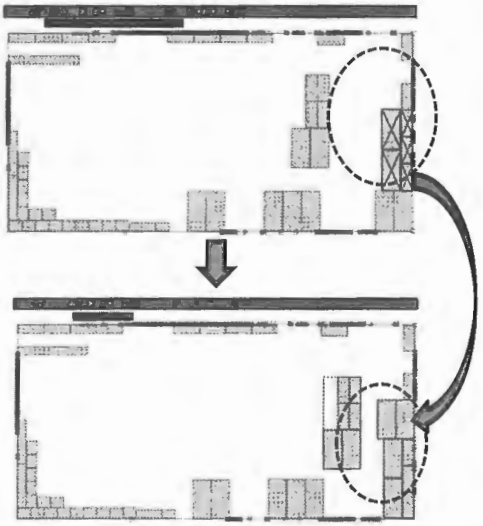
Long Net length



DC Graphical: Improved timing



DCG-DP link:
select and move macros as group



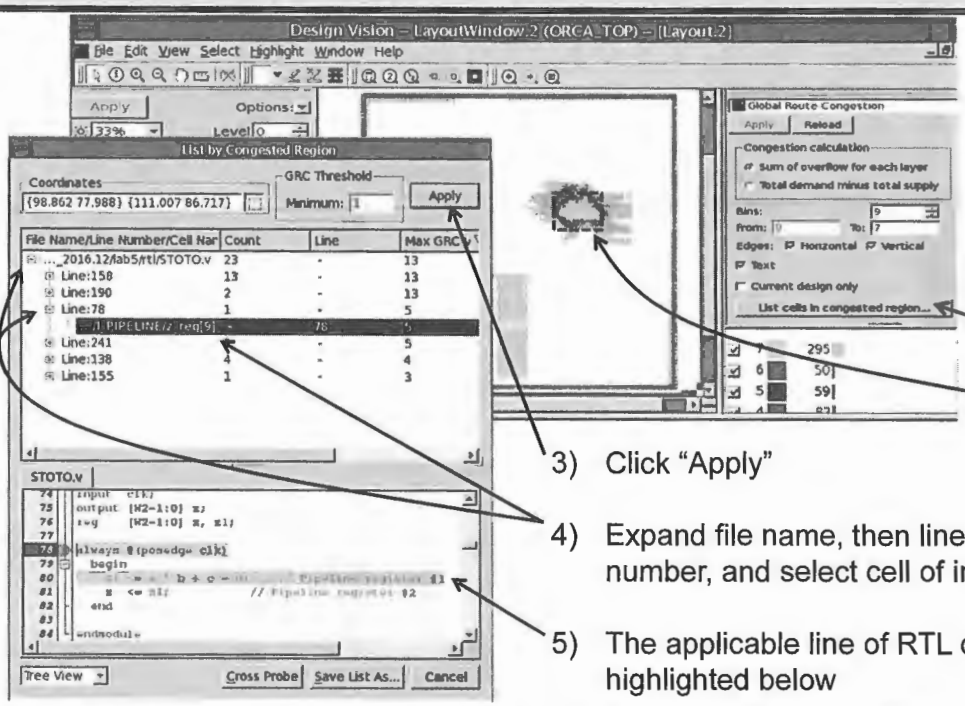


See Appendix for details on invoking the DCG-DP link

8-31

©2018 Synopsys, Inc. All Rights Reserved.

Trace from GUI back to RTL source



1) Click "List cells …"

2) Select area of interest

3) Click "Apply"

4) Expand file name, then line number, and select cell of interest

5) The applicable line of RTL code is highlighted below

8-32

Try to minimize structures that are concentrated areas with much data throughput

Lookup table logic

- Coded as case statements
- Create highly-connected structures in the netlist

Large MUX trees

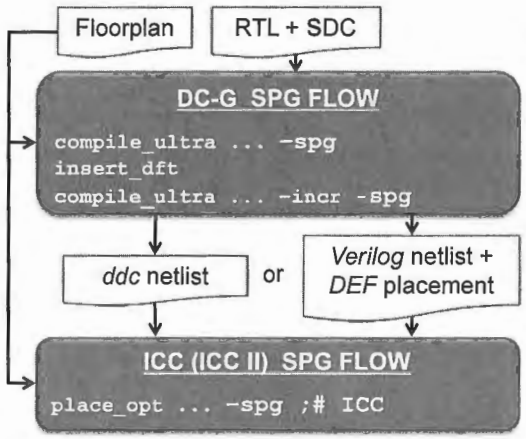
- Many ways to select
- Large busses
- Cause many high-fanout nets

Large sums of products (ROMS)

©2018 Synopsys, Inc. All Rights Reserved.

Physical Guidance

- **Physical guidance allows:**
  - Coarse placement to be written to *ddc* or *DEF* for ICC (II)
  - *SPG* placement in ICC (II) to use DC-G placement as the start-point for optimization and legalization
- **Benefits:**
  - Improved timing correlation between DC and ICC (II)
  - Improved `place_opt` runtime (coarse placement is skipped)



8-33

Synthesis with `compile_ultra -spg` performs timing- and congestion-driven coarse placement (ideal, but not legal cell placement), similar to coarse placement in *IC Compiler/IC Compiler II*, and allows the placement to be saved with the *ddc* netlist or in a *DEF* file. Synthesis in topographical mode, without the `-spg` option, performs coarse placement, but does not allow the placement to be written out. The `-spg` option also performs netlist topology or structural congestion optimization during compile, which will be explained in the next few pages.

In ICC/ICC II, the `place_opt` command also performs timing- and congestion-driven coarse placement, by default, followed by AHFS or automatic high-fanout net synthesis (buffering of high fanout nets like enable, set/reset, select), placement legalization (moving cells into placement rows, separating overlapping cells, moving cells out of placement blockage areas), and logic optimization (cell sizing, buffer insertion, pin swapping). With *SPG* `place_opt`, coarse placement is skipped, but it performs the other optimization and legalization steps, using DC-G's coarse placement as its startpoint. This helps to achieve better timing and area correlation between DC and ICC/ICC II, and also reduces ICC/ICC II placement runtime.

In *ICC*, SPG placement is performed by:

```
place_opt -spg
```

In *ICC II*, SPG placement is performed by:

```
set_app_options -name place_opt.flow.do_spg -value true
place_opt
```


©2018 Synopsys, Inc. All Rights Reserved.

Improved Timing Correlation with ICC or ICC II

- **Apply the following command before compile – may improve timing correlation between DC and ICC or ICC II:**

```
set_compile_spg_mode [ICC | ICC2]
compile_ultra ... -spg
```

- **Aligns DC's timer settings with those of ICC (II)**
  - When targeting ICC II, for example, the enhanced physical guidance command has the following effects in DC:
    - Enables enhanced timing modeling for RC estimation
    - Removes long net pessimism
    - Enables placement pre-clustering
    - Enables recovery removal checks
- **Does not override user-applied settings**

8- 34

©2018 Synopsys, Inc. All Rights Reserved.

Summary

You should now be able to:

- Describe the features of *DC Graphical*
- Execute the *Synopsys Physical Guidance* flow
- Control *SPG* optimization features
- Analyze congestion

Reference document: Search for "*Using Synopsys Physical Guidance*" in the "Design Compiler User Guide"

8-35

©2018 Synopsys, Inc. All Rights Reserved.

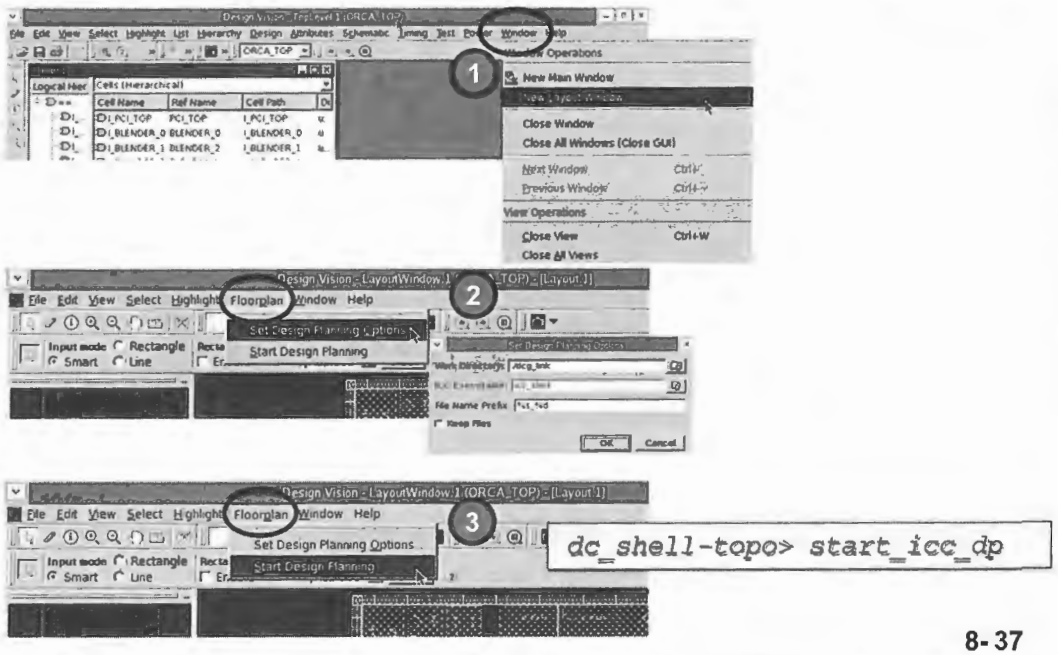
Appendix

Invoking the DCG-DP Link for ICC (and ICC II)

©2018 Synopsys, Inc. All Rights Reserved.

Invoking ICC Design Planning

- **Windows → New Layout Window**
- **Floorplan → Set Design Planning Options**
- **Floorplan → Start Design Planning**



8-37

The DCG-DP link for IC Compiler II is invoked (step 3 above) using `start_icc2`.

"Set Design Planning Options"

Work Directory

For storing the temporary files like scripts, floorplan files

ICC Executable

For specifying the path to the IC Compiler (II) shell binary

File Name Prefix

For defining prefix names of the temporary files

The default format is "`%s_%d`"

`%s` is design name and `%d` is process id

Keep Files

For retaining the temporary files even after the DCG session ends

After selecting "Floorplan-> Start Design Planning"

IC Compiler Floorplanning GUI opens with simplified menus

The design & floorplan gets loaded into IC Compiler floorplanning automatically

Transfers only DCT supported floorplan data to ICC floorplanning

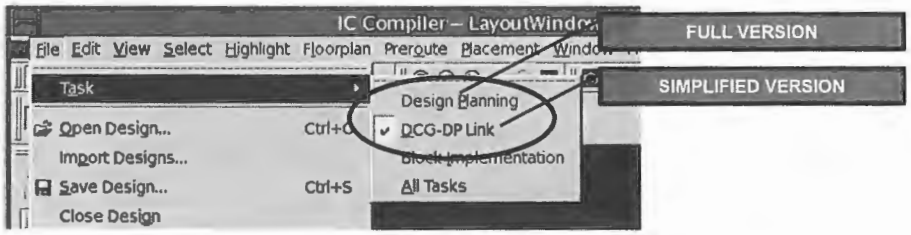
Automatically transfer SDC, attributes, library setup to ICC floorplanning

All DCG windows and shells are disabled

©2018 Synopsys, Inc. All Rights Reserved.

ICC-DP Simplified and Full Version

- IC Compiler Design Planning GUI opens with simplified menus for ease of use
- Designers can also access full version of IC Compiler Design Planning GUI



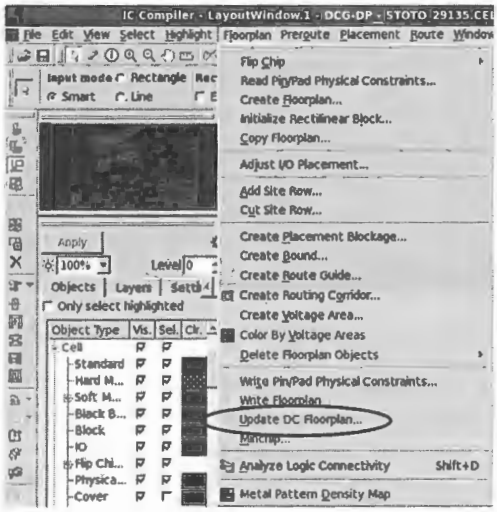
- Floorplan changes are retained when switching between simplified and full IC Compiler Design Planning version

8-38

©2018 Synopsys, Inc. All Rights Reserved.

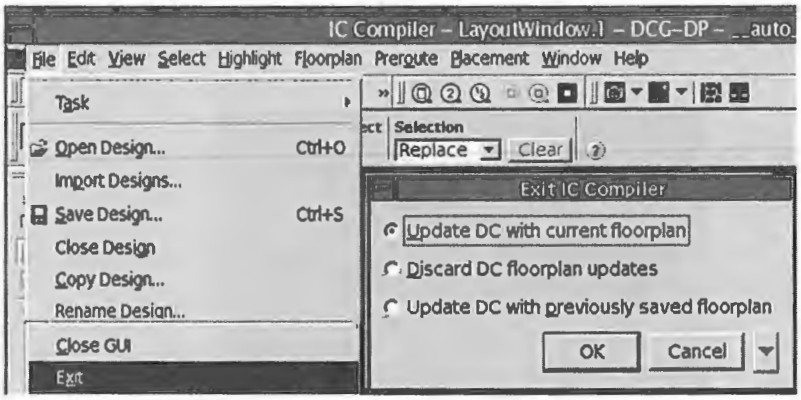
Saving the Floorplan

- Use "`Floorplan→Update DC Floorplan`" to save floorplan for synthesis



©2018 Synopsys, Inc. All Rights Reserved.

Return to Design Compiler



Option	Function
Update DC with current floorplan	Updates DCG with current floorplan
Discard DC floorplan updates	Dose not update DCG with new floorplan (including any previously generated floorplan)
Update DC with previously saved floorplan	Updates previously saved floorplan (generated with "Update DC Floorplan") into DCG

8-40

©2018 Synopsys, Inc. All Rights Reserved.

Exiting IC Compiler Floorplanning

- **After exiting IC Compiler Floorplanning:**
  - Control returns to DCG and DCG windows, shell becomes active
  - The floorplan is either updated or not updated into DCG
    - Depends on your selection in the "Exit IC Compiler" dialog
  - The original layout window disappears when there is floorplan updates
    - User needs to re-open layout window for viewing the floorplan changes

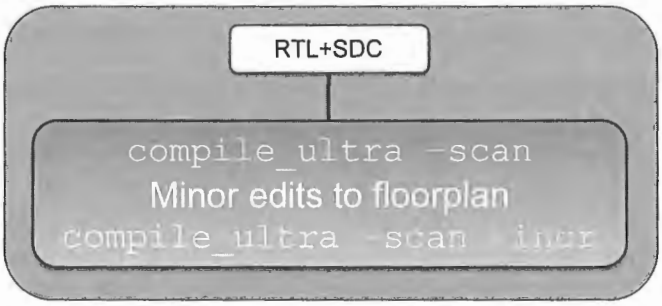
NOTE: Only floorplan changes are transferred back to DCG from IC Compiler Floorplanning. Design or attributes or ports changes, are not transferred back or updated into DCG

8-41

©2018 Synopsys, Inc. All Rights Reserved.

Incremental Synthesis

- **Incremental synthesis recommended for**
  - Small floorplan changes
  - Quick QoR assessment

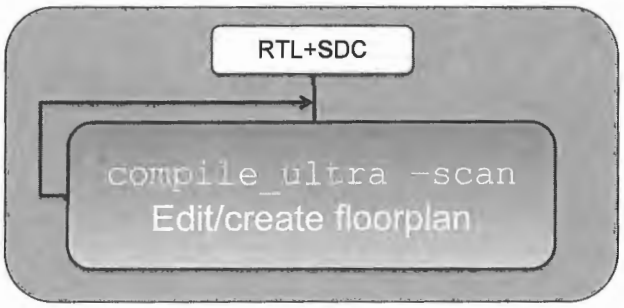


8-42

©2018 Synopsys, Inc. All Rights Reserved.

Full Re-synthesis

- **Full re-synthesis recommended for**
  - Initial floorplan creation or large changes to floorplan
  - Final or QoR implementation flow



8-43

©2018 Synopsys, Inc. All Rights Reserved.

This page was intentionally left blank

©2018 Synopsys, Inc. All Rights Reserved.

Agenda

DAY 3

6	Timing Analysis
7	Constraints: Multiple Clocks and Exceptions
8	DC Graphical: SPG Flow, Congestion, Layout GUI
9	Constraints: Complex Design Considerations
10	Post-Synthesis Output Data
11	Conclusion

© 2018 Synopsys, Inc. All Rights Reserved

Lab9: Optional lab that may be done after completing the workshop

©2018 Synopsys, Inc. All Rights Reserved.

Unit Objectives



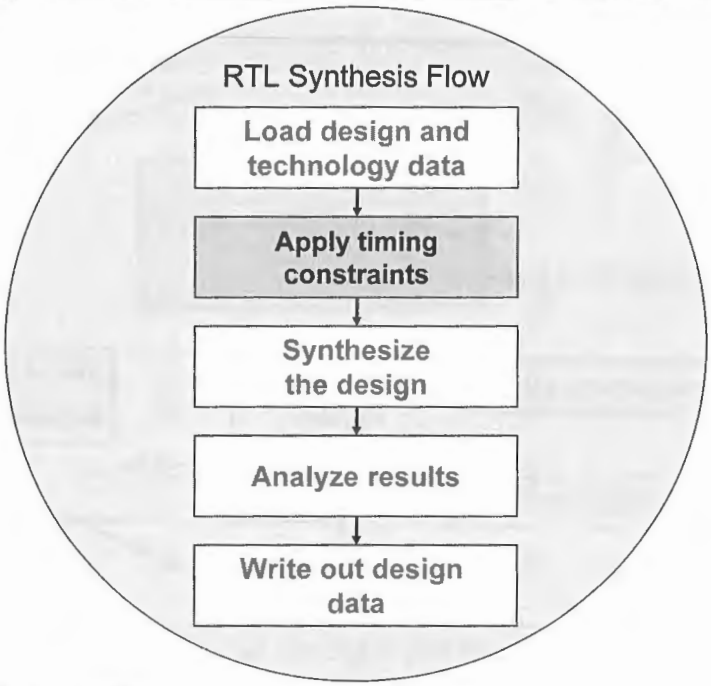
After completing this unit, you should be able to apply constraints that address the following complex design considerations:

- Clocks with non-default (50%) duty cycle
- Multiple external input/output paths
- External inputs/outputs clocked by negative edge
- Clock latencies independent of pre- versus post-CTS timing analysis

9-2

©2018 Synopsys, Inc. All Rights Reserved.

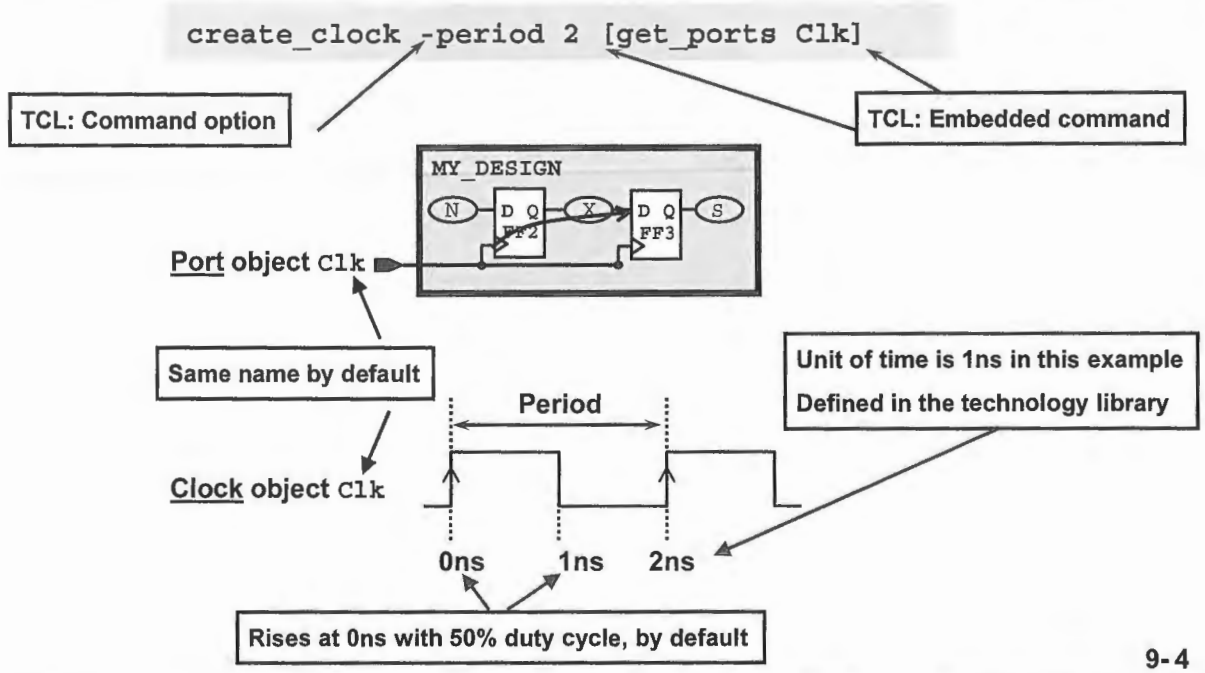
RTL Synthesis Flow



9-3

©2018 Synopsys, Inc. All Rights Reserved.

Defining a Clock: Recall Default Behavior



9-4

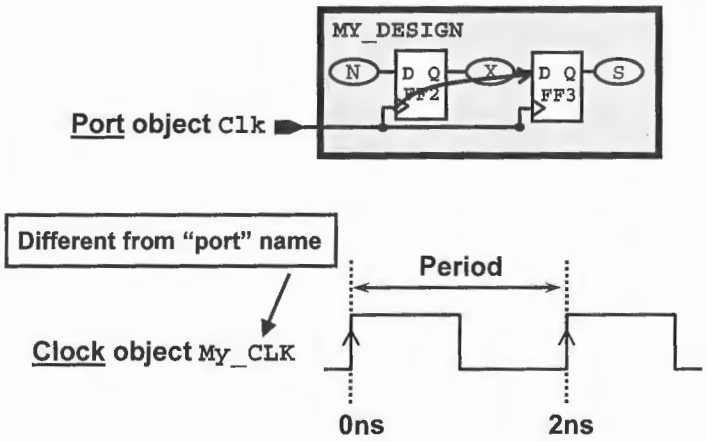
Unless explicitly defined with the `-waveform` option, the clock object that is created starts off going high at 0ns, comes down at 50% of the clock period, and repeats every clock period. The clock object's name is the same as the port/pin object to which it is assigned, unless explicitly defined by the `-name` option.

The unit of time is defined in the technology library. It is commonly 1 nano-second, as in the example above, but it may also be defined differently, for example: 0.1 nano-second, 10 pico-seconds, or 1 pico-second. You should verify the unit of time by looking at the top of the report generated by `report_lib <library_name>`.

©2018 Synopsys, Inc. All Rights Reserved.

Defining a Clock: Different Clock Name

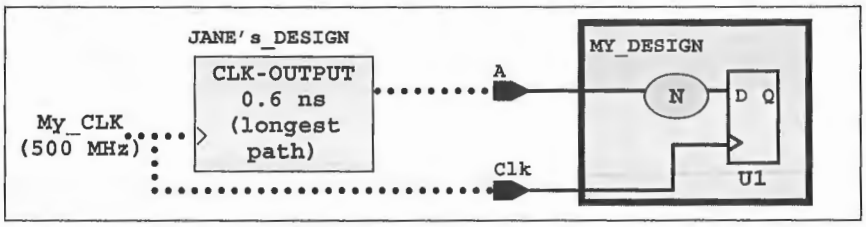
```
create_clock -period 2 -name My_CLK [get_ports Clk]
```

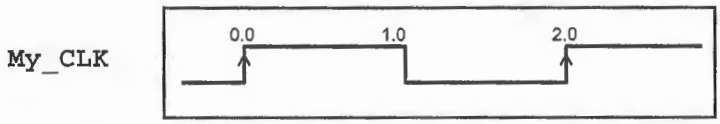


9-5

©2018 Synopsys, Inc. All Rights Reserved.

Input Delay with Different Clock Name





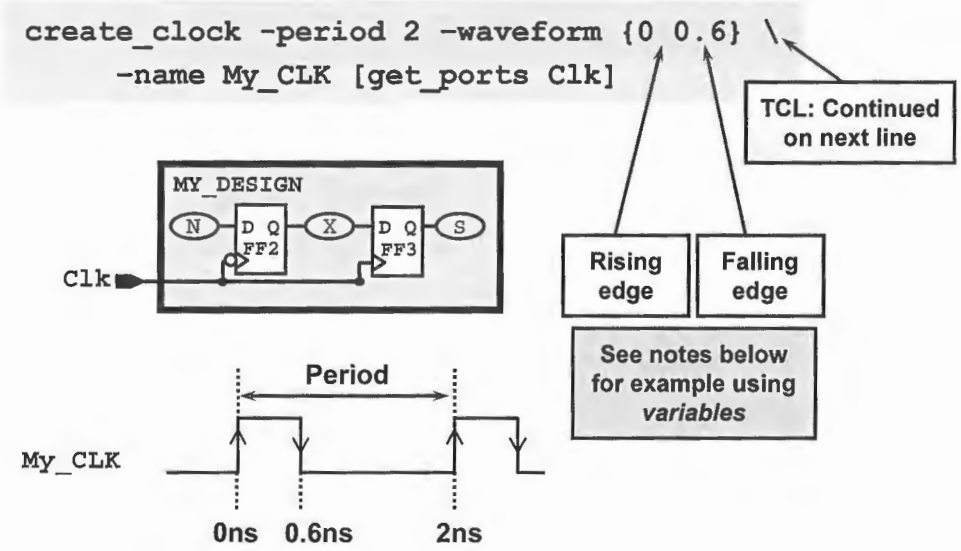

```
create_clock -period 2 -name My_CLK [get_ports Clk]
set_input_delay -max 0.6 -clock My_CLK [get_ports A]
```

JANE's clock object name, not port name

9-6

©2018 Synopsys, Inc. All Rights Reserved.

Defining a Clock: Duty-cycle



9-7

This waveform has a 30% duty cycle.

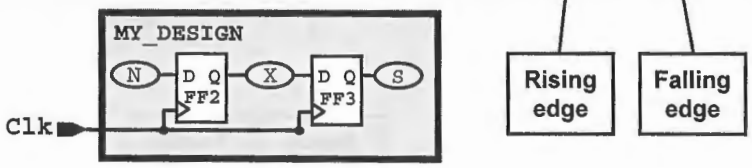
Alternatively, variables can be used with the `create_clock` command, as follows:

```
set P 2
set R 0
set F [expr {0.3 * $P}]
set N My_Clk
create_clock -period $P -waveform "$R $F" -name $N [get_ports Clk]
```


©2018 Synopsys, Inc. All Rights Reserved.

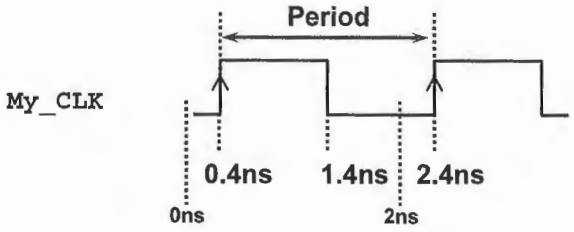
Defining a Clock: Offset

```
create_clock -period 2 -waveform {0.4 1.4} \
     -name My_CLK [get_ports Clk]
```



Rising edge

Falling edge



Defining an offset with `-waveform` is NOT recommended!

See Notes for the alternative

9-8

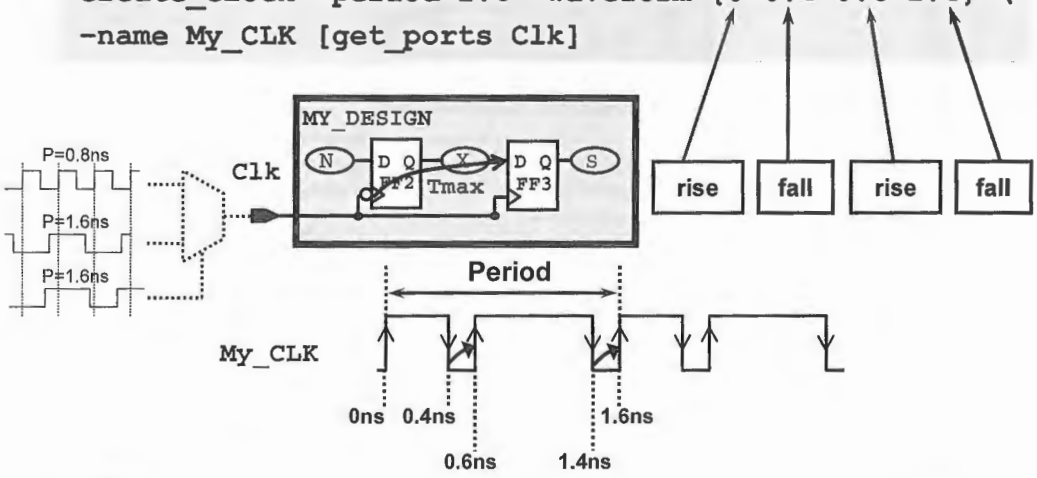
It is NOT recommended to use the `-waveform` option to offset the clock waveform. In a multi-clock environment this offset method can cause DC to pick unintended clock launch and/or capture edges during synthesis which may over- or under-constrain your design. The `-waveform` option should be used only to modify the duty-cycle and/or invert the waveform. To introduce an offset it is recommended to use `set_clock_latency`. It can be done as follows:

```
create_clock -period 2 -name My_CLK [get_ports Clk]
set_clock_latency <-source> -max 0.4 [get_clocks My_CLK]
```


©2018 Synopsys, Inc. All Rights Reserved.

Defining a Clock: Complex

```
create_clock -period 1.6 -waveform {0 0.4 0.6 1.4} \
-name My_CLK [get_ports Clk]
```



? If $FF3_{setup}$= 0.03ns, what is T_{max} for the Reg-to-Reg path?

$T_{max,\ Reg\text{-}to\text{-}Reg}$ = _____

9-9

Answer: $0.6 - 0.4 - 0.03 = 0.17$ns.

©2018 Synopsys, Inc. All Rights Reserved.

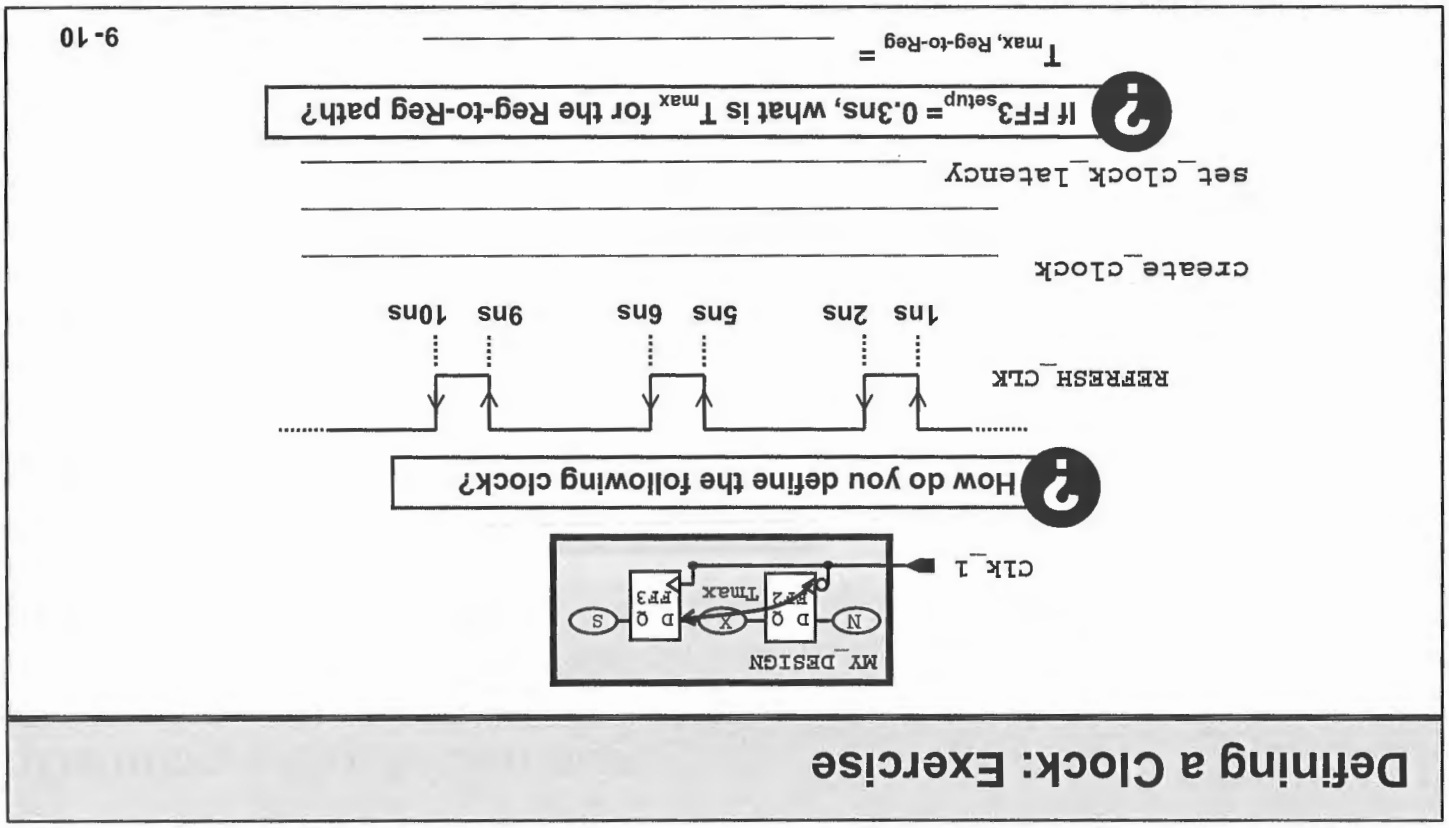
```
create_clock -period 4 -waveform {0 3} -name REFRESH_CLK \
                    [get_ports Clk_1]
set_clock_latency  <-source> -max 2 [get_clocks REFRESH_CLK]
```

If the entire 2ns latency comes from an "external" source (like a clock generator), then use the `-source` option; If part of the 2ns latency is external, and the rest is from "internal" network latency, model with two separate `set_clock_latency` commands – one for the source, and the other for the network latency portion that makes up the total 2ns latency.

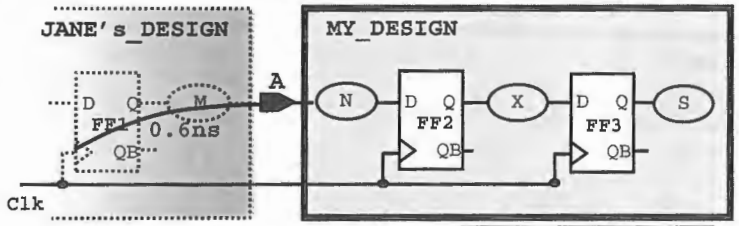
An alternative approach:

```
create_clock -period 4 -name REFRESH_CLK [get_ports Clk_1]
set_clock_latency -max -rise 2 [get_clocks REFRESH_CLK]
set_clock_latency -max -fall 3 [get_clocks REFRESH_CLK]
```

$T_{FF3,\ capture} - FF3_{Setup} - T_{FF2,\ launch} = (4+2) - 0.3 - (3+2) = 0.7$ ns.



Input Delay: Recall – Basic Options



External delay: JANE's maximum output delay

```
set_input_delay -max 0.6 -clock Clk [get_ports A]
```

Constrains MY input path for setup time

w.r.t. JANE's positive-edge triggered clock

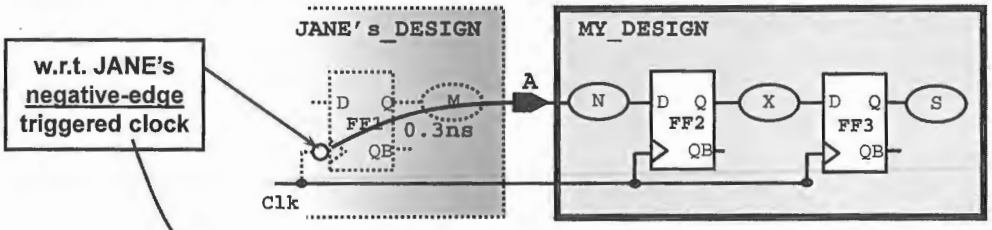
You specify how much time is used by external logic...

DC calculates how much time is left for the internal logic.

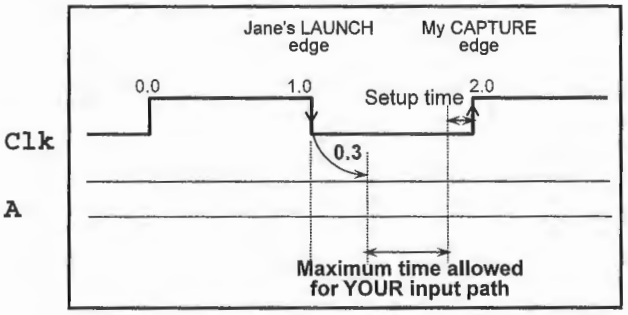
9-11

©2018 Synopsys, Inc. All Rights Reserved.

Input Delay: Falling Clock Edge



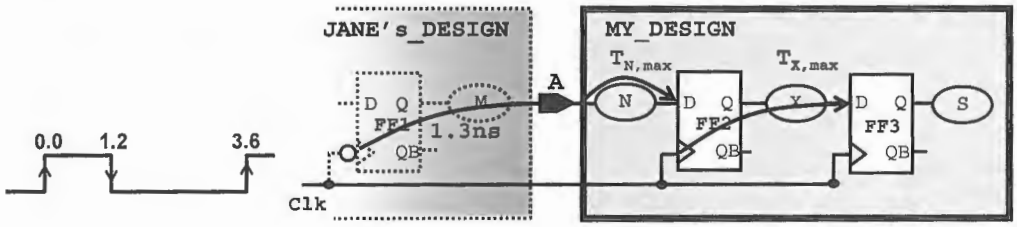

```
create_clock -period 2 [get_ports Clk]
set_input_delay -max 0.3 -clock Clk -clock_fall [get_ports A]
```



9-12

©2018 Synopsys, Inc. All Rights Reserved.

Input Delay: Falling Clock Edge Exercise




```
create_clock -period _____________________________[get_ports Clk]
set_input_delay ___________________________________ [get_ports A]
```

If FF2 and FF3 have a 0.2 ns setup requirement:

What is the maximum delay $T_{N, max}$? _____

What is the maximum delay $T_{X, max}$? _____

9-13

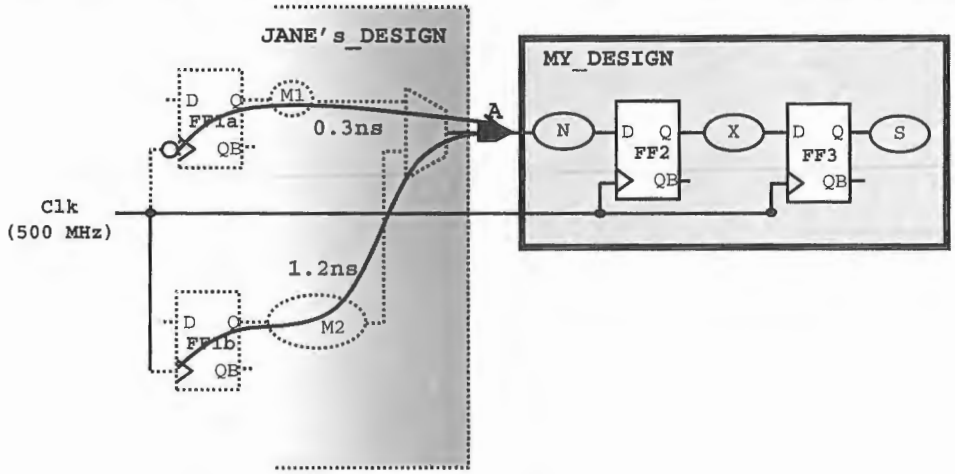
$Tn, max = 3.6 - 1.2 - 1.3 - 0.2 = 0.9ns$

$Tx, max = 3.6 - 0.2 = 3.4ns$

```
create_clock -period 3.6 -waveform {0.0 1.2} [get_ports Clk]
set_input_delay -max 1.3 -clock Clk -clock_fall [get_ports A]
```


©2018 Synopsys, Inc. All Rights Reserved.

Input Delay: Multiple Input Paths

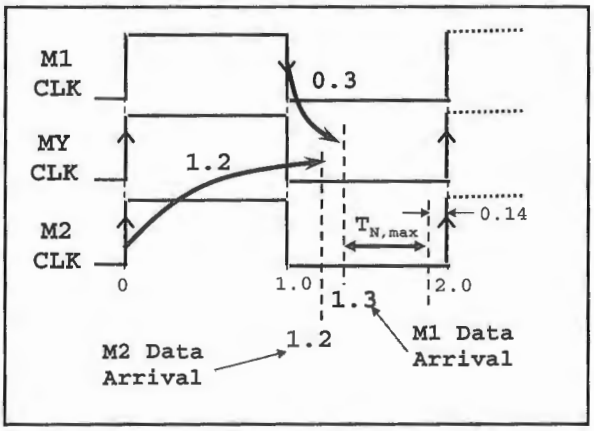



```
create_clock -period 2 [get_ports Clk]
set_input_delay -max 0.3 -clock Clk -clock_fall [get_ports A]
set_input_delay -max 1.2 -clock Clk -add_delay  [get_ports A]
```

9-14

©2018 Synopsys, Inc. All Rights Reserved.

Multiple Input Path Timing Analysis



Design Compiler **analyzes both paths and constrains input logic path `N` with the more restrictive of the two – path `M1` in this example**

If FF2 has a 0.14 ns setup requirement:

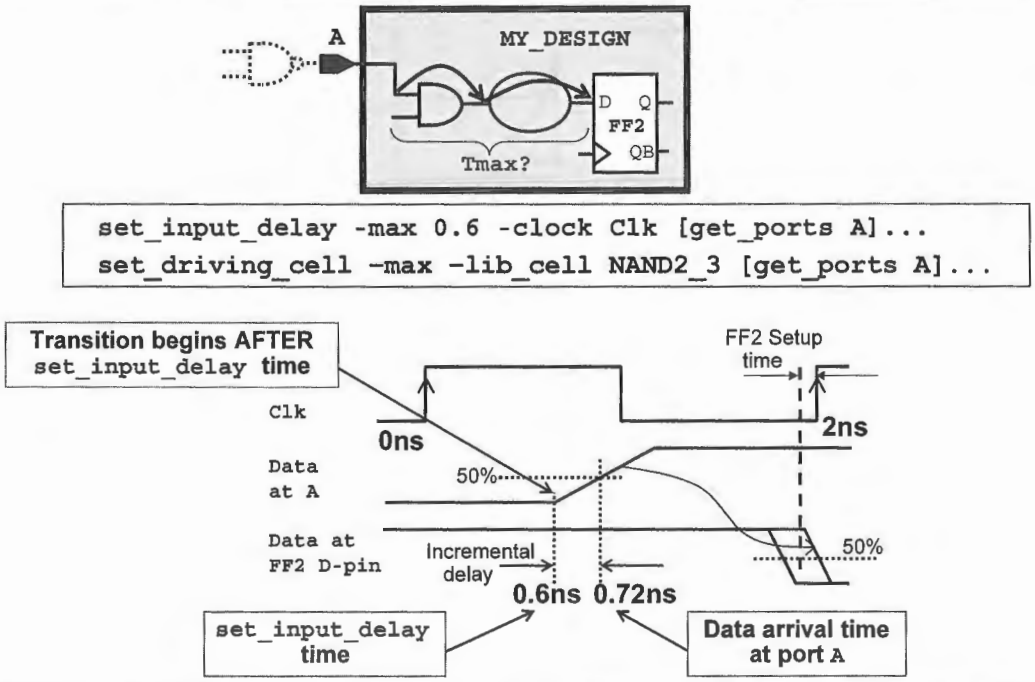
What is the maximum delay $T_{N,\,max}$? _____

9-15

$T_{n,\,max} = 2.0 - 1.3 - 0.14 = 0.56ns$

©2018 Synopsys, Inc. All Rights Reserved.

Effect of *Driving Cell* on Input Delay



9-16

By specifying a driving cell DC calculates an accurate rise/fall transition time on the input port, based on your actual internal capacitive loading. This allows DC to calculate an accurate cell delay for the input gate, and the input path.

DC assumes, however, that the transition on the input port begins at the "set_input_delay time". DC therefore adds an incremental delay to the data arrival time specified by `set_input_delay`.

©2018 Synopsys, Inc. All Rights Reserved.

set_driving_cell Recommendation

If your are confident that your design constraint specs are accurate, and you want to model your input data arrival time with precision, make sure that:

1. The `set_input_delay` number you apply is based on zero output load on Jane's block (the intrinsic delay to Jane's output port)
2. The `set_driving_cell` gate matches Jane's output driver

If not, your input constraints will include some built-in pessimism

Spec: Latest Data Arrival Time at Port A, after Jane's launching clock:

Use this number! → 0.60ns, with 50fF load

0.48ns, with 0.0fF load

Driving cell on input port A: `Qn` pin of `FD1` flip-flop

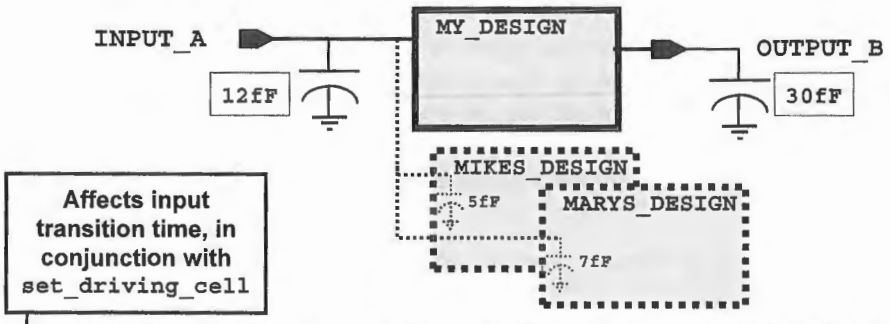
```
create_clock -period 2 [get_ports Clk]
set_input_delay -max 0.48 -clock Clk [get_ports A]
set_driving_cell -max -lib_cell FD1 -pin Qn [get_ports A]
```

9-17

©2018 Synopsys, Inc. All Rights Reserved.

Modeling External Capacitive Load on Inputs

Spec: Input A Drives two other blocks with 12fF total input capacitance



Affects input transition time, in conjunction with set_driving_cell

```
create_clock -period 2 [get_ports Clk]
set_input_delay -max 0.6 -clock Clk [get_ports A]
set_output_delay -max 0.8 -clock Clk [get_ports B]
set_load -max 0.030 [get_ports OUTPUT_B]
set_driving_cell -max -lib_cell NAND2_3 [get_ports INPUT_A]
set_load -max 0.012 [get_ports INPUT_A]
```

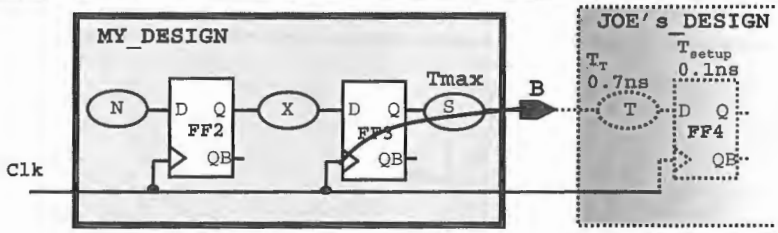
9-18

The additional load on the input port will affect the transition time on port INPUT_A.

If you have a `set_input_transition` on INPUT_A instead of `set_driving_cell`, then the `set_load` on the input port will have no effect on the input transition.

©2018 Synopsys, Inc. All Rights Reserved.

Output Delay: Recall – Basic Options



External delay: JOE's maximum setup time: input delay (T) + FF setup time

```
set_output_delay -max 0.8 -clock Clk [get_ports B]
```

Constrains MY output path for maximum delay, or setup time w.r.t. Joe's input

w.r.t. JOE's positive-edge triggered clock

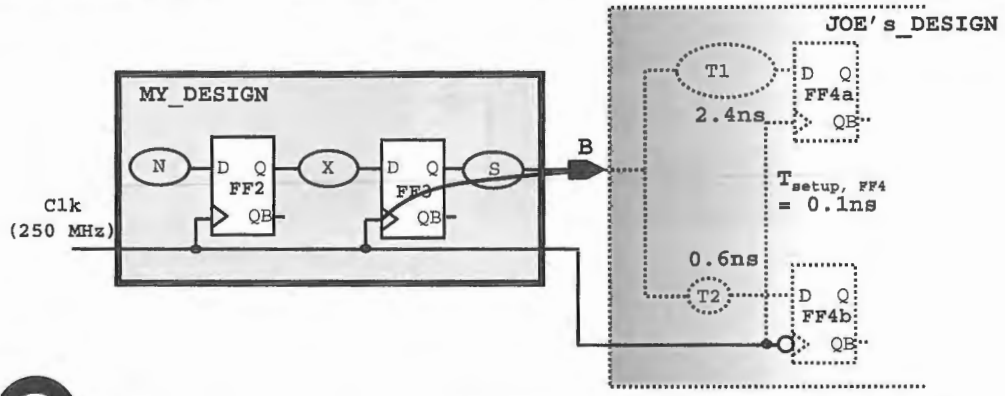
You specify how much time is used by external logic...

DC calculates how much time is left for the internal logic.

9-19

©2018 Synopsys, Inc. All Rights Reserved.

Output Delay: Complex Output Paths



How do you constrain MY_DESIGN for the indicated path?

```
create_clock -period 4 [get_ports Clk]
set_output_delay _____________________________________________
set_output_delay _____________________________________________
```

9-20

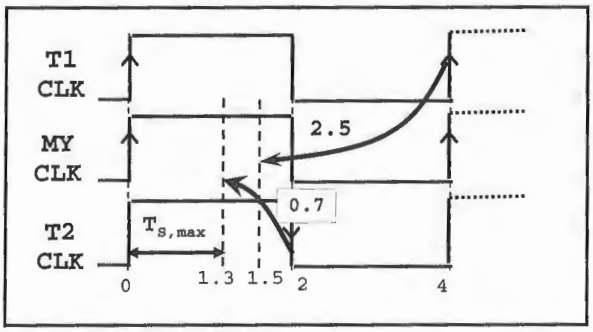
Or alternatively, if you recognize that the T2 path is the more restrictive one:

```
set_output_delay -max 2.5 -clock Clk [get_ports B]
set_output_delay -max 0.7 -clock Clk - clock_fall -add_delay \
                                                  [get_ports B]
set_output_delay -max 0.7 -clock Clk -clock_fall [get_ports B]
```


©2018 Synopsys, Inc. All Rights Reserved.

Complex Output Path Timing Analysis

```
create_clock -period 4 [get_ports Clk]
set_output_delay -max 2.5 -clock Clk [get_ports B]
set_output_delay -max 0.7 -clock Clk -clock_fall -add_delay [get_ports B]
```



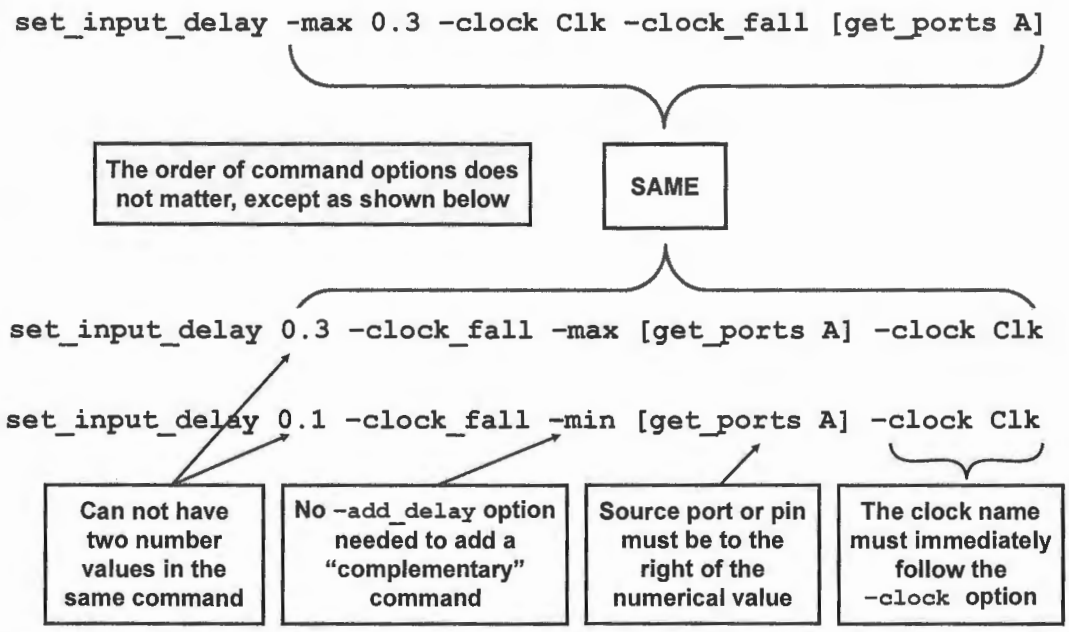
Design Compiler analyzes both paths and constrains output logic path S with the more restrictive of the two. $T_{S, max}$ = _____

9-21

Ts, max = smaller of: (4.0 − 2.5) or (2.0 − 0.7) ← = 1.3ns

©2018 Synopsys, Inc. All Rights Reserved.

Argument Ordering of TCL Commands



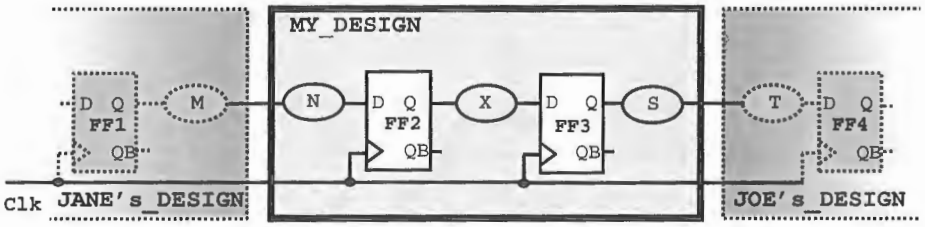
9-22

"Complementary" commands are commands with complementary numerical option values, for example: `-max` and `-min`; `-rise` and `-fall`; `-setup` and `-hold`

©2018 Synopsys, Inc. All Rights Reserved.

Pre- versus Post-CTS Clock Latencies

The remaining pages cover "ideal" (pre-CTS) versus "propagated" (post-CTS) clock latencies



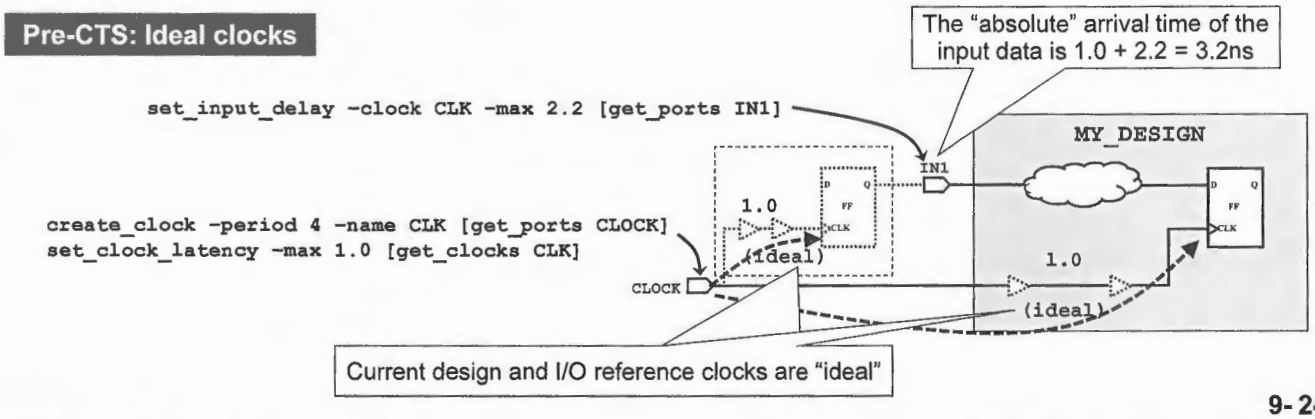
9-23

©2018 Synopsys, Inc. All Rights Reserved.

Pre-CTS: Ideal Clock Latencies

Prior to *clock tree synthesis* (*CTS*), clock networks are *ideal*:

- Timing analysis in *Design Compiler*, *IC Compiler(II)* and *PrimeTime* use estimated **ideal** clock latency values defined by `set_clock_latency`
- *Ideal* clock network latencies apply to current design and I/O reference clocks (defined by `set_input_delay`/`set_output_delay`)



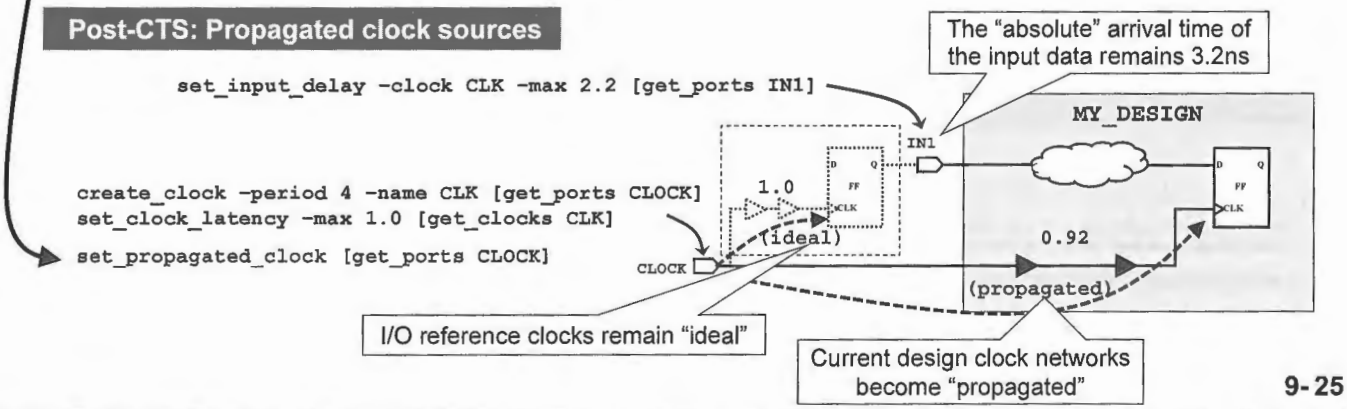
9-24

©2018 Synopsys, Inc. All Rights Reserved.

Post-CTS: Propagated Clock Sources

After CTS, clock networks become *propagated*:

- ***IC Compiler(II)*** automatically applies `set_propagated_clock` constraint to clock sources (ports/pins)
- Actual **propagated** clock network latencies are calculated inside the current design
- Estimated **ideal** clock network latencies apply to I/O reference clocks



9-25

IC Compiler (II) automatically applies the `set_propagated_clock` command to the clock ports and pins in the current design, after clock tree synthesis. Any post-CTS timing analysis in *IC Compiler (II)* will exhibit the above clock latency timing. If the updated constraints are written out of *IC Compiler (II)* and used in *Design Compiler* or *PrimeTime* (`write_sdc`), then *Design Compiler* and *PrimeTime* will also exhibit this same clock latency timing.

©2018 Synopsys, Inc. All Rights Reserved.

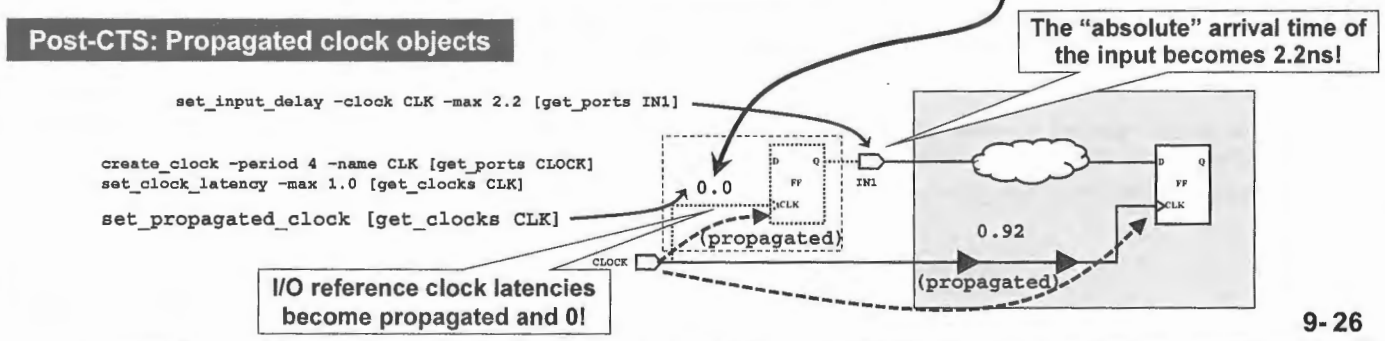
Post-CTS: Propagated Clock Objects

Post CTS, some *IC Compiler(II)* or *PrimeTime* users inadvertently propagate the clock objects (`all_clocks`) by explicitly applying:

```
set_propagated_clock [all_clocks]
```

- I/O reference clocks also become **propagated**, but there are no actual clock paths
  → **Calculated I/O clock latencies are 0!**

Dangerous: Can cause missed or false timing violations!



Applying `set_propagated_clock [all_clocks]` is dangerous:
With the `[all_clocks]` argument the command is applied to all clock objects. Since I/O delay constraints reference clock objects (through their `-clock` option), the clock latency for the "external I/O registers" also becomes propagated. For propagated clock networks, *IC Compiler (II)* and *Primetime* ignore clock network latency constraints: These tools calculate the actual propagation delay through the synthesized clock tree, to determine the clock network latency. Since there is no synthesized clock network for the I/O's, the clock latency of the I/O's becomes zero: On input paths, the zero clock latencies create optimistic *setup* timing, and pessimistic *hold* timing; On output paths, the zero clock latencies create pessimistic *setup* timing, and optimistic *hold* timing, all of which can lead to missed, as well as false *setup* and *hold* timing violations!

There are a couple of different ways to constrain the I/O's from the start of the design cycle, with this issue in mind, such that the problem can be avoided – see next pages.

©2018 Synopsys, Inc. All Rights Reserved.

Independent I/O Clock Latencies (1 of 2)

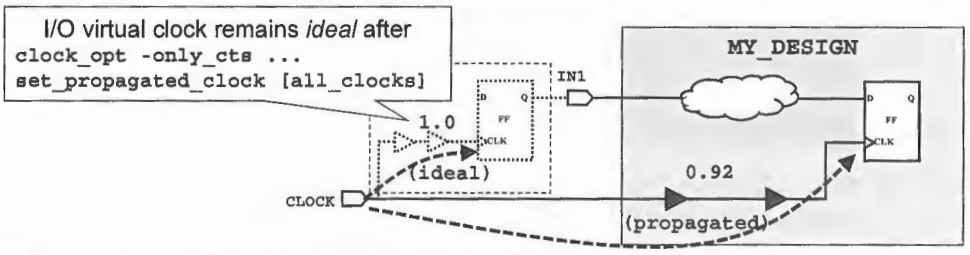
I/O's can be constrained in one of two ways to ensure that I/O reference clock latencies are correct, independent of how `set_propagated_clock` is applied:

1) Use independent *virtual* clocks for the I/O's, with their own latencies

- *Virtual* clocks remain *ideal* so their network latencies are determined by their `set_clock_latency` constraints

```
create_clock -period 4 -name CLK [get_ports CLOCK]
set_clock_latency -max 1.0 [get_clocks CLK]

create_clock -period 4 -name V_CLK
set_clock_latency -max 1.0 [get_clocks V_CLK]
set_input_delay -clock V_CLK -max 2.2 [get_ports IN1]
```



9-27

©2018 Synopsys, Inc. All Rights Reserved.

Independent I/O Clock Latencies (2 of 2)

2) Include the clock network latency in the I/O data path delay

- Example: Instead of doing this …

```
create_clock -period 4 -name CLK [get_ports CLK]
set_clock_latency -max  1.0 [get_clocks CLK]

set_input_delay -clock CLK -max 2.2 [get_ports IN1]
```

- … account for I/O clock network latency by including it in the "data" delay time, and using `-network_latency_included`:

```
create_clock -period 4 -name CLK [get_ports CLOCK]
set_clock_latency -max 1.0 [get_clocks CLK]

set_input_delay -clock CLK -max 3.2 \
          -network_latency_included [get_ports IN1]
```

The pre- and post-CTS "absolute" arrival time of the input data is 3.2ns

- The `-network_latency_included` option ensures that the I/O data arrival times are the same for both *ideal* and *propagated* clocks

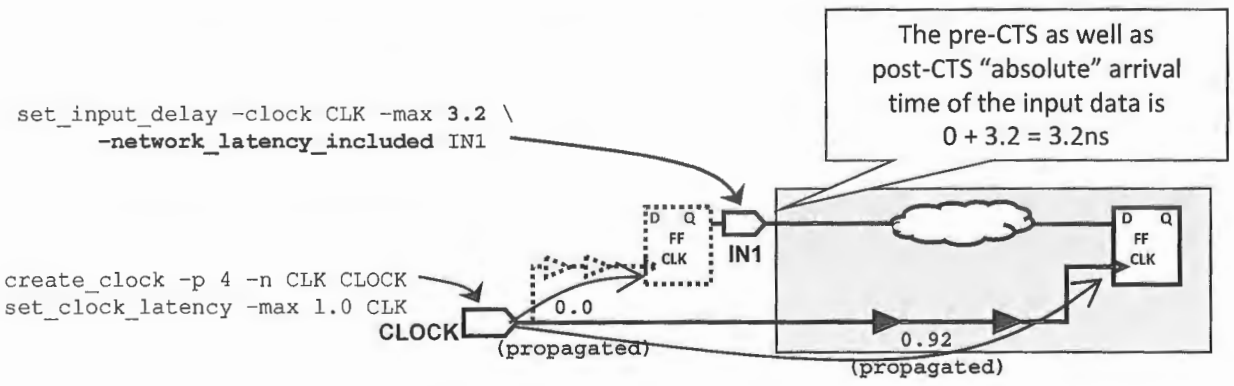
9-28

When the `-network_latency_included` option is used with `set_input/output_delay`, timing analysis assumes that the network latency for the clock of the register driving/capturing the data on the input/output port, respectively, is zero. This option assumes that the user has "included" the effect of the network latency in the data arrival/required time at the input/output port, by adjusting the `-max/-min` delay numbers accordingly.

In the example shown below, because of the `-network_latency_included` option, the "absolute" arrival time (relative to time zero) of the data at input port `IN1` is the same 3.2ns before CTS, when the clocks are "ideal", or when "propagated" post-CTS after executing:

Post-CTS:

```
set_propagated_clock [all_clocks]
```



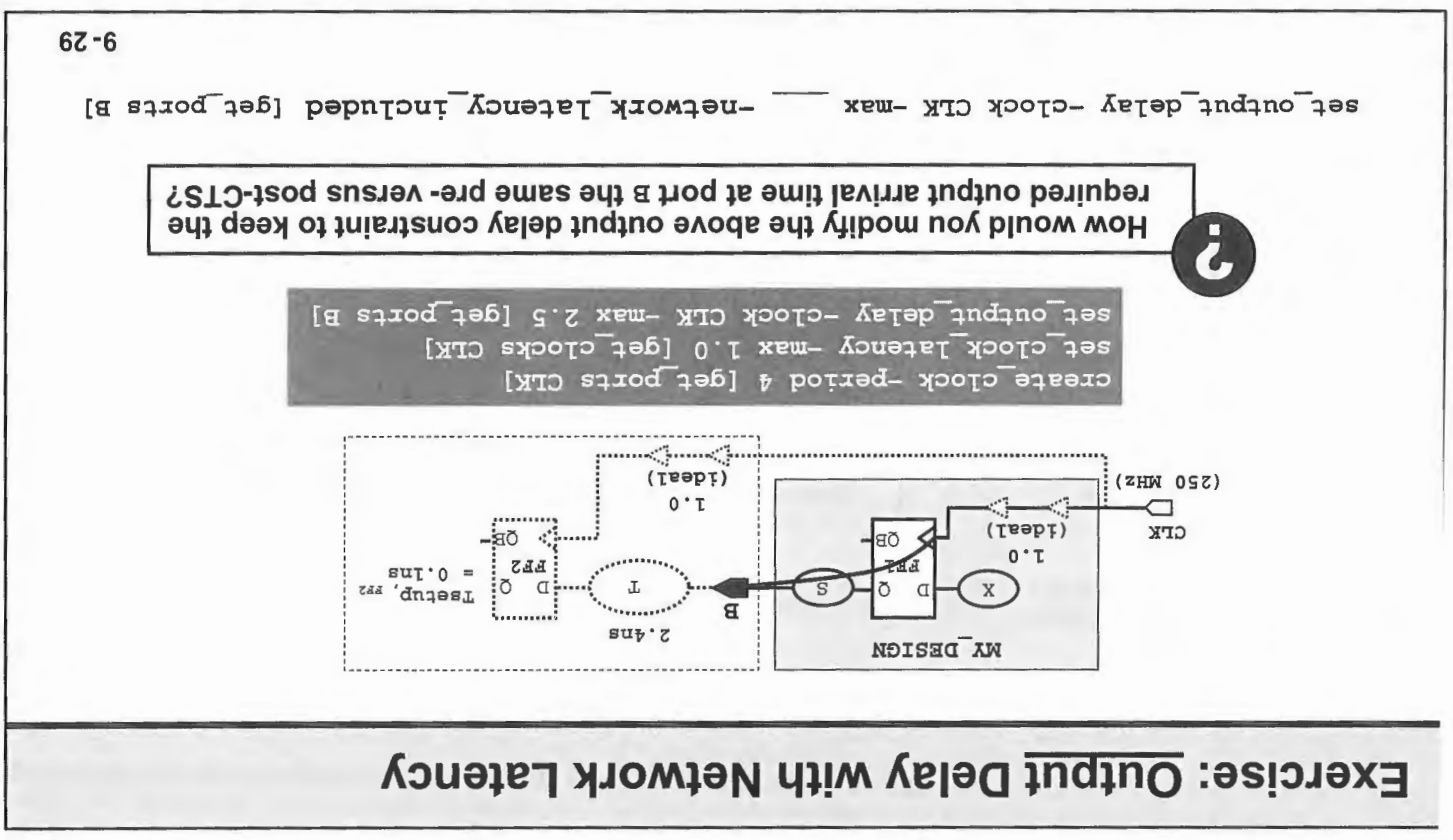
©2018 Synopsys, Inc. All Rights Reserved.

©2018 Synopsys, Inc. All Rights Reserved.


```
set_output_delay -clock CLK -max 1.5  \
     -network_latency_included [get_ports B]
```

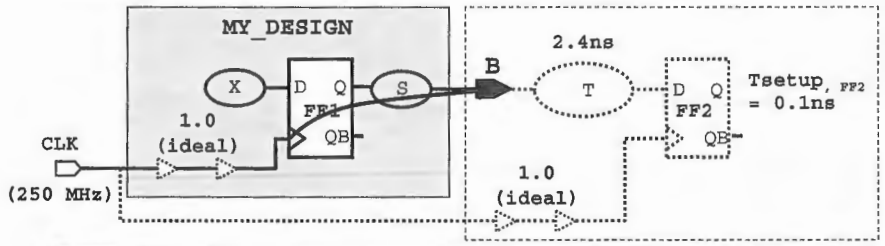
Using the original `set_output_delay -clock -max 2.5 [get_ports B]` constraint:
Pre-CTS, when the clocks are ideal, the latest allowable absolute arrival time of the output data at port B is: 4 + 1 – 2.5 = 2.5ns.
Post-CTS, if the clock objects become "propagated" (`set_propagated_clock [all_clocks]`), the latest allowable absolute arrival time of the output data at port B becomes: 4 + 0 – 2.5 = 1.5ns.

Using the modified
`set_output_delay -clock -max 1.5 -network_latency_included [get_ports B]`
constraint:
For pre-CTS ideal clocks, as well as post-CTS (with `set_propagated_clock [all_clocks]`, or with `set_propagated_clock [get_ports CLK*]`), the latest allowable absolute arrival time of the output data at port B is: 4 + 0 – 1.5 = 2.5ns.



Exercise: Output Delay with Network Latency

Exercise: Output Delay with Source Latency




```
create_clock -period 4 [get_ports CLK]
set_clock_latency -source -max 0.4 [get_clocks CLK]
set_clock_latency -max 1.0 [get_clocks CLK]
set_output_delay -clock CLK -max 2.5 [get_ports B]
```

How would you modify the above output delay constraint to include both source- and network-latency?

```
set_output_delay -clock CLK -max ____  \
                 -source_latency_included \
                 -network_latency_included [get_ports B]
```

9-30

```
set_output_delay -clock CLK -max 1.1      \
                 -source_latency_included \
                 -network_latency_included [get_ports B]
```

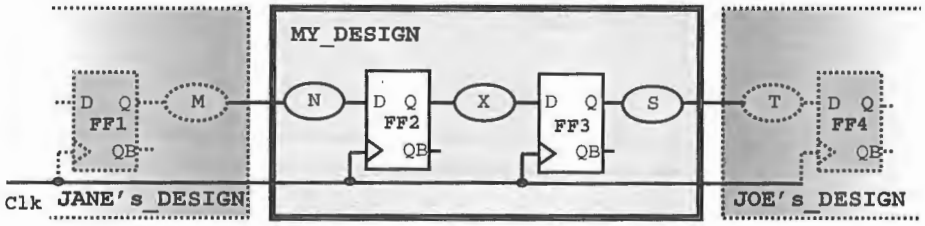

©2018 Synopsys, Inc. All Rights Reserved.

Different I/O versus Internal Latencies



How do we model "ideal" external or I/O latencies that are different than the current design's latencies?

Spec:	JANE	ME	JOE
Source latency	0.18	0.30	0.30
Network latency	0.12	0.12	0.07



9-31

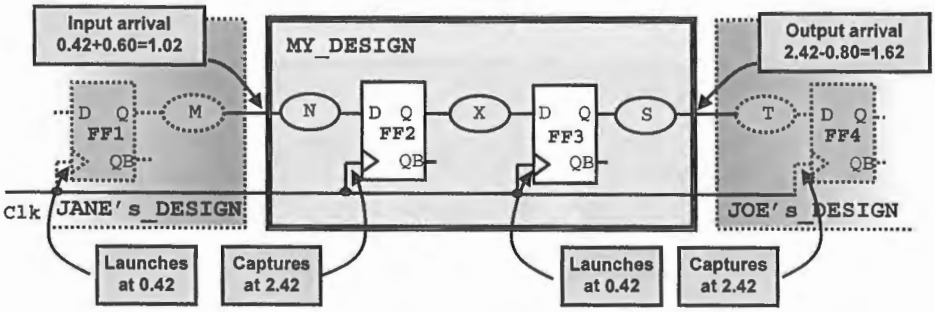
©2018 Synopsys, Inc. All Rights Reserved.

Default External or I/O Clock Latencies

Example: Default "ideal" I/O latencies, and input/output data arrival times, based on `MY_DESIGN`'s source and network latencies:

Spec:	JANE	ME	JOE
Source latency	0.30	0.30	0.30
Network latency	0.12	0.12	0.12

```
create_clock -period 2 [get_ports Clk]
set_clock_latency -source -max 0.3 [get_clocks Clk]
set_clock_latency -max 0.12 [get_clocks Clk]
set_input_delay -max 0.6 -clock Clk [all_inputs]
set_output_delay -max 0.8 -clock Clk [all_outputs]
```



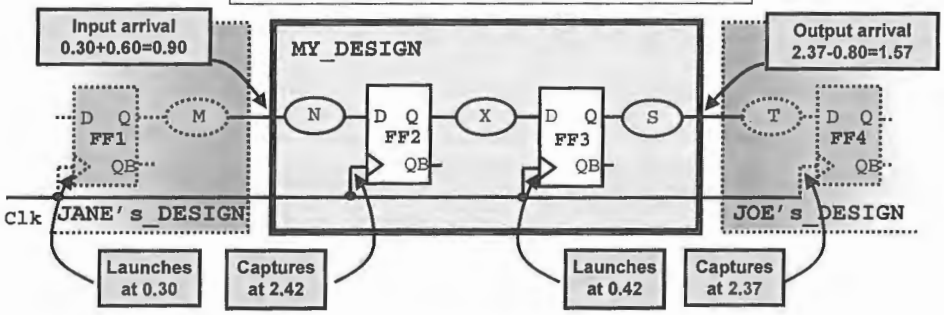
9-32

©2018 Synopsys, Inc. All Rights Reserved.

Handling Different I/O vs Internal Latencies

Example: Based on the latency spec below, these are the actual "ideal" input/output data arrival times we would like to model:

Spec:	JANE	ME	JOE
Source latency	0.18	0.30	0.30
Network latency	0.12	0.12	0.07



This is easily accomplished using either the "virtual" or "included" clock latency methods – shown next

9-33

©2018 Synopsys, Inc. All Rights Reserved.

Virtual External Clock Latencies

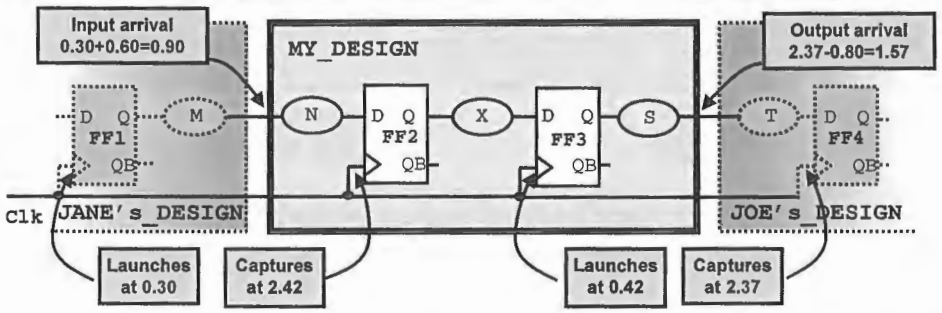
```
create_clock -period 2 [get_ports Clk]
create_clock -period 2 -name Clk_Jane  ; # Launch Virtual clock
create_clock -period 2 -name Clk_Joe   ; # Capture Virtual clock

set_clock_latency -source -max 0.30 [get_clocks Clk]
set_clock_latency -source -max 0.18 [get_clocks Clk_Jane]
set_clock_latency -source -max 0.30 [get_clocks Clk_Joe]

set_clock_latency -max 0.12 [get_clocks Clk]
set_clock_latency -max 0.12 [get_clocks Clk_Jane]
set_clock_latency -max 0.07 [get_clocks Clk_Joe]

set_input_delay -max 0.6 -clock Clk_Jane [all_inputs]
set_output_delay -max 0.8 -clock Clk_Joe [all_outputs]
```

Spec:	JANE	ME	JOE
Source latency	0.18	0.30	0.30
Network latency	0.12	0.12	0.07



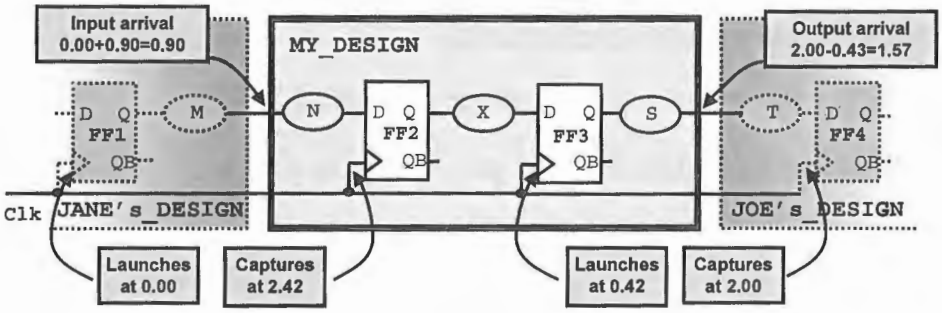
9-34

©2018 Synopsys, Inc. All Rights Reserved.

Included External Clock Latencies

Spec:	JANE	ME	JOE
Source latency	0.18	0.30	0.30
Network latency	0.12	0.12	0.07

```
create_clock -period 2 [get_ports Clk]
set_clock_latency -source -max 0.3 [get_clocks Clk]
set_clock_latency -max 0.12 [get_clocks Clk]
set_input_delay  -max [expr {0.6 + 0.18 + 0.12}] -clock Clk \
 -source_latency_included -network_latency_included [all_inputs]
set_output_delay -max [expr {0.8 - (0.30 + 0.07)}] -clock Clk \
 -source_latency_included -network_latency_included [all_outputs]
```



9-35

©2018 Synopsys, Inc. All Rights Reserved.

Unit Objectives Review



You should now be able to apply constraints that address the following complex design considerations:

- **Clock with non-default (50%) duty cycle**
- **Multiple external input/output paths**
- **External inputs/outputs clocked by negative edge**
- **Clock latencies independent of pre- versus post-CTS timing analysis**

9- 36

©2018 Synopsys, Inc. All Rights Reserved.

OPTIONAL Lab 9: Constraints: Complex Design Considerations

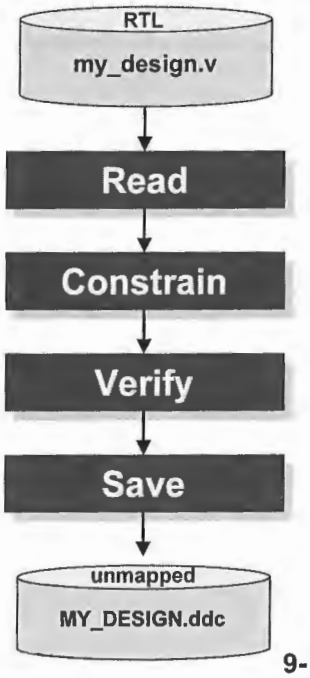


75 minutes

OPTIONAL Lab 9: Constraints: Complex Design Considerations

Constrain a design with complex design considerations

(Run this optional lab after completion of the workshop)



9- 37

©2018 Synopsys, Inc. All Rights Reserved.

This page was intentionally left blank

©2018 Synopsys, Inc. All Rights Reserved.

Agenda

DAY 3

6	Timing Analysis
7	Constraints: Multiple Clocks and Exceptions
8	DC Graphical: SPG Flow, Congestion, Layout GUI
9	Constraints: Complex Design Considerations
10	Post-Synthesis Output Data
11	Conclusion

© 2018 Synopsys, Inc. All Rights Reserved

©2018 Synopsys, Inc. All Rights Reserved.

Objectives



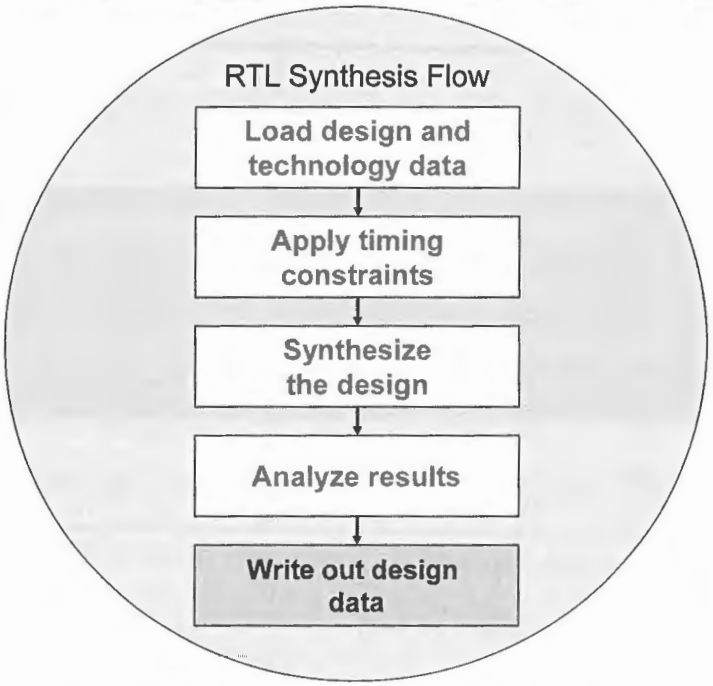
After completing this section, you should be able to:

Generate required post-synthesis output data files that take into account the abilities and restrictions of downstream physical design tools

10-2

©2018 Synopsys, Inc. All Rights Reserved.

RTL Synthesis Flow



10-3

©2018 Synopsys, Inc. All Rights Reserved.

Unit Agenda

Data needed for physical design

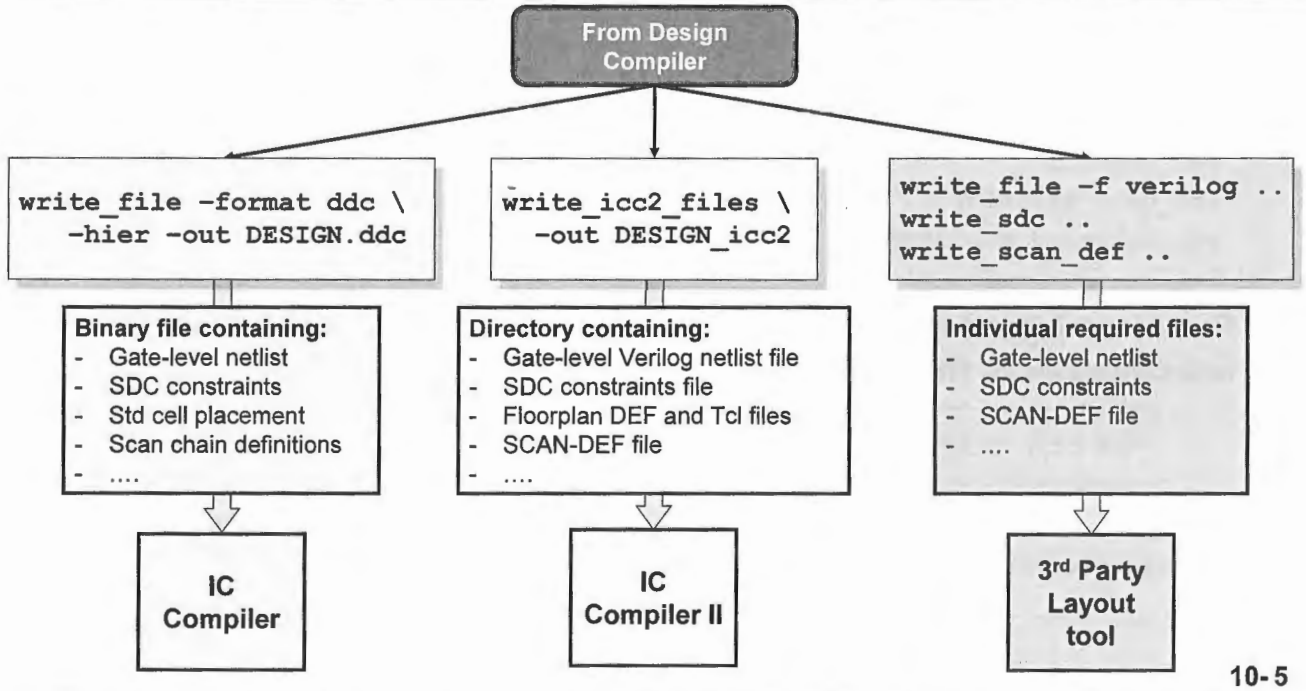
Netlist restrictions

- `assign` statement
- Special characters

10-4

©2018 Synopsys, Inc. All Rights Reserved.

Data Needed for Physical Design or Layout



The *ddc* netlist contains all the design data that is needed by IC Compiler, including but not limited to the netlist, constraints, scan chain definitions and standard cell placement information.

ICC II does not read *ddc*, so instead, the `write_icc2_files` command writes ASCII data files and *Tcl* scripts for all the design data needed by IC Compiler II. Files are written into the directory specified by the `-output` option. The files include:

- o Netlist in Verilog format
- o Tcl floorplan and DEF files
- o Scan chain information in SCANDEF format
- o Constraints in SDC format
- o Propagate backward SAIF
- o Design's power intent as a UPF command script
- o Timing contexts (scenarios)
- o Design Compiler settings in IC Compiler II format
- o The top-level script to load all data generated in IC Compiler II

You can write out just the standard cell placement to ICC II for use with a golden floorplan file:
`write_icc2_files -output ABC -golden_floorplan golden_floorplan.tcl`
The golden floorplan constraints should be applied to DC and the golden floorplan file should be passed to the `write_icc2_files` command using `-golden_floorplan` option.

Non-Synopsys 3rd party back-end or layout tools do not read in the *ddc* format. They require a standard Verilog/VHDL netlist as input. Since the Verilog/VHDL netlist does not contain any constraints, the constraints are provided as a separate SDC format file. The SDC file, explained in the next slide, contains a sub-set of the full-blown constraints and directives applied to the design. If these 3rd party layout tools have the ability to re-order scan chains, the SCANDEF file should also be provided.

©2018 Synopsys, Inc. All Rights Reserved.

What is *SDC*?

- **3rd party layout tools do not use or understand DC-specific *compile directives* such as:**

```
group_path
set_ungroup
set_cost_priority
set_optimize_registers
```

- **For these layout tools, write out a 'constraints-only' version of DC's full constraints file[1] with:**

```
write_sdc <my_design.sdc>
```

- **The *SDC* file contains the normal constraints, with expanded[2] arguments**

```
create_clock; set_in/output_delay; set_false_path ...
```

10-6

[1]The full set of constraints and directives can be written out with `write_script -out <file_name>`

[2]Besides stripping out compile directives, the `write_sdc` command also 'expands' the command arguments, for example:
The argument `[get_ports sd_* -filter "port_direction == in"]` will be expanded to `[get_ports sd_in[0]]` through `[get_ports sd_in[7]]` .

The argument `[load_of [get_lib_pins max_lib/pc3b03/I]]` will be expanded to its load value, e.g. `0.0325`.

©2018 Synopsys, Inc. All Rights Reserved.

What is *SCAN-DEF*?

- Contains scan chain information
- Used by physical design or layout tools to optimize the grouping and ordering of existing scan chains
- Standard *DEF* format used by ICC/ICC II and other 3rd party layout tools
- Note: Must write out the *SCAN-DEF* file prior to writing out the *ddc* netlist to include scan chain information in *ddc*!

```
write_scan_def -out <my_design.def>
```

10-7

©2018 Synopsys, Inc. All Rights Reserved.

Unit Agenda

Data needed for physical design

Netlist restrictions

- `assign` statement
- Special characters

10-8

©2018 Synopsys, Inc. All Rights Reserved.

Problem: Verilog `assign` Statements

The Problem: Layout tools may not be able to handle `assign` statements in the Verilog netlist

The Solution: Prevent `assign` statements in Verilog netlists caused by:

- Multiple port nets
- Verilog *tri* declarations

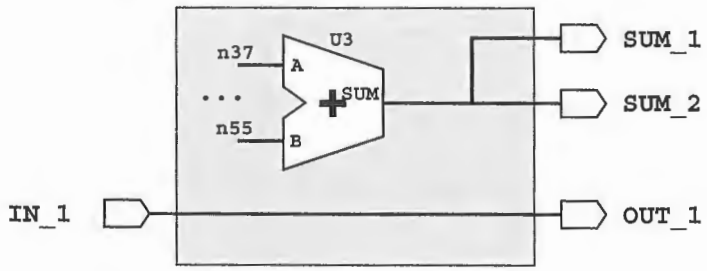
10-9

©2018 Synopsys, Inc. All Rights Reserved.

Multiple Port Nets Cause `assign` Statements

Multiple ports or hierarchy pins connected to the same signal, constant nets, as well as feed-throughs, use `assign`:

```
input IN_1, ...
output SUM_1, SUM_2, OUT_1, CONST_1, CONST_2 ...
...
DW01_add U3 (.SUM(SUM_1), .A(n37), .B(n55);
assign SUM_2 = SUM_1;
assign OUT_1 = IN_1;
```



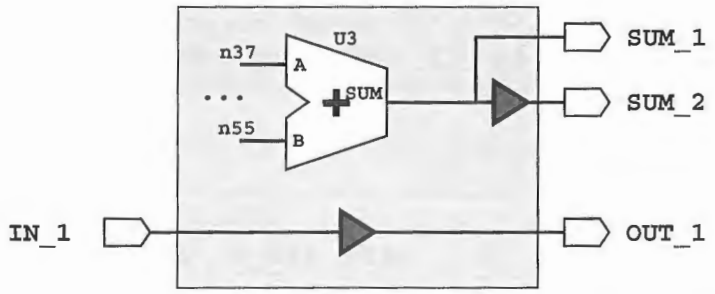
10-10

©2018 Synopsys, Inc. All Rights Reserved.

Preventing Multiple Port Nets

To ensure that your final netlist does not contain `assign` statements, separate the multiple port nets during compile:

```
set_fix_multiple_port_nets -all
compile_ultra ...
```



10-11

The `-all` option acts like `-feedthroughs`, `-outputs`, and `-constants` together: Inserts buffers to isolate input ports from output ports at all levels of the hierarchy; Inserts buffers so that no cell driver pin drives more than one output port at any level of the hierarchy; Logic constants are duplicated, not buffered, however, with `-buffer_constants`, logic constants are buffered instead of being duplicated.

By default `set_fix_multiple_port_nets` is applied to the entire design. You can optionally include a design list.

The code for the above design looks like this – notice – no `assign` statements:

```
DW01_add U3 ( .SUM(SUM_1), .A(n37), .B(n55) );
inv U1 ( .I(SUM_1), .Z(SUM_2) );
inv U2 ( .I(IN_1), .Z(OUT_1) );
```


©2018 Synopsys, Inc. All Rights Reserved.

Verilog `tri` Declarations Cause `assign` Statements

- DC uses `assign` statements for `tri` signals, but not for `wire` signals – either can be used to model connections with no logic function
- Solution: Automatically convert `tri` declarations to `wire` declarations before writing out the netlist

```
compile_ultra ...
...
set_app_var verilogout_no_tri true
write_file -f verilog -out ...
```

```
input IN_1, ...
output SUM_1,  OUT_1 ...
reg SUM_1, OUT_1, ...
tri SIG_1, SIG_2;      →    wire SIG_1, SIG_2;
```

10-12

©2018 Synopsys, Inc. All Rights Reserved.

Unit Agenda

Data needed for physical design

Netlist restrictions

- `assign` statement
- Special characters

10-13

©2018 Synopsys, Inc. All Rights Reserved.

Special Characters in Netlists

- ***Special characters*** **in port, cell and net names: Anything other than a number, letter, or underscore**

- **When DC writes out a netlist it inserts backslashes, if needed, to *escape* special characters in port, net and cell names - for example:**
  - The net `bus[7:0]` is expanded into 'scalar' names and becomes `\bus[7]`, `\bus[6]...`
    - The brackets in this case are just part of a scalar net name, not a special character denoting the 'slice' of a bus
  - VHDL multi-dimensional arrays use square brackets as word subscript delimiters: `\reg[0][19], \reg[0][18]..`

- **<u>The Problem</u>: Layout tools may not recognize DC's "\" *escape* convention and may therefore misinterpret the special characters**

10-14

©2018 Synopsys, Inc. All Rights Reserved.

Special Characters Solution: `change_names`

The Solution: Ensure that the netlist is free of special characters by automatically replacing *special characters* with *non-special* ones before writing out the netlist

```
compile_ultra...
...
set_app_var verilogout_no_tri true

change_names -rules verilog -hier

write_file -f verilog -out ...
write_scan_def ...
write_sdc ...
```

```
\bus[7]       →   bus_7_
\reg[0][19]   →   reg_0__19_
```

10-15

Note: The `verilog` / `vhdl` rules files can handle most common Verilog/VHDL netlist changes. It is possible to create customized "rules" for special needs.

©2018 Synopsys, Inc. All Rights Reserved.

Avoiding Bit-Blasted Ports: change_names

To avoid bit-blasting ports in SystemVerilog and VHDL records (packed or unpacked):

```
define_name_rules verilog -preserve_struct_ports
change_names -hierarchy -rules verilog
```

10-16

As shown in the following examples, the Verilog netlist 2 preserves the struct ports using the naming rules specified by the preceding two commands:

RTL

```
typedef struct {logic blue; logic red;} color;
module top (
   input  color i_color,
   output color o_color
);
assign o_color = i_color;
endmodule
```

Netlist 1 with bit-blasted struct ports

```
module top ( i_color_blue_, i_color_red_, o_color_blue_, o_color_red_ );
  input  i_color_blue_, i_color_red_;
  output o_color_blue_, o_color_red_;
  wire   o_color_blue_, o_color_red_;
  assign o_color_blue_ = i_color_blue_;
  assign o_color_red_  = i_color_red_;
endmodule
```

Netlist 2 preserving the struct ports

```
module top ( i_color, o_color );
  input  [1:0] i_color;
  output [1:0] o_color;
assign o_color[1] = i_color[1];
assign o_color[0] = i_color[0];
endmodule
```


©2018 Synopsys, Inc. All Rights Reserved.

Unit Objectives Review

You should now be able to:

Generate required post-synthesis output data files that take into account the abilities and restrictions of downstream physical design tools

10-17

©2018 Synopsys, Inc. All Rights Reserved.

This page was intentionally left blank

©2018 Synopsys, Inc. All Rights Reserved.

Agenda

DAY 3

6 Timing Analysis

7 Constraints: Multiple Clocks and Exceptions 

8 DC Graphical: SPG Flow, Congestion, Layout GUI

9 Constraints: Complex Design Considerations 

10 Post-Synthesis Output Data

11 Conclusion

© 2018 Synopsys, Inc. All Rights Reserved

©2018 Synopsys, Inc. All Rights Reserved.

Recall: Workshop Goal



- **Use *Design Compiler* to synthesize a block-level RTL design to generate a gate-level netlist with acceptable post-placement timing and congestion by:**
  - Reading hierarchical block-level RTL designs
  - Loading libraries, technology data and floorplan constraints
  - Applying and checking timing constraints
  - Using timing- and congestion-focused DC Ultra and SPG optimization features
  - Analyzing timing reports
  - Analyzing layout congestion (DC Graphical feature)
  - Generating output required by physical design tools

11-2

©2018 Synopsys, Inc. All Rights Reserved.

Achieving Best Timing, Congestion and Correlation

For best timing and congestion results, and post-placement correlation

- Good quality RTL code - optimum for DC
- Constrain and compile "top-down"
- Complete and accurate constraints
- Execute synthesis in *Topographical mode*
- Use all applicable *Ultra* optimization techniques
- Execute the *SPG* flow (*DC Graphical* feature)
- Analyze and reduce congestion (*DC Graphical* features)

Let's review each of the above in more detail ...

11-3

©2018 Synopsys, Inc. All Rights Reserved.

RTL Code & Steps After Loading the Design

Good quality RTL code – optimum for DC

- **This step can have a huge impact on the timing, area and power dissipation of your design!**
- **Refer to Online Documentation on SolvNet**
  - HDL Compiler User Guide: *VHDL*, *Verilog*, *SystemVerilog*
  - SolvNet Doc Id: 015771 "Coding Guidelines for Datapath Synthesis"
- **Good practice steps after loading the design:**

```
read_verilog {A.v B.v TOP.v}
current_design MY_TOP
link
check_design
list_libs
source TOP.con
```

11-4

©2018 Synopsys, Inc. All Rights Reserved.

Top-down Compile Strategy and Constraints

Constrain and compile "top-down"

- Simplifies the synthesis flow – no guessing of sub-block constraints
- Requires fewer compiles and iterations

Complete and accurate constraints

- I/O constraints represent external logic (remember Jane/Joe)
- If accurate spec is not available, use time- and load-budgeting for reasonable estimates
  - Do NOT over-constrain!
- Refine constraints as more accurate information is available
  - Always re-compile from RTL or unmapped ddc

11-5

©2018 Synopsys, Inc. All Rights Reserved.

Constraints – continued 1/2

Complete and accurate constraints - continued

- Use *master*, *virtual* and *generated* clocks to constrain multi-clock designs

```
create_clock -period 3 [get_ports CLK1]
create_clock -period 4 -name V_CLK2
create_generated_clock -divide_by 2 -name CLK1_SLW \
        -source [get_ports CLK1] [get_pins FF1_reg/QB]
```

- Apply and check all *timing exceptions* before compiling

```
set_false_path -from [get_clocks CLK1] -to [get_clocks CLK2]
set_false_path -from [get_clocks CLK2] -to [get_clocks CLK1]
set_multicycle_path -setup 6 -from A_reg[*] -to C_reg[*]
set_multicycle_path -hold  5 -from A_reg[*] -to C_reg[*]
report_timing_requirements -ignored
reset_path <copy-paste path from report>
```

- Define *clock groups* instead of *false paths* (when possible) for fewer constraints and better post-synthesis timing analysis with crosstalk

```
set_clock_groups -group CLK1 -group CLK2 \
  -logically_exclusive | -physically_exclusive | -asynchronous
```

11-6

©2018 Synopsys, Inc. All Rights Reserved.

Constraints – continued 2/2

Complete and accurate constraints - continued

- Check the syntax of constraint (and other) scripts

```
unix% dcprocheck TOP.con
```

- Verify that all internal and I/O paths are constrained, and all clocks are fully defined

```
dc_shell-topo> check_timing
dc_shell-topo> report_clock -skew -attr
dc_shell-topo> report_port -verbose
```

- Good-practice steps for a constraint file

```
# Include comments to document what you did
reset_design; # Remove pre-existing constraints
create_clock -period 5 [get_clocks CLK1]
```

11-7

©2018 Synopsys, Inc. All Rights Reserved.

Topographical Mode

Execute synthesis in *Topographical* mode

Placement during synthesis allows more accurate RC parasitic estimation, for better post-placement correlation

- Physical data required:
  - Physical standard cell and macro libraries (*Milkyway*)
  - Technology file
  - TLU Plus files + Layer mapping file
  - Floorplan `report_physical_constraints`
- Use two-pass synthesis flow to generate the netlist for floorplanning and then for placement
- Align DC-T settings with that of ICC/ICC II for best correlation:
  - SolvNET Doc ID 026366: DC-T versus ICC consistency
  - SolvNET Doc ID 2112936: DC-T/ICC versus ICC II consistency

```
set_compile_spg_mode icc | icc2
compile_ultra ...
```

11-8

©2018 Synopsys, Inc. All Rights Reserved.

Ultra Optimization techniques

Use all applicable *Ultra* optimization techniques

- Auto ungrouping
- Boundary optimization
- Test-ready synthesis
- Adaptive retiming
- High effort timing
- Prioritizing setup timing
- Disabling clock DRCs
- Path groups
- TNS-driven placement
- Pipeline retiming
- Multi-core optimization
- Final area recovery

```
set_ungroup [get_designs B] false
set_boundary_optimization [get_cells I_TEST] false
set_scan_configuration -style clocked_scan
set_optimize_registers true -design Pipeline
set_dont_retime [get_cells U_Pipe/P3_reg*] true
set_cost_priority -delay
set_auto_disable_drc_nets -on_clock_network true
set_app_var compile_timing_high_effort true
set_app_var placer_tns_driven true
group_path -name CLK1 -critical 0.3 -weight 5
group_path -name INPUTS  -from [all_inputs]
set_host_options -max_cores 8

compile_ultra -retime -scan [-no_autoungroup] [-no_boundary] [-incremental]

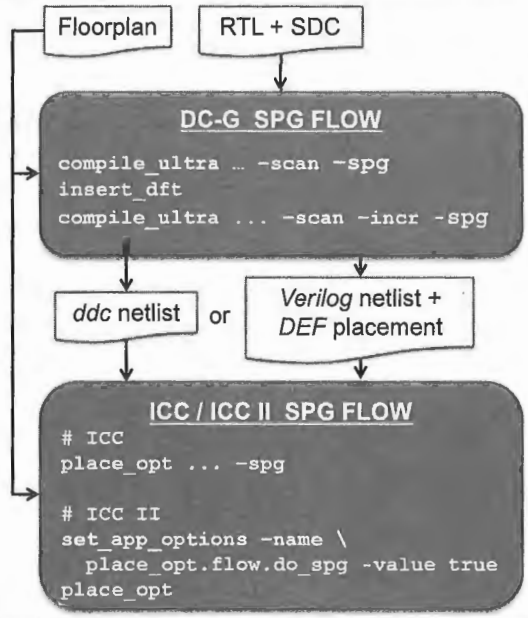
optimize_netlist -area
```

11-9

©2018 Synopsys, Inc. All Rights Reserved.

Synopsys Physical Guidance Flow

Execute the *Synopsys Physical Guidance* (SPG) flow



`compile_ultra .. -spg`

- Invokes *net layer optimization*
- Congestion-friendly netlist topology optimization
- Automatic register duplication
- Allows coarse placement to be saved in the *ddc* netlist or *DEF* file, which is sent to *ICC/ ICC II*

`place_opt` with *SPG* enabled

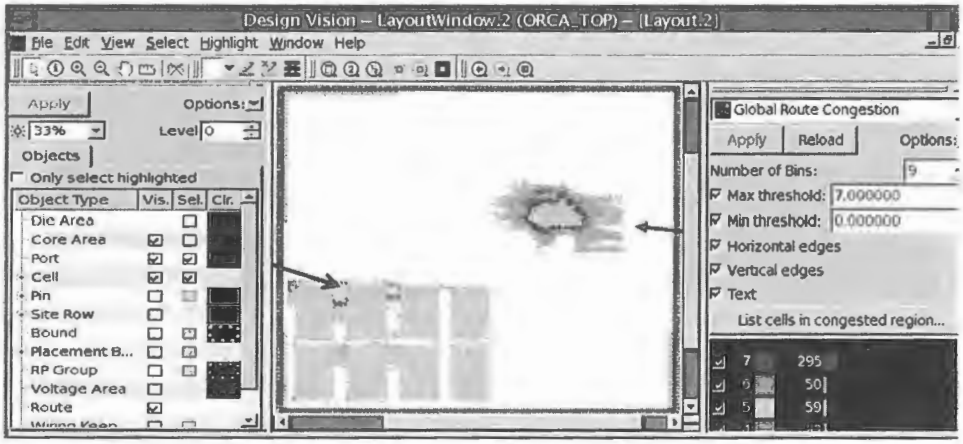
- Starts with *DC's* coarse placement and continues with logic optimization and legalization

11-10

©2018 Synopsys, Inc. All Rights Reserved.

Congestion Analysis

Analyze congestion after synthesis



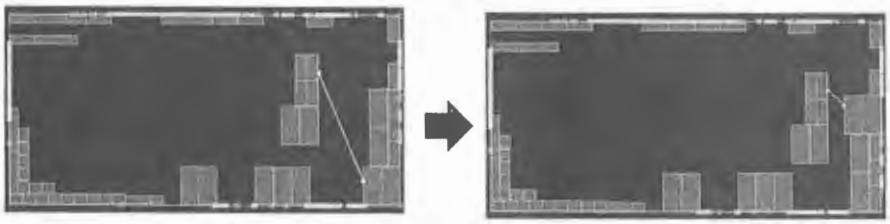
Identify floorplan and non-floorplan related congestion

11-11

©2018 Synopsys, Inc. All Rights Reserved.

DCG-DP Link

Modify the floorplan using DCG-DP link



Invoke the ICC/ICC II's Design Planning tool to modify the floorplan as needed to improve congestion and timing

11-12

©2018 Synopsys, Inc. All Rights Reserved.

Log Files and *RM* Scripts

- **Always create a run log file (Batch/Interactive shell)**

```
unix% dc_shell -topo | tee -i dc.log
unix% dc_shell -topo -f dc.tcl | tee -i dc.log
```

- **Redirect the output of key steps for later review**

```
read_verilog {A.v B.v TOP.v}
current_design MY_TOP
redirect -tee -file precompile.rpt {link}
redirect -append -tee -file precompile.rpt {check_design}
```

- **Use DC Reference Methodology Scripts**
  `https://solvnet.synopsys.com/rmgen/`
  - `common_setup.tcl`
  - `dc_setup.tcl`
  - `dc.tcl`

11-13

©2018 Synopsys, Inc. All Rights Reserved.

Post-Synthesis Analysis

- **After synthesis generate appropriate reports to analyze**
  - Logic DRC violations (`max_tran`, `max_cap`)
  - Timing violations
  - Area (in *db* as well as *Milkyway* library area units)
  - Congestion statistics

```
report_constraint -all
report_timing
report_qor
report_area -physical
report_congestion
```

11-14

©2018 Synopsys, Inc. All Rights Reserved.

Generate Output for Layout Tool

```
set_fix_multiple_port_nets -all; # Buffer multiple port nets - eliminate assign
compile_ultra ...
set_app_var verilogout_no_tri true; # Convert tri to wire - eliminate assign
define_name_rules verilog -preserve_struct_ports; # Avoid bit-blasting ports
change_names -rules verilog -hierarchy; # Eliminate special characters

# For ICC (and other Synopsys tools): Write out single ddc file
write_scan_def -out my_design.def[1]; # Annotate scan chain information to ddc
write_file -f ddc -hierarchy -output my_ddc.ddc
# For ICC II: Create directory with all required ASCII files
write_icc2_files  -out DESIGN_icc2
# For 3rd party tools:
write_file -f verilog -hierarchy -output my_verilog.v
write_sdc my_design.sdc; # Write out the constraints-only sdc file
write_scan_def -out my_design.def; # Write out scan chain information
```

11-15

©2018 Synopsys, Inc. All Rights Reserved.

Getting Help Within DC

- **For variables:**

```
printvar *_library;    # Lists all matching variables
                       # and corresponding values

man target_library;    # Complete 'man page'
```

- **For commands:**

```
help *clock;           # Lists all matching commands

create_clock -help;    # Lists command options

man create_clock;      # Complete 'man page'

# List command(s) containing the specified string,
# e.g. "period", in the command name or option(s):

apropos -symbols_only period;
```

11-16

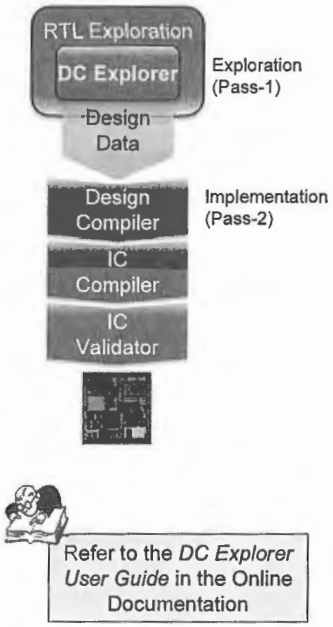
The apropos command performs pattern matching, like the Unix 'grep' utility. You can provide either the full or partial keyword. The search is performed on the entire 'man page' listing of commands – Returns any command that contains the keyword in the command name, the command option, or elsewhere in the command's man page description.

```
dc_shell-topo> apropos -symbols_only  period
 create_clock          # create clock
   [-name clock_name]      (name for the clock)
   [-period period_value] (period of the clock:
                           Value >= 0)
...
dc_shell-topo> apropos -symbols_only exclusive
 set_clock_groups      # set clock groups
   [-name name]            (Name for clock grouping)
   -logically_exclusive    (Specify exclusive clock groups)
   -physically_exclusive   (Specify exclusive clock groups)
 ...
dc_shell-topo> apropos -symbols_only clock
 all_clocks            # return all clocks in the current design
 all_inputs            # return input ports
   [-clock clock]          (ports with input delay relative to clock)
...
```


©2018 Synopsys, Inc. All Rights Reserved.

Introducing DC Explorer

- **Explores 1st pass netlist quicker**
  - 5-10X faster runtime, providing QoR within 10% of DC
  - Quickly explore early RTL and constraints
  - Send early netlist to *ICC / ICC II* for floorplan exploration
- **Tolerates missing/infeasible inputs**
  - Handles dirty netlist with missing/incorrect/mismatched pins and library cells, and missing/incomplete sub-designs
  - Reports and creates missing constraints
  - Optimizes feasible critical paths in the presence of infeasible paths
- **Provides script and flow compatibility**
  - Run existing Design Compiler script in *DC Explorer*
- **Works like DCT**
  - Reads floorplan if one is provided
  - Provides link to ICC/ICC II-DP to create/modify floorplan
  - Has congestion analysis capability



Refer to the *DC Explorer User Guide* in the Online Documentation

11-17

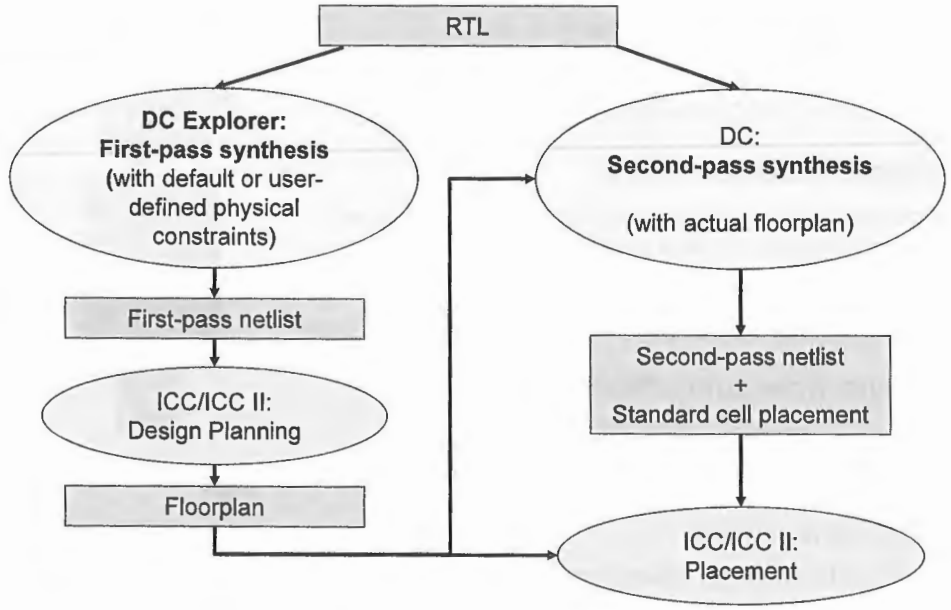
DC-Explorer is used during the "exploration phase" of the design: Can be used to quickly explore multiple versions of RTL, constraints, the floorplan, etc. Early netlist can be sent to IC Compiler / IC Compiler II for early floorplan exploration and physical implementation when exploration is completed, you take your "fleshed-out" design data (RTL, constraints and floorplan) to DC-Topo for actual synthesis.

Infeasible paths are paths which are not likely to meet timing. This can occur due to unachievable constraints, missing false or multi-cycle path constraints, or because there are simply too many levels of logic (which can be addressed in DC-T with register retiming techniques). By default, DC-T will ignore optimization of sub-critical paths, even very feasible sub-critical paths. DC-E will optimize "feasible paths" which would otherwise be "blocked" by the presence of "infeasible paths".

DC Explorer has a similar look and feel as using DC-Topographical:
- Uses a setup file named `.synopsys_de.setup`
- Is invoked using the `de_shell` command
- Use same or similar commands, for example: `compile_exploration` (instead of `compile_ultra` in DC-T)
- Accepts "physical constraints", just like DC-T
- Automatically performs command translation if an existing DC-T script is sourced

©2018 Synopsys, Inc. All Rights Reserved.

Design Explorer in Two-pass Synthesis Flow



11-18

In the above flow, *DC Explorer* is used for the first pass synthesis. There are no other changes to the flow.

©2018 Synopsys, Inc. All Rights Reserved.

SYNOPSYS®

Customer Support

© 2018 Synopsys, Inc. All Rights Reserved

20181001

© 2018 Synopsys, Inc. All Rights Reserved.

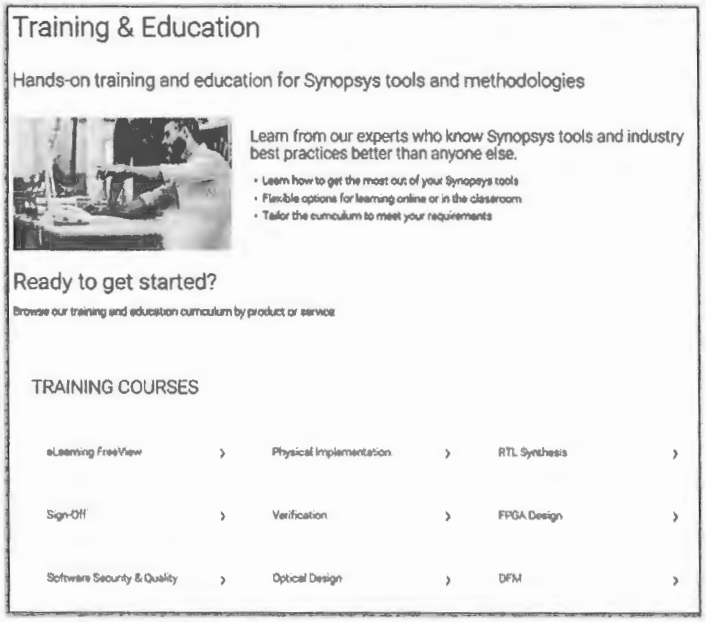
Synopsys Support Resources

- **Build a solid foundation:**
  Hands-on training for Synopsys tools and methodologies

  https://synopsys.com/support/training.html
  - Workshop Schedule and Registration
  - Download Labs
    (SolvNet ID required)
- **Drill down to areas of interest:**
  SolvNet online support

  https://solvnet.synopsys.com
  - Online technical information and access to support resources
  - Documentation & Media
- **Ask an Expert:**
  Synopsys Support Center

  https://onlinecase.synopsys.com



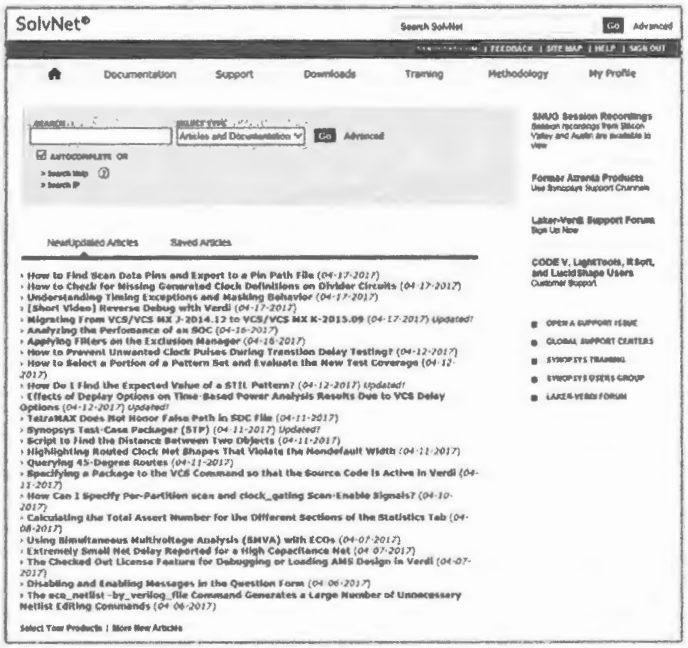
training.synopsys.com

CS - 2

© 2018 Synopsys, Inc. All Rights Reserved.

SolvNet Online Support

- Immediate access to the latest technical information
- Product Update Training
- Methodology Training
- Thousands of expert-authored articles, Q&As, scripts and tool tips
- Open a Support Center Case
- Release information
- Online documentation
- License keys
- Electronic software downloads
- Synopsys announcements (latest tool, event and product information)



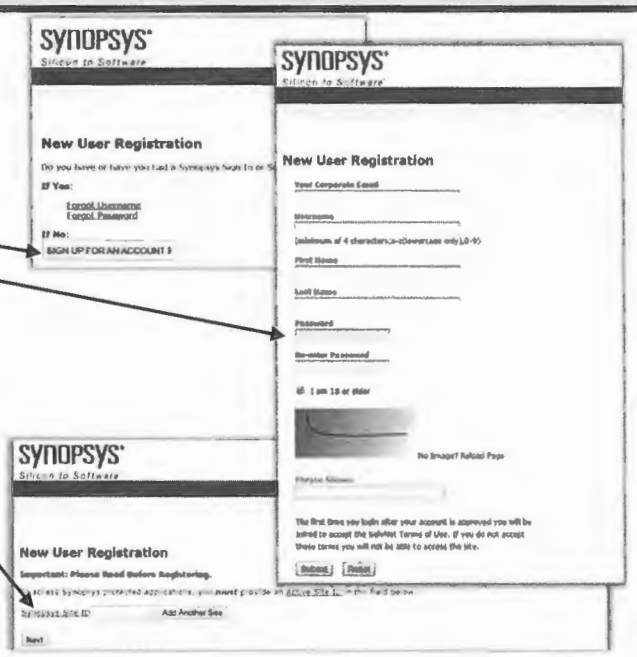
https://solvnet.synopsys.com

CS - 3

© 2018 Synopsys, Inc. All Rights Reserved.

SolvNet Registration

1. **Go to SolvNet page:**
   - https://solvnet.synopsys.com/
2. **Click on:**
   - "Sign Up for an Account"
3. **Pick a username and password.**
4. **You will need your "Site ID"**
   - For Information on how to find your Site ID, select the "Synopsys Site ID" link
5. **Authorization typically takes just a few minutes.**



https://solvnet.synopsys.com/ProcessRegistration

© 2018 Synopsys, Inc. All Rights Reserved.

Support Center

- **Industry seasoned Application Engineers:**
  - 50% of the support staff has >5 years applied experience
  - Many tool specialist AEs with >12 years industry experience
  - Engineers located worldwide
- **Great wealth of applied knowledge:**
  - Service >2000 issues per month
- **Remote access, and interactive debug, available via WebEx**

Contact us:
Open a support case



http://www.synopsys.com/Support/GlobalSupportCenters

CS - 5

© 2018 Synopsys, Inc. All Rights Reserved.

Other Technical Sources

- **Application Consultants (ACs):**
  - Tool and methodology pre-sales support
  - Contact your Sales Account Manager for more information
- **Synopsys Professional Services (SPS) Consultants:**
  - Available for in-depth, on-site, dedicated, custom consulting
  - Contact your Sales Account Manager for more details
- **SNUG (Synopsys Users Group):**
  https://www.synopsys.com/community/snug.html

CS - 6

© 2018 Synopsys, Inc. All Rights Reserved.

Summary: Getting Support

- **Customer Training**

  https://www.synopsys.com/support/training.html
  - Register for a Class
  - Download Labs
- **SolvNet**

  https://solvnet.synopsys.com
  - Tool Documentation and Support Articles
  - Product Update and Methodology Information / Training
  - Open a Support Case (Support Center)
- **Other Technical Resources**
  - Synopsys Users Group (SNUG)
  - Application Consultants
  - Synopsys Professional Services

CS - 7

© 2018 Synopsys, Inc. All Rights Reserved.

This page was intentionally left blank.

© 2018 Synopsys, Inc. All Rights Reserved.